Fine-grained, privacy-augmenting LI-compliance in the LAKE standard

Pascal Lafourcade*, Elsa López Pérez⋆, Charles Olivier-Anclin*, Cristina Onete†, Clément Papon†, and Mališa Vučinić⋆

* LIMOS, Université Clermont Auvergne, France
⋆ Inria, France
† XLIM, Université de Limoges, France

Abstract

The Internet Engineering Task Force and its LAKE working group standardized the Ephemeral Diffie-Hellman over COSE (EDHOC) authenticated key-exchange protocol for use in constrained Internet of Things deployments. The use cases include cellular networks, such as NB-IoT, but also non-cellular networks such as 6TiSCH, and LoRaWAN. As a result of its use in cellular networks, EDHOC will be subject to Lawful Interception (LI), which allows a group of authorities to break, if equipped with a warrant, the end-to-end (E2E) security of the channel established through EDHOC. Current implementations of EDHOC would only allow lawful interception by using the cellular network operator as a legitimate endpoint, essentially running a Person-in-the-Middle attack against the protocol. In this work, we focus on a privacy-preserving, fine-grained LI-compliant modification of EDHOC for all four authentication methods that this protocol currently supports. We achieve this via a careful white-box composition of EDHOC with the Lawful Interception Key-Exchange approach of Arfaoui et al. (ESORICS 2021) and Bultel and Onete (SAC 2022). Our resulting construction not only achieves strong key-security, but also non-frameability, and LI-compliance, without breaking the identity-protection property of EDHOC. Our implementation results show that, while LIKE adds an overhead to a standard EDHOC implementation in Rust, the resulting protocol remains practical while achieving much better privacy and LI-compliance.

This is an extended version of the article published as:
P. Lafourcade, E. López Pérez, C. Olivier-Anclin, C. Onete, C. Papon, M. Vučinić, "Fine-grained, privacy-augmenting LI-compliance in the LAKE standard", in Proceedings of the 30th European Symposium on Research in Computer Security (ESORICS) 2025, Springer Nature, 2026.
https://doi.org/10.1007/978-3-032-07901-5_13
This extended version includes additional results, detailed proofs, and new discussions not present in the original publication.

Table of contents

1 Introduction **5**
1.1 LAKE . . . 5
1.2 Our contribution . . . 6
1.3 Related work . . . 6

2 The EDHOC protocol **7**
2.1 Protocol outline . . . 7

3 Background **10**
3.1 Notations . . . 10
3.2 Cryptographic background . . . 10
3.2.1 Classical group assumptions . . . 10
3.2.2 The Oracle-DLog problem . . . 11
3.2.3 Signature scheme . . . 12
3.2.4 Zero-knowledge tools . . . 13
3.2.5 Encryption scheme . . . 14
3.2.6 Hash function . . . 17
3.3 The *identical-until-bad* lemma . . . 17
3.4 The LIKE framework . . . 18

4 LI-compliant EDHOC **19**
4.1 Setup and Key generation . . . 20
4.2 Precomputation stage . . . 21
4.3 Handshake - First case . . . 22
4.4 Handshake - Second case . . . 28
4.5 Handshake - Third case . . . 32
4.6 Handshake - Fourth case . . . 35

5 Implementation and evaluation **39**
5.1 Implementation setup . . . 39
5.2 Implementation results . . . 40

6 Security Model **42**
6.1 Execution environment . . . 42
6.2 Oracles . . . 44
6.3 Security properties . . . 46
6.3.1 Key-security . . . 46
6.3.2 LI-compliance . . . 46
6.3.3 Non-frameability . . . 49
6.3.4 Identity protection . . . 49

7 Security results **52**
7.1 Key security . . . 52
7.2 LI-compliance . . . 81
7.3 Non-frameability. . . . 96
7.4 Identity protection. . . . 101
7.4.1 Responder identity protection . . . 101
7.4.2 Initiator identity protection . . . 112

8 Conclusion **121**

9 Acknowledgments **121**

References **122**

1 Introduction

Privacy is a human right. The Universal Declaration for Human Rights [34], states: "*No one shall be subjected to arbitrary interference with his privacy [...] or correspondence [...]. Everyone has the right to the protection of the law against such [...] attacks.*" The United Nations describes privacy as a cornerstone of other basic human rights, like "*the free development and expression of an individual's personality, identity, and beliefs, and their ability to participate in political, economic, social and cultural life*" [35].

Edward Snowden's 2013 revelations of widespread mass surveillance engendered public outcry. Advocates for sacrificing privacy for the sake of (inter-)national security clashed with privacy supporters warning of censorship and autocracy. Examples of illegal mass-surveillance abound today. The NSA once collected call data from all Verizon customers, and data pertaining to calls in the Bahamas and Afghanistan [17]. During the COVID-19 pandemic, contact-tracing data was used in order to facilitate criminal investigations [16]. A recently-proposed EU regulation on moderating child sexual abuse material (CSAM) in encrypted communications enables mass client-side scanning, in spite of outspread criticism from both scientists [30] and socio-economical entities [12].

Mass-surveillance is a threat to basic human rights. Yet, even strong privacy advocates agree that investigations *limited in scope and motivation* (by lawfully-obtained warrants) can be legitimate [4]. This is the type of limited Lawful Interception (LI) that the *EU resolution on security through encryption and security despite encryption* supports [15].

Lawful Interception (LI) has been a legal and technical requirement for over 30 years in mobile communications, featuring prominently in 3GPP specifications [2, 1, 3]. Every operator that provides end-to-end encrypted user communication is subject to such requirements.

1.1 LAKE

The Lightweight Authenticated Key Exchange (LAKE) working group of the Internet Engineering Task Force (IETF) standardized a lightweight secure-channel establishment for use in NB-IoT, 6TiSCH, and LoRaWAN. Since it will be used in mobile environments, LAKE must comply with LI.

The solution proposed by LAKE is EDHOC [31]: a mutually-authenticated lightweight secure-channel establishment scheme, guaranteeing identity-protection as a form of privacy. The initiator and responder may choose to authenticate either through signatures or by using static DH-keys. As described in section 2, the protocol completes in 1.5, optionally 2, round trips. Starting from the 2nd message, communications may be authenticated and/or encrypted using intermediate secrets.

EDHOC is not LI-compliant by design. Naïvely, this can be achieved by turning mobile infrastructure nodes into endpoints – as is the case for other protocols, like AKA; this comes, however, at the cost of an *unnecessary loss of privacy*, making mobile network operators complicit to mass surveillance.

In this paper, we aim to render the EDHOC protocol LI-compliant without this massive loss of privacy. Ideally, our LI-EDHOC scheme must guarantee the security of exchanged messages except with respect to their sender, the receiver, and the collaborative efforts of *all* the Lawful Interception authorities[1] permitted to intercept. Moreover, interception should be fine-grained, limited only to one session at a time.

We use the pairing-free Lawful-Interception Key-Exchange (LIKE) approach [9] described in section 3. Applying LIKE to a real-world protocol such as EDHOC, which features four methods of authentication and complex key schedules, while also preserving EDHOC's lightweight character and identity-protection, is far from trivial. An important challenge, for instance, stems from the fact that EDHOC features two types of authentication, which can be combined arbitrarily. While signature-based authentication is compatible with

[1] LI is performed differently from one country to another. For instance in France, Lawful Interception requires the technical cooperation of the operator, the legislative branch, and law-enforcement. We model each participant that requires a cryptographic key as an authority – which allows us to construct our protocol in an elegant manner. In other countries, potentially more authorities might be required. Our system is flexible and can address all such scenarios. Our protocol is designed to allow parties to be aware of potential LI (as we believe transparency should be reinforced) – but this could be technically adjusted to allow proxies to indicate, in a prior message, the authorities that must be used for each exchange.

past LIKE approaches, it is not clear that MAC-based authentication will provide provable non-framewability. Another important challenge is combining LIKE with the complex key-schedule of the EDHOC protocol.

1.2 Our contribution

We describe LI-compliant extensions for EDHOC's four authentication methods, and guarantee:

- **Key-secrecy:** session traffic keys are indistinguishable from random except for that session's initiator, responder, and the collusion of all the authorities allowed to perform LI for that session.
- **Non-Frameability:** it is impossible to falsely accuse a user of taking part in a session that it did not run.
- **LI Compliance:** the mobile infrastructure nodes (proxies) forwarding the communication can prove keys computed in accepting sessions are lawfully-interceptable by the correct set of LI authorities.
- **Identity-protection:** an attacker (active for Initiator and passive for Responder) cannot learn the identities of the two endpoints.

Each property provably holds in the LIKE models of [5, 9], and a (modified) identity-protection model of [14]. Moreover, we provide an open-source implementation of our scheme in Rust optimized for constrained devices. We demonstrate the feasibility of our scheme by evaluating it on two hardware platforms that are typical examples of hardware used in the LAKE use cases.

1.3 Related work

Lawful Interception initially relied on Key-Escrow: the idea of entrusting communication to a managing trusted third party (TTP), which could learn the session key. Unfortunately, Key-Escrow often requires the online presence of authorities and can easily be pushed to mass surveillance. In spite of new LI techniques [6], current LI still relies on Key Escrow [36, 22].

Our work comes closest to a different approach [5] called LIKE, which aimed to achieve fine-grained LI-compliant Authenticated Key-Exchange (AKE) with better privacy. A first pairing-based instantiation [5] was rendered more efficient, pairing-free, and usable in roaming scenarios by Bultel and Onete [9]. One property provided by LIKE is non-frameability, which is also underlined by recent related work [8] in the context of 5G (and beyond) network architectures.

Some techniques for connection-monitoring and data-encryption [19, 21] could be extended to the LI scenario, though this is not their original goal. As they rely on pairings, they are incompatible with lightweight AKE, however. Finally, a worthwhile privacy-preserving alternative for LI is provided by CRUMPLE [37], which provides LI through a proof of work. This interesting approach is, unfortunately, not sufficiently efficient to comply with current LI requirements (which demand that interception yield "timely" results).

Finally note that throughout the standardization process of EDHOC [31], the protocol was analysed in various versions. A formal analysis using SAPIC+ identified weaknesses and proposed modifications in EDHOC version 12 [23]; later, the authors verified draft 14, showing that many previously identified issues had been addressed.

Cottier and Pointcheval then provided an analysis of EDHOC draft 15 [14, 13] and suggested further improvements. In 2023, Günther and Mukendi [18] analyzed draft 17 and proposed changes to the key schedule and to the construction of transcript hashes.

2 The EDHOC protocol

In response to a need for lightweight secure-channel establishment in the context of the Internet of Things (IoT), the Internet Engineering Task Force (IETF) is currently standardizing Lightweight Authenticated Key Exchange (LAKE), centered around the Ephemeral Diffie-Hellman Over COSE (EDHOC) protocol. EDHOC is a two-party mutually-authenticated Diffie-Hellman-based AKE scheme relying on Krawczyk's SIGMA-I protocol [25]. It features three mandatory messages and an optional fourth. The handshake is run in sessions between an Initiator I and a Responder R, which can choose how they want to authenticate: either by using signature schemes, or by using a certified static Diffie-Hellman (DH) key. The four combinations of authentication mechanisms are called methods 0, 1, 2, and 3, as described in Table 1. We denote by $(\mathsf{ssk_P}, \mathsf{spk_P})$ the private/public signature keys of party P, and by (p, g^p) party P's long-term static DH private/public keys. The parties indicate their preferred means of authentication method in `message_1`.

ID	Initiator	Credential I	Responder	Credential R
0	Signature	$(\mathsf{ssk_I}, \mathsf{spk_I})$	Signature	$(\mathsf{ssk_R}, \mathsf{spk_R})$
1	Signature	$(\mathsf{ssk_I}, \mathsf{spk_I})$	Static Diffie-Hellman	(r, g^r)
2	Static Diffie-Hellman	(i, g^i)	Signature	$(\mathsf{ssk_R}, \mathsf{spk_R})$
3	Static Diffie-Hellman	(i, g^i)	Static Diffie-Hellman	(r, g^r)

Table 1 – Current authentication methods registered by IANA (0-1-2-3).

In addition to these two authentication methods, the Working Group is currently working on the standardization of Pre-Shared Key (PSK) authentication for EDHOC, where the PSK can be either distributed using an out-of-band channel (fresh sessions), or it can be derived from previous handshakes using any of the already supported methods (0-1-2-3) (resumption session).

2.1 Protocol outline

Figure 1 depicts the EDHOC message flow. The first message, always sent unencrypted, consists of parameters `METHOD` and `SUITES_I`, indicating the authentication method and desired cipher suite, a *connection identifier* $\mathsf{C_I}$ which acts as a session identifier for peers running multiple sessions, some additional information called external authorization data $\mathsf{EAD_1}$, and a fresh ephemeral public key g^x (we denote the corresponding private key as x).

If the Responder supports the method and cipher suite, it computes `message_2`, which consists of an ephemeral DH element g^y sent in clear (the private key is denoted as y) and a ciphertext, encrypted with a key denoted as $\mathsf{sk_2}$, derived from g^x and g^y. The plaintext consists of: a connection identifier $\mathsf{C_R}$, an identifier indicating the Responder's long-term authentication credential, some additional data $\mathsf{EAD_2}$, and an authentication by the Responder, consisting of either a signature (methods 1 and 3) or a MAC (methods 0 and 2). The signature is computed with the Responder's private key $\mathsf{ssk_R}$ over the credential identifier ID_CRED_R, a transcript hash $\mathsf{TH_2}$ detailed in Figure 2, the public key $\mathsf{spk_R}$, and $\mathsf{EAD_2}$. If the Responder uses static-DH authentication, then the MAC key is derived from both the ephemeral DH product of g^x and g^y and from the semi-static DH product of g^x and the static element g^r. The Initiator decrypts the ciphertext in `message_2` with the key $\mathsf{sk_2}$, derived from g^x and g^y (the latter is sent in clear). Then, it recovers and verifies the Responder's authentication, according to the authentication method. `message_3` is somewhat similar to `message_2`, except that it is AEAD-encrypted using a key derived either solely from the ephemeral DH values g^x and g^y (methods 0 and 2) or from both the ephemeral DH product of g^x and g^y, and the semi-static DH product of g^x and g^r (methods 1 and 3).

The optional `message_4` provides explicit key-confirmation and is an AEAD-encryption on additional data $\mathsf{EAD_4}$. The encryption key is derived from the previously computed ephemeral (and potentially semi-static)

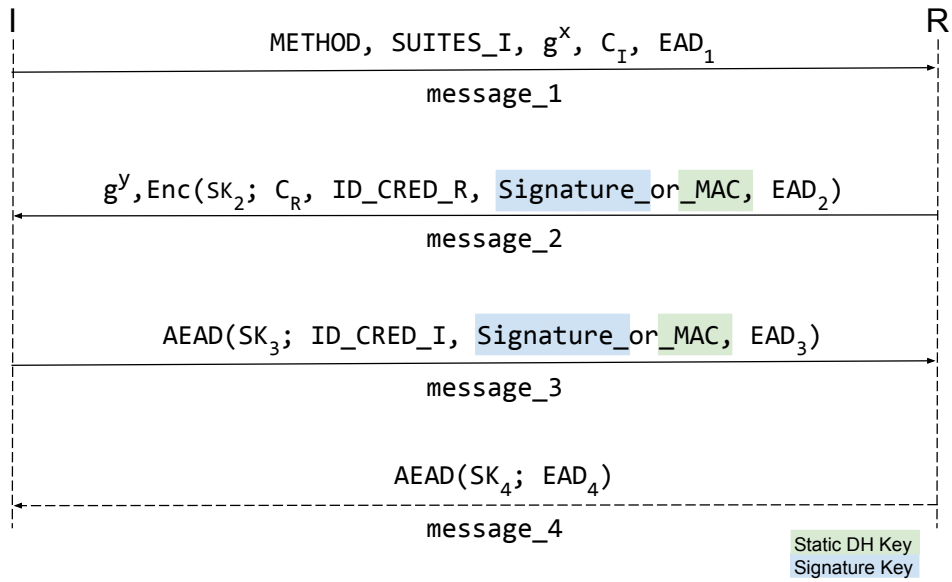

FIGURE 1 – The EDHOC message flow [28].

DH products, and additionally, for methods 2 or 3, a new semi-static DH product between the ephemeral value g^y and the Initiator's static DH element g^i.

The full key-schedule of the protocol is depicted in FIGURE 2. EDHOC uses two functions, `EDHOC_Expand` and `EDHOC_Extract`, with the EDHOC hash algorithm in the selected cipher suite to derive keys used in message processing. `EDHOC_Extract` is used to derive fixed-length uniformly pseudorandom keys (PRKs) from Elliptic Curve DIFFIE-HELLMAN (ECDH) shared secrets.
`EDHOC_Expand` is used to define a key derivation function, `EDHOC_KDF`, for generating MACs and for deriving output keying material (OKM) from PRKs.

There are three main secret values that are computed, labelled, respectively PRK_{2e}, PRK_{3e2m} and PRK_{4e3m}. The notations indicate the purposes of these secrets: the 2e subscript indicates the key is used to encrypt `message_2` content. The 3e2m subscript indicates the value is used to encrypt `message_3` and (depending on the authentication method) compute the MAC in `message_2`. Note that in case of signature-only authentication (method 0), the entire key-schedule is derived from the ephemeral DH values g^x and g^y. If both parties use static DH authentication, the key schedule resembles X3DH [27], using the ephemeral DH product and two semi-static ones.

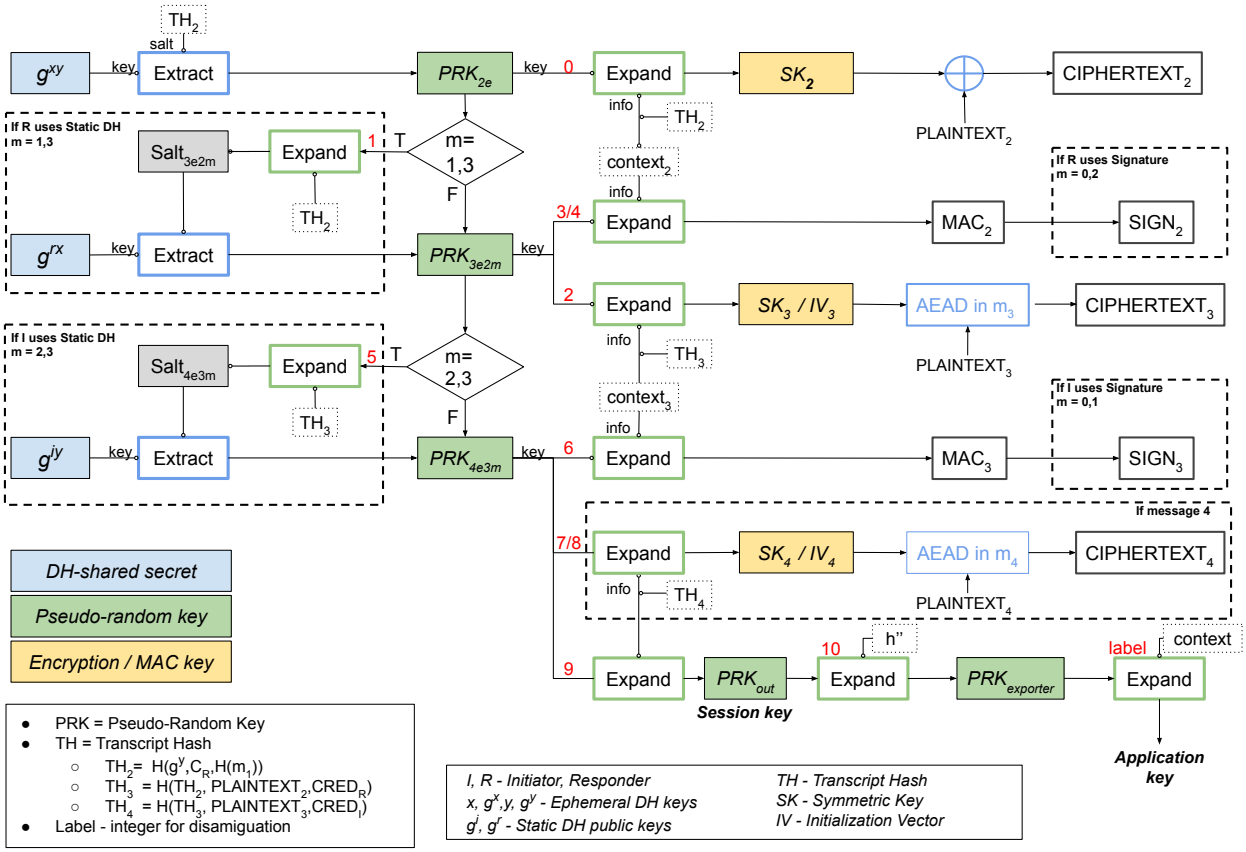

FIGURE 2 – EDHOC's key scheduling. The figure shows how keys are derived for all four authentication methods [29].

3 Background

We present here all the notations and the necessary cryptographic background to build our protocol in a solid and elegant manner, as well as to provide detailed and rigorous proofs.

3.1 Notations

With this short paragraph, we summarize the general notations we will use. These are mostly standard notations.

Throughout this paper, $\lambda \in \mathbb{N}^*$ is a security parameter, and we write $x \leftarrow y$ to denote that variable x is assigned value y. By $x \leftarrow\$ X$ we express the fact that x is sampled uniformly at random from the set X. The notation $y \leftarrow \texttt{Alg}(x)$ signifies that algorithm $\texttt{Alg}$ outputs y on input x.

When we talk about the EDHOC protocol, we will use the notations of the standardization [32], and during our modification, for the writing of the LI-EDHOC protocol, those that we will add will try to be in the continuity of the style. We also use some of [14] in order to lighten certain algorithmic notations.

Since we mix the EDHOC protocol with the LIKE protocol, we also use notations from [5, 9]. To refer to a general party, we use the letter P, but when we need to be more specific we use: E to denote an endpoint, Prox to denote a proxy and Λ to denote an authority. Some parties have attributes and to refer to them we use $\mathsf{P}.\{\text{the attribute}\}$. Finally if F is an event, we denote by $\mathbb{P}(F)$ the probability that the event F occurs.

3.2 Cryptographic background

3.2.1 Classical group assumptions

While EDHOC is a generic protocol which can be instantiated with different primitives, currently its main use relies on elliptic-curve cryptography (ECC) [29]. Moreover, as it will be illustrated several times in this paper, we use Diffie-Hellman elements, from a well chosen group $\mathbb{G}$. We thus exploit the cryptographic power of groups. For that we formalize the cryptographic assumptions needed for our security analysis. To begin with we want that in our group the DDH problem is hard to solve, the CDH assumption holds and the GDH problem is hard to solve. In what follows, $\mathbb{G}$ denotes a cyclic group of prime order p and g denotes a generator of $\mathbb{G}$.

Definition 1 (Discret Logarithm problem) *The* DL*-advantage of a p.p.t adversary $\mathcal{A}$ in a group $\mathbb{G}$ is defined with the experiment depicted in* Figure 3 *by:*

$$\mathsf{Adv}^{\mathrm{DL}}_{\mathbb{G},\mathcal{A}}(\lambda) = \mathbb{P}\Big[\mathbb{E}^{\mathrm{DL}}_{\mathbb{G},\mathcal{A}}(\lambda) \Longrightarrow 1\Big]$$

By $\mathsf{Adv}^{\mathrm{DL}}_{\mathbb{G}}(\lambda)$ we thus denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$. The DL *problem for the group $\mathbb{G}$ is said to be hard if $\mathsf{Adv}^{\mathrm{DL}}_{\mathbb{G}}(\lambda)$ is negligible in λ.*

Definition 2 (Decisional Diffie-Hellman problem) *The* DDH*-advantage of p.p.t. adversary $\mathcal{A}$ in a group $\mathbb{G}$ is defined thanks to the experiment depicted in* Figure 3 *by:*

$$\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G},\mathcal{A}}(\lambda) = \Big|\mathbb{P}\Big[\mathbb{E}^{\mathrm{DDH},1}_{\mathbb{G},\mathcal{A}}(\lambda) \Longrightarrow 1\Big] - \mathbb{P}\Big[\mathbb{E}^{\mathrm{DDH},0}_{\mathbb{G},\mathcal{A}}(\lambda) \Longrightarrow 1\Big]\Big|.$$

We then denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$ by $\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}}(\lambda)$. The DDH *problem for the group $\mathbb{G}$ is said to be hard if $\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}}(\lambda)$ is negligible in λ.*

Definition 3 (Computational Diffie-Hellman assumption) *The* CDH*-advantage of p.p.t. adversary $\mathcal{A}$ in a group $\mathbb{G}$ is defined thanks to the experiment depicted in* Figure 3 *by:*

$$\mathsf{Adv}^{\mathrm{CDH}}_{\mathbb{G},\mathcal{A}}(\lambda) = \mathbb{P}\Big[\mathbb{E}^{\mathrm{CDH}}_{\mathbb{G},\mathcal{A}}(\lambda) \Longrightarrow 1\Big].$$

We then denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$ by $\mathsf{Adv}^{\mathrm{CDH}}_{\mathbb{G}}(\lambda)$. The CDH *assumption for the group $\mathbb{G}$ states that $\mathsf{Adv}^{\mathrm{CDH}}_{\mathbb{G}}(\lambda)$ is negligible in λ.*

Definition 4 (Gap Diffie-Hellman problem) *The* Gap Diffie-Hellman *problem for the group $\mathbb{G}$ is defined thanks to the experiment depicted in* Figure 3 *played against a p.p.t. adversary $\mathcal{A}$, having access to a* DDH *oracle $\mathcal{O}_{\mathrm{DDH}}$.*
The GDH *problem is said to be hard for $\mathbb{G}$ if:*

$$\mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G},\mathcal{A}}(\lambda) = \mathbb{P}\Big[\mathbb{E}^{\mathrm{GDH}}_{\mathbb{G},\mathcal{A}}(\lambda) \Longrightarrow 1\Big]$$

is negligible in λ for all p.p.t. adversary $\mathcal{A}$. We then denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$ by $\mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda)$.

$\mathbb{E}^{\mathrm{DL}}_{\mathbb{G},\mathcal{A}}(\lambda)$

```
u ←$ Z_{p-1}, U ← g^u
u' ← A(U)
if g^{u'} = h then
    return 1
else
    return 0
```

$\mathbb{E}^{\mathrm{CDH}}_{\mathbb{G},\mathcal{A}}(\lambda)$

```
u, v ←$ Z_{p-1}
(U, V) ← (g^u, g^v)
W ← A(U, V)
if W = g^{uv} then
    return 1
else
    return 0
```

$\mathbb{E}^{\mathrm{GDH}}_{\mathbb{G},\mathcal{A}}(\lambda)$

```
u, v ←$ Z_{p-1}
(U, V) ← (g^u, g^v)
W ← A^{O_DDH(·,·,·)}(U, V)
if W = g^{uv} then
    return 1
else
    return 0
```

$\mathbb{E}^{\mathrm{DDH},0}_{\mathbb{G},\mathcal{A}}(\lambda)$

```
u, v, w ←$ Z_{p-1}
(U, V, W) ← (g^u, g^v, g^w)
b' ← A(U, V, W)
return b'
```

$\mathbb{E}^{\mathrm{DDH},1}_{\mathbb{G},\mathcal{A}}(\lambda)$

```
u, v ←$ Z_{p-1}
(U, V, W) ← (g^u, g^v, g^{uv})
b' ← A(U, V, W)
return b'
```

$\mathcal{O}_{\mathrm{DDH}}(g^u, g^v, g^w)$

```
if g^w = g^{uv} then
    return 1
else
    return 0
```

Figure 3 – The DL, DDH, CDH and GDH experiments

3.2.2 The Oracle-DLog problem

Interestingly, allowing both static-DH-based and signature-based authentication yields both a positive and a negative outcome. On the plus side, if one credential is lost or corrupted, the endpoint need not immediately re-initialize its public keys. Moreover, note that EDHOC's static-DH authentication mode is still forward-secure, since it is combined with an ephemeral secret. On the downside, however, the use of static-DH authentication renders non-frameability difficult to prove, requiring either heavier computations, or a slight modification of the standard Discrete Logarithm problem (we will require access to a CDH oracle, akin to how GDH requires DDH oracle access). We dub this new problem *Oracle-DLog*, which is similar to the *Q-One-More Diffie–Hellman* problem defined in [24].

Definition 5 (Oracle-DLog problem) *The* Oracle-DLog *problem is defined as follows. Let* $x \leftarrow\$ \mathbb{Z}_p^*$ *and* $X \leftarrow g^x$*. The* Oracle-DLog *problem challenges an adversary in possession of* X*, with oracle access to* $\mathcal{O}_{\mathsf{CDH}}^X(\cdot)$ *– returning, on input* $Y = g^y \in \mathbb{G}$*, the value* $g^{xy} \in \mathbb{G}$ *– to retrieve* x*, as depicted with the experiment of* FIGURE 4 *below.*

As for the previous group problems, we define the Oracle-DLog advantage of a p.p.t. adversary.

Definition 6 *The* Oracle-DLog-*advantage of p.p.t. adversary* $\mathcal{A}$ *in a group* $\mathbb{G}$ *is defined thanks to the experiment depicted in* FIGURE 4 *by:*

$$\mathsf{Adv}_{\mathbb{G},\mathcal{A}}^{\text{Oracle-DLog}}(\lambda) = \mathbb{P}\Big[\mathbb{E}_{\mathbb{G},\mathcal{A}}^{\text{Oracle-DLog}}(\lambda) \Longrightarrow 1\Big].$$

By $\mathsf{Adv}_{\mathbb{G}}^{\text{Oracle-DLog}}(\lambda)$ *we denote the maximum advantage over all the p.p.t. adversary* $\mathcal{A}$*. The* Oracle-DLog *problem is said to be hard in* $\mathbb{G}$ *if* $\mathsf{Adv}_{\mathbb{G}}^{\text{Oracle-DLog}}(\lambda)$ *is negligible in* λ*.*

$\mathbb{E}_{\mathbb{G},\mathcal{A}}^{\text{Oracle-DLog}}(\lambda)$	$\mathcal{O}_{\mathsf{CDH}}^{g^u}(g^v)$
$u \leftarrow\$ \mathbb{Z}_{p-1}, U \leftarrow g^u$	$W \leftarrow g^{uv} = V^u$
$u' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{CDH}}^U(\cdot)}(U)$	**return** W
if $u = u'$ **then**	
return 1	
else	
return 0	

FIGURE 4 – The Oracle-DLog experiment

3.2.3 Signature scheme

Signature-based authentication methods (*i.e.,* methods 0, 1, 2) in EDHOC require the use of an EUF-CMA-secure signature scheme (usually instantiated by EdDSA or ES256), while static-DH-based authentication is combined with a MAC scheme, which in EDHOC in fact implements as `HKDF_Expand` (thus, we idealize this call as a random oracle). We extend the use of digital signature scheme to proxies. In order to do that properly, we define what is a digital signature scheme and also what is the EUF-CMA property expected from it.

Definition 7 (Digital signature scheme) *A digital signature scheme* DS *consists of three algorithms:* DS.`Gen` *and* DS.`Sign` *which are probabilistic and* DS.`Verif` *which is deterministic. The first one,* DS.`Gen`*, takes as input the security parameter* 1^λ *and outputs a pair of public/private signing keys* $(\mathsf{PK}, \mathsf{SK})$*. The second one,* DS.`Sign`*, takes as input the secret key* SK *and a message* m *to sign, and outputs a signature* σ *for* m*. Finally,* DS.`Verif` *takes as input the public key* PK*, the message* m*, and the signature* σ*, and outputs* 1 *if* σ *is a valid signature for* m*, or* 0 *otherwise.*
Moreover, we require that DS *has* perfect correctness*, i.e., that for all* $(\mathsf{PK}, \mathsf{SK})$ *output by* DS.`Gen`*, for all message* m*,* $\mathsf{DS.Verif}(\mathsf{PK}, m, \mathsf{DS.Sign}(\mathsf{SK}, m)) = 1$*.*

Definition 8 (EUF-CMA property) *The* Existential Unforgeability against Chosen Messages Attacks *property against a p.p.t. adversary* $\mathcal{A}$ *for a digital signature scheme* $\mathsf{DS} = (\mathsf{DS.Gen}, \mathsf{DS.Sign}, \mathsf{DS.Verif})$*,*

having access to a signing oracle Sig*, is defined thanks to the experiment depicted in* FIGURE 5*.*
The digital signature scheme DS *is said existentially unforgeable against chosen messages attacks if:*

$$\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda) = \mathbb{P}\Big[\mathbb{E}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda) \Longrightarrow 1\Big]$$

is negligible in λ *for all p.p.t. adversary* $\mathcal{A}$*. We then denote the maximum advantage over all the p.p.t. adversary* $\mathcal{A}$ *by* $\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda)$*.*

$\mathbb{E}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda)$

1 : $(\mathsf{PK}, \mathsf{SK}) \leftarrow \mathsf{DS}.\mathtt{Gen}(1^\lambda)$
2 : $\mathcal{M} \leftarrow \emptyset$
3 : $(m, \sigma) \leftarrow \mathcal{A}^{\mathsf{Sig}(\cdot)}(\mathsf{PK})$
4 : **if** $\mathsf{DS}.\mathtt{Verif}(\mathsf{PK}, m, \sigma) = 1$ **and** $m \notin \mathcal{M}$
5 : **then return** 1
6 : **else**
7 : **return** 0

$\mathsf{Sig}(m)$

1 : $\sigma \leftarrow \mathsf{DS}.\mathtt{Sign}(m)$
2 : $\mathcal{M} \leftarrow \mathcal{M} \cup \{m\}$
3 : **return** σ

FIGURE 5 – The EUF-CMA experiment

3.2.4 Zero-knowledge tools

Following the work established in [5, 9], we embed in the EDHOC protocol LI elements, whihc are proved well constructed by the endpoint thanks to a signature of knowledge, a well-known primitive. We also use non-interactive proof of knowledge to add security on long-term key and trapdoors. We now define properly signature of knowledge and proof of knowledge, with the required properties for the security of the protocol. For that, let $\mathcal{R}$ be a binary relation and let $\mathcal{L}$ be a language such that $s \in \mathcal{L} \Longleftrightarrow \big(\exists w, (s, w) \in \mathcal{R}\big)$.

The objective of a prover with a Non-Interactive Proof of Knowledge (NIPoK) is to convince a verifier, possessing a statement s, that he knows a witness w such that $(s, w) \in \mathcal{R}$. The nature of the Zero-Knowledge (ZK) property of NIPoK additionally certifies that no information about w is revealed with the proof. Signature of knowledge (SoK) are similar to NIPoK. Here a prover will use its witness w as a signature private key to sign the message m, and then the verifier uses the statement as a public signature key to check the validity of the signature. If the verification is positive, the verifier is convinced by the prover that the latter know a witness for the statement. We can again here ask for a ZK property. In what follows, for a statement s, a witness w and a message m, we denote by $\mathsf{NIPoK}\{w : (w, s) \in \mathcal{R}\}$ and $\mathsf{SoK}_m\{w : (w, s) \in \mathcal{R}\}$ respectively NIPoK and SoK. We require that the one we use are ZK. We will give in that paper an example of such a signature of knowledge, using practical objects for implementation (but any kind of proof/signature of knowledge satisfying the security properties can be used).

Definition 9 (Non-interactive proof of knowledge) *A* Non-Interactive Proof of Knowledge *(NIPoK) for the language* $\mathcal{L}$ *is a pair of algorithms* (NIPoK, $\mathsf{NIPoKver}$) *verifying: (1)* $\mathsf{NIPoK}\{w : (s, w) \in \mathcal{R}\} \to \nu$*; and (2)* $\mathsf{NIPoKver}(s, \nu) \to b$*, where* b *is either* 1 *or* 0 *if the proof is valid or not.*
For memory, s *denotes the statement for which a prover knows a witness* w *and* ν *denotes the proof outputs by* NIPoK *and verified by a verifier.*

We require that our NIPoK has the following properties:

† *Completeness*: for all pair $(s, w) \in \mathcal{R}$, we have the equation:

$$\mathsf{NIPoKver}(s, \mathsf{NIPoK}\{w : (s, w) \in \mathcal{R}\}) = 1.$$

† *Perfect Zero Knowledge*: there exists a polynomial-time algorithm called the simulator and denoted $\mathtt{Sim}_{\mathsf{NIPoK}}$ taking as input a statement, such that for all pair $(s, w) \in \mathcal{R}$, $\mathtt{Sim}_{\mathsf{NIPoK}}$ and $\mathsf{NIPoK}\{w : (s, w) \in \mathcal{R}\}$ follow the same probability distribution. This is a formal reformulation of the zero-knowledge idea presented above. We require the simulator to generate a proof, without knowing w, that is indistinguishable from the one generated by the prover. If this is the case, it means that the real proof reveals no information about the secret.

† *Knowledge extractor*: there exists a probabilistic polynomial time (p.p.t.) algorithm called the extractor and denoted $\mathtt{Ext}_{\mathsf{NIPoK}}$ as well as a negligible function $\epsilon_{\mathsf{NIPoK}}$ of λ, such that for any algorithm $\mathcal{A}(\lambda)$ having access to the previous simulator $\mathtt{Sim}$ that forges signatures for chosen statement and that outputs a fresh tuple (s, ν) verifying $\mathsf{NIPoKver}(s, \nu) = 1$, then the extractor $\mathtt{Ext}_{\mathsf{NIPoK}}(\lambda)$, having access to $\mathcal{A}(\lambda)$, outputs a witness w such that $(s, w) \in \mathcal{R}$ with probability at least $1 - \epsilon_{\mathsf{NIPoK}}(\lambda)$.

The definition on signature of knowledge follows the same construction as NIPoK.

Definition 10 (Signature of knowledge) *A* Signature of Knowledge *(SoK) for the language $\mathcal{L}$ is then a pair of algorithms* $(\mathsf{SoK}, \mathsf{SoKver})$ *verifying: (1)* $\mathsf{SoK}\{m|w : (s, w) \in \mathcal{R}\} \to \sigma$*, where σ is a signature of the message m; and (2)* $\mathsf{SoKver}(m, s, \sigma) \to b$*, where b is either* 1 *or* 0 *if the signature is valid or not.*
For memory, m denotes the message to sign, s denotes the statement for which we know a witness w and σ denotes the signature outputs by SoK*.*

We require that our SoK verifies the following properties:

† *Completeness*: for all pair $(s, w) \in \mathcal{R}$, for all message m, we have the equation: $\mathsf{SoKver}(m, s, \mathsf{SoK}\{m|w : (s, w) \in \mathcal{R}\}) = 1$.

† *Perfect Zero Knowledge*: there exists a polynomial-time algorithm called the simulator and denoted $\mathtt{Sim}_{\mathsf{SoK}}$ taking as input a message and a statement, such that for all message m, for all pair $(s, w) \in \mathcal{R}$, $\mathtt{Sim}_{\mathsf{SoK}}(m, s)$ and $\mathsf{SoK}\{m|w : (s, w) \in \mathcal{R}\}$ follow the same probability distribution.

† *Knowledge extractor*: there exists a probabilistic polynomial time (p.p.t.) algorithm called the extractor and denoted $\mathtt{Ext}_{\mathsf{SoK}}$ as well as a negligible function ϵ_{SoK} of λ, such that for any algorithm $\mathcal{A}(\lambda)$ having access to the previous simulator $\mathtt{Sim}_{\mathsf{SoK}}$ that forges signatures for chosen message/statement pairs and that outputs a fresh tuple (m, s, σ) verifying $\mathsf{SoKver}(m, s, \sigma) = 1$, then the extractor $\mathtt{Ext}_{\mathsf{SoK}}(\lambda)$, having access to $\mathcal{A}(\lambda)$, outputs a witness w such that $(s, w) \in \mathcal{R}$ with probability at least $1 - \epsilon_{\mathsf{SoK}}(\lambda)$.

In this work, we rely on proofs of knowledge of discrete logarithms, which are known to be efficiently constructible [11]. Subsequently, signatures of knowledge can be derived from non-interactive zero-knowledge proofs using the Fiat–Shamir heuristic [10].

3.2.5 Encryption scheme

Key-derivation for EDHOC employs HKDF [26], which comes with an Extract-and-Expand mechanism. We idealize both steps as a random oracle. Once expanded, keys are used in two fundamental ways: for stream-cipher encryption (XOR-ing key-bits with plaintext-bits), and for AEAD-encryption (which we require to be AEAD-secure). Following [14], we describe in more detail the properties of the two encryption schemes

we use in the protocol. The first encryption is made with a symmetric encryption scheme $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ verifying the two following properties.

We require that our XOR-ing encryption scheme $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ satisfies the common and implicit following property: *injectivity*.

Definition 11 (Injective property) *The symmetric encryption scheme* $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ *satisfies the* Injective property *if :*

$$\forall \mathsf{sk} \in \mathcal{K}, \forall m_0, m_1 \in \mathcal{M},\ \mathsf{Enc}(\mathsf{sk}, m_0) = \mathsf{Enc}(\mathsf{sk}, m_1) \Longrightarrow m_0 = m_1$$

where $\mathcal{K}$ denotes the key space and $\mathcal{M}$ the message space.
In other words, for each fixed key and each pair of messages, if the encryption with the key the two messages gives the same ciphertext, then this implies that the two messages are the same, or said another way, for a fixed key, it is impossible to find two different messages which give the same ciphertext when encrypted with this key.

The identity protection property of the LI-EDHOC protocol on the Responder point of view, defined in [14], consist generally speaking in distinguishing the identity of two given Responder. This property requiries that our XOR-ing encryption scheme satisfies a very common property, close to the IND-CPA property, but only allowing the adversary to chose one pair of plaintext messages, the IND property.

Definition 12 (Indistinguishability property) *The* Indistinguishability property *of the encryption scheme* $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ *against a p.p.t. adversary $\mathcal{A}$ is defined thanks to the experiment* IND *depicted in* FIGURE 7. *The encryption scheme* $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ *satisfies the indistinguishability property if:*

$$\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec}),\mathcal{A}}(\kappa) = \mathbb{P}\Big[\mathbb{E}^{\mathrm{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec})}(\kappa) \Longrightarrow 1\Big]$$

is a function negligible in κ (κ a security parameter) for all p.p.t. adversary $\mathcal{A}$. We then denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$ by $\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec})}(\kappa)$.

For example, the OTP encryption scheme verifies these two properties. For the second encryption, we use an Authenticated Encryption with Associated Data (AEAD) encryption scheme, generally defined as follows.

Definition 13 (Authenticated Encryption with Associated Data) *An* Authenticated Encryption with Associated Data *encryption scheme is an encryption scheme* $(\mathsf{KGen}', \mathsf{Enc}', \mathsf{Dec}')$ *working as follows. The key-generation algorithm takes as input a security parameter κ and picks a random a secret key* sk *(from a key space $\mathcal{K}$) and an initialization vector* IV *(from an $\mathcal{IV}$ space) and returns* $(\mathsf{sk}, \mathsf{IV})$. *The encryption algorithm* Enc', *on input* $(\mathsf{sk}, \mathsf{IV})$, *a message m (of a message space $\mathcal{M}$) and some associated data a (from a data space $\mathcal{AD}$), outputs the ciphertext $c = \mathsf{Enc}'(\mathsf{sk}, \mathsf{IV}, m, a)$. Finally the decryption algorithm* Dec' *takes as input* $(\mathsf{sk}, \mathsf{IV})$, *a ciphertext c (from a ciphertext space $\mathcal{C}$) and some associated data a, returns $m = \mathsf{Dec}'(\mathsf{sk}, \mathsf{IV}, c, a)$ if c is a valid ciphertext for* sk, IV *and a, or $\perp$ otherwise.*

We need our AEAD encryption scheme to be secure in the following way.

Definition 14 *We say that our* AEAD *encryption scheme is secure if, for all p.p.t. adversary $\mathcal{A}$, the advantage* $\mathsf{Adv}^{\mathrm{Secure}}_{\mathrm{AEAD},\,\mathcal{A}}(\kappa)$, *defined thanks to the experiment depicted in* FIGURE 6 *by :*

$$\mathsf{Adv}^{\mathrm{Secure}}_{\mathrm{AEAD},\,\mathcal{A}}(\kappa) = \Big|\mathbb{P}\Big[\mathbb{E}^{\text{Secure-1}}_{\mathrm{AEAD}}(\kappa) \Longrightarrow 1\Big] - \mathbb{P}\Big[\mathbb{E}^{\text{Secure-0}}_{\mathrm{AEAD}}(\kappa) \Longrightarrow 1\Big]\Big|$$

is negligible compared to κ.
In other words, an adversary $\mathcal{A}$ has access, for a fixed key generated by the challenger, to an encryption and a decryption oracle. The adversary may query both. Depending on the game, the encryption oracle return a faithfully encrypt message or a random ciphertext of the expected length. On the other side, the decryption oracle return either the faithful decryption of a ciphertext (not previously generated by $\mathcal{A}$) or $\perp$. The advantage of $\mathcal{A}$ is then its probability of distinguishing which experiment he played.

$\mathbb{E}^{\text{Secure-1}}_{\text{AEAD}}(\kappa)$

```
sk ← KGen'(κ)
C' ← ∅
b' ← A^{O^1_{Enc'_sk}(·,·,·), O^1_{Dec'_sk}(·,·,·)}(κ)
if b' = 1 then
    return 1
else
return 0
```

$\mathcal{O}^b_{\mathsf{Enc}'_{\mathsf{sk}}}(\mathsf{IV}, m, a)$

```
if b = 1 then
    c ← Enc'(sk, IV, m, a)
else
    c' ← Enc'(sk, IV, m, a)
    c ←$ C^{|c'|}
C' ← C' ∪ {c}
    return c
```

$\mathbb{E}^{\text{Secure-0}}_{\text{AEAD}}(\kappa)$

```
sk ← KGen'(κ)
C' ← ∅
b' ← A^{O^0_{Enc'_sk}(·,·,·), O^0_{Dec'_sk}(·,·,·)}(κ)
if b' = 0 then
    return 1
else
    return 0
```

$\mathcal{O}^b_{\mathsf{Dec}'_{\mathsf{sk}}}(\mathsf{IV}, c, a)$

```
if b = 1 then
    if c ∉ C' then
        m ← Dec'(sk, IV, c, a)
    else
        return ⊥
else
    return ⊥
```

FIGURE 6 – AEAD secure games (here $\mathcal{C}^{|c'|}$ denotes the set of ciphertexts of length $|c'|$).

Then for the identity protection property on the point of view of the Initiator, we require that our AEAD encryption scheme satisfies the following two properties, ensuring that the adversary may not be able to distinguish between two Initiator's identities.

Definition 15 (IND-CPA property) *The* Indistinguishability under Chosen Plaintext Attack property *against a p.p.t. adversary $\mathcal{A}$ of an* AEAD *encryption scheme* $(\mathsf{KGen}', \mathsf{Enc}', \mathsf{Dec}')$ *is defined thanks to the experiment* IND *depicted in* FIGURE 7. *The* AEAD *encryption scheme satisfies the* IND-CPA *security property if:*

$$\mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD},\,\mathcal{A}}(\kappa) = \mathbb{P}\Big[\mathbb{E}^{\text{IND-CPA}}_{\text{AEAD}}(\kappa) \Longrightarrow 1\Big]$$

is negligible in κ (κ a security parameter) for all p.p.t. adversary $\mathcal{A}$. We then denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$ by $\mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD}}(\kappa)$.

Definition 16 (UF-CMA property) *The* UnForgeability under Chosen-Message Attack security property *of an* AEAD *encryption scheme* $(\mathsf{KGen}', \mathsf{Enc}', \mathsf{Dec}')$ *against a p.p.t. adversary $\mathcal{A}$, having access to an* AEAD *encryption oracle $\mathcal{O}_{\mathsf{Enc}'}$, is defined thanks to the experiment* UF-CMA *depicted in* FIGURE 7. *The* AEAD *encryption scheme satisfies the* UF-CMA *property if:*

$$\mathsf{Adv}^{\text{UF-CMA}}_{\text{AEAD},\,\mathcal{A}}(\kappa) = \mathbb{P}\Big[\mathbb{E}^{\text{UF-CMA}}_{\text{AEAD}}(\kappa) \Longrightarrow 1\Big]$$

is a function negligible in κ for all p.p.t. adversary $\mathcal{A}$. We then denote the maximum advantage over all the p.p.t. adversary $\mathcal{A}$ by $\mathsf{Adv}^{\text{UF-CMA}}_{\text{AEAD}}(\kappa)$.

$\mathbb{E}^{\text{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec}),\mathcal{A}}(\kappa)$

$b \leftarrow\$ \{0,1\}$
$\mathsf{sk} \leftarrow \mathsf{KGen}(\kappa)$
$(m_0, m_1) \leftarrow \mathcal{A}(\mathcal{M})$
$c \leftarrow \mathsf{Enc}(\mathsf{sk}, m_b)$
$b' \leftarrow \mathcal{A}(c)$
if $b = b'$ **then**
  return 1
else
  return 0

$\mathbb{E}^{\text{IND-CPA}}_{\text{AEAD},\mathcal{A}}(\kappa)$

$b \leftarrow\$ \{0,1\}$
$\mathsf{sk} \leftarrow \mathsf{KGen}'(\kappa)$
$(\mathsf{IV}_j, a_j, m_{0,j}, m_{1,j})_{j=1}^n \leftarrow \mathcal{A}(\kappa)$
$(c_j)_{j=1}^n \leftarrow (\mathsf{Enc}'(\mathsf{sk}, \mathsf{IV}_j, m_{b,j}, a_j))_{j=1}^n$
$b' \leftarrow \mathcal{A}((c_j)_{j=1}^n)$
if $b = b'$ **then**
  return 1
else
  return 0

$\mathbb{E}^{\text{UF-CMA}}_{\text{AEAD},\mathcal{A}}(\kappa)$

$\mathsf{sk} \leftarrow \mathsf{KGen}'(\kappa)$
$\mathcal{C}' \leftarrow \emptyset$
$(\mathsf{IV}, a, c) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Enc}'_{\mathsf{sk}}}(\cdot,\cdot,\cdot)}(\kappa)$
if $\mathsf{Dec}'(\mathsf{sk}, \mathsf{IV}, c, a) \neq \perp$ **and** $c \notin \mathcal{C}'$
  then return 1
else
  return 0

$\mathcal{O}_{\mathsf{Enc}'_{\mathsf{sk}}}(\mathsf{IV}, m, a)$

$c \leftarrow \mathsf{Enc}'(\mathsf{sk}, \mathsf{IV}, m, a)$
$\mathcal{C}' \leftarrow \mathcal{C}' \cup \{c\}$
return c

FIGURE 7 – The IND, IND-CPA and UF-CMA experiments

3.2.6 Hash function

We also need a hash function $\mathcal{H}$ at different stages. In particular, the key-derivation steps rely on the computation and input of updated session hashes.

We idealize this hash function $\mathcal{H}$ as a random oracle (in practice, instantiated as SHA256).

The choice of precise primitives as described in [29] makes it possible to respect, in particular, the physical implementation constraints. In our case, we consider more general and theoretical primitives possessing some specific cryptographic properties, while ensuring that those used in [29] verify these properties.

3.3 The *identical-until-bad* lemma

Another important tool that will be used in our security proofs is the famous *identical-until-bad* lemma (also known as *the fundamental lemma of game-playing* in [7], or as (an adaptation of) *the difference lemma* in [33]).

Theorem 1 (The *identical-unitl-bad* lemma) *Let $\mathbb{G}_q$ and $\mathbb{G}_j$ be two games and let F be an event. We consider Z the set of all possible outputs common to both games $\mathbb{G}_q$ and $\mathbb{G}_j$. We assume that for all $\zeta \in Z$, the events "$\mathbb{G}_q \Longrightarrow \zeta$", "$\mathbb{G}_j \Longrightarrow \zeta$" and F are defined on the same probability space. We also assume that $\mathbb{G}_q$ and $\mathbb{G}_j$ behave identically unless the event F occurs in one of them, i.e., for all $\zeta \in Z$, $\big[(\mathbb{G}_q \Longrightarrow \zeta) \wedge \neg F\big] \Longleftrightarrow \big[(\mathbb{G}_j \Longrightarrow \zeta) \wedge \neg F\big]$.*
Under the previous assumptions, we have, for all $\zeta \in Z$:

$$\big|\mathbb{P}(\mathbb{G}_q \Longrightarrow \zeta) - \mathbb{P}(\mathbb{G}_j \Longrightarrow \zeta)\big| \leq \mathbb{P}(F).$$

Proof 1 The proof is quite direct. Let $\zeta \in Z$. First, by the law of total probability, we have:

$$\left|\mathbb{P}\big(\mathbb{G}_q \Longrightarrow \zeta\big) - \mathbb{P}\big(\mathbb{G}_j \Longrightarrow \zeta\big)\right| = \Big|\mathbb{P}\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge F\right] + \mathbb{P}\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge \neg F\right] - \mathbb{P}\left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge F\right] - \mathbb{P}\left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge \neg F\right]\Big|.$$

Since by assumption $\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge \neg F\right] \iff \left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge \neg F\right]$, it comes that $\mathbb{P}\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge \neg F\right] = \mathbb{P}\left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge \neg F\right]$, and we now obtain:

$$\left|\mathbb{P}\big(\mathbb{G}_q \Longrightarrow \zeta\big) - \mathbb{P}\big(\mathbb{G}_j \Longrightarrow \zeta\big)\right| = \Big|\mathbb{P}\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge F\right] - \mathbb{P}\left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge F\right]\Big|.$$

But it is clear that $\mathbb{P}\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge \neg F\right] \in [0; \mathbb{P}(F)]$ and $\mathbb{P}\left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge \neg F\right] \in [0; \mathbb{P}(F)]$. So the absolute value of the difference cannot exceed $\mathbb{P}(F)$, which can be rewritten as:

$$\left|\mathbb{P}\big(\mathbb{G}_q \Longrightarrow \zeta\big) - \mathbb{P}\big(\mathbb{G}_j \Longrightarrow \zeta\big)\right| = \Big|\mathbb{P}\left[\big(\mathbb{G}_q \Longrightarrow \zeta\big) \wedge F\right] - \mathbb{P}\left[\big(\mathbb{G}_j \Longrightarrow \zeta\big) \wedge F\right]\Big| \leq \mathbb{P}(F).$$

This is what was announced, and this concludes the proof. ■

3.4 The LIKE framework

Originally introduced in the context of mobile authenticated key-exchange protocols, Lawful Interception Key Exchange (LIKE) is a two-party AKE scheme featuring two proxies, representing the serving networks of the two endpoints [5]. The system also features a number of authorities, which need not be online for the duration of each protocol session. Each handshake is lawfully-interceptable by the collaboration of all the members of an authority set, whose size and composition depend on current legislations.

Formally, LIKE is defined as $\mathsf{LIKE} = (\texttt{Setup}, \texttt{UKeyGen}, \texttt{OpKeyGen}, \texttt{AKeyGen}, \texttt{AKE}, \texttt{Verify}, \texttt{TDGen}, \texttt{Open})$. `Setup` provides a global setup for the choice of universal parameters. The next three algorithms allow mobile users, operators, and authorities to generate long-term public keys. The LIKE protocol is run in sessions via the `AKE` algorithm. The verification algorithm allows for the verification of the soundness of the handshake and for the validation of the participants, whereas the last two algorithms provide mechanisms for LI.

LIKE assumes a scenario in which the two endpoints can only communicate through the proxies, and the protocol guarantees: key-security, non-frameability, and LI compliance (also called "honest-operator" in [5, 9]). In this paper, we start from [9], a follow-up work of [5] which achieved pairing-free LIKE in the roaming scenario, by using an Elgamal-like encryption of the session secret.

Our approach carefully combines this second LIKE approach, and makes use of both (non-interactive) proofs and signatures of knowledge.

4 LI-compliant EDHOC

In order to empower mobile network operators (MNOs) with the ability to provide privacy-preserving LI-compliant secure-channel establishment, we encrypt session secrets in the session state, such that only the collaboration of the entire authority-set will be able to decrypt these values. For that purpose, we essentially combine the LIKE approach and the EDHOC protocol (section 2).

The composition is not black-box, for several important reasons. The first is our wish to preserve as many of the EDHOC properties as possible, including identity-protection. The latter requires that network adversaries (passive, for Responders, active, for Initiators) be unable to distinguish the endpoint's identity. As a result, some of the LIKE messages will need to be incorporated in EDHOC's encrypted messages, modifying them in a non-trivial fashion. As a result, we prefer to use different notations for the two protocols, referring to EDHOC's first and second messages as `message_1` and `message_2` (as in section 2), as opposed to M_1 and M_2 for our protocol LI-EDHOC. Another problem is verifying the consistency of the elements input in the key-computation, which we discuss below. Finally, we choose white-box composition for the sake of efficiency – since some LIKE elements are already present in EDHOC.

An important modification we make to the original LIKE [5, 9] is that the session keys take in input an additional nonce n_r (and we use the exporter keys featured in EDHOC). This artifice of our construction is necessary in order to enforce LI-compliance on the initiator's side. Thus, we include the nonce n_r in the context of the exporter PRK expansion (bottom right of Figure 2). Encrypting part of the handshake also raises a serious concern with respect to LI-compliance, in which proxies must be able to prove that the elements required to compute the key are consistent with the encrypted secrets and the ciphertexts. A naïve solution would be to send the entire messages unencrypted (thus nullifying the identity-protection afforded by EDHOC). Our solution tries to reconcile both LI requirements and identity-protection. We make endpoints send their encryption keys (not secrets) for handshake messages, encrypted with a key shared between the endpoint and the proxy[2], which allows the latter to verify the consistency of the transcript[3].

In order to provide LI-recovery of session keys, we essentially encrypt some component session secrets, as specified in Table 2. In the rest of this section, we will explain how, using elements H (and optionally H^1 and H^2), authorities may recover g^{xy} (and g^{rx} and g^{iy}). This could be done naïvely by running the protocol of [9] for each secret in parallel; that, however, would be suboptimal and could potentially break the LI-compliance guarantee, since EDHOC features an inter-dependence between the ephemeral and semi-static secrets. Each secret will be recovered by the authorities from an ElGamal-like encryption of it under the product of the authorities' keys. During the protocol, each authority derives a trapdoor (essentially a local decryption with just its own private key) – and by using the homomorphic properties of ElGamal, the combination of all trapdoors yields the session's secret. Crucially, if even one authority withholds its trapdoor, the secret remains hidden – even from a collusion of all other authorities.

Our white-box approach to combining the LIKE approach and EDHOC enables us to minimise the overhead introduced to EDHOC, while reusing its elements (such as the ephemeral DH values g^x, g^y) as components in both protocols. To further reduce computational overhead, we use Schnorr-based proofs and signatures of knowledge, applying the Fiat–Shamir heuristic to make them non-interactive.

We proceed to describe our protocol, whose handshake is also depicted in Figure 8. This is a high level description. Much more details, for each version of the protocol, are depicted in Figures 9 and 10 ($\mathsf{ID} = 0$), in Figures 11 and 12 ($\mathsf{ID} = 1$), in Figures 13 and 14 ($\mathsf{ID} = 2$) and in Figures 15 and 16 ($\mathsf{ID} = 3$).

[2]This is not a strong assumption, as the mobile protocol stack feature a hierarchy of keys usable to that effect.

[3]We considered two alternative approaches. The key under which the keys are encrypted could be negotiated during the handshake, at the expense of additional complexity. We discarded this alternative since a shared key is already present in the 5G stack. A second, cleaner alternative would be to encrypt handshake messages with a different key, which is computable by the proxies and endpoints alike. Unfortunately this would require modifying the key schedule of EDHOC (already standardized) and potentially introducing cumbersome key-computation tools including pairings.

Mode	g^{xy}	g^{xr}	g^{iy}	Secret	Initiator/Responder SoK statement stm for the associated secret
Sign-Sign (0)	✓	✗	✗	x y	$X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x$ $Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y$
Sign-Static (1)	✓	✓	✗	x $(\boxed{y}, \boxed{r})$	$X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^r)^x$ $\boxed{Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y} \wedge \boxed{Y_1 = g^r \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^x)^r}$
Static-Sign (2)	✓	✗	✓	$(\boxed{x}, \boxed{i})$ y	$\boxed{X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x} \wedge \boxed{X_1 = g^i \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^y)^i}$ $Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^i)^y$
Static-Static (3)	✓	✓	✓	$(\boxed{x}, \boxed{i})$ $(\boxed{y}, \boxed{r})$	$\boxed{X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^r)^x} \wedge \boxed{X_1 = g^i \wedge \mathsf{H_I^2} = (\mathsf{h_I} g^y)^i}$ $\boxed{Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^i)^y} \wedge \boxed{Y_1 = g^r \wedge \mathsf{H_R^2} = (\mathsf{h_R} g^x)^r}$

TABLE 2 – Secrets that are openable by LI and associated SoK statement for the Initiator and the Responder.

Initiator		Proxy $\mathsf{P_I}$		Proxy $\mathsf{P_R}$		Responder
$\mathtt{InitRun1}(\mathsf{ID_I})$	$\xrightarrow{M_1}$	$\mathtt{Prox_I Run1}(\mathsf{ID_I}, M_1)$	$\xrightarrow{M_1}$	$\mathtt{Prox_R Run1}(\mathsf{ID_R}, M_1)$	$\xrightarrow{M_1}$	$\mathtt{RespRun1}(\mathsf{ID_R}, M_1)$
$\mathtt{InitRun2}(\mathsf{ID_I}, M_2)$	$\xleftarrow{M_2}$	$\mathtt{Prox_I Run2}(\mathsf{ID_I}, M_2)$	$\xleftarrow{M_2}$	$\mathtt{Prox_R Run2}(\mathsf{ID_R}, M_2)$	$\xleftarrow{M_2}$	
	$\xrightarrow{M_3}$	$\mathtt{Prox_I Run3}(\mathsf{ID_I}, M_3)$	$\xrightarrow{M_3'}$	$\xrightarrow{M_3'}$	$\xrightarrow{M_3'}$	$\mathtt{RespRun2}(\mathsf{ID_R}, M_3')$
$\mathtt{InitRun3}(\mathsf{ID_I}, M_4'')$	$\xleftarrow{M_4''}$	$\mathtt{Prox_I Run4}(\mathsf{ID_I}, M_4')$	$\xleftarrow{M_4'}$	$\mathtt{Prox_R Run3}(\mathsf{ID_R}, M_4)$	$\xleftarrow{M_4}$	

FIGURE 8 – Message flow of the LI-EDHOC handshake protocols.

4.1 Setup and Key generation

To generate the security public parameter pp from the security parameter 1^λ, we use the $\mathtt{Setup}$ algorithm from [9].

$\underline{\mathtt{Setup}(1^\lambda)}$: Based on λ, it chooses a group $\mathbb{G}$ of prime order p, with a generator g of $\mathbb{G}$, and outputs $\mathsf{pp} = (\lambda, \mathbb{G}, p, g)$.

In this protocol, each authority Λ involved in the interception will need to generate a pair of public/private keys $(\Lambda.\mathsf{SK}, \Lambda.\mathsf{PK})$. The public keys will be used to constructed a tool, which will be inserted into the message flow and which will later enable lawful interception, using the private keys to generate trapdoors. The two proxies participating in the exchange also need to generate a pair of public/private signing keys $(\mathsf{ssk_{Prox}}, \mathsf{spk_{Prox}})$. Indeed, during the exchange they store (public) data, useful for lawful interception when transmitted to authorities. To ensure that these data are correct, they need to sign it with their private key. When lawfully intercepting, authorities check that the signature is correct with proxy's public key. To generate these key pairs, we use the key generator algorithm $\mathsf{DS}.\mathtt{Gen}$ from a fixed digital signature scheme DS. We now describe how these algorithms work.

$\underline{\mathtt{PKeyGen}(\mathsf{pp})}$: Run $(\mathsf{ssk_{Prox}}, \mathsf{spk_{Prox}}) \leftarrow \mathsf{DS}.\mathtt{Gen}(1^\lambda)$ and return $(\mathsf{ssk_{Prox}}, \mathsf{spk_{Prox}})$.

$\underline{\mathtt{AKeyGen}(\mathsf{pp})}$: Pick $\Lambda.\mathsf{SK} \leftarrow\$ \mathbb{Z}_p^*$. Compute $\Lambda.\mathsf{pk} \leftarrow g^{\Lambda.\mathsf{SK}}$ and $\Lambda.\mathsf{ni} \leftarrow \mathsf{NIPoK}\{\Lambda.\mathsf{SK} : \Lambda.\mathsf{pk} = g^{\Lambda.\mathsf{SK}}\}$. Set $\Lambda.\mathsf{PK} \leftarrow (\Lambda.\mathsf{pk}, \Lambda.\mathsf{ni})$ and return $(\Lambda.\mathsf{PK}, \Lambda.\mathsf{SK})$.

We also need to generate the public and private keys of the endpoints (users), namely the Initiator and the Responder. Depending on the version of the LI-EDHOC protocol (*i.e.,* $\mathsf{ID} = 0, 1, 2$ or 3), the keys pair

has a different shape (either a DIFFIE-HELLMAN element, or signature keys). The $\mathtt{EKeyGen}$ algorithm, for an endpoint E, generates these two pairs of keys, an latter when running a protocol session, E selects the appropriate pair.

$\underline{\mathtt{EKeyGen}(\mathsf{pp})}$: This algorithm generates a signing keys pair $(\mathsf{ssk_E}, \mathsf{spk_E})$ using the $\mathsf{DS}.\mathtt{Gen}$ algorithm from DS, and also sample $e \leftarrow\$ \mathbb{Z}_p^*$ and generates (e, g^e) $(:= (\mathsf{E.SK}, \mathsf{E.PK}))$, a static DIFFIE-HELLMAN keys pair. It then returns $((\mathsf{ssk_E}, \mathsf{spk_E}), (e, g^e))$. Then depending on the method used for the key-exchange, the user selects one of the two pairs of (long term) keys that will be used for the specific protocol run.

4.2 Precomputation stage

Handshakes are preceded by a precomputation phase, during which users verify all authorities' public keys and proofs. Note that due to roaming, LI can take place for an independent set of authorities at each end (Initiator and Responder). In addition, at this point we assume that, if the proxy does not already have a shared key $\mathsf{pcsk_{E,Prox_E}}$ with its endpoint (E), then we assume both the proxy and endpoint generate this common key[4].

On their side, the Initiator and its proxy $\mathsf{Prox_I}$ run the $\mathtt{PreComp_I}$ algorithm, which parses $\mathsf{APK_I} = \left(\Lambda_j^{\mathsf{I}}.\mathsf{PK}\right)_{j=1}^{n_\mathsf{I}}$ (vector of the public keys of the Initiator's authorities involved in the interception process) as $\left((\Lambda_j^{\mathsf{I}}.\mathsf{pk}, \Lambda_j^{\mathsf{I}}.\mathsf{ni})\right)_{j=1}^{n_\mathsf{I}}$ and checks $\Lambda_j^{\mathsf{I}}.\mathsf{ni}$ for all $j \in [\![1, n_\mathsf{I}]\!]$.

It sets $\omega_\mathsf{I} \leftarrow (\Lambda_j^{\mathsf{I}})_{j=1}^{n_\mathsf{I}} \| \mathsf{ID_CRED_I}$[5] and $\mathsf{h_I} \leftarrow \prod_{j=1}^{n_\mathsf{I}} \Lambda_j^{\mathsf{I}}.\mathsf{pk} = \prod_{j=1}^{n_\mathsf{I}} g^{\Lambda_j^{\mathsf{I}}.\mathsf{SK}}$.

If the Initiator and its proxy does not already share a long-term encryption key $\mathsf{pcsk_{I,Prox_I}}$, $\mathtt{PreComp_I}$ generates such a key (in our modeling we assume that this long term key is used for a secure encryption scheme $(\mathsf{KGen}'', \mathsf{Enc}'', \mathsf{Dec}'')$).

Similarly, the Responder and its proxy $\mathsf{Prox_R}$ do the same check routine, and they set $\omega_\mathsf{R} \leftarrow (\Lambda_j^{\mathsf{R}})_{j=1}^{n_\mathsf{R}} \|$ $\mathsf{ID_CRED_R}$ and $\mathsf{h_R} \leftarrow \prod_{j=1}^{n_\mathsf{R}} \Lambda_j^{\mathsf{R}}.\mathsf{pk} = \prod_{j=1}^{n_\mathsf{R}} g^{\Lambda_j^{\mathsf{R}}.\mathsf{SK}}$ thanks to the $\mathtt{PreComp_R}$ algorithm. If necessary, they also negotiate a long-term encryption key $\mathsf{pcsk_{R,Prox_R}}$.

The values $(\mathsf{h_I}, \omega_\mathsf{I})$ and $\mathsf{pcsk_{I,Prox_I}}$ (for the Initiator), and $(\mathsf{h_R}, \omega_\mathsf{R})$ and $\mathsf{pcsk_{R,Prox_R}}$ (for the Responder), are shared between the proxies and the endpoints. We assume moreover that the proxies know the identity of the endpoints (as is already the case following an initial AKA/Handshake). This stage can be done once and then reused for multiple sessions (for the same set of authorities of an endpoint).

Elements $\mathsf{h_I}$ and $\mathsf{h_R}$ are publicly known, but their insertion in the protocol will allow us to do secure lawful interception. Indeed, their role is to help authorities to retrieve *e.g.,* g^{xy}, a necessary element to compute the session key.

Formally expressed as an algorithm, this results in the following process, which must be executed as a precomputation by both the Initiator and the Responder.

$\underline{\mathtt{PreComp_I}(M_\mathsf{I})}$: Parse M_I as $(\Lambda_j^{\mathsf{I}})_{j=1}^{n_\mathsf{I}} \| \mathsf{ID_I}$, and verify each authority's proof via $\mathsf{NIPoKver}(\Lambda_j^{\mathsf{I}}.\mathsf{pk}, \Lambda_j^{\mathsf{I}}.\mathsf{ni})$. Upon failure, the procedure aborts with $\perp$. Upon success, compute $\mathsf{h_I} \leftarrow \prod_{j=1}^{n_\mathsf{I}} \Lambda_j^{\mathsf{I}}.\mathsf{pk}$, and $\omega_\mathsf{I} \leftarrow (\Lambda_j^{\mathsf{I}})_{j=1}^{n_\mathsf{I}} \| \mathsf{ID_I}$, and return $(\mathsf{h_I}, \omega_\mathsf{I})$.

Algorithm $\mathtt{PreComp_R}$ for the Responder is naïvely obtain from $\mathtt{PreComp_I}$ by adapting the notations.

We will now describe the four versions of the LI compliant EDHOC protocol, assuming all precomputations are done. In what follows, we present the modifications we made on the original protocol, and we provide in four figures an algorithmic description of the protocols, respecting the specification of [32].

[4]This would be done at the beginning of each handshake anyway, within the 5G protocol stack.

[5]This notation, $\mathsf{ID_CRED_E}$, comes from [32] and denotes the identity credentials of an user E. In the rest of the paper we will abbreviate it as $\mathsf{ID_E}$, to make it mode readable.

4.3 Handshake - First case

Message flow. With elements of the pre-computation stage, we can now integrate lawful interception in the EDHOC protocol. For this we will insert into the message flow a tool which will subsequently allow us to generate trapdoors and make fine-grained lawful interception by recovering the session key $\mathsf{PRK_{out}}$.
We first consider the case where both the Initiator and the Responder use signature as current authentication method ($\mathsf{ID} = 0$ in TABLE 1). We will later explain the changes to be made in case this method changes ($\mathsf{ID} = 1, 2, 3$ in TABLE 1). The protocol is depicted in FIGURES 8 and 9 below.

Messages between the endpoints, the Initiator and the Responder, pass through proxies we added to the scheme. The first task of the Initiator remains unchanged. He sample its session nonce (connexion identifier) and its DIFFIE-HELLMAN pair. He then sends message M_1, to the Responder, via proxies $\mathsf{Prox_I}$ and $\mathsf{Prox_R}$ (`InitRun1`). To sum up M_1 contains, g^x, its connection identifier $\mathsf{C_I}$, the authentication method (here it is 0), the cipher suite which will be used, and the external authorization data $\mathsf{EAD_1}$. The proxy $\mathsf{Prox_I}$ (with $\mathtt{Prox_I Run1}$) initializes τ_I as $\omega_\mathsf{I} \| \mathsf{MCS} \| g^x$ (g^x, and latter g^y, are necessary elements for trapdoors generation) and $\mathsf{Prox_R}$ initializes τ_R as $\omega_\mathsf{R} \| \mathsf{MCS} \| g^x$ ($\mathtt{Prox_R Run1}$)(here MCS denotes the method and cipher suite that the Initiator wants to use for the handshake).

The first actions of the Responder remains also unchanged, except that he returns to its proxy the usual EDHOC second message, concatenated with the transcript hash, *i.e.,* he sends its ephemeral DIFFIE-HELLMAN element g^y and the ciphertext c_2 (which is m_2 encrypted with the key $\mathsf{sk_2}$ (`RespRun1`)).

The proxy $\mathsf{Prox_R}$ updates τ_R with g^y, and sends g^y and c_2 to $\mathsf{Prox_I}$, which also update τ_I with g^y, before returning the message he received to the Initiator ($\mathtt{Prox_R Run2}$ and $\mathtt{Prox_I Run2}$).

The first important addition is made on the Initiator side (in `InitRun1`). Besides its usual calculations, we ask him to compute $\mathsf{H_I}$ with his private DIFFIE-HELLMAN element x, the shared ephemeral DIFFIE-HELLMAN element g^y and the public value $\mathsf{h_I}$: $\mathsf{H_I} = (\mathsf{h_I} g^y)^x$, and to sign it with a signature of knowledge of discrete logarithm. This signature is important because $\mathsf{H_I}$ is necessary for retrieving the session key $\mathsf{PRK_{out}}$ (so we must ensure that $\mathsf{H_I}$ as been constructed well). The signed message contains the identities of the involved parties (authorities and endpoints) as well as the three transcript hashes. These elements are important for the lawful interception process. Indeed, if the signature of knowledge is valid, this ensure that the two endpoints well take part to the handshakes and that the elements used for the computation of $\mathsf{PRK_{out}}$, which will be transmitted to the authorities, are correct. This signature is a central object in the implementations presented in section 5. In addition to this, the Initiator encrypt the two secret key $\mathsf{sk_2}$ and $(\mathsf{sk_3}, \mathsf{IV_3})$, and the HMAC t_2 with the key $\mathsf{pcsk_{I,Prox_I}}$ as S_I[6].

Then the Initiator sends (in clear in M_3) to its proxy $\mathsf{Prox_I}$, in addition to the usual ciphertext c_3 (which is m_3 encrypted using $\mathsf{sk_3}$), the element $\mathsf{H_I}$ and the signature of knowledge $\mathsf{ni_I}$, along with S_I. When receiving this message, the proxy decipher S_I. Since he now gets M_1, c_2 and c_3, along with $\mathsf{sk_2}$ and $(\mathsf{sk_3}, \mathsf{IV_3})$, he can on his own compute $\mathsf{TH_2}, \mathsf{TH_3}$, and $\mathsf{TH_4}$ and verify the signature σ_2 with t_2 and m_2; he also obtains $\mathsf{ID_R}$ from c_2. With these elements, he recreates the signed message of the SoK $\mathsf{ni_I}$ and verified it. If it is valid, this means that all elements are correct so far. He updates τ_I by storing $\mathsf{TH_2}$, $\mathsf{TH_3}$,$\mathsf{TH_4}$, M_1, m_2 and m_3, t_2, $\mathsf{H_I}$, $\mathsf{ni_R}$, and $\mathsf{ID_R}$ (so far we have $\tau_\mathsf{I} = \omega_\mathsf{I} \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4} \| M_1 \| m_2 \| m_3 \| \mathsf{H_I} \| \mathsf{ni_I} \| \mathsf{ID_R} \| t_2$) ($\mathtt{Prox_I Run2}$). Finally it forwards c_3 to $\mathsf{Prox_R}$ which simply forwards it to the Responder.

The Responder, on his side (`RespRun2`), performs its usual calculations and the same additional operations as the Initiator with the element $\mathsf{H_R}$ and the signature of knowledge $\mathsf{ni_R}$. He also ciphers the keys $\mathsf{sk_2}$ and $(\mathsf{sk_3}, \mathsf{IV_3})$ along with the two HMAC t_2 and t_3, as S_R. There is also another important modification here. In the normal EDHOC protocol, this fourth message is optional. In our situation we must make it mandatory because we must add randomness in the computation of the session key $\mathsf{PRK_{out}}$ in the following way. The Responder picks at random a nonce n_r and inserts it in the computation of $\mathsf{PRK_{out}}$. He also computes a verification value (whose usefulness will be explained below) $\mathsf{val} = \mathcal{H}(g^{xy}, \text{“ ”}, n_r)$ and the value

[6]This addition is not trivial and useless; it results from extensive reflexion and optimization efforts aimed at resolving an issue that emerged during the proof of the non-frameability security property. In particular, it concerns the ability of verify a signature during the verification phase. Moreover, both HMACs must be sent separately on the Initiator's side in order to prevent an impersonation attack.

$\mathsf{TH}_5 = \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$. In this way, the Initiator, when receiving M_4 obtains n_r and is able to compute the same $\mathsf{PRK}_{\mathsf{out}}$. The natural question is then : why is this necessary ?

To make is simple, if we do not insert this fourth message, including randomness, the Initiator computes the key $\mathsf{PRK}_{\mathsf{out}}$ before the Responder and the latter can also compute $\mathsf{PRK}_{\mathsf{out}}$ before proving to its proxy he formed well its lawful interception element H_{R}. So in this situation, the two endpoints may engaged a communication which is not completely interceptable[7]. The addition of the randomness and the fourth obligatory message prevent this situation[8].

As the Initiator did, the Responder now needs to transmit these values to its proxy. So he sends M_4 composed of the ciphertext c_4 (which is m_4 encrypted with sk_4), transcript hashes $\mathsf{TH}_2, \mathsf{TH}_3$ and TH_4, the element H_{R} along with the signature of knowledge ni_{R}, and also n_r, TH_5 and val.

At this point, we must explain how which party finishes its run of the exchange. Firstly $\mathsf{Prox}_{\mathsf{R}}$, as $\mathsf{Prox}_{\mathsf{I}}$ did, deciphers S_{R} and computes m_2, m_3 and the transcript hashes TH_2, TH_3 and TH_4 on its own (he also obtains ID_{I} from c_3). He verifies the two signatures σ_2 and σ_3 with t_2 and t_3. He also verifies the signature of knowledge he received, as well as the validity of n_r with val, TH_5 and the value he computed TH_4. If it is valid, he then updates in τ_{R} (we have $\tau_{\mathsf{R}} = \omega_{\mathsf{R}} \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| t_2 \| t_3 \| n_r \| \mathsf{val}$), and then signed τ_{R} with its private signature key $\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{R}}}$. The session state $\mathsf{sst}_{\mathsf{R}}$ he stores and provides eventually to authorities is τ_{R} concatenated with this signature ($\mathtt{Prox_R Run3}$). He finally forwards c_4, n_r, val, TH_5 and t_3 to $\mathsf{Prox}_{\mathsf{I}}$, which checks the validity of this message and the signature σ_3 (as $\mathsf{Prox}_{\mathsf{R}}$ did) and performs the same computations as $\mathsf{Prox}_{\mathsf{R}}$ to generate $\mathsf{sst}_{\mathsf{I}}$ ($\mathtt{Prox_I Run4}$) after updating τ_{I}. This last element sst is useful and important for trapdoors generation and in the procedure to recover the key (that is why we added a signature from the proxy, in order to ensure that the element τ is well formed). Finally $\mathsf{Prox}_{\mathsf{I}}$ forwards c_4, n_r, val and TH_5 to the Initiator which performs the same verification as the proxies, checks the ciphertext c_4 and computes $\mathsf{PRK}_{\mathsf{out}}$ with the valid n_r.

Session state. At the end of the protocol, $\mathsf{Prox}_{\mathsf{d}}$ returns :

$$\mathsf{sst}_{\mathsf{d}} = \omega_{\mathsf{d}} \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{d}} \| \mathsf{ni}_{\mathsf{d}} \| \mathsf{pid}_{\mathsf{d}} \| t_2 \| t_3 \| n_r \| \mathsf{val} \| \sigma_{\mathsf{d}}$$

where $\mathsf{pid}_{\mathsf{d}}$ denotes the partner identity of ID_{d}. We parse $\mathsf{sst}_{\mathsf{d}}$ as

$$\omega_{\mathsf{d}} \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{d}} \| \mathsf{ni}_{\mathsf{d}} \| \mathsf{pid}_{\mathsf{d}} \| t_2 \| t_3 \| n_r \| \mathsf{val} \| \sigma_{\mathsf{d}}$$

To keep it simple, the dashed boxes are used to check the validity of the session, *i.e.,* verify the identity of the participants and verify that the tools for lawful interception have been well implemented. Next, the black boxes are used to compute g^{xy}, and then with the highlighted boxes we compute $\mathsf{PRK}_{\mathsf{out}}$ (this will be explained below).

Verification of sst. Before extracting trapdoors or opening the session, the session state sst must be verified. It consists of: $\omega \| \mathsf{MCS} \| g^x \| g^y \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H} \| \mathsf{ni} \| \mathsf{pid} \| t_2 \| t_3 \| n_r \| \mathsf{val} \| \sigma$. The plaintext messages M_1, m_2 and m_3 allow for the verification of the transcript hashes and the authentication of both parties (through signatures of knowledge). The verification algorithm also extracts, from ω the public keys of the authorities, for which the proofs of knowledge of the discrete logarithm must be verified. Subsequently the signature of knowledge is verified with respect to both the signed message and to the equality of the discrete logarithm used to compute H. In this version of the protocol, since both endpoints use signature authentication, the verification algorithm obtains, from m_2, m_3, t_2 and t_3 all the necessary information to verify the signatures σ_2 and σ_3. Finally, parsing sst as $\tau \| \sigma$, the signature σ generated by the proxy is verified with respect to the message τ.

[7]Note that this problem appears in [9], but does not impact the security properties proved or the proposed conditions of use presented in the paper.

[8]This fourth additional message could be seen as a reduction of the lightweight nature of the protocol. But it remains reasonable, and we except that with this addition, we will be able to prove soon another security property: explicit authentication.

$\texttt{PreComp}_{\texttt{I}}(M_{\mathsf{I}})$

Parse M_{I} as $((\Lambda^{\mathsf{I}}_j)^{n_{\mathsf{I}}}_{j=1} \| \mathsf{ID}_{\mathsf{I}})$
$\forall j \in [\![1, n_{\mathsf{I}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{I}}_j.\mathsf{pk}, \Lambda^{\mathsf{I}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{I}} \leftarrow \prod_{j=1}^{n_{\mathsf{I}}} \Lambda^{\mathsf{I}}_j.\mathsf{pk}, \omega_I \leftarrow (\Lambda^{\mathsf{I}}_j)^{n_{\mathsf{I}}}_{j=1} \| \mathsf{ID}_{\mathsf{I}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}$
return $(\mathsf{h}_{\mathsf{I}}, \omega_{\mathsf{I}})$

$\texttt{Prox}_{\texttt{I}}\texttt{Run1}(\mathsf{ID}_{\mathsf{I}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{I}} \leftarrow \omega_{\mathsf{I}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{I}
return M_1

$\texttt{Prox}_{\texttt{I}}\texttt{Run2}(\mathsf{ID}_{\mathsf{I}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^y$ // *Update* τ_{I}
return M_2

$\texttt{Prox}_{\texttt{I}}\texttt{Run3}(\mathsf{ID}_{\mathsf{I}}, M_3)$

Parse M_3 as $(c_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| S_{\mathsf{I}})$
// *Decipher* S_{I} *and verify transcript hashes*
$K_{\mathsf{I}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}, S_{\mathsf{I}})$, parse K_{I} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3 \| t_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2)$, get $\mathsf{ID}_{\mathsf{R}}, \sigma_2$ from m_2
$\mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1)), \mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}})$
$\mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$
// *Verify SoK and HMAC, and update* τ_{I}
if $\mathsf{DS}.\texttt{Verif}(\mathsf{spk}_{\mathsf{R}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_2 \| t_2), \sigma_2) \neq 1$
  then return $\perp$
if $\mathsf{SoKver}(\omega_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{I}}, \mathsf{ni}_{\mathsf{I}}) \neq 1$
  then return $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}} \| t_2$
return $M'_3 \leftarrow c_3$

$\texttt{Prox}_{\texttt{I}}\texttt{Run4}(\mathsf{ID}_{\mathsf{I}}, M'_4)$

Parse M'_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| t_3)$
if $M'_4 = \perp$ **then return** $\perp$
if $\mathsf{TH}_5 = \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
if $\mathsf{DS}.\texttt{Verif}(\mathsf{spk}_{\mathsf{I}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_3 \| t_3), \sigma_3) \neq 1$
  then return $\perp$
// *Update and sign* τ_{I}
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| t_3 \| n_r \| \mathsf{val}, \sigma_{\mathsf{I}} \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{I}}}, \tau_{\mathsf{I}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| \sigma_{\mathsf{I}}$
return $M''_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5)$

$\texttt{PreComp}_{\texttt{R}}(M_{\mathsf{R}})$

Parse M_{R} as $((\Lambda^{\mathsf{R}}_j)^{n_{\mathsf{R}}}_{j=1} \| \mathsf{ID}_{\mathsf{R}})$
$\forall j \in [\![1, n_{\mathsf{R}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{R}}_j.\mathsf{pk}, \Lambda^{\mathsf{R}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{R}} \leftarrow \prod_{j=1}^{n_{\mathsf{R}}} \Lambda^{\mathsf{R}}_j.\mathsf{pk}, \omega_R \leftarrow (\Lambda^{\mathsf{R}}_j)^{n_{\mathsf{R}}}_{j=1} \| \mathsf{ID}_{\mathsf{R}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}$
return $(\mathsf{h}_{\mathsf{R}}, \omega_{\mathsf{R}})$

$\texttt{Prox}_{\texttt{R}}\texttt{Run1}(\mathsf{ID}_{\mathsf{R}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{R}} \leftarrow \omega_{\mathsf{R}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{R}
return M_1

$\texttt{Prox}_{\texttt{R}}\texttt{Run2}(\mathsf{ID}_{\mathsf{R}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| g^y$ // *Update* τ_{R}
return M_2

$\texttt{Prox}_{\texttt{R}}\texttt{Run3}(\mathsf{ID}_{\mathsf{R}}, M_4)$

Parse M_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| S_{\mathsf{R}})$
if $M_4 = \perp$ **then return** $\perp$
// *Decipher* S_{R} *and verify transcript hashes*
$K_{\mathsf{R}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}, S_{\mathsf{R}})$
Parse K_{R} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3 \| t_2 \| t_3)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2), \mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}}), \mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3)$
$m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$, get $\mathsf{ID}_{\mathsf{I}}, \sigma_3$ from m_3
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$
// *Verify SoK and HMACs, update and sign* τ_{R}
if $\mathsf{DS}.\texttt{Verif}(\mathsf{spk}_{\mathsf{R}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_2 \| t_2), \sigma_2) \neq 1$
  then return $\perp$
if $\mathsf{DS}.\texttt{Verif}(\mathsf{spk}_{\mathsf{I}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_3 \| t_3), \sigma_3) \neq 1$
  then return $\perp$
if $\mathsf{SoKver}(\omega_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{R}}, \mathsf{ni}_{\mathsf{R}}) \neq 1$
  then return $\perp$
if $\mathsf{TH}_5 \neq \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| t_2 \| t_3 \| n_r \| \mathsf{val}$
$\sigma_{\mathsf{R}} \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{R}}}, \tau_{\mathsf{R}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| \sigma_{\mathsf{R}}$
return $M'_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| t_3)$

FIGURE 9 – Lawful Interception in the EDHOC protocol with ID = 0. The notation MCS denotes the method (here 0 = **Sign-sign**) and the cipher suite that the Initiator wants to use.

$\texttt{InitRun1}(\mathsf{ID_I})$

$(x \leftarrow\$ \mathbb{Z}_p^*, g^x), \mathsf{C_I} \leftarrow\$ \{0,1\}^{l_C}$
return $M_1 \leftarrow (\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$

$\texttt{InitRun2}(\mathsf{ID_I}, M_2)$

Parse M_2 as $(g^y \| c_2), g^{xy} \leftarrow (g^y)^x$
// *Compute* $\mathsf{sk_2}$ *and decrypt* c_2
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk_2}, c_2)$, parse m_2 as $(\mathsf{C_R} \| \mathsf{ID_R} \| \sigma_2 \| \mathsf{EAD_2})$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{EAD_2}), \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$t_2' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Responder *authentication*
if $\mathsf{DS.Verif}(\mathsf{spk_R}, (l_{\mathsf{sig}} \| \mathsf{CTX_2} \| t_2'), \sigma_2) \neq 1$ **then**
  return $\perp$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 3rd message*
$\mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R}), \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| \mathsf{EAD_3})$
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3})$
$t_3 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Authenticate (signature in method 0)*
$\sigma_3 \leftarrow \mathsf{DS.Sign}(\mathsf{ssk_I}, (l_{\mathsf{sig}} \| \mathsf{CTX_3} \| t_3))$
// *Ensure LI-compliance and assemble* M_3
$m_3 \leftarrow (\mathsf{ID_I} \| \sigma_3 \| \mathsf{EAD_3}), c_3 \leftarrow \mathsf{Enc}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, m_3)$
$\mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$
$\mathsf{H_I} \leftarrow (\mathsf{h_I} g^y)^x$
$\mathsf{stm_I} \leftarrow [X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x]$
$\mathsf{ni_I} \leftarrow \mathsf{SoK}_{\omega_I \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{x : \mathsf{stm_I}\}$
$K_I \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3} \| t_2), S_I \leftarrow \mathsf{Enc}''(\mathsf{pcsk}_{\mathsf{I,Prox_I}}, K_I)$
return $M_3 \leftarrow (c_3 \| \mathsf{H_I} \| \mathsf{ni_I} \| S_I)$

$\texttt{InitRun3}(\mathsf{ID_I}, M_4'')$

Parse M_4'' as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5})$
if $c_4 = \perp$ **then return** $\perp$
if $\mathsf{TH_5} \neq \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r)$ **then return** $\perp$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4}), m_4 \leftarrow \mathsf{Dec}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, c_4)$
Parse m_4 as $\mathsf{EAD_4}$, **if** $m_4 = \perp$ **then return** $\perp$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1$

$\texttt{RespRun1}(\mathsf{ID_R}, M_1)$

$(y \leftarrow\$ \mathbb{Z}_p^*, g^y), \mathsf{C_R} \leftarrow\$ \{0,1\}^{l_C}$
Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$
// *Compute key-schedule for 2nd message*
$g^{xy} \leftarrow (g^x)^y, \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{EAD_2})$
$t_2 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Authenticate (signature in method 0)*
$\sigma_2 \leftarrow \mathsf{DS.Sign}(\mathsf{ssk_R}, (l_{\mathsf{sig}} \| \mathsf{CTX_2} \| t_2))$
// *Assemble* M_2
$m_2 \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \sigma_2 \| \mathsf{EAD_2}), c_2 \leftarrow \mathsf{Enc}(\mathsf{sk_2}, m_2)$
return $M_2 \leftarrow (g^y \| c_2)$

$\texttt{RespRun2}(\mathsf{ID_R}, M_3')$

Parse M_3' as $c_3, \mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R})$
// *Compute* $\mathsf{sk_3}$ *and decrypt* c_3
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3}), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, c_3)$
Parse m_3 as $(\mathsf{ID_I} \| \sigma_3 \| \mathsf{EAD_3}), \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| \mathsf{EAD_3})$
$t_3' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Initiator *authentication*
if $\mathsf{DS.Verif}(\mathsf{spk_I}, (l_{\mathsf{sig}} \| \mathsf{CTX_3} \| t_3'), \sigma_3) \neq 1$ **then**
  return $\perp$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 4th message*
$\mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4})$
// *Ensure LI-compliance and assemble* M_4
$\mathsf{H_R} \leftarrow (\mathsf{h_R} g^x)^y$
$\mathsf{stm_R} \leftarrow [Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y]$
$\mathsf{ni_R} \leftarrow \mathsf{SoK}_{\omega_R \| \mathsf{ID_I} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{y : \mathsf{stm_R}\}$
$K_R \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3} \| t_2 \| t_3), S_R \leftarrow \mathsf{Enc}''(\mathsf{pcsk}_{\mathsf{R,Prox_R}}, K_R)$
$m_4 \leftarrow (\mathsf{EAD_4} \| n_r), c_4 \leftarrow \mathsf{Enc}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, m_4)$
$n_r \leftarrow\$ \{0,1\}^{l_{\mathsf{rand}}}, \mathsf{val} \leftarrow \mathcal{H}(g^{xy}, \text{" "}, n_r)$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1, \mathsf{TH_5} \leftarrow \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r)$
return $M_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5} \| \mathsf{H_R} \| \mathsf{ni_R} \| S_R)$

FIGURE 10 – Lawful Interception in the EDHOC protocol with ID = 0 (continued).

Formally the verification algorithm `Verify` works as follows.

$\underline{\texttt{Verify}(\mathsf{pp}, \mathsf{sst}, \mathsf{I}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{Prox}.\overline{\mathsf{PK}}, \mathsf{APK}) \to b}$: Parse APK as $(\Lambda_j.\mathsf{pk}, \Lambda_j.\mathsf{ni})_{j=1}^n$, set $m_0 \leftarrow (\Lambda_j)_{j=1}^n \| \mathsf{ID_I} \| \mathsf{ID_R}$ and $\mathsf{h} \leftarrow \prod_{j=1}^n \Lambda_j.\mathsf{pk}$. Parse sst as $\omega \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4} \| M_1 \| m_2 \| m_3 \| t_2 \| t_3 \| \mathsf{H} \| \mathsf{ni} \| \mathsf{pid} \| n_r \| \mathsf{val} \| \sigma$. Set $m \leftarrow \omega' \| \mathsf{pid} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}$.
If $m_0 = \omega' \| \mathsf{pid}$ and :

- $\mathsf{TH_2} = \mathcal{H}(g^y, M_1)$, $\mathsf{TH_3} = \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R})$ and $\mathsf{TH_4} = \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$;
- Forall $j \in [\![1, n]\!]$, $\mathsf{NIPoKver}(\Lambda_j.\mathsf{pk}, \Lambda_j.\mathsf{ni}) = 1$;
- $\mathsf{SoKver}(m, \mathsf{stm}, \mathsf{ni}) = 1$;
- $\mathsf{DS}.\texttt{Verif}(\mathsf{spk_R}, (l_{\mathsf{sig}} \| \mathsf{CTX_2} \| t_2), \sigma_2) = 1$;
- $\mathsf{DS}.\texttt{Verif}(\mathsf{spk_I}, (l_{\mathsf{sig}} \| \mathsf{CTX_3} \| t_3), \sigma_3) = 1$;
- $\mathsf{DS}.\texttt{Verif}(\mathsf{Prox}.\overline{\mathsf{PK}}, \omega \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4} \| M_1 \| m_2 \| m_3 \| t_2 \| t_3 \| \mathsf{H} \| \mathsf{ni} \| \mathsf{pid} \| n_r \| \mathsf{val}, \sigma) = 1$;

then returns 1, else returns 0.

Generation of trapdoors and opening the session. We now focus on the generation of trapdoors, which will allow us to find the session key $\mathsf{PRK_{out}}$. For this we use and modify the `TDGen` and the `Open` algorithms from [9]. Algorithm `TDGen` takes as input the public parameter pp, the session state sst output by the proxy, and the authority's secret key $\Lambda.\mathsf{SK}$. It then outputs an authority's trapdoor $\Lambda.\mathsf{td}$ as follows.

$\underline{\texttt{TDGen}(\mathsf{pp}, \mathsf{sst_d}, \Lambda^{\mathsf{d}}.\mathsf{SK})}$: Parse $\mathsf{sst_d}$ as $\omega_{\mathsf{d}} \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4} \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{ni_d} \| \mathsf{pid_d} \| t_2 \| t_3 \| n_r \| \mathsf{val} \| \sigma_{\mathsf{d}}$ (with $\mathsf{d} \in \{\mathsf{I}, \mathsf{R}\}$) and set $\mathsf{A_I} \leftarrow g^x$ and $\mathsf{A_R} \leftarrow g^y$. Set $\Lambda^{\mathsf{d}}.\mathsf{td_1} \leftarrow {\mathsf{A_d}}^{\Lambda^{\mathsf{d}}.\mathsf{SK}}$ and $\Lambda^{\mathsf{d}}.\mathsf{td_0} \leftarrow \mathsf{NIPoK}\{\Lambda^{\mathsf{d}}.\mathsf{SK} : \Lambda^{\mathsf{d}}.\mathsf{pk} = g^{\Lambda^{\mathsf{d}}.\mathsf{SK}} \wedge \Lambda^{\mathsf{d}}.\mathsf{td_1} = {\mathsf{A_d}}^{\Lambda^{\mathsf{d}}.\mathsf{SK}}\}$. Set $\Lambda^{\mathsf{d}}.\mathsf{td} \leftarrow (\Lambda^{\mathsf{d}}.\mathsf{td_0}, \Lambda^{\mathsf{d}}.\mathsf{td_1})$. Return $\Lambda^{\mathsf{d}}.\mathsf{td}$.

In order to retrieve the session key, using the trapdoors previously generated, we run the `Open` algorithm from [9], which we modify a little. It takes as input the public parameter pp, the session state sst output by the proxy, the vector APK, where $\mathsf{APK_d} = (\Lambda_j^{\mathsf{d}}.\mathsf{PK})_{j=1}^{n_{\mathsf{d}}}$ and the vector $\mathcal{T}$, where $\mathcal{T}_{\mathsf{d}} = \left(\Lambda_j^{\mathsf{d}}.\mathsf{td}\right)_{j=1}^{n_{\mathsf{d}}}$. It then outputs the session key $\mathsf{PRK_{out}}$.

$\underline{\texttt{Open}(\mathsf{pp}, \mathsf{sst_d}, \mathsf{APK_d}, \mathcal{T}_{\mathsf{d}})}$: Parse $\mathsf{sst_d}$ as $\omega_{\mathsf{d}} \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4} \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{ni_d} \| \mathsf{pid_d} \| t_2 \| t_3 \| n_r \| \mathsf{val} \| \sigma_{\mathsf{d}}$ and set $\mathsf{A_I} \leftarrow g^x$ and $\mathsf{A_R} \leftarrow g^y$. Parse $\mathsf{APK_d}$ as $\left((\Lambda_j^{\mathsf{d}}.\mathsf{pk}, \Lambda_j^{\mathsf{d}}.\mathsf{ni})\right)_{j=1}^{n_{\mathsf{d}}}$ and $\mathcal{T}_{\mathsf{d}}$ as $\left((\Lambda_j^{\mathsf{d}}.\mathsf{td_0}, \Lambda_j^{\mathsf{d}}.\mathsf{td_1}\right)_{j=1}^{n_{\mathsf{d}}}$.
Verify that $\mathsf{NIPoKver}\big((g, \Lambda_j^{\mathsf{d}}.\mathsf{pk}, g^x, g^y, \Lambda_j^{\mathsf{d}}.\mathsf{td_1}), \Lambda_j^{\mathsf{d}}.\mathsf{td_0}\big) = 1$ for all $j \in [\![1, n_{\mathsf{d}}]\!]$. If any verification fails, return special symbol $\perp$.
Else compute: $g^{xy} \leftarrow \dfrac{\mathsf{H_d}}{\prod_{j=1}^{n_{\mathsf{d}}} \Lambda_j^{\mathsf{d}}.\mathsf{td_1}}$ and if $\mathcal{H}(g^{xy}, \text{“ ”}, n_r) \neq \mathsf{val}$, return $\perp$. Otherwise with g^{xy}, $\mathsf{TH_2}$, $\mathsf{TH_3}$, $\mathsf{TH_4}$ and n_r, compute $\mathsf{PRK_{2e}}$, the salts, and $\mathsf{PRK_{out}}$.

Note that during the opening, the authorities check the validity of the last unverified element of the sst: n_r. To do this they simply compute $\mathcal{H}(g^{xy}, \text{“ ”}, n_r)$ (not computable by proxies) and compare this value to the provided val value in sst. This verification ensures that this n_r, transmitted by the proxy, is indeed the one used by the endpoints to compute their session key $\mathsf{PRK_{out}}$. So with this fourth message and the values $\mathsf{TH_5}$ and val we ensure that the element n_r is always the good one, and is never forged by any party. This verification is performed solely by the authorities (*i.e.,* it is not done into the `Verify` algorithm). Its purpose is to ensure that the proxies do not cheat on the value n_r, and thus confirming that the session is interceptable. This verification is of minor importance. Indeed, if a proxy cheated, the authorities would

detect it with this verification, and prevent it from communicating in the futur. Note that in our situation, such a scenario is unlikely. In our use case within 5G networks, proxies are operators, and it is not in their interest to cheat. Also note that if a proxy cheats in this way, it does not compromise the security properties we are going to establish.

Correctness of the opening. Let's quickly detail the previous calculation. For example we have with $\mathsf{d} = \mathsf{I}$:

$$\mathsf{H_I} = (\mathsf{h_I} g^y)^x = \Big(\prod_{j=1}^{n_\mathsf{I}} \Lambda_j^\mathsf{I}.\mathsf{pk}\Big)^x g^{xy} = \Big(\prod_{j=1}^{n_\mathsf{I}} g^{\Lambda_j^\mathsf{I}.\mathsf{SK}}\Big)^x g^{xy} = \prod_{j=1}^{n_\mathsf{I}} g^{\Lambda_j^\mathsf{I}.\mathsf{SK}x} g^{xy} = \prod_{j=1}^{n_\mathsf{I}} g^{x\Lambda_j^\mathsf{I}.\mathsf{SK}} g^{xy}.$$

And remind that $\Lambda_j^\mathsf{I}.\mathsf{td_1} = (g^x)^{\Lambda_j^\mathsf{I}.\mathsf{SK}} = g^{x\Lambda_j^\mathsf{I}.\mathsf{SK}}$, so

$$\prod_{j=1}^{n_\mathsf{I}} \Lambda_j^\mathsf{I}.\mathsf{td_1} = \prod_{j=1}^{n_\mathsf{I}} g^{x\Lambda_j^\mathsf{I}.\mathsf{SK}}.$$

Finally we have as desired:

$$\frac{\mathsf{H_I}}{\prod_{j=1}^{n_\mathsf{I}} \Lambda_j^\mathsf{I}.\mathsf{td_1}} = \frac{\prod_{j=1}^{n_\mathsf{I}} g^{x\Lambda_j^\mathsf{I}.\mathsf{SK}} g^{xy}}{\prod_{j=1}^{n_\mathsf{I}} g^{x\Lambda_j^\mathsf{I}.\mathsf{SK}}} = g^{xy}.$$

Signature of knowledge. In the protocol, the $\mathsf{Initiator}$ and the $\mathsf{Responder}$ have to do a signature of knowledge. Formally, it is a proof of knowledge of a discrete logarithm to two different bases, in order to certify that the element $\mathsf{H_d}$ is well formed (and note that we previously emphasize the importance and the role of the signed message) [11]. We can proceed as follows, for example with $\mathsf{d} = \mathsf{I}$.

We consider a hash function $\mathcal{H}_{\mathsf{SoK}} : \mathbb{G} \times \{0,1\}^* \to \mathbb{Z}_p^*$ (idealize as a random oracle as previously explained), the statement $\mathsf{stm_I} = \big[X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x\big]$, a (secret) witness x for that statement and a (public) message $m_\mathsf{I} = \omega_\mathsf{I} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}$. The $\mathsf{Initiator}$ then runs the SoK algorithm as follows.

$\underline{\mathsf{SoK}(m_\mathsf{I}, x, \mathsf{stm_I})}$: Pick at random $r_\mathsf{I} \leftarrow\$ \mathbb{Z}_p^*$. Compute $I_1 \leftarrow g^{r_\mathsf{I}}$, $I_2 \leftarrow \big(\mathsf{h_I} g^y\big)^{r_\mathsf{I}}$, $\alpha_\mathsf{I} \leftarrow \mathcal{H}_{\mathsf{SoK}}(I_1 I_2, m_\mathsf{I})$ and $\beta_\mathsf{I} \leftarrow r_\mathsf{I} - \alpha_\mathsf{I} x \pmod p$. Return $\mathsf{ni_I} = (\alpha_\mathsf{I}, \beta_\mathsf{I})$.

Next, when he checks $\mathsf{ni_I}$, the proxy $\mathsf{Prox_I}$ runs the SoKver algorithm as follows.

$\underline{\mathsf{SoKver}(m_\mathsf{I}, \mathsf{stm_I}, \mathsf{ni_I})}$: Compute $I_1' \leftarrow g^{\beta_\mathsf{I}} (g^x)^{\alpha_\mathsf{I}}$ and $I_2' \leftarrow (\mathsf{h_I} g^y)^{\beta_\mathsf{I}} (\mathsf{h_I}^x g^{xy})^{\alpha_\mathsf{I}} = (\mathsf{h_I} g^y)^{\beta_\mathsf{I}} (\mathsf{H_I})^{\alpha_\mathsf{I}}$. Return 1 if $\alpha_\mathsf{I} = \mathcal{H}_{\mathsf{SoK}}(I_1' I_2', m_\mathsf{I})$, else return 0.

This signature of knowledge works. Indeed, if $\mathsf{ni_I}$ is a valid signature we will have:

$$I_1' = g^{\beta_\mathsf{I}} g^{x\alpha_\mathsf{I}} = g^{r_\mathsf{I} - \alpha_\mathsf{I} x} g^{\alpha_\mathsf{I} x} = g^{r_\mathsf{I}} g^{-\alpha_\mathsf{I} x} g^{\alpha_\mathsf{I} x} = g^{r_\mathsf{I}} = I_1$$

and

$$\begin{aligned} I_2' = (\mathsf{h_I} g^y)^{\beta_\mathsf{I}} \mathsf{h_I}^{x\alpha_\mathsf{I}} g^{xy\alpha_\mathsf{I}} &= \mathsf{h_I}^{\beta_\mathsf{I}} g^{y\beta_\mathsf{I}} \mathsf{h_I}^{x\alpha_\mathsf{I}} g^{xy\alpha_\mathsf{I}} = \mathsf{h_I}^{r_\mathsf{I} - \alpha_\mathsf{I} x} g^{r_\mathsf{I} y - \alpha_\mathsf{I} xy} \mathsf{h_I}^{x\alpha_\mathsf{I}} g^{xy\alpha_\mathsf{I}} \\ &= \mathsf{h_I}^{r_\mathsf{I}} \mathsf{h_I}^{-\alpha_\mathsf{I} x} g^{r_\mathsf{I} y} g^{-\alpha_\mathsf{I} xy} \mathsf{h_I}^{x\alpha_\mathsf{I}} g^{xy\alpha_\mathsf{I}} = \mathsf{h_I}^{r_\mathsf{I}} g^{r_\mathsf{I} y} = I_2 \end{aligned}$$

On their side, the $\mathsf{Responder}$ and his proxy $\mathsf{Prox_R}$ do the same with the statement $\mathsf{stm_R} = \big[Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y\big]$, the (private) witness y and the (public) message $m_\mathsf{R} = \omega_\mathsf{R} \| \mathsf{ID_I} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}$.

Proof of knowledge. As [9], we use two kinds of proofs of knowledge. The first one is used for authorities' long-term public keys and proves the knowledge of a discrete logarithm. This is done to secure the opening

procedure, ensuring that it happens only when all the authorities imbedded in the session state cooperate to retrieve the session key. The second one is used to prove the knowledge of a discrete logarithm to two different bases. This is done to ensure that when cooperate to open session, no authority cheats and gives a false trapdoor, potentially leading to recovering the wrong session key. The proofs are non-interactive and zero-knowledge, and can be build in a very similar way as the signature of knowledge we presented above.

4.4 Handshake - Second case

Message flow. We are now interested in the case where the $\mathsf{Responder}$ uses Static Diffie-Hellman as current authentication method ($\mathsf{ID} = 1$). Compared to the previous version changes are minimal. The $\mathsf{Initiator}$ (resp. the $\mathsf{Responder}$) only needs to compute the element H_I^1 (resp. H_R^1) and to add it to his signature of knowledge ni_I (resp. ni_R). This element, *e.g.,* on the $\mathsf{Initiator}$'s side is composed of the public value h_I, the secret ephemeral Diffie-Hellman element x and the longterm secret key of the $\mathsf{Responder}$ g^r: $\mathsf{H}_\mathsf{I}^1 = (\mathsf{h}_\mathsf{I} g^r)^x$. Similarly $\mathsf{H}_\mathsf{R}^1 = (\mathsf{h}_\mathsf{R} g^x)^r$.

On their side, proxies need to store the static Diffie-Hellman element g^r (this element is useful for the authorities to build an additional trapdoor, as we will see in the `TDGen` algorithm). Prox_I gets it from the $\mathsf{Initiator}$ in message M_3 and Prox_R knows it from its database since he knows the $\mathsf{Responder}$. The rest of the protocol does not undergo further modifications. In this case, since t_2 is not signed, we do not need to make it verified by proxies and stored in sst, thus we don't need to encrypt it in S_I and S_R. This is depicted in Figures 11 and 12 below. Moreover, some changes are necessary for the `TDGen` and the `Open` algorithms.

Session state. In this case, the way we constructed the session state sst is slightly different from the first case. At the end of the protocol, we have

$$\mathsf{sst_d} = \omega_\mathsf{d} \| \mathsf{MCS} \| g^x \| g^y \| g^r \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{H}_\mathsf{d}^1 \| \mathsf{ni_d} \| \mathsf{pid_d} \| t_3 \| n_r \| \mathsf{val} \| \sigma_\mathsf{d}$$

where $\mathsf{pid_d}$ denotes the partner identity of $\mathsf{ID_d}$. We parse $\mathsf{sst_d}$ as

$$\boxed{\omega_\mathsf{d}} \| \boxed{\mathsf{MCS}} \| \boxed{g^x} \| \boxed{g^y} \| \boxed{g^r} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| \boxed{M_1} \| \boxed{m_2} \| \boxed{m_3} \| \boxed{\mathsf{H_d}} \| \boxed{\mathsf{H}_\mathsf{d}^1} \| \boxed{\mathsf{ni_d}} \| \boxed{\mathsf{pid_d}} \| \boxed{t_3} \| n_r \| \boxed{\mathsf{val}} \| \boxed{\sigma_\mathsf{d}}$$

As before, the dashed boxes are used to check the validity of the session. Next, the black boxes are used to compute g^{xy} and g^{rx}, and then with the highlighted boxes we compute $\mathsf{PRK_{out}}$.

Verification of sst. The verification of sst works in a very similar way as for the previous case. The algorithm `Verify` just did not check the signature σ_2.

Generation of trapdoors and opening the session. In this situation, we have to make some changes to the `TDGen` and `Open` algorithms. For the `TDGen` algorithm, the inputs remain the same, but it now works as follows.

<u>$\mathtt{TDGen}(\mathsf{pp}, \mathsf{sst_d}, \Lambda^\mathsf{d}.\mathsf{SK})$:</u> Parse $\mathsf{sst_d}$ as $\omega_\mathsf{d} \| \mathsf{MCS} \| g^x \| g^y \| g^r \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{H}_\mathsf{d}^1 \| \mathsf{ni_d} \| \mathsf{pid_d} \| t_3 \| n_r \| \mathsf{val} \| \sigma_\mathsf{d}$ (with $\mathsf{d} \in \{\mathsf{I}, \mathsf{R}\}$) and set $\mathsf{A_I} \leftarrow g^x$ and $\mathsf{A_R} \leftarrow g^y$. Set $\Lambda^\mathsf{d}.\mathsf{td}_1 \leftarrow \mathsf{A_d}^{\Lambda^\mathsf{d}.\mathsf{SK}}$, $\Lambda^\mathsf{d}.\mathsf{td}_2 \leftarrow g^{r\Lambda^\mathsf{d}.\mathsf{SK}}$ and $\Lambda^\mathsf{d}.\mathsf{td}_0 \leftarrow \mathsf{NIPoK}\{\Lambda^\mathsf{d}.\mathsf{SK} : \Lambda^\mathsf{d}.\mathsf{pk} = g^{\Lambda^\mathsf{d}.\mathsf{SK}} \wedge \Lambda^\mathsf{d}.\mathsf{td}_1 = \mathsf{A_d}^{\Lambda^\mathsf{d}.\mathsf{SK}} \wedge \Lambda^\mathsf{d}.\mathsf{td}_2 = g^{r\Lambda^\mathsf{d}.\mathsf{SK}}\}$. Set $\Lambda^\mathsf{d}.\mathsf{td} \leftarrow (\Lambda^\mathsf{d}.\mathsf{td}_0, \Lambda^\mathsf{d}.\mathsf{td}_1, \Lambda^\mathsf{d}.\mathsf{td}_2)$. Return $\Lambda^\mathsf{d}.\mathsf{td}$.

For the `Open` algorithm, the inputs remain the same, as well as a large part of the algorithm. We simply add some computations.

<u>$\mathtt{Open}(\mathsf{pp}, \mathsf{sst_d}, \mathsf{APK_d}, \mathcal{T}_\mathsf{d})$:</u> Parse $\mathsf{sst_d}$ as $\omega_\mathsf{d} \| \mathsf{MCS} \| g^x \| g^y \| g^r \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{H}_\mathsf{d}^1 \| \mathsf{ni_d} \| \mathsf{pid_d} \| t_3 \| n_r \|$

$\texttt{PreComp}_{\texttt{I}}(M_{\mathsf{I}})$

Parse M_{I} as $((\Lambda^{\mathsf{I}}_j)_{j=1}^{n_{\mathsf{I}}} \| \mathsf{ID}_{\mathsf{I}})$
$\forall j \in [\![1, n_{\mathsf{I}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{I}}_j.\mathsf{pk}, \Lambda^{\mathsf{I}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{I}} \leftarrow \prod_{j=1}^{n_{\mathsf{I}}} \Lambda^{\mathsf{I}}_j.\mathsf{pk}, \omega_I \leftarrow (\Lambda^{\mathsf{I}}_j)_{j=1}^{n_{\mathsf{I}}} \| \mathsf{ID}_{\mathsf{I}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}$
return $(\mathsf{h}_{\mathsf{I}}, \omega_{\mathsf{I}})$

$\texttt{PreComp}_{\texttt{R}}(M_{\mathsf{R}})$

Parse M_{R} as $((\Lambda^{\mathsf{R}}_j)_{j=1}^{n_{\mathsf{R}}} \| \mathsf{ID}_{\mathsf{R}})$
$\forall j \in [\![1, n_{\mathsf{R}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{R}}_j.\mathsf{pk}, \Lambda^{\mathsf{R}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{R}} \leftarrow \prod_{j=1}^{n_{\mathsf{R}}} \Lambda^{\mathsf{R}}_j.\mathsf{pk}, \omega_R \leftarrow (\Lambda^{\mathsf{R}}_j)_{j=1}^{n_{\mathsf{R}}} \| \mathsf{ID}_{\mathsf{R}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}$
return $(\mathsf{h}_{\mathsf{R}}, \omega_{\mathsf{R}})$

$\texttt{Prox}_{\texttt{I}}\texttt{Run1}(\mathsf{ID}_{\mathsf{I}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{I}} \leftarrow \omega_{\mathsf{I}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{I}
return M_1

$\texttt{Prox}_{\texttt{R}}\texttt{Run1}(\mathsf{ID}_{\mathsf{R}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{R}} \leftarrow \omega_{\mathsf{R}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{R}
return M_1

$\texttt{Prox}_{\texttt{I}}\texttt{Run2}(\mathsf{ID}_{\mathsf{I}}, M'_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^y$ // *Update* τ_{I}
return M_2

$\texttt{Prox}_{\texttt{R}}\texttt{Run2}(\mathsf{ID}_{\mathsf{R}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| g^y \| g^r$ // *Update* τ_{R}
return M_2

$\texttt{Prox}_{\texttt{I}}\texttt{Run3}(\mathsf{ID}_{\mathsf{I}}, M_3)$

Parse M_3 as $(c_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{H}^1_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| S_{\mathsf{I}})$
// *Decipher* S_{I} *and verify transcript hashes*
$K \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}, S_{\mathsf{I}})$, parse K as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2)$, get ID_{R} from m_2
$\mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1)), \mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}})$
$\mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$, get g^r from ID_{R}
// *Verify SoK and update* τ_{I}
if $\mathsf{SoKver}(\omega_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{I}}, \mathsf{ni}_{\mathsf{I}}) \neq 1$
  then return $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^r \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{H}^1_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}}$
return $M'_3 \leftarrow c_3$

$\texttt{Prox}_{\texttt{R}}\texttt{Run3}(\mathsf{ID}_{\mathsf{R}}, M_4)$

Parse M_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{H}^1_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| S_{\mathsf{R}})$
if $M_4 = \perp$ **then return** $\perp$
// *Decipher* S_{R} *and verify transcript hashes*
$K_{\mathsf{R}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}, S_{\mathsf{R}})$, parse K_{R} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3 \| t_3)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2), \mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}}), \mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3)$
$m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$, get $\mathsf{ID}_{\mathsf{I}}, \sigma_3$ from m_3
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$
// *Verify SoK and HMACs, update and sign* τ_{R}
if $\mathsf{DS.Verif}(\mathsf{spk}_{\mathsf{I}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_3 \| t_3), \sigma_3) \neq 1$
  then return $\perp$
if $\mathsf{SoKver}(\omega_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{R}}, \mathsf{ni}_{\mathsf{R}}) \neq 1$
  then return $\perp$
if $\mathsf{TH}_5 \neq \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{H}^1_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| t_3 \| n_r \| \mathsf{val}$
$\sigma_{\mathsf{R}} \leftarrow \mathsf{DS.Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{R}}}, \tau_{\mathsf{R}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| \sigma_{\mathsf{R}}$
return $M'_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| t_3)$

$\texttt{Prox}_{\texttt{I}}\texttt{Run4}(\mathsf{ID}_{\mathsf{I}}, M'_4)$

Parse M'_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| t_3)$
if $M'_4 = \perp$ **then return** $\perp$
if $\mathsf{TH}_5 = \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
if $\mathsf{DS.Verif}(\mathsf{spk}_{\mathsf{I}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_3 \| t_3), \sigma_3) \neq 1$
  then return $\perp$
// *Update and sign* τ_{I}
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| t_3 \| n_r \| \mathsf{val}, \sigma_{\mathsf{I}} \leftarrow \mathsf{DS.Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{I}}}, \tau_{\mathsf{I}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| \sigma_{\mathsf{I}}$
return $M''_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5)$

FIGURE 11 – Lawful Interception in the EDHOC protocol with ID = 1. The notation MCS denotes the method (here 1 = **Sign-static**) and the cipher suite that the $\mathsf{Initiator}$ wants to use.

$\texttt{InitRun1}(\mathsf{ID_I})$

$(x \leftarrow\$ \mathbb{Z}_p^*, g^x), \mathsf{C_I} \leftarrow\$ \{0,1\}^{l_C}$
$\textbf{return } M_1 \leftarrow (\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$

$\texttt{InitRun2}(\mathsf{ID_I}, M_2)$

Parse M_2 as $(g^y \| c_2), g^{xy} \leftarrow (g^y)^x$
// *Compute* $\mathsf{sk_2}$ *and decrypt* c_2
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk_2}, c_2)$, parse m_2 as $(\mathsf{C_R} \| \mathsf{ID_R} \| t_2 \| \mathsf{EAD_2})$
Get g^r from $\mathsf{ID_R}, g^{rx} \leftarrow (g^r)^x, \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| g^r \| \mathsf{EAD_2})$
// *Verify* Responder *authentication*
$\mathsf{salt_{3e2m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 1 \| \mathsf{TH_2} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{3e2m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{3e2m}}, g^{xr})$
$t_2' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
$\boxed{\textbf{if } t_2' \neq t_2 \textbf{ then return } \perp}$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 3rd message*
$\mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R}), \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| \mathsf{EAD_3})$
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$t_3 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Authenticate (signature in method 1)*
$\boxed{\sigma_3 \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk_I}, (l_{\mathsf{sig}} \| \mathsf{CTX_3} \| t_3))}$
// *Ensure LI-compliance and assemble* M_3
$m_3 \leftarrow (\mathsf{ID_I} \| \sigma_3 \| \mathsf{EAD_3}), \mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3})$
$c_3 \leftarrow \mathsf{Enc}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, m_3), \mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$
$\mathsf{H_I} \leftarrow (\mathsf{h_I} g^y)^x, \mathsf{H_I^1} \leftarrow (\mathsf{h_I} g^r)^x$
$\mathsf{stm_I} \leftarrow \left[X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^r)^x\right]$
$\mathsf{ni_I} \leftarrow \mathsf{SoK}_{\omega_{\mathsf{I}} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{x : \mathsf{stm_I}\}$
$K_{\mathsf{I}} \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3}), S_{\mathsf{I}} \leftarrow \mathsf{Enc}''(\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox_I}}, K_{\mathsf{I}})$
$\textbf{return } M_3 \leftarrow (c_3 \| \mathsf{H_I} \| \mathsf{H_I^1} \| \mathsf{ni_I} \| S_{\mathsf{I}})$

$\texttt{InitRun3}(\mathsf{ID_I}, M_4'')$

Parse M_4'' as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5})$
$\textbf{if } c_4 = \perp \textbf{ then return } \perp$
$\textbf{if } \mathsf{TH_5} \neq \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r) \textbf{ then return } \perp$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4}), m_4 \leftarrow \mathsf{Dec}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, c_4)$
Parse m_4 as $\mathsf{EAD_4}, \textbf{if } m_4 = \perp \textbf{ then return } \perp$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1$

$\texttt{RespRun1}(\mathsf{ID_R}, M_1)$

$(y \leftarrow\$ \mathbb{Z}_p^*, g^y), \mathsf{C_R} \leftarrow\$ \{0,1\}^{l_C}$
Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$
// *Compute key-schedule for 2nd message*
$g^{xy} \leftarrow (g^x)^y, g^{xr} \leftarrow (g^x)^r, \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(m_1))$
$\mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$\mathsf{salt_{3e2m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 1 \| \mathsf{TH_2} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{3e2m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{3e2m}}, g^{xr})$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| g^r \| \mathsf{EAD_2})$
// *Authenticate (static DH in method 1)*
$\boxed{t_2 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})}$
// *Assemble* M_2
$m_2 \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| t_2 \| \mathsf{EAD_2}), c_2 \leftarrow \mathsf{Enc}(\mathsf{sk_2}, m_2)$
$\textbf{return } M_2 \leftarrow (g^y \| c_2)$

$\texttt{RespRun2}(\mathsf{ID_R}, M_3')$

Parse M_3' as $c_3, \mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R})$
// *Compute* $\mathsf{sk_3}$ *and decrypt* c_3
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3}), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, c_3)$
Parse m_3 as $(\mathsf{ID_I} \| \sigma_3 \| \mathsf{EAD_3}), \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| \mathsf{EAD_3})$
$t_3' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Initiator *authentication*
$\boxed{\textbf{if } \mathsf{DS}.\texttt{Verif}(\mathsf{spk_I}, (l_{\mathsf{sig}} \| \mathsf{CTX_3} \| t_3'), \sigma_3) \neq 1 \textbf{ then return } \perp}$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 4th message*
$\mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4})$
// *Ensure LI-compliance and assemble* M_4
$\mathsf{H_R} \leftarrow (\mathsf{h_R} g^x)^y, \mathsf{H_R^1} \leftarrow (\mathsf{h_R} g^x)^r$
$\mathsf{stm_R} \leftarrow \left[Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge Y_1 = g^r \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^x)^r\right]$
$\mathsf{ni_R} \leftarrow \mathsf{SoK}_{\omega_{\mathsf{R}} \| \mathsf{ID_I} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{(y, r) : \mathsf{stm_R}\}$
$K_{\mathsf{R}} \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3} \| t_3), S_{\mathsf{R}} \leftarrow \mathsf{Enc}''(\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox_R}}, K_{\mathsf{R}})$
$m_4 \leftarrow (\mathsf{EAD_4} \| n_r), c_4 \leftarrow \mathsf{Enc}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, m_4)$
$n_r \leftarrow\$ \{0,1\}^{l_{\mathsf{rand}}}, \mathsf{val} \leftarrow \mathcal{H}(g^{xy}, \text{" "}, n_r)$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1, \mathsf{TH_5} \leftarrow \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r)$
$\textbf{return } M_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5} \| \mathsf{H_R} \| \mathsf{H_R^1} \| \mathsf{ni_R} \| S_{\mathsf{R}})$

FIGURE 12 – Lawful Interception in the EDHOC protocol with ID = 1 (continued).

$\mathsf{val}\|\sigma_{\mathsf{d}}$ and set $\mathsf{A_I} \leftarrow g^x$ and $\mathsf{A_R} \leftarrow g^y$. Parse $\mathsf{APK_d}$ as $\big((\Lambda_j^{\mathsf{d}}.\mathsf{pk}, \Lambda_j^{\mathsf{d}}.\mathsf{ni})\big)_{j=1}^{n_{\mathsf{d}}}$ and $\mathcal{T}_{\mathsf{d}}$ as $\big((\Lambda_j^{\mathsf{d}}.\mathsf{td_0}, \Lambda_j^{\mathsf{d}}.\mathsf{td_1}, \Lambda_j^{\mathsf{d}}.\mathsf{td_2})\big)_{j=1}^{n_{\mathsf{d}}}$. Verify that $\mathsf{NIPoKver}\big((g, \Lambda_j^{\mathsf{d}}.\mathsf{pk}, \mathsf{A_d}, g^r, \Lambda_j^{\mathsf{d}}.\mathsf{td_1}, \Lambda_j^{\mathsf{d}}.\mathsf{td_2}), \Lambda_j^{\mathsf{d}}.\mathsf{td_0}\big) = 1$ for all $j \in [\![1, n_{\mathsf{d}}]\!]$. If any verification fails, return special symbol $\perp$. Else:

† for $\mathsf{d} = \mathsf{I}$, compute: $g^{xy} \leftarrow \dfrac{\mathsf{H_I}}{\prod_{j=1}^{n_{\mathsf{I}}} \Lambda_j^{\mathsf{I}}.\mathsf{td_1}}$ and $g^{rx} \leftarrow \dfrac{\mathsf{H_I^1}}{\prod_{j=1}^{n_{\mathsf{I}}} \Lambda_j^{\mathsf{I}}.\mathsf{td_1}}$

† for $\mathsf{d} = \mathsf{R}$, compute: $g^{xy} \leftarrow \dfrac{\mathsf{H_R}}{\prod_{j=1}^{n_{\mathsf{R}}} \Lambda_j^{\mathsf{R}}.\mathsf{td_1}}$ and $g^{rx} \leftarrow \dfrac{\mathsf{H_R^1}}{\prod_{i=1}^{n_{\mathsf{R}}} \Lambda_j^{\mathsf{R}}.\mathsf{td_2}}$

and if $\mathcal{H}(g^{xy}, \text{“ ”}, n_r) \neq \mathsf{val}$, return $\perp$. Otherwise, with g^{xy}, g^{rx}, $\mathsf{TH_2}$, $\mathsf{TH_3}$, $\mathsf{TH_4}$ and n_r, compute $\mathsf{PRK_{2e}}$, $\mathsf{PRK_{3e2m}}$, and $\mathsf{PRK_{out}}$.

Note that here the same verification of n_r, as in the previous case, is performed.

Correctness of the opening. Let's explain what changes. In this situation, to compute $\mathsf{PRK_{out}}$, we need to retrieve g^{xy} and g^{rx}. Due to the way we constructed the elements $\mathsf{H_I}$ and $\mathsf{H_I^1}$, we just need to "divide" them by $\mathsf{h_I^x}$ to find g^{xy} and g^{rx}. That is why we only use trapdoors $\Lambda_j^{\mathsf{I}}.\mathsf{td_1}$ for these computations. However, elements $\mathsf{H_R}$ and $\mathsf{H_R^1}$ have been constructed in a slightly different way. So in order to find g^{xy}, we “divide" $\mathsf{H_R}$ by $\mathsf{h_R^x}$ and to find g^{rx}, we “divide" $\mathsf{H_R^1}$ by $\mathsf{h_R^r}$. That is in this situation we use $\Lambda_j^{\mathsf{R}}.\mathsf{td_1}$ and $\Lambda_j^{\mathsf{R}}.\mathsf{td_2}$. So we do not always use the trapdoors $\Lambda_j^{\mathsf{I}}.\mathsf{td_2}$, but we let them appear to facilitate readability.

Signature of knowledge. In this situation, the Initiator and the Responder don't have the same statement to sign. That leads to few modifications in the signature scheme.
The signature of knowledge made by the Initiator is a proof of knowledge of a discrete logarithm to three different bases. On his side, the Responder makes a proof of knowledge of two discrete logarithms each to two different bases. Again, we make this in order to certify that the elements $\mathsf{H_I}$, $\mathsf{H_I^1}$, $\mathsf{H_R}$ and $\mathsf{H_R^1}$ are well formed. We can proceed as follows. We continue to consider a hash function $\mathcal{H}_{\mathsf{SoK}} : \mathbb{G} \times \{0,1\}^* \to \mathbb{Z}_p^*$, still idealized as a random oracle

On the Initiator side, we consider the statement $\mathsf{stm_I} = \big[X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^r)^x\big]$, a witness x for that statement and the message $m_{\mathsf{I}} = \omega_{\mathsf{I}}\|\mathsf{ID_R}\|\mathsf{TH_2}\|\mathsf{TH_3}\|\mathsf{TH_4}$. The SoK algorithm proceed in the same way as in the previous case, except that we also compute $I_3 \leftarrow \big(\mathsf{h_I} g^r\big)^{r_{\mathsf{I}}}$ and add it in the hash: $\alpha_{\mathsf{I}} \leftarrow \mathcal{H}_{\mathsf{SoK}}(I_1 I_2 I_3, m_{\mathsf{I}})$.
The SoKver algorithm undergoes the same modifications. We compute $I_3' \leftarrow (\mathsf{h_I} g^r)^{\beta_{\mathsf{I}}} (\mathsf{h_I^x} g^{xr})^{\alpha_{\mathsf{I}}} = (\mathsf{h_I} g^r)^{\beta_{\mathsf{I}}} (\mathsf{H_I^1})^{\alpha_{\mathsf{I}}}$ and return 1 if $\alpha_{\mathsf{I}} = \mathcal{H}_{\mathsf{SoK}}(I_1' I_2' I_3', m_{\mathsf{I}})$.

On their side, the Responder and his proxy $\mathsf{Prox_R}$ work with the statement $\mathsf{stm_R} = \big[Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge Y_1 = g^r \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^x)^r\big]$. Here we need two witnesses: y and r, and the message $m_{\mathsf{R}} = \omega_{\mathsf{R}}\|\mathsf{ID_R}\|\mathsf{TH_2}\|\mathsf{TH_3}\|\mathsf{TH_4}$. The Responder runs the SoK algorithm as follows.

$\underline{\mathsf{SoK}(m_{\mathsf{R}}, (y, r), \mathsf{stm_R})}$: Pick at random $r_{\mathsf{R}}, s_{\mathsf{R}} \leftarrow\$ \mathbb{Z}_p^*$. Compute $R_1 \leftarrow g^{r_{\mathsf{R}}}$, $R_2 \leftarrow \big(\mathsf{h_R} g^x\big)^{r_{\mathsf{R}}}$, $R_3 \leftarrow g^{s_{\mathsf{R}}}$, $R_4 \leftarrow \big(\mathsf{h_R} g^x\big)^{s_{\mathsf{R}}}$, $\alpha_{\mathsf{R}} \leftarrow \mathcal{H}_{\mathsf{SoK}}(R_1 R_2 R_3 R_4, m_{\mathsf{R}})$, $\beta_{\mathsf{R}} \leftarrow r_{\mathsf{R}} - \alpha_{\mathsf{R}} y \pmod p$ and $\gamma_{\mathsf{R}} \leftarrow s_{\mathsf{R}} - \alpha_{\mathsf{R}} r \pmod p$. Return $\mathsf{ni_R} = (\alpha_{\mathsf{R}}, \beta_{\mathsf{R}}, \gamma_{\mathsf{R}})$.

Next, when he checks $\mathsf{ni_R}$, the proxy $\mathsf{Prox_R}$ runs the SoKver algorithm as follows.

$\underline{\mathsf{SoKver}(m_{\mathsf{R}}, \mathsf{stm_R}, \mathsf{ni_R})}$: Compute $R_1' \leftarrow g^{\beta_{\mathsf{R}}} (g^y)^{\alpha_{\mathsf{R}}}$, $R_2' \leftarrow (\mathsf{h_R} g^x)^{\beta_{\mathsf{R}}} (\mathsf{h_R^y} g^{xy})^{\alpha_{\mathsf{R}}} = (\mathsf{h_R} g^x)^{\beta_{\mathsf{R}}} (\mathsf{H_R})^{\alpha_{\mathsf{R}}}$, $R_3' \leftarrow g^{\gamma_{\mathsf{R}}} (g^r)^{\alpha_{\mathsf{R}}}$ and $R_4' \leftarrow (\mathsf{h_R} g^x)^{\gamma_{\mathsf{R}}} (\mathsf{h_R^r} g^{xr})^{\alpha_{\mathsf{R}}} = (\mathsf{h_R} g^x)^{\gamma_{\mathsf{R}}} (\mathsf{H_R^1})^{\alpha_{\mathsf{R}}}$. Return 1 if $\alpha_{\mathsf{R}} = \mathcal{H}_{\mathsf{SoK}}(R_1' R_2' R_3' R_4', m_{\mathsf{R}})$, else return 0.

Proof of knowledge. The proofs of knowledge for authorities' long-term public keys remains the same in this case. Only the ones for trapdoors are slightly modified, as we need to prove the discrete logarithm equality in another base. But this can be done in a similar way as the previous ones.

4.5 Handshake - Third case

Message flow. We are now interested in the case where the Initiator uses Static Diffie-Hellman as current authentication method and the Responder a signature ($\mathsf{ID} = 2$). For this version, as well as the last one, the changes compare to the first case, are minimal. In this version, both the Initiator and the Responder need to compute the additional element H^1_I or H^1_R. On the Initiator's side for example, H^1_I is formed of h_I, the longterm secret i and the shared ephemeral Diffie-Hellman element g^y: $\mathsf{H}^1_\mathsf{I} = (\mathsf{h}_\mathsf{I} g^y)^i$. Similarly, $\mathsf{H}^1_\mathsf{R} = (\mathsf{h}_\mathsf{R} g^i)^y$. The rest of the message flow remains the same.

Again on their side, proxies store the element g^i (for the same reason as in the previous case), Prox_R obtaining it with the fourth message of the Responder and Prox_I from its database since he knows the Initiator. In this case, since t_3 is not signed, we do not need to make it verified by proxies and stored in sst, thus we don't need to encrypt it in S_R and send it to Prox_I. This is depicted in Figures 13 and 14 below. Moreover, few changes are necessary for the `TDGen` and the `Open` algorithms as it is detailed after.

Session state. The way we constructed the session state sst in this case is similar to the second case. At the end of the protocol, we have

$$\mathsf{sst_d} = \omega_\mathsf{d} \| \mathsf{MCS} \| g^x \| g^y \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{H}^1_\mathsf{d} \| \mathsf{ni_d} \| \mathsf{pid_d} \| t_2 \| n_r \| \mathsf{val} \| \sigma_\mathsf{d}$$

where $\mathsf{pid_d}$ denotes the partner identity of $\mathsf{ID_d}$. So we parse $\mathsf{sst_d}$ as

$$\boxed{\omega_\mathsf{d}} \| \boxed{\mathsf{MCS}} \| \boxed{g^x} \| \boxed{g^y} \| \boxed{g^i} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| \boxed{M_1} \| \boxed{m_2} \| \boxed{m_3} \| \boxed{\mathsf{H_d}} \| \boxed{\mathsf{H}^1_\mathsf{d}} \| \boxed{\mathsf{ni_d}} \| \boxed{\mathsf{pid_d}} \| \boxed{t_2} \| n_r \| \boxed{\mathsf{val}} \| \boxed{\sigma_\mathsf{d}}$$

As usual, the dashed boxes are used to check the validity of the session. Next, the black boxes are used to compute g^{xy} and g^{iy}, and then with the highlighted boxes we compute $\mathsf{PRK_{out}}$.

Verification of sst. The verification of sst works in a very similar way as for the first case. The algorithm `Verify` just did not check the signature σ_3 in this situation.

Generation of trapdoors and opening the session. We discuss here the changes that must be done to the `TDGen` and `Open` algorithms.

For the `TDGen` algorithm, the inputs remain the same, as well as the entire protocol. We just replace the element g^r by g^i in the computation of $\Lambda.\mathsf{td}_2$ (and so in the statement for the NIPoK ($\Lambda.\mathsf{td}_0$)).

For the `Open` algorithm, the inputs remain the same, as wall as a large part of the algorithm. The only modification appears in the computation of g^{iy}. In this case, to retrieve g^{iy}, it is the Initiator authorities which use the trapdoors $\Lambda^\mathsf{I}_j.\mathsf{td}_2$, and the Responder authorities use only the trapdoors $\Lambda^\mathsf{R}_j.\mathsf{td}_1$.

Signature of knowledge. In this case, in terms of signature, compared to the second case, the role of the Initiator and the Responder are reversed, *i.e.,* the Responder signs with the SoK algorithm that was used by the Initiator, and the latter signs with the SoK algorithm that was used by the Responder.

Proof of knowledge. The same remark as the previous case applies here. As we explained before, in the case $\mathsf{ID} = 1$, for the case $\mathsf{ID} = 2$, only one set of authorities needs of the two trapdoors to retrieve the key. For a practical implementation, in this situation, the set which only needs one trapdoor can compute a simpler proof of knowledge, by not including the second trapdoor. Again for readability, we let it as it is.

$\texttt{PreComp}_{\texttt{I}}(M_{\mathsf{I}})$

Parse M_{I} as $((\Lambda^{\mathsf{I}}_j)^{n_{\mathsf{I}}}_{j=1} \| \mathsf{ID}_{\mathsf{I}})$
$\forall j \in [\![1, n_{\mathsf{I}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{I}}_j.\mathsf{pk}, \Lambda^{\mathsf{I}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{I}} \leftarrow \prod_{j=1}^{n_{\mathsf{I}}} \Lambda^{\mathsf{I}}_j.\mathsf{pk}, \omega_I \leftarrow (\Lambda^{\mathsf{I}}_j)^{n_{\mathsf{I}}}_{j=1} \| \mathsf{ID}_{\mathsf{I}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}$
return $(\mathsf{h}_{\mathsf{I}}, \omega_{\mathsf{I}})$

$\texttt{Prox}_{\texttt{I}}\texttt{Run1}(\mathsf{ID}_{\mathsf{I}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{I}} \leftarrow \omega_{\mathsf{I}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{I}
return M_1

$\texttt{Prox}_{\texttt{I}}\texttt{Run2}(\mathsf{ID}_{\mathsf{I}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^y$ // *Update* τ_{I}
return M_2

$\texttt{Prox}_{\texttt{I}}\texttt{Run3}(\mathsf{ID}_{\mathsf{I}}, M_3)$

Parse M_3 as $(c_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{H}^1_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| S_{\mathsf{I}})$
  // *Decipher* S_{I} *and verify transcript hashes*
$K_{\mathsf{I}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}, S_{\mathsf{I}})$, parse K_{I} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3 \| t_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2)$, get $\mathsf{ID}_{\mathsf{R}}, \sigma_2$ from m_2
$\mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1)), \mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}})$
$\mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$
  // *Verify SoK and HMAC, and update* τ_{I}
if $\mathsf{DS}.\texttt{Verif}(\mathsf{spk}_{\mathsf{R}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_2 \| t_2), \sigma_2) \neq 1$
  then return $\perp$
if $\mathsf{SoKver}(\omega_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{I}}, \mathsf{ni}_{\mathsf{I}}) \neq 1$
  then return $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{H}^1_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}} \| t_2$
return $M'_3 \leftarrow c_3$

$\texttt{Prox}_{\texttt{I}}\texttt{Run4}(\mathsf{ID}_{\mathsf{I}}, M'_4)$

Parse M'_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5)$
if $M'_4 = \perp$ **then return** $\perp$
if $\mathsf{TH}_5 = \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
  // *Update and sign* τ_{I}
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| n_r \| \mathsf{val}, \sigma_{\mathsf{I}} \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{I}}}, \tau_{\mathsf{I}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| \sigma_{\mathsf{I}}$
return $M''_4 \leftarrow M'_4$

$\texttt{PreComp}_{\texttt{R}}(M_{\mathsf{R}})$

Parse M_{R} as $((\Lambda^{\mathsf{R}}_j)^{n_{\mathsf{R}}}_{j=1} \| \mathsf{ID}_{\mathsf{R}})$
$\forall j \in [\![1, n_{\mathsf{R}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{R}}_j.\mathsf{pk}, \Lambda^{\mathsf{R}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{R}} \leftarrow \prod_{j=1}^{n_{\mathsf{R}}} \Lambda^{\mathsf{R}}_j.\mathsf{pk}, \omega_R \leftarrow (\Lambda^{\mathsf{R}}_j)^{n_{\mathsf{R}}}_{j=1} \| \mathsf{ID}_{\mathsf{R}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}$
return $(\mathsf{h}_{\mathsf{R}}, \omega_{\mathsf{R}})$

$\texttt{Prox}_{\texttt{R}}\texttt{Run1}(\mathsf{ID}_{\mathsf{R}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{R}} \leftarrow \omega_{\mathsf{R}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{R}
return M_1

$\texttt{Prox}_{\texttt{R}}\texttt{Run2}(\mathsf{ID}_{\mathsf{R}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| g^y$ // *Update* τ_{R}
return M_2

$\texttt{Prox}_{\texttt{R}}\texttt{Run3}(\mathsf{ID}_{\mathsf{R}}, M_4)$

Parse M_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{H}^1_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| S_{\mathsf{R}})$
if $M_4 = \perp$ **then return** $\perp$
  // *Decipher* S_{R} *and verify transcript hashes*
$K_{\mathsf{R}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}, S_{\mathsf{R}})$, parse K_{R} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3 \| t_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2), \mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}}), \mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3)$
$m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$, get ID_{I} from m_3
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$, get g^i from ID_{I}
  // *Verify SoK and HMAC, update and sign* τ_{R}
if $\mathsf{DS}.\texttt{Verif}(\mathsf{spk}_{\mathsf{R}}, (l_{\mathsf{sig}} \| \mathsf{CTX}_2 \| t_2), \sigma_2) \neq 1$
  then return $\perp$
if $\mathsf{SoKver}(\omega_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{R}}, \mathsf{ni}_{\mathsf{R}}) \neq 1$
  then return $\perp$
if $\mathsf{TH}_5 \neq \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{H}^1_{\mathsf{R}} \|$
  $\mathsf{ni}_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| t_2 \| n_r \| \mathsf{val}$
$\sigma_{\mathsf{R}} \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{R}}}, \tau_{\mathsf{R}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| \sigma_{\mathsf{R}}$
return $M'_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5)$

FIGURE 13 – Lawful Interception in the EDHOC protocol with ID = 2. The notation MCS denotes the method (here 2 = **Static-sign**) and the cipher suite that the Initiator wants to use.

$\texttt{InitRun1}(\mathsf{ID_I})$

$(x \leftarrow\$ \mathbb{Z}_p^*, g^x), \mathsf{C_I} \leftarrow\$ \{0,1\}^{l_C}$
return $M_1 \leftarrow (\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$

$\texttt{InitRun2}(\mathsf{ID_I}, M_2)$

Parse M_2 as $(g^y \| c_2), g^{xy} \leftarrow (g^y)^x, g^{yi} \leftarrow (g^y)^i$
// *Compute* $\mathsf{sk_2}$ *and decrypt* c_2
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk_2}, c_2)$, parse m_2 as $(\mathsf{C_R} \| \mathsf{ID_R} \| \sigma_2 \| \mathsf{EAD_2})$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{EAD_2}), \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$t_2' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Responder *authentication*
if $\mathsf{DS.Verif}(\mathsf{spk_R}, (l_{\mathsf{sig}} \| \mathsf{CTX_2} \| t_2'), \sigma_2) \neq 1$ **then return** $\perp$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule ofr 3rd message*
$\mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R}), \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| g^i \| \mathsf{EAD_3})$
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{salt_{4e3m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 5 \| \mathsf{TH_3} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{4e3m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{4e3m}}, g^{yi})$
// *Authenticate (static DH in method 2)*
$t_3 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Ensure LI-compliance and assemble* M_3
$m_3 \leftarrow (\mathsf{ID_I} \| t_3 \| \mathsf{EAD_3}), \mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3})$
$c_3 \leftarrow \mathsf{Enc}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, m_3), \mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$
$\mathsf{H_I} \leftarrow (\mathsf{h_I} g^y)^x, \mathsf{H_I^1} \leftarrow (\mathsf{h_I} g^y)^i$
$\mathsf{stm_I} \leftarrow [X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge X_1 = g^i \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^y)^i]$
$\mathsf{ni_I} \leftarrow \mathsf{SoK}_{\omega_\mathsf{I} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{(x, i) : \mathsf{stm_I}\}$
$K_\mathsf{I} \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3} \| t_2), S_\mathsf{I} \leftarrow \mathsf{Enc}''(\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox_I}}, K_\mathsf{I})$
return $M_3 \leftarrow (c_3 \| \mathsf{H_I} \| \mathsf{H_I^1} \| \mathsf{ni_I} \| S_\mathsf{I}))$

$\texttt{InitRun3}(\mathsf{ID_I}, M_4'')$

Parse M_4'' as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5})$
if $c_4 = \perp$ **then return** $\perp$
if $\mathsf{TH_5} \neq \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r)$ **then return** $\perp$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4}), m_4 \leftarrow \mathsf{Dec}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, c_4)$
Parse m_4 as $\mathsf{EAD_4}$, **if** $m_4 = \perp$ **then return** $\perp$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1$

$\texttt{RespRun1}(\mathsf{ID_R}, M_1)$

$(y \leftarrow\$ \mathbb{Z}_p^*, g^y), \mathsf{C_R} \leftarrow\$ \{0,1\}^{l_C}$
Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$
// *Compute key-schedule for 2nd message*
$g^{xy} \leftarrow (g^x)^y, \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{EAD_2})$
$t_2 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Authenticate (signature in method 2)*
$\sigma_2 \leftarrow \mathsf{DS.Sign}(\mathsf{ssk_R}, (l_{\mathsf{sig}} \| \mathsf{CTX_2} \| t_2))$
// *Assemble* M_2
$m_2 \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \sigma_2 \| \mathsf{EAD_2}), c_2 \leftarrow \mathsf{Enc}(\mathsf{sk_2}, m_2)$
return $M_2 \leftarrow (g^y \| c_2)$

$\texttt{RespRun2}(\mathsf{ID_R}, M_3')$

Parse M_3' as $c_3, \mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R})$
// *Compute* $\mathsf{sk_3}$ *and decrypt* c_3
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3}), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, c_3)$
Parse m_3 as $(\mathsf{ID_I} \| t_3 \| \mathsf{EAD_3})$, get g^i from $\mathsf{ID_I}$
$g^{yi} \leftarrow (g^i)^y, \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| g^i \| \mathsf{EAD_3})$
$\mathsf{salt_{4e3m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 5 \| \mathsf{TH_3} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{4e3m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{4e3m}}, g^{yi})$
$t_3' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Initiator *authentication*
if $t_3 \neq t_3'$ **then return** $\perp$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 4th message*
$\mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I}), \mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4})$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
// *Ensure LI-compliance and assemble* M_4
$\mathsf{H_R} \leftarrow (\mathsf{h_R} g^x)^y, \mathsf{H_R^1} \leftarrow (\mathsf{h_R} g^i)^y$
$\mathsf{stm_R} \leftarrow [Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^i)^y]$
$\mathsf{ni_R} \leftarrow \mathsf{SoK}_{\omega_\mathsf{R} \| \mathsf{ID_I} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{y : \mathsf{stm_R}\}$
$K_\mathsf{R} \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3} \| t_2), S_\mathsf{R} \leftarrow \mathsf{Enc}''(\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox_R}}, K_\mathsf{R})$
$m_4 \leftarrow \mathsf{EAD_4}, c_4 \leftarrow \mathsf{Enc}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, m_4)$
$n_r \leftarrow\$ \{0,1\}^{l_{\mathsf{rand}}}, \mathsf{val} \leftarrow \mathcal{H}(g^{xy}, \text{" "}, n_r)$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1, \mathsf{TH_5} \leftarrow \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r)$
return $M_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5} \| \mathsf{H_R} \| \mathsf{H_R^1} \| \mathsf{ni_R} \| S_\mathsf{R})$

FIGURE 14 – Lawful Interception in the EDHOC protocol with ID = 2 (continued).

4.6 Handshake - Fourth case

Message flow. We study here the last case, where the $\mathsf{Initiator}$ and the $\mathsf{Responder}$ use Static Diffie-Hellman as current authentication method ($\mathsf{ID} = 3$). Changes are minimal compare to the previous case. Indeed the $\mathsf{Initiator}$, when receiving the second message M_2 gets g^r and then compute elements H^1_I and H^2_I: $\mathsf{H}^1_\mathsf{I} = (\mathsf{h}_\mathsf{I} g^r)^x$ and $\mathsf{H}^2_\mathsf{I} = (\mathsf{h}_\mathsf{I} g^y)^i$. He adds those elements to its signature of knowledge and transmits them to its proxy Prox_I. On his side, the $\mathsf{Responder}$ obtains g^i after deciphering c_3 and then compute elements H^1_R and H^2_R: $\mathsf{H}^1_\mathsf{R} = (\mathsf{h}_\mathsf{R} g^i)^y$ and $\mathsf{H}^2_\mathsf{R} = (\mathsf{h}_\mathsf{R} g^x)^r$. Similarly he adds those elements to its signature of knowledge and also transmits them to its proxy Prox_R along with its usual last message. This modification are depicted in Figures 15 and 16 below. Note that it is mainly a merge of the two previous methods (from the static point of view). Concerning the proxies, they both store the static Diffie-Hellman elements g^i and g^r (obtain in the same way we describe for the two previous cases), which will be useful in the `Open` algorithm (see details below). In this case, since t_2 and t_3 are not signed, we do not need to make them verified by proxies and thus encrypt it in S_I or S_R, which simplifies a little the protocol. Moreover, some changes are again necessary for the `TDGen` and the `Open` algorithms. We will explain latter in more details the importance of these elements H^1 and H^2 in addition to H. But here to make it simple, for methods involving a static Diffie-Hellman element, we need to derive another PRK compute with a Diffie-Hellman key involving this static Diffie-Hellman. As we need to retrieve g^{xy} to compute the key $\mathsf{PRK}_\mathsf{out}$ in the first case, we will need here to retrieve this Diffie-Hellman key, and that is why we introduce these elements.

Session state. At the end of the protocol, we have

$$\mathsf{sst_d} = \omega_\mathsf{d} \| \mathsf{MCS} \| g^x \| g^y \| g^r \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{H}^1_\mathsf{d} \| \mathsf{H}^2_\mathsf{d} \| \mathsf{ni_d} \| \mathsf{pid_d} \| n_r \| \mathsf{val} \| \sigma_\mathsf{d}$$

where $\mathsf{pid_d}$ denotes the partner identity of $\mathsf{ID_d}$. So we parse sst_d as

$$\boxed{\omega_\mathsf{d}} \| \boxed{\mathsf{MCS}} \| \boxed{g^x} \| \boxed{g^y} \| \boxed{g^r} \| \boxed{g^i} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| \boxed{M_1} \| \boxed{m_2} \| \boxed{m_3} \| \boxed{\mathsf{H_d}} \| \boxed{\mathsf{H}^1_\mathsf{d}} \| \boxed{\mathsf{H}^2_\mathsf{d}} \| \boxed{\mathsf{ni_d}} \| \boxed{\mathsf{pid_d}} \| n_r \| \boxed{\mathsf{val}} \| \boxed{\sigma_\mathsf{d}}$$

Again, the dashed boxes are used to check the validity of the session. Next, the black boxes are used to compute g^{xy}, g^{iy} and g^{rx}; then with the highlighted boxes we compute $\mathsf{PRK}_\mathsf{out}$.

Verification of sst. The verification of sst works in a very similar way as for the first case. The algorithm `Verify` just did not check signatures σ_2 and σ_3 in this situation.

So we may think to generalized `Verify` algorithm, which based on the MCS it gets from sst (parsed in the good way after analyzing MCS) adapts to each case, performing the appropriate verifications for the method selected in MCS. This is the modify `Verify` algorithm we will consider in our security analysis.

Generation of trapdoors and opening the session. As for the previous cases, we make some changes to the `TDGen` and `Open` algorithms.
For the `TDGen` algorithm, the inputs remain the same, and as in the previous case, the trapdoor vector of an authority Λ is $(\Lambda.\mathsf{td}_0, \Lambda.\mathsf{td}_1, \Lambda.\mathsf{td}_2)$. Elements $\Lambda.\mathsf{td}_0$ and $\Lambda.\mathsf{td}_1$ are constructed in the same way as in the previous case, unlike $\Lambda.\mathsf{td}_2$. Indeed, here we set $\mathsf{B_I} \leftarrow g^i$ and $\mathsf{B_R} \leftarrow g^r$ and $\Lambda.\mathsf{td}_2 \leftarrow \mathsf{B_d}^{\Lambda.\mathsf{SK}}$

For the `Open` algorithm, the inputs remain the same, but the usual disjunction we made in the two previous cases is more complex here.

$\underline{\mathtt{Open}(\mathsf{pp}, \mathsf{sst_d}, \mathsf{APK_d}, \mathcal{T}_\mathsf{d})}$: Parse $\mathsf{sst_d}$ as $\omega_\mathsf{d} \| \mathsf{MCS} \| g^x \| g^y \| g^r \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{H}^1_\mathsf{d} \| \mathsf{H}^2_\mathsf{d} \| \mathsf{ni_d} \| \mathsf{pid_d} \| n_r \| \mathsf{val} \| \sigma_\mathsf{d}$ (with $\mathsf{d} \in \{\mathsf{I}, \mathsf{R}\}$) and set $\mathsf{A_I} \leftarrow g^x$, $\mathsf{A_R} \leftarrow g^y$, $\mathsf{B_I} \leftarrow g^i$ and $\mathsf{B_R} \leftarrow g^r$. Parse $\mathsf{APK_d}$ as $\big((\Lambda^\mathsf{d}_j.\mathsf{pk}, \Lambda^\mathsf{d}_j.\mathsf{ni})\big)_{j=1}^{n_\mathsf{d}}$ and $\mathcal{T}_\mathsf{d}$ as $\big((\Lambda^\mathsf{d}_j.\mathsf{td}_0, \Lambda^\mathsf{d}_j.\mathsf{td}_1, \Lambda^\mathsf{d}_j.\mathsf{td}_2)\big)_{j=1}^{n_\mathsf{d}}$.
Verify that $\mathsf{NIPoKver}\big((g, \Lambda^\mathsf{d}_j.\mathsf{pk}, \mathsf{A_d}, \mathsf{B_d}, \Lambda^\mathsf{d}_j.\mathsf{td}_1, \Lambda^\mathsf{d}_j.\mathsf{td}_2), \Lambda^\mathsf{d}_j.\mathsf{td}_0\big) = 1$ for all $j \in [\![1, n_\mathsf{d}]\!]$. If any verification fails, return special symbol $\perp$. Else:

$\texttt{PreComp}_{\texttt{I}}(M_{\mathsf{I}})$

Parse M_{I} as $((\Lambda^{\mathsf{I}}_j)^{n_{\mathsf{I}}}_{j=1} \| \mathsf{ID}_{\mathsf{I}})$
$\forall j \in [\![1, n_{\mathsf{I}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{I}}_j.\mathsf{pk}, \Lambda^{\mathsf{I}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{I}} \leftarrow \prod_{j=1}^{n_{\mathsf{I}}} \Lambda^{\mathsf{I}}_j.\mathsf{pk}, \omega_I \leftarrow (\Lambda^{\mathsf{I}}_j)^{n_{\mathsf{I}}}_{j=1} \| \mathsf{ID}_{\mathsf{I}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}$
return $(\mathsf{h}_{\mathsf{I}}, \omega_{\mathsf{I}})$

$\texttt{Prox}_{\texttt{I}}\texttt{Run1}(\mathsf{ID}_{\mathsf{I}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{I}} \leftarrow \omega_{\mathsf{I}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{I}
return M_1

$\texttt{Prox}_{\texttt{I}}\texttt{Run2}(\mathsf{ID}_{\mathsf{I}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^y$ // *Update* τ_{I}
return M_2

$\texttt{Prox}_{\texttt{I}}\texttt{Run3}(\mathsf{ID}_{\mathsf{I}}, M_3)$

Parse M_3 as $(c_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{H}^1_{\mathsf{I}} \| \mathsf{H}^2_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| S_{\mathsf{I}})$
$K_{\mathsf{I}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{I},\mathsf{Prox}_{\mathsf{I}}}, S_{\mathsf{I}})$, parse K_{I} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2)$, get ID_{R} from m_2
$\mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1)), \mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}})$
$\mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$, get g^r from ID_{R}
// *Verify SoK and update* τ_{I}
if $\mathsf{SoKver}(\omega_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{I}}, \mathsf{ni}_{\mathsf{I}}) \neq 1$
  then return $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| g^r \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \|$
  $\mathsf{H}_{\mathsf{I}} \| \mathsf{H}^1_{\mathsf{I}} \| \mathsf{H}^2_{\mathsf{I}} \| \mathsf{ni}_{\mathsf{I}} \| \mathsf{ID}_{\mathsf{R}}$
return $M'_3 \leftarrow c_3$

$\texttt{Prox}_{\texttt{I}}\texttt{Run4}(\mathsf{ID}_{\mathsf{I}}, M'_4)$

Parse M'_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5)$
if $M'_4 = \perp$ **then return** $\perp$
if $\mathsf{TH}_5 = \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
// *Update and sign* τ_{I}
$\tau_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| n_r \| \mathsf{val}, \sigma_{\mathsf{I}} \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{I}}}, \tau_{\mathsf{I}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{I}} \leftarrow \tau_{\mathsf{I}} \| \sigma_{\mathsf{I}}$
return $M''_4 \leftarrow M'_4$

$\texttt{PreComp}_{\texttt{R}}(M_{\mathsf{R}})$

Parse M_{R} as $((\Lambda^{\mathsf{R}}_j)^{n_{\mathsf{R}}}_{j=1} \| \mathsf{ID}_{\mathsf{R}})$
$\forall j \in [\![1, n_{\mathsf{R}}]\!]$ **if** $\mathsf{NIPoKver}(\Lambda^{\mathsf{R}}_j.\mathsf{pk}, \Lambda^{\mathsf{R}}_j.\mathsf{ni}) \neq 1$
  then return $\perp$
$\mathsf{h}_{\mathsf{R}} \leftarrow \prod_{j=1}^{n_{\mathsf{R}}} \Lambda^{\mathsf{R}}_j.\mathsf{pk}, \omega_R \leftarrow (\Lambda^{\mathsf{R}}_j)^{n_{\mathsf{R}}}_{j=1} \| \mathsf{ID}_{\mathsf{R}}$
Agreement on secret key $\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}$
return $(\mathsf{h}_{\mathsf{R}}, \omega_{\mathsf{R}})$

$\texttt{Prox}_{\texttt{R}}\texttt{Run1}(\mathsf{ID}_{\mathsf{R}}, M_1)$

Parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1)$
$\tau_{\mathsf{R}} \leftarrow \omega_{\mathsf{R}} \| \mathsf{MCS} \| g^x$ // *Initialize* τ_{R}
return M_1

$\texttt{Prox}_{\texttt{R}}\texttt{Run2}(\mathsf{ID}_{\mathsf{R}}, M_2)$

Parse M_2 as $(g^y \| c_2)$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| g^y \| g^r$ // *Update* τ_{R}
return $M_2 \leftarrow (g^y \| c_2)$

$\texttt{Prox}_{\texttt{R}}\texttt{Run3}(\mathsf{ID}_{\mathsf{R}}, M_4)$

Parse M_4 as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{H}^1_{\mathsf{R}} \| \mathsf{H}^2_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| S_{\mathsf{R}})$
if $M_4 = \perp$ **then return** $\perp$
// *Decipher* S_{R} *and verify transcript hashes*
$K_{\mathsf{R}} \leftarrow \mathsf{Dec}''(\mathsf{pcsk}_{\mathsf{R},\mathsf{Prox}_{\mathsf{R}}}, S_{\mathsf{R}})$, parse K_{R} as $(\mathsf{sk}_2 \| \mathsf{sk}_3 \| \mathsf{IV}_3)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk}_2, c_2), \mathsf{TH}_2 \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1))$
$\mathsf{TH}_3 \leftarrow \mathcal{H}(\mathsf{TH}_2, m_2, \mathsf{ID}_{\mathsf{R}}), \mathsf{ad}_3 \leftarrow (l_{\mathsf{aead}} \| \text{“ ”} \| \mathsf{TH}_3)$
$m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk}_3, \mathsf{IV}_3, \mathsf{ad}_3, c_3)$, get ID_{I} from m_3
$\mathsf{TH}_4 \leftarrow \mathcal{H}(\mathsf{TH}_3, m_3, \mathsf{ID}_{\mathsf{I}})$, get g^i from ID_{I}
// *Verify SoK, update and sign* τ_{R}
if $\mathsf{SoKver}(\omega_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4, \mathsf{stm}_{\mathsf{R}}, \mathsf{ni}_{\mathsf{R}}) \neq 1$
  then return $\perp$
if $\mathsf{TH}_5 \neq \mathcal{H}(\mathsf{TH}_4, \mathsf{val}, n_r)$ **then return** $\perp$
$\texttt{accepted} \leftarrow 1$
$\tau_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| g^r \| g^i \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \|$
  $\mathsf{H}_{\mathsf{R}} \| \mathsf{H}^1_{\mathsf{R}} \| \mathsf{H}^2_{\mathsf{R}} \| \mathsf{ni}_{\mathsf{R}} \| \mathsf{ID}_{\mathsf{I}} \| n_r \| \mathsf{val}$
$\sigma_{\mathsf{R}} \leftarrow \mathsf{DS}.\texttt{Sign}(\mathsf{ssk}_{\mathsf{Prox}_{\mathsf{R}}}, \tau_{\mathsf{R}})$
$\texttt{terminated} \leftarrow 1, \mathsf{sst}_{\mathsf{R}} \leftarrow \tau_{\mathsf{R}} \| \sigma_{\mathsf{R}}$
return $M'_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5)$

FIGURE 15 – Lawful Interception in the EDHOC protocol with ID = 3.The notation MCS denotes the method (here 3 = **Static-static**) and the cipher suite that the $\mathsf{Initiator}$ wants to use.

$\texttt{InitRun1}(\mathsf{ID_I})$

$(x \leftarrow\$ \mathbb{Z}_p^*, g^x), \mathsf{C_I} \leftarrow\$ \{0,1\}^{l_C}$
return $M_1 \leftarrow (\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$

$\texttt{InitRun2}(\mathsf{ID_I}, M_2)$

Parse M_2 as $(g^y \| c_2), g^{xy} \leftarrow (g^y)^x, g^{yi} \leftarrow (g^y)^i$
// Compute $\mathsf{sk_2}$ and decrypt c_2
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1)), \mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$m_2 \leftarrow \mathsf{Dec}(\mathsf{sk_2}, c_2)$, parse m_2 as $(\mathsf{C_R} \| \mathsf{ID_R} \| t_2 \| \mathsf{EAD_2})$
Get g^r from $\mathsf{ID_R}, g^{xr} \leftarrow (g^r)^x$
$\mathsf{salt_{3e2m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 1 \| \mathsf{TH_2} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{3e2m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{3e2m}}, g^{xr})$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| g^r \| \mathsf{EAD_2})$
$t_2' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Responder *authentication*
$\boxed{\textbf{if } t_2 \neq t_2' \textbf{ then return } \perp}$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 3rd message*
$\mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R}), \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| g^i \| \mathsf{EAD_3})$
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{salt_{4e3m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 5 \| \mathsf{TH_3} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{4e3m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{4e3m}}, g^{yi})$
// *Authenticate (static DH in method 3)*
$\boxed{t_3 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})}$
// *Ensure LI-compliance and assemble M_3*
$m_3 \leftarrow (\mathsf{ID_I} \| t_3 \| \mathsf{EAD_3}), \mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I})$
$\mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3}), c_3 \leftarrow \mathcal{E}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, m_3)$
$\mathsf{H_I} \leftarrow (\mathsf{h_I} g^y)^x, \mathsf{H_I^1} \leftarrow (\mathsf{h_I} g^r)^x, \mathsf{H_I^2} \leftarrow (\mathsf{h_I} g^y)^i$
$\mathsf{stm_I} \leftarrow [X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^r)^x \wedge X_1 = g^i \wedge \mathsf{H_I^2} = (\mathsf{h_I} g^y)^i]$
$\mathsf{ni_I} \leftarrow \mathsf{SoK}_{\omega_\mathsf{I} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{(x, i) : \mathsf{stm_I}\}$
$K_\mathsf{I} \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3}), S_\mathsf{I} \leftarrow \mathcal{E}''(\mathsf{pcsk}_{\mathsf{I,Prox_I}}, K_\mathsf{I})$
return $M_3 \leftarrow (c_3 \| \mathsf{H_I} \| \mathsf{H_I^1} \| \mathsf{H_I^2} \| \mathsf{ni_I} \| S_\mathsf{I})$

$\texttt{InitRun3}(\mathsf{ID_I}, M_4'')$

Parse M_4'' as $(c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5})$
if $c_4 = \perp$ **then return** $\perp$
if $\mathsf{TH_5} \neq \mathcal{H}(\mathsf{TH_4}, \mathsf{val}, n_r)$ **then return** $\perp$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4}), m_4 \leftarrow \mathsf{Dec}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, c_4)$
Parse m_4 as $\mathsf{EAD_4}$, **if** $m_4 = \perp$ **then return** $\perp$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1$

$\texttt{RespRun1}(\mathsf{ID_R}, M_1)$

$(y \leftarrow\$ \mathbb{Z}_p^*, g^y), \mathsf{C_R} \leftarrow\$ \{0,1\}^{l_C}$
parse M_1 as $(\mathsf{MCS} \| g^x \| \mathsf{C_I} \| \mathsf{EAD_1})$
// *Compute key-schedule for 2nd message*
$g^{xy} \leftarrow (g^x)^y, g^{xr} \leftarrow (g^x)^r, \mathsf{sid} \leftarrow (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$
$\mathsf{TH_2} \leftarrow \mathcal{H}(g^y, \mathcal{H}(M_1)), \mathsf{PRK_{2e}} \leftarrow \texttt{HKDF_Extract}(\mathsf{TH_2}, g^{xy})$
$\mathsf{sk_2} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 0 \| \mathsf{TH_2} \| l_2, l_2)$
$\mathsf{salt_{3e2m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{2e}}, 1 \| \mathsf{TH_2} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{3e2m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{3e2m}}, g^{xr})$
$\mathsf{CTX_2} \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| \mathsf{TH_2} \| g^r \| \mathsf{EAD_2})$
// *Authenticate (static DH in method 3)*
$\boxed{t_2 \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 2 \| \mathsf{CTX_2} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})}$
// *Assemble M_2*
$m_2 \leftarrow (\mathsf{C_R} \| \mathsf{ID_R} \| t_2 \| \mathsf{EAD_2}), c_2 \leftarrow \mathcal{E}(\mathsf{sk_2}, m_2)$
return $M_2 \leftarrow (g^y \| c_2)$

$\texttt{RespRun2}(\mathsf{ID_R}, M_3')$

Parse M_3' as $c_3, \mathsf{TH_3} \leftarrow \mathcal{H}(\mathsf{TH_2}, m_2, \mathsf{ID_R})$
// *Compute $\mathsf{sk_3}$ and decrypt c_3*
$\mathsf{sk_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 3 \| \mathsf{TH_3} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_3} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 4 \| \mathsf{TH_3} \| l_{\mathsf{iv}}, l_{\mathsf{iv}})$
$\mathsf{ad_3} \leftarrow (l_{\mathsf{aead}} \| \text{" "} \| \mathsf{TH_3}), m_3 \leftarrow \mathsf{Dec}'(\mathsf{sk_3}, \mathsf{IV_3}, \mathsf{ad_3}, c_3)$
Parse m_3 as $(\mathsf{ID_I} \| t_3 \| \mathsf{EAD_3})$, get g^i from $\mathsf{ID_I}$
$g^{yi} \leftarrow (g^i)^y, \mathsf{CTX_3} \leftarrow (\mathsf{ID_I} \| \mathsf{TH_3} \| g^i \| \mathsf{EAD_3})$
$\mathsf{salt_{4e3m}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{3e2m}}, 5 \| \mathsf{TH_3} \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\mathsf{PRK_{4e3m}} \leftarrow \texttt{HKDF_Extract}(\mathsf{salt_{4e3m}}, g^{yi})$
$t_3' \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 6 \| \mathsf{CTX_3} \| l_{\mathsf{mac}}, l_{\mathsf{mac}})$
// *Verify* Initiator *authentication*
$\boxed{\textbf{if } t_3 \neq t_3' \textbf{ then return } \perp}$
$\texttt{accepted} \leftarrow 1$
// *Compute key-schedule for 4th message*
$\mathsf{TH_4} \leftarrow \mathcal{H}(\mathsf{TH_3}, m_3, \mathsf{ID_I}), \mathsf{ad_4} \leftarrow (l_{\mathsf{aead}}, \| \text{" "} \| \mathsf{TH_4})$
$\mathsf{sk_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 8 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
$\mathsf{IV_4} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 9 \| \mathsf{TH_4} \| l_{\mathsf{key}}, l_{\mathsf{key}})$
// *Ensure LI-compliance and assemble M_4*
$\mathsf{H_R} \leftarrow (\mathsf{h_R} g^x)^y, \mathsf{H_R^1} \leftarrow (\mathsf{h_R} g^i)^y, \mathsf{H_R^2} \leftarrow (\mathsf{h_R} g^x)^r$
$\mathsf{stm_R} \leftarrow [Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^i)^y \wedge Y_1 = g^r \wedge \mathsf{H_R^2} = (\mathsf{h_R} g^x)^r]$
$\mathsf{ni_R} \leftarrow \mathsf{SoK}_{\omega_\mathsf{R} \| \mathsf{ID_I} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}}\{(y, r) : \mathsf{stm_R}\}$
$K_\mathsf{R} \leftarrow (\mathsf{sk_2} \| \mathsf{sk_3} \| \mathsf{IV_3}), S_\mathsf{R} \leftarrow \mathcal{E}''(\mathsf{pcsk}_{\mathsf{R,Prox_R}}, K_\mathsf{R})$
$n_r \leftarrow\$ \{0,1\}^{l_{\mathsf{rand}}}, \mathsf{val} \leftarrow \mathcal{H}(g^{xy}, \text{" "}, n_r)$
$m_4 \leftarrow \mathsf{EAD_4}, c_4 \leftarrow \mathsf{Enc}'(\mathsf{sk_4}, \mathsf{IV_4}, \mathsf{ad_4}, m_4)$
$\mathsf{PRK_{out}} \leftarrow \texttt{HKDF_Expand}(\mathsf{PRK_{4e3m}}, 7 \| \mathsf{TH_4} \| \lceil n_r \rceil \| l_{\mathsf{hash}}, l_{\mathsf{hash}})$
$\texttt{terminated} \leftarrow 1, \mathsf{TH_5} \leftarrow \mathcal{H}(\mathsf{TH_4}, \text{" "}, n_r)$
return $M_4 \leftarrow (c_4 \| n_r \| \mathsf{val} \| \mathsf{TH_5} \| \mathsf{H_R} \| \mathsf{H_R^1} \| \mathsf{H_R^2} \| \mathsf{ni_R} \| S_\mathsf{R})$

FIGURE 16 – Lawful Interception in the EDHOC protocol with ID = 3 (continued).

† for $\mathsf{d} = \mathsf{I}$, compute: $g^{xy} \leftarrow \dfrac{\mathsf{H_I}}{\prod_{j=1}^{n_\mathsf{I}} \Lambda_j^\mathsf{I}.\mathsf{td_1}}$, $g^{rx} \leftarrow \dfrac{\mathsf{H_I^1}}{\prod_{j=1}^{n_\mathsf{I}} \Lambda_j^\mathsf{I}.\mathsf{td_1}}$ and $g^{iy} \leftarrow \dfrac{\mathsf{H_I^2}}{\prod_{j=1}^{n_\mathsf{I}} \Lambda_j^\mathsf{I}.\mathsf{td_2}}$,

† for $\mathsf{d} = \mathsf{R}$, compute $g^{xy} \leftarrow \dfrac{\mathsf{H_R}}{\prod_{j=1}^{n_\mathsf{R}} \Lambda_j^\mathsf{R}.\mathsf{td_1}}$, $g^{rx} \leftarrow \dfrac{\mathsf{H_R^2}}{\prod_{j=1}^{n_\mathsf{R}} \Lambda_j^\mathsf{R}.\mathsf{td_2}}$ and $g^{iy} \leftarrow \dfrac{\mathsf{H_R^1}}{\prod_{j=1}^{n_\mathsf{R}} \Lambda_j^\mathsf{R}.\mathsf{td_1}}$,

and if $\mathcal{H}(g^{xy}, \text{“ ”}, n_r) \neq \mathsf{val}$, return $\perp$. Otherwise, with g^{xy}, g^{rx}, g^{iy}, $\mathsf{TH_2}$, $\mathsf{TH_3}$, $\mathsf{TH_4}$ and n_r, compute $\mathsf{PRK_{2e}}$, $\mathsf{PRK_{3e2m}}$, $\mathsf{PRK_{4e3m}}$ and $\mathsf{PRK_{out}}$.

Let's quickly explain importance of the changes we made. This is similar to what we have done before. We want to retrieve g^{xy}, g^{iy} and g^{rx}. However the elements $\mathsf{H_I}, \mathsf{H_I^1}, \mathsf{H_I^2}$ and $\mathsf{H_R}, \mathsf{H_R^1}, \mathsf{H_R^2}$ have been constructed in different ways. So to retrieve g^{xy} with $\mathsf{H_I}$ (resp. $\mathsf{H_R}$), we need to "divide" $\mathsf{H_I}$ by $\mathsf{h_I^x}$ (resp. $\mathsf{H_R}$ by $\mathsf{h_R^y}$), to retrieve g^{iy} with $\mathsf{H_I^1}$ (resp. $\mathsf{H_R^1}$), we "divide" $\mathsf{H_I^1}$ by $\mathsf{h_I^i}$ (resp. $\mathsf{H_R^1}$ by $\mathsf{h_R^y}$), and to retrieve g^{rx} with $\mathsf{H_I^2}$ (resp. $\mathsf{H_R^2}$, we "divide" $\mathsf{H_I^2}$ by $\mathsf{h_I^x}$ (resp. $\mathsf{H_R^2}$ by $\mathsf{h_R^r}$).

Signature of knowledge. In this case we mix the two signature protocols of the second case. The signature of knowledge made by the Initiator and the Responder is a proof of knowledge of a discrete logarithm to three different bases and of a discrete logarithm to two different bases in order to certify that the elements $\mathsf{H_d}$, $\mathsf{H_d^1}$ and $\mathsf{H_d^2}$ are well formed. We continue to consider a hash function $\mathcal{H}_{\mathsf{SoK}} : \mathbb{G} \times \{0,1\}^* \to \mathbb{Z}_p^*$. We can proceed as follows, for example on the Initiator side.

We work with the statement $\mathsf{stm_I} = \left[X = g^x \wedge \mathsf{H_I} = (\mathsf{h_I} g^y)^x \wedge \mathsf{H_I^1} = (\mathsf{h_I} g^r)^x \wedge X_1 = g^i \wedge \mathsf{H_I^2} = (\mathsf{h_I} g^y)^i\right]$, witnesses x and i for that statement and the message $m_\mathsf{I} = \omega_\mathsf{I} \| \mathsf{ID_R} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}$. The Initiator runs the SoK algorithm as follows.

$\underline{\mathsf{SoK}(m_\mathsf{I}, (x,i), \mathsf{stm_I})}$: Pick at random $r_\mathsf{I}, s_\mathsf{I} \leftarrow\$ \mathbb{Z}_p^*$. Compute $I_1 \leftarrow g^{r_\mathsf{I}}$, $I_2 \leftarrow \left(\mathsf{h_I} g^y\right)^{r_\mathsf{I}}$, $I_3 \leftarrow \left(\mathsf{h_I} g^r\right)^{r_\mathsf{I}}$, $I_4 \leftarrow g^{s_\mathsf{I}}$, $I_5 \leftarrow \left(\mathsf{h_I} g^y\right)^{s_\mathsf{I}}$, $\alpha_\mathsf{I} \leftarrow \mathcal{H}_{\mathsf{SoK}}(I_1 I_2 I_3 I_4 I_5, m_\mathsf{I})$, $\beta_\mathsf{I} \leftarrow r_\mathsf{I} - \alpha_\mathsf{I} x \pmod p$ and $\gamma_\mathsf{I} \leftarrow s_\mathsf{I} - \alpha_\mathsf{I} i \pmod p$. Return $\mathsf{ni_I} = (\alpha_\mathsf{I}, \beta_\mathsf{I}, \gamma_\mathsf{I})$.

Next, when he checks $\mathsf{ni_I}$, the proxy $\mathsf{Prox_I}$ runs the SoKver algorithm as follows.

$\underline{\mathsf{SoKver}(m_\mathsf{I}, \mathsf{stm_I}, \mathsf{ni_I})}$: Compute $I_1' \leftarrow g^{\beta_\mathsf{I}} (g^x)^{\alpha_\mathsf{I}}$, $I_2' \leftarrow (\mathsf{h_I} g^y)^{\beta_\mathsf{I}} (\mathsf{h_I^x} g^{xy})^{\alpha_\mathsf{I}} = (\mathsf{h_I} g^y)^{\beta_\mathsf{I}} (\mathsf{H_I})^{\alpha_\mathsf{I}}$, $I_3' \leftarrow (\mathsf{h_I} g^r)^{\beta_\mathsf{I}} (\mathsf{h_I^x} g^{xr})^{\alpha_\mathsf{I}} = (\mathsf{h_I} g^r)^{\beta_\mathsf{I}} (\mathsf{H_I^1})^{\alpha_\mathsf{I}}$, $I_4' \leftarrow g^{\gamma_\mathsf{I}} (g^i)^{\alpha_\mathsf{I}}$ and $I_5' \leftarrow (\mathsf{h_I} g^y)^{\gamma_\mathsf{I}} (\mathsf{h_I^i} g^{iy})^{\alpha_\mathsf{I}} = (\mathsf{h_I} g^y)^{\gamma_\mathsf{I}} (\mathsf{H_I^2})^{\alpha_\mathsf{I}}$. Return 1 if $\alpha_\mathsf{I} = \mathcal{H}_{\mathsf{SoK}}(I_1' I_2' I_3' I_4' I_5', \omega_\mathsf{I})$, else return 0.

On their side, the Responder and his proxy $\mathsf{Prox_R}$ work with the statement $\mathsf{stm_R} = \left[Y = g^y \wedge \mathsf{H_R} = (\mathsf{h_R} g^x)^y \wedge \mathsf{H_R^1} = (\mathsf{h_R} g^i)^y \wedge Y_1 = g^r \wedge \mathsf{H_R^2} = (\mathsf{h_R} g^x)^r\right]$, the witnesses y and r, and the message $m_\mathsf{R} = \omega_\mathsf{R} \| \mathsf{ID_I} \| \mathsf{TH_2} \| \mathsf{TH_3} \| \mathsf{TH_4}$. The SoK and SoKver algorithms work in the same way.

Proof of knowledge. In this version of the protocol, each set of authorities needs to compute the two trapdoors. Thus the proofs of knowledge for them are a little bit more substantial to calculate, but works in the same way as the previous two cases.

5 Implementation and evaluation

As method 3 (**Static-static**) of authentication is likely to be the most expensive, we implement LI-EDHOC in this particular setting. We complement the `lakers` implementation of EDHOC in Rust, with bindings for C and Python. As an online addition to this article, we release the LIKE implementation as open source[9].

5.1 Implementation setup

Our implementation transports LIKE elements within the External Authorization Data (EAD) fields of EDHOC. This allows for selective activation of lawful interception maintaining EDHOC's base structure. There are two main additions to the lakers implementation of EDHOC: (1) the cryptographic primitives supporting the generation and verification of the Signature of Knowledge. We instantiate the SoK as described in section 4.6. (2) dedicated EAD handlers that generate and process LIKE elements during the EDHOC handshake. These handlers populate the EAD field of the message with the necessary additional elements of LIKE. The implementation includes the pre-computation phase performed once for a given set of peers and authorities, during which the values $(\mathsf{h}_\mathsf{I}, \omega_\mathsf{I})$ are computed.

We evaluate the implementation on two typical constrained platforms, based on the ARM Cortex-M core[10]: (1) nRF52840 development board featuring an ARM Cortex-M4F processor running at 64 MHz with 256 kB RAM and 1 MB flash memory, and (2) STM32WBA55CG development board featuring an ARM Cortex-M33 processor running at 32 MHz with 128 kB RAM and 512 kB flash memory. To precisely measure execution time we used the Saleae Logic 8 logic analyzer connected to specific GPIO pins on both development boards, that are toggled at the beginning and end of given code sections (see Figure 17). The sampling frequency is set to 40 MS/s (Mega Samples per second).

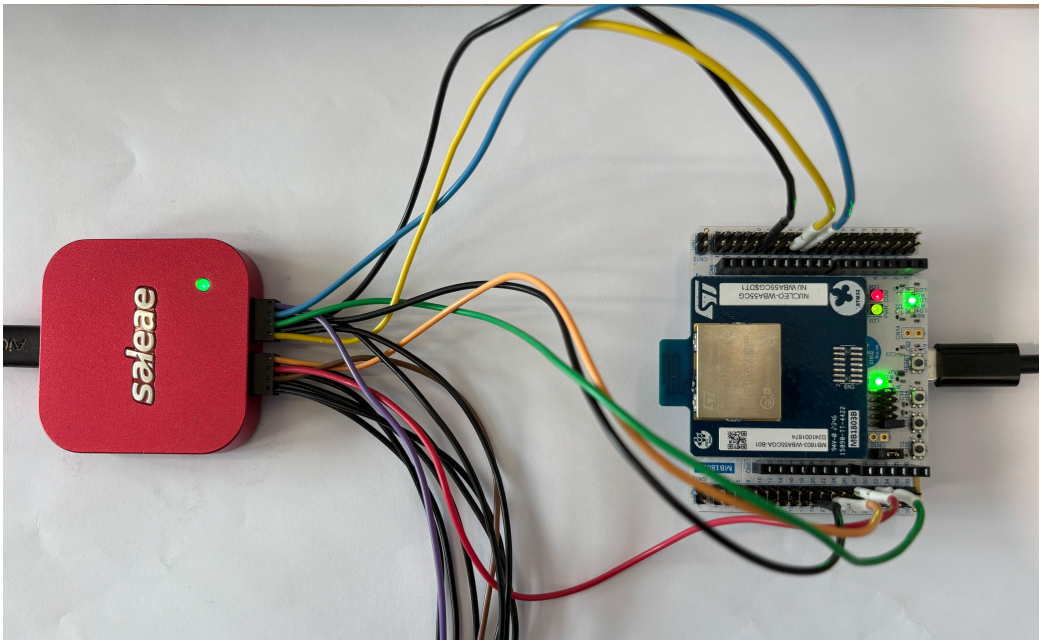

Figure 17 – Our evaluation setup. We benchmark the execution on the embedded evaluation board STM32WBA55CG (on the right) by toggling General Purpose Input/Output pins and recording the execution times using a logic analyzer (on the left). The logic analyzer software used to process the data runs on a PC.

[9] `https://github.com/ElsaLopez133/lakers.git`

[10] Note that we are not making a comparison here, but simply presenting implementation results on commonly used platforms.

For the evaluation, the Initiator and the Responder are located on the same physical device, which removes the network communication overhead and allows us to focus on computational performance. Note that the evaluation does not include the proxies which run on non-constrained devices.

5.2 Implementation results

We present the results in Tables 3 and 4. "Software LIKE" corresponds to a software-based implementation, with all cryptographic operations performed in software, without any hardware acceleration. "Hardware LIKE" corresponds to a hardware-accelerated implementation where elliptic curve operations (point addition and point multiplication), field operations (modular multiplication and modular addition) and hash operations are performed using hardware acceleration.

Message	**EDHOC with LIKE (bytes)**	**EDHOC base (bytes)**
Message_1	37	37
Message_3	115	19
Total	**152**	**56**

Table 3 – Message overhead for the Initiator running EDHOC authenticated with DH Static Keys with and without implementing LIKE.

Table 3 shows the bytes carried in each message for the Initiator (messages 1 and 3). The extra bytes compared to the standard EDHOC implementation in the third message come from the additional signature of knowledge.

Table 4 shows that the implementation of LIKE comes with a significant overhead, mainly due to the expensive ECC point multiplications and additions which figure in the SoK. We note an (expected) effect of the clock frequency and processor architecture on performance: the software-based implementation on Cortex M4F is comparable with the hardware-based implementation on Cortex M33. Finally, we observe that hardware acceleration substantially reduces the execution time differences observed between base EDHOC and LI-EDHOC for the Cortex M33 device, making lawful-interception more practical for deployment in constrained devices. Future iterations of this implementation could implement hardware acceleration as well in the nRF52840 (Cortex M4F) to further improve performance and reduce computation times.

Moreover, it would be interesting to run future experiments on hardware dedicated for ECC computations, such as [20]. Finally note that, whereas static authentication requires a lengthy signature of knowledge, complexity drops for authentication methods relying only on ephemeral secrets (*i.e.,* where peers use signatures to authenticate).

Operation	**Cortex M4F @ 64 MHz**		**Cortex M33 @ 32 MHz**			
	SW LIKE	**SW base**	**SW LIKE**	**SW base**	**HW LIKE**	**HW base**
Precomputation Phase						
Precomputation (vok_log_auth)	1.8095	-	7.1282	-	0.5798	-
Precomputation (h)	0.0015	-	0.0057	-	0.00005	-
Precomputation (w)	0.0015	-	0.0055	-	0.0009	-
Subtotal	**1.8125**	**-**	**7.1394**	**-**	**0.5808**	**-**
Initiator						
Message_1	0.0016	0.0016	0.0062	0.0062	0.0016	0.0016
Message_3	9.1231	2.8495	35.9294	11.2157	4.1256	1.2649
(SoK)	(6.2477)	-	(24.6097)	-	(2.8484)	-
Subtotal	**9.1247**	**2.8511**	**35.9356**	**11.2219**	**4.1272**	**1.2665**

Table 4 – LI-EDHOC handshake (LIKE) vs. EDHOC (base) runtime evaluation (in seconds), for implementations fully run in software (SW) or featuring ECC, field operations, and hashes that are hardware accelerated (HW).

6 Security Model

Our LI-EDHOC protocol guarantees the key-security, non-frameability, and LI-compliance security properties from LIKE [5, 9], but also a stronger property of EDHOC: identity protection. We provide here a description of the security model.

6.1 Execution environment

LIKE includes three types of participants: endpoints belonging to a set ESet, proxies from a set $\mathsf{PROXSet}$, and authorities in a set $\mathsf{AUTHSet}$. Each party P has private and public parameters $(\mathsf{P}.\overline{\mathsf{SK}}, \mathsf{P}.\overline{\mathsf{PK}})$ – which for EDHOC are in fact private/public *keysets* instead of just *one key*[11]. We also make endpoints and proxies store a set of symmetric keys shared between them in an attribute $\mathsf{P}.\mathtt{PCSK} = \{(\mathsf{Q}, \mathsf{pcsk}_{\mathsf{P},\mathsf{Q}})\}, \mathsf{Q} \in \mathsf{ShareSet}$ (if $\mathsf{P} \in \mathsf{ESet}$, then $\mathsf{ShareSet} = \mathsf{PROXSet}$ and vice-versa). Any party may be corrupted, yielding to the adversary one element of the private keyset. The party P stores a corruption bit $\mathsf{P}.\mathsf{SK}.\gamma$ for each key $\mathsf{SK} \in \mathsf{P}.\overline{\mathsf{SK}}$, initially set to 0 and raised to 1 either if the adversary reveals the long term key SK during the handshake, or if he registers a malicious party.

Protocol sessions are always run by *instances* of four parties: two endpoints, playing the parts of $\mathsf{Initiator}$ and $\mathsf{Responder}$; and two proxies. We denote by π^q_{P} the qth instance of P. Each endpoint/proxy instance keeps track of *attributes*, including: the session identifier $\pi^q_{\mathsf{P}}.\mathsf{sid}$; the computed session key $\pi^q_{\mathsf{P}}.\mathsf{PRK}_{\mathsf{out}}$; a session state $\pi^q_{\mathsf{P}}.\mathsf{sst}$; partner identifiers ($\mathsf{PNID}$ for the partnering endpoint (or endpoints if this is a proxy instance), AID for the authorities chosen by that party as legitimate for Lawful Interception, and ProxID for the partnering proxies); a flag indicating authentication acceptance (denoted $\pi^q_{\mathsf{P}}.\alpha$); and $\pi^q_{\mathsf{P}}.\mathsf{role}$ (the role $\mathsf{role} \in \{\mathsf{Initiator}, \mathsf{Responder}, \mathsf{Proxy}_{\mathsf{I}}, \mathsf{Proxy}_{\mathsf{R}}\}$ of P in the handshake).

To better capture EDHOC, endpoint party instances require several additional values: $\pi^q_{\mathsf{P}}.\mathsf{meth}$ (the method that will be used for a session involving this instance); $\pi^q_{\mathsf{P}}.\mathsf{SK}$ (the private key used by P to authenticate); a flag $\pi^q_{\mathsf{P}}.\mathsf{SK}.\gamma$ (indicating the corruption of the long term key SK used by π^q_{P}); a flag indicating the revelation of the session key (denoted $\pi^q_{\mathsf{P}}.\rho$); and $\pi^q_{\mathsf{P}}.\mathsf{pid}.\mathsf{PK}$ (the public key that P expects its partner to use for authentication). Both proxy and endpoint instances also store the current symmetric encryption key $\pi^q_{\mathsf{P}}.\mathsf{pcsk} \in \mathsf{P}.\mathtt{PCSK}$ used during the handshake. Finally proxy instances store three specific values : $\pi^q_{\mathsf{P}}.\mathsf{sst}$ (the session state built during the protocol session run); $\pi^q_{\mathsf{P}}.\mathsf{tr}$ (the transcript of the protocol session run) and $\pi^q_{\mathsf{P}}.\mathsf{ctn}$ (an indicative value (in $[\![1,4]\!]$ ou in $[\![1,3]\!]$) used to indicate which proxy algorithm should be executed next in the protocol run).

So to sum up, we list the data associated to each instance π^q_{P} as follows:

† <u>sid:</u> a vector $(\mathsf{C}_{\mathsf{I}}, \mathsf{C}_{\mathsf{R}}, g^x, g^y)$ that contains the ephemeral public keys g^x and g^y, as well as the connection identifiers C_{I} and C_{R}, initially set to $\perp$, and update during the session run (for example if the two endpoint instances do not agree on the method to use during the handshake, sid is maintained to $\perp$).

† <u>$\mathsf{PRK}_{\mathsf{out}}$:</u> the session key, initially set to $\perp$, and update when the protocol terminate correctly. This information is specific to the endpoint instances.

† <u>PNID:</u> this set contains the identity of all other endpoints taking part to the same session as π^q_{P}. There is two cases for this: either it contains only the name of the partner π^q_{P} is trying to exchange with if $\mathsf{P} \in \mathsf{ESet}$; or it contains the names of the two users trying to exchange if $\mathsf{P} \in \mathsf{PROXSet}$ (in this case the list is ordered : $\mathsf{PNID} = [\mathsf{ID_Initiator}, \mathsf{ID_Responder}]$).

† <u>ProxID:</u> this set stores the identity of all other proxies taking part to the same session as π^q_{P}. There is also two cases here: either this contains the identity of the two proxies taking part to the exchange if $\mathsf{P} \in \mathsf{ESet}$ (in this case the list is ordered : $\mathsf{ProxID} = [\mathsf{ID_Proxy_I}, \mathsf{ID_Proxy_R}]$); or it contains the identity of the other proxy involved in the exchange if $\mathsf{P} \in \mathsf{PROXSet}$.

[11] For an authority, these sets contains solely one element, respectively a and g^a. For a proxy, there are naturally the private and public signing key, but also symmetric encryption keys, shared with endpoints. Finally, concerning an endpoint, there are the two types of long-term keys, and also the symmetric encryption key shared with the associated proxy.

† AID: this set is unilateral, it only contains the identities of the authorities, (potentially) involved in the lawful interception, associated to the instance π^q_{P}.

† role: the $\mathsf{role} \in \{\mathsf{Initiator}, \mathsf{Responder}, \mathsf{Proxy_I}, \mathsf{Proxy_R}\}$ of P in the handshake.

† α: a flag initially set to one of the five values $\{\perp, \mathsf{running}, \mathsf{accepted}, \mathsf{rejected}, \mathsf{terminated}\}$ depending on the state of the protocol run.

† ρ: a reveal bit, initially set to 0 and raised to 1 if the adversary reveals the instance's session key. This information is specific to endpoint instances.

† meth: the method $\mathsf{meth} \in \{0, 1, 2, 3\}$ that will be used for a session involving this instance (initially set to $\perp$).

† SK: the long term secret key used by P during the handshake to authenticate. This information is specific to endpoint instances.

† SK.γ: a reveal bit set to 0 and raised to 1 if the adversary reveals the long-term secret key $\pi^q_{\mathsf{P}}.\mathsf{SK}$ during the handshake, or if P is a malicious party registered by $\mathcal{A}$. This information is specific to endpoint instances.

† pid.PK: the public key P expects its partner to use during the handshake to authenticate (according to the method selected). This information is specific to endpoint instances.

† pcsk: the symmetric encryption key ($\mathsf{pcsk} \in \mathsf{P}.\mathtt{PCSK}$) used during the handshake to cipher data between an endpoint and its associated proxy instances.

† sst: the session state sst, used by proxies to store and verify data, and useful for the key-recovery process. This information is specific to proxy instances.

† tr: the transcript of the protocol session run, storing, in order, all messages sent and received. This information is specific to proxy instances.

† cnt: this is an integer $f \in [\![1, 4]\!]$ (for $\mathsf{Initiator}$ proxies) or $f \in [\![1, 3]\!]$ (for $\mathsf{Responder}$ proxies) used to indicate which proxy algorithm ($\mathtt{Prox_dRun}$) should be executed next in the protocol run. It has no impact on the security analysis.

Moreover each authority registered is equipped with a trapdoor reveal bit $\mathtt{tdrev}$, initially set to 0 and raised to 1 if, in any session, the adversary reveals the trapdoor of an authority on one instance.

Following [5, 9] we define here the notion of instances *have matching conversation.*

Definition 17 *Let* (P, Q) *be two endpoints with* $\mathsf{P} \neq \mathsf{Q}$ *and let* q, j *be two natural integers. We says that* π^q_{P} *have matching conversation with* π^j_{Q} *(or that* π^q_{P} *matches* π^j_{Q}*) if* $\pi^q_{\mathsf{P}}.\mathsf{sid} \neq \perp$, $\pi^q_{\mathsf{P}}.\mathsf{sid} = \pi^j_{\mathsf{Q}}.\mathsf{sid}$, $\mathsf{Q} = \pi^q_{\mathsf{P}}.\mathsf{PNID}$ *and* $\mathsf{P} = \pi^j_{\mathsf{Q}}.\mathsf{PNID}$.

For *correctness*, accepting endpoint instances that have matching conversation compute the same session key; accepting endpoint instance keys are lawfully-interceptable by the collaboration of all the authorities in the instance's AID.

6.2 Oracles

Similarly to [5, 9], we use game-based security notions and give the adversary access to the following oracles:

† $\underline{\texttt{oRegister}(\mathsf{P}, \mathsf{type}, \mathsf{P}.\overline{\mathsf{PK}})}$: this oracle allows $\mathcal{A}$ to register either honest parties (with $\mathsf{P}.\overline{\mathsf{PK}} = \perp$) or malicious parties (with malicious $\mathsf{P}.\overline{\mathsf{PK}}(\neq\perp)$). The value type belongs to $\{\mathsf{endpoint}, \mathsf{proxy}, \mathsf{authority}\}$.

† $\underline{\texttt{oSend}(\pi^q_{\mathsf{P}}, m)}$: with this oracle, $\mathcal{A}$ can send a message m to an endpoint instance $\pi^j_{\mathsf{P}'}$, following the logical order of messages send during the protocol session run.

† $\underline{\texttt{oReveal}(\pi^q_{\mathsf{P}})}$: with this oracle $\mathcal{A}$ learns the session key of endpoint instance π^q_{P}. It also sets the corruption flag ρ to 1.

† $\underline{\texttt{oTest}_b(\pi^q_{\mathsf{P}})}$: This oracle can only be queried once during a security game, and returns either the real session key computed by endpoint instance π^q_{P} or a random key, depending on the value of a bit b (selected at the beginning of the game by the challenger).

† $\underline{\texttt{oRevealTD}(\mathsf{sst}, \mathsf{I}, \mathsf{R}, \mathsf{Prox}, \mathsf{AID}, \ell)}$: thanks to this oracle, $\mathcal{A}$ learns the trapdoor of authority Λ_ℓ registered in authority set AID for a session with state sst, if, and only if, the `Verify` algorithm called on sst and the associated data outputs 1. It also sets the trapdoor reveal flag $\Lambda_\ell.\texttt{tdrev}$ to 1.

The previous oracles come from [9], and we need to modify two oracles in our work, extending them to capture multiple authentication modes and credentials:

† $\underline{\texttt{oNewSession}(\mathsf{P}, \mathsf{role}, \mathsf{PNID}, \mathsf{PK_I}, \mathsf{PK_R}, \mathsf{ProxID}, \mathsf{AID})}$: with this oracle $\mathcal{A}$ creates endpoint/proxy instances of an endpoint/proxy P with role $\mathsf{role} \in \{\mathsf{Initiator}, \mathsf{Responder}, \mathsf{Proxy_I}, \mathsf{Proxy_R}\}$, with initiator authentication using the initiator's public credential $\mathsf{PK_I}$ and responder authentication using responder credential $\mathsf{PK_R}$. This allow to create an instance using a specific method (either 0, 1, 2 or 3) (given $(\mathsf{PK_I}, \mathsf{PK_R})$, we identify the method and we implicitly associate an integer corresponding to the method (either 0, 1, 2 or 3).

† $\underline{\texttt{oCorrupt}(\mathsf{P}, \ell)}$: this oracle allows the adversary to corrupt the ℓth long-term key of party P, and sets the corruption bit γ of each instance π^q_{P} using this long-term key to 1.

The oracles accessible to the adversary, along with their functionalities, are all depicted in Figure 18. Moreover as in [5, 9], we need an auxiliary function which takes as input a sst and an instance π, and returns 1 if the sst is the resulting of a π run. This allows us to establish a link between a session state and an instance (instead of just a party). We denote this function `IdentifySession`.

$\texttt{oRegister}(u, \mathsf{type}, \overline{\mathsf{PK}})$

if $\mathsf{type} = \mathsf{endpoint}$ **then**
  $\mathsf{ESet} \leftarrow \mathsf{ESet} \cup \{u\}$
elseif $\mathsf{role} = \mathsf{proxy}$ **then**
  $\mathsf{PROXSet} \leftarrow \mathsf{PROXSet} \cup \{u\}$
else
  $\mathsf{AUTHSet} \leftarrow \mathsf{AUTHSet} \cup \{u\}$
$(\widetilde{\mathsf{PK}}, \widetilde{\mathsf{SK}}) \leftarrow (\mathsf{type})\texttt{KeyGen}(u, \mathsf{pp})$
if $\overline{\mathsf{PK}} = \{\perp\}$ **then**
  $u.\overline{\mathsf{PK}} \leftarrow \widetilde{\mathsf{PK}}, u.\overline{\mathsf{SK}} \leftarrow \widetilde{\mathsf{SK}}$
  for all $\mathsf{SK} \in \overline{\mathsf{SK}}, \mathsf{SK}.\gamma = 0$
else $\overline{\mathsf{PK}} \neq \{\perp\}$ **then**
  $u.\overline{\mathsf{PK}} \leftarrow \overline{\mathsf{PK}}$
  for all $\mathsf{SK} \in \overline{\mathsf{SK}}, \mathsf{SK}.\gamma = 1$
return $u.\overline{\mathsf{PK}}$

$\texttt{oNewSession}(u, q, \mathsf{role}, \mathsf{PK_I}, \mathsf{PK_R}, \mathsf{PNID}, \mathsf{ProxID}, \mathsf{AID})$

if $\pi_u^q = \perp$ **then**
  $\pi_u^q.\mathsf{sid} \leftarrow \perp, \pi_u^q.\alpha \leftarrow \perp$
  $\pi_u^q.\mathsf{role} \leftarrow \mathsf{role}, \pi_u^q.\mathsf{PNID} \leftarrow \mathsf{PNID}$
  $\pi_u^q.\mathsf{ProxID} \leftarrow \mathsf{ProxID}, \pi_u^q.\mathsf{AID} \leftarrow \mathsf{AID}$
  $\pi_u^q.\mathsf{meth} \leftarrow (\mathsf{PK_I}, \mathsf{PK_R}), \pi_u^q.\mathsf{PRK_{out}} \leftarrow \perp$
  if $\mathsf{role} \in \{\mathsf{Initiator}, \mathsf{Responder}\}$ **then**
    $\pi_u^q.\rho \leftarrow 0$
    if $\mathsf{role} = \mathsf{Initiator}$ **then**
      $\pi_u^q.\mathsf{SK} \leftarrow u.\overline{\mathsf{SK}}[\mathsf{PK_I}]$
      $\pi_u^q.\mathsf{pid.PK} \leftarrow \mathsf{PK_R}$
      $\pi_u^q.\mathsf{pcsk} \leftarrow \mathsf{pcsk}_{\mathsf{u,ProxID[I]}}$
    else
      $\pi_u^q.\mathsf{SK} \leftarrow u.\overline{\mathsf{SK}}[\mathsf{PK_R}]$
      $\pi_u^q.\mathsf{pid.PK} \leftarrow \mathsf{PK_I}$
      $\pi_u^q.\mathsf{pcsk} \leftarrow \mathsf{pcsk}_{\mathsf{u,ProxID[R]}}$
    if $\mathsf{role} \in \{\mathsf{Proxy_I}, \mathsf{Proxy_R}\}$ **then**
      $\pi_u^q.\mathsf{sst} \leftarrow \perp$
      $\pi_u^q.\mathsf{ctn} \leftarrow 1$
      $\pi_u^q.\mathsf{tr} \leftarrow \perp$
      if $\mathsf{role} = \mathsf{Proxy_I}$ **then**
        $\pi_u^q.\mathsf{pcsk} \leftarrow \mathsf{pcsk}_{\mathsf{PNID[I],u}}$
      else
        $\pi_u^q.\mathsf{pcsk} \leftarrow \mathsf{pcsk}_{\mathsf{PNID[R],u}}$
return π_u^q

$\texttt{oCorrupt}(u, \ell)$

$u.\overline{\mathsf{SK}}[\ell].\gamma \leftarrow 1$
$\forall q \in \mathbb{N}^*,\ \pi_u^q.\mathsf{SK} = u.\overline{\mathsf{SK}}[\ell]$ **then** $\pi_u^q.\mathsf{SK}.\gamma = 1$
return $u.\overline{\mathsf{SK}}[\ell]$

$\texttt{oTest}_b(\pi_u^q)$

if $\pi_u^q = \perp$ **or** $\pi_u^q.\alpha \neq \texttt{terminated}$
  or $\pi_u^q.\mathsf{role} \notin \{\mathsf{Initiator}, \mathsf{Responder}\}$ **then**
  return $\perp$
else
  $T \leftarrow T \cup \{\pi_u^q\}$
  $\mathsf{k}_0 \leftarrow \pi_u^q.\mathsf{PRK_{out}}$
  $\mathsf{k}_1 \leftarrow\$ \mathcal{K}$
return k_b

$\texttt{oSend}(\pi_u^q, m)$

if $\pi_u^q.\mathsf{role} = \mathsf{Initiator}$ **then**
  if $\pi_u^q.\alpha = \perp$ **then**
    $m' \leftarrow \texttt{InitRun1}(m)$
  else
    $m' \leftarrow \texttt{InitRun2}(m)$
  return m'
elseif $\pi_u^q.\mathsf{role} = \mathsf{Responder}$ **then**
  if $\pi_u^q.\mathsf{sid} = \perp$ **then**
    $m' \leftarrow \texttt{RespRun1}(m)$
  else
    $m' \leftarrow \texttt{RespRun2}(m)$
  return m'
else
  if $\pi_u^q.\mathsf{cnt} = f \in [\![1, 4]\!]$ **and** $\pi_u^q.\mathsf{role} = \mathsf{Proxy_I}$ **then**
    $m' \leftarrow \texttt{Prox}_\texttt{I}\texttt{Run}f(m)$
    $\pi_u^q.\mathsf{cnt} = \pi_u^q.\mathsf{cnt} + 1$
    return m'
  elseif $\pi_u^q.\mathsf{cnt} = f \in [\![1, 3]\!]$ **and** $\pi_u^q.\mathsf{role} = \mathsf{Proxy_R}$ **then**
    $m' \leftarrow \texttt{Prox}_\texttt{R}\texttt{Run}f(m)$
    $\pi_u^q.\mathsf{cnt} = \pi_u^q.\mathsf{cnt} + 1$
    return m'
  else
    return $\perp$

$\texttt{oReveal}(\pi_u^q)$

if $\pi_u^q.\mathsf{role} \notin \{\mathsf{Proxy_I}, \mathsf{Proxy_R}\}$ **then**
  if $\pi_u^q = \perp$ **or** $\pi_u^q.\mathsf{status} \neq \texttt{terminated}$ **then**
    return $\perp$
  else
    $\pi_u^q.\rho \leftarrow 1$
    return $\pi_u^q.\mathsf{PRK_{out}}$

$\texttt{oRevealTD}(\mathsf{sst}, u, v, \mathsf{Prox}, (\Lambda_o)_{o=1}^n, \ell)$

if $\texttt{Verify}(\mathsf{pp}, \mathsf{sst}, u.\overline{\mathsf{PK}}, v.\overline{\mathsf{PK}}, \mathsf{Prox}.\overline{\mathsf{PK}}, (\Lambda_o.\overline{\mathsf{PK}})_{o=1}^n) = 1$
  then $\Lambda_\ell.\texttt{tdrev} \leftarrow 1$
  return $\Lambda_\ell.\mathsf{td}$
else
  return $\perp$

FIGURE 18 – Security model oracles (by $u.\overline{\mathsf{SK}}[\mathsf{PK_I}]$ we denote the long-term secret key of u associated to the long-term public key $\mathsf{PK_I}$).

6.3 Security properties

6.3.1 Key-security

In the key-security game, the adversary uses all the oracles presented above in order to learn the value of the bit b (selected by the challenger) used for the single $\texttt{oTest}_b$ query. The adversary wins if, and only if, it guesses b correctly, if its $\texttt{oTest}$ query target Fresh instances as defined below (following [9]), and if he respects the Sound property. This property ensures that only two endpoint instances have matching conversation with the same session identifier sid (no collision among the sid of different sessions occurs) and that two endpoint instances having matching conversation have opposite roles and derive the same session key. The Sound and Fresh properties are depicted in FIGURE 22, as well as the key-security game. (i.e., for which)

Definition 18 *An instance π_u^q is said to be* Fresh *if the long-term authentication credential used by both endpoints in that instance is uncorrupted at the time of the test query, the session key has not been revealed, and at least one authority in* AID *is honest and its trapdoor has not been revealed. More formally the endpoint instance π_u^q is* Fresh *if:*

- † $\pi_u^q.\alpha = \mathsf{terminated}$, $\pi_u^q.\mathsf{SK}.\gamma = 0$ *for the long-term key* $u.\mathsf{SK}$ *used by the instance and* $\pi_u^q.\rho = 0$*;*
- † *If there exists $j \in \mathbb{N}$ such that π_u^q and π_v^j have matching conversation (where $v = \pi_u^q.\mathsf{PNID}$), then* $\pi_v^j.\alpha = \mathsf{terminated}$, $\pi_v^j.\mathsf{SK}.\gamma = 0$ *for the long-term key* $v.\mathsf{SK}$ *used in the session, and* $\pi_v^j.\rho = 0$*;*
- † *If there exists no instance π_v^j such that π_u^q has matching conversation with π_v^j, then* $\pi_v^j.\mathsf{SK}.\gamma = 0$ *for the key* $v.\mathsf{SK}$ *supposed to be used in a potential exchange with π_u^q (regarding* $\pi_u^q.\mathsf{meth}$*);*
- † *It exists* $\Lambda \in \mathsf{AUTHSet}$ *such that $\mathcal{A}$ made no* $\texttt{oRevealTD}(\mathsf{sst}, \mathsf{I}, \mathsf{R}, \mathsf{Prox}, (\Lambda_i)_{i=1}^n, \ell)$ *query such that:*
  - ‡ $\Lambda \in \pi_u^q.\mathsf{AID}$, $\Lambda.\mathsf{SK}.\gamma = 0$ *for all* $\mathsf{SK} \in \Lambda.\overline{\mathsf{SK}}$ *and* $\Lambda = \Lambda_l$*;*
  - ‡ $\texttt{IdentifySession}(\mathsf{sst}, \pi_u^q) = 1$.
- † *It exists* $\Lambda' \in \mathsf{AUTHSet}$ *such that for all $j \in \mathbb{N}$ such that π_u^q has matching conversation with π_v^j, $\mathcal{A}$ made no* $\texttt{oRevealTD}(\mathsf{sst}, \mathsf{I}, \mathsf{R}, \mathsf{Prox}, (\Lambda_i)_{i=1}^n, \ell)$ *query such that:*
  - ‡ $\Lambda' \in \pi_v^j.\mathsf{AID}$, $\Lambda'.\mathsf{SK}.\gamma = 0$ *for all* $\mathsf{SK} \in \Lambda'.\overline{\mathsf{SK}}$ *and* $\Lambda' = \Lambda_l$*;*
  - ‡ $\texttt{IdentifySession}(\mathsf{sst}, \pi_v^j) = 1$.

Definition 19 (Key-security) *We define the* key-security *advantage of a p.p.t. adversary $\mathcal{A}$ as:*

$$\mathsf{Adv}^{\mathrm{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) = \left| \mathbb{P}\left[\mathbb{E}^{\mathsf{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\right] - \frac{1}{2} \right|.$$

The experiment is depicted in FIGURE 22 *along with the* Sound *and* Fresh *properties* (FIGURE 19)*. We then said that the* LI-EDHOC *protocol is* key-secure *if for any p.p.t. adversary $\mathcal{A}$, the advantage* $\mathsf{Adv}^{\mathrm{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ *is negligible compared to λ.*

6.3.2 LI-compliance

In the LI-compliance security game, $\mathbb{E}^{\mathsf{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$, the goal of the adversary is to generate a protocol session that completes successfully, but for which the lawful interception process fails—either by recovering no session key or by retrieving a key that differs from the one actually established during the session.

$\mathbb{E}^{\mathsf{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
$\mathsf{ESet} \leftarrow \emptyset, \mathsf{PROXSet} \leftarrow \emptyset, \mathsf{AUTHSet} \leftarrow \emptyset, T \leftarrow \emptyset$
$b \leftarrow\$ \{0,1\}$
$\mathcal{O}_{\mathsf{KS}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot,\cdot,\cdot), \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot,\cdot), \mathtt{SessionKeyReveal}(\cdot,\cdot), \\ \mathtt{oRevealTD}(\cdot,\cdot,\cdot,\cdot,\cdot), \mathtt{oTest}_b(\cdot,\cdot), \mathtt{oSend}(\cdot,\cdot) \end{array} \right\}$
$(e, \Xi, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{KS}}}(\lambda, \mathsf{pp})$
if $\neg\mathsf{Sound}$ **then return** 1
if $\neg\mathsf{Fresh}(\pi^e_\Xi)$ **then** $b' \leftarrow\$ \{0,1\}$
if $b = b'$ **return** 1, **else return** 0

Sound

for all $u, v, w \in \mathsf{ESet}$
  if $\exists$ **distinct** $\pi^q_u, \pi^j_v, \pi^k_w$ **with**
    $\pi^q_u.\mathsf{sid} = \pi^j_v.\mathsf{sid} = \pi^k_w.\mathsf{sid}$ **then**
    return `false`
  if $\exists\ \pi^q_u, \pi^j_v$ **with**
    $\pi^q_u.\alpha = \pi^j_v.\alpha = \mathtt{terminated}$
    and $\pi^q_u.\mathsf{sid} = \pi^q_u.\mathsf{sid}$
    and $\pi^q_u.\mathsf{meth} = \pi^q_u.\mathsf{meth}$
    and $\pi^q_u.\mathsf{PNID} = v$ **and** $\pi^j_v.\mathsf{PNID} = u$
    and $\pi^q_u.\mathsf{role} \neq \pi^j_v.\mathsf{role}$
    and $\pi^q_u.\mathsf{PRK}_{\mathsf{out}} \neq \pi^j_v.\mathsf{PRK}_{\mathsf{out}}$
    then return `false`
  return `true`

$\mathsf{Fresh}(\pi^q_u)$

if π^q_u **in** T
  if $\pi^q_u.\rho = 1$ **or** $\pi^q_u.\alpha \neq \mathtt{terminated}$
    or $\pi^q_u.\mathsf{SK}.\gamma = 1$ **or** $u.\overline{\mathsf{SK}}[\pi^q_u.\mathsf{SK}].\gamma = 1$ **then**
    return `false`
  if $\exists \pi^j_v \neq \pi^q_u$ **with** $\pi^j_v.\mathsf{role} \in \{\mathsf{Initiator}, \mathsf{Responder}\} \setminus \pi^q_u.\mathsf{role}$
    and $\pi^q_u.\mathsf{sid} = \pi^j_v.\mathsf{sid}$ **and** $(\pi^j_v.\alpha \neq \mathtt{terminated}$
    or $\pi^j_v.\rho = 1$ **or** $\pi^j_v.\mathsf{SK}.\gamma = 1$ **or** $v.\overline{\mathsf{SK}}[\pi^j_v.\mathsf{SK}].\gamma = 1)$
    then return `false`
  if $\forall \pi^j_v \neq \pi^q_u$ **with** $\pi^j_v.\mathsf{role} \in \{\mathsf{Initiator}, \mathsf{Responder}\} \setminus \pi^q_u.\mathsf{role}$,
    $\pi^q_u.\mathsf{sid} \neq \pi^j_v.\mathsf{sid}$ **and** $(\pi^j_v.\mathsf{SK}.\gamma = 1$ **or** $v.\overline{\mathsf{SK}}[\pi^j_v.\mathsf{SK}].\gamma = 1)$
    then return `false`
  if $\forall \Lambda \in \pi^q_u.\mathsf{AID}, (\Lambda.\mathsf{tdrev} = \mathtt{true}$ **or** $\Lambda.\mathsf{SK}.\gamma = 1\ (\forall\ \mathsf{SK} \in \Lambda.\overline{\mathsf{SK}}))$
    then return `false`
  if $\exists \pi^j_v$ **matching** π^q_u **s.t.** $\forall \Lambda' \in \pi^j_v.\mathsf{AID}$,
    $(\Lambda'.\mathsf{tdrev} = \mathtt{true}$ **or** $\Lambda'.\mathsf{SK}.\gamma = 1\ (\forall\ \mathsf{SK} \in \Lambda'.\overline{\mathsf{SK}}))$
    then return `false`
  return `true`
else
  return `false`

FIGURE 19 – The LI-EDHOC key-security security game and the Sound and Fresh properties.

To model this property, we use a key-extractor algorithm, as originally proposed in [9]. This algorithm derives from a complete protocol session between an Initiator and a Responder (who may be malicious) an outputs which is the session key established during the execution. To ensure that key security is preserved, the extractor must be unbounded. Based on [9], we adapt the general structure of the key-extractor algorithm as follows.

Definition 20 (Key-extractor) *A key-extractor algorithm* $\texttt{KeyExtract}$ *is a deterministic and unbounded algorithm which takes as input a proxy instance and a vector of public keys, and outputs a session key* $\mathsf{PRK_{out}}$*. Moreover, we require that for any* $m, n \in \mathbb{N}^*$*, for any* Initiator I *and* Responder R*, for any proxies* $\mathsf{Prox_I}$ *and* $\mathsf{Prox_R}$*, for any sets of authorities* $(\Lambda^{\mathsf{I}}_j)^m_{j=1}$ *and* $(\Lambda^{\mathsf{R}}_j)^n_{j=1}$*, and for any set* $\{\mathsf{pp}, \mathsf{I}.\overline{\mathsf{PK}}, \mathsf{I}.\overline{\mathsf{SK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{SK}}, \mathsf{Prox_I}.\overline{\mathsf{PK}}, \mathsf{Prox_I}.\overline{\mathsf{SK}}, \mathsf{Prox_R}.\overline{\mathsf{PK}}, \mathsf{Prox_R}.\overline{\mathsf{SK}}, \mathsf{PRK_{out}}, \mathsf{sst_I}, \mathsf{sst_R}, \mathsf{APK_I} = (\Lambda^{\mathsf{I}}_j.\overline{\mathsf{PK}})^m_{j=1}, \mathsf{ASK_I} = (\Lambda^{\mathsf{I}}_j.\overline{\mathsf{SK}})^m_{j=1}, \mathsf{APK_R} = (\Lambda^{\mathsf{R}}_j.\overline{\mathsf{PK}})^n_{j=1}, \mathsf{ASK_R} = (\Lambda^{\mathsf{R}}_j.\overline{\mathsf{SK}})^n_{j=1}, \mathsf{tr_I}, \mathsf{tr_R}, \mathsf{LIC}.\overline{\mathsf{PK}}\}$ *generated as follows:*

- $\mathsf{pp} \leftarrow \texttt{Setup}(\lambda)$*;*
- $(\mathsf{I}.\overline{\mathsf{PK}}, \mathsf{I}.\overline{\mathsf{SK}})$*,* $(\mathsf{R}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{SK}}) \leftarrow \texttt{EKeyGen}(\mathsf{pp})$*;*
- $(\mathsf{Prox_I}.\overline{\mathsf{PK}}, \mathsf{Prox_I}.\overline{\mathsf{SK}})$*,* $(\mathsf{Prox_R}.\overline{\mathsf{PK}}, \mathsf{Prox_R}.\overline{\mathsf{SK}}) \leftarrow \texttt{ProxKeyGen}(\mathsf{pp})$*;*
- $\forall j \in [\![1, m]\!], \forall k \in [\![1, n]\!]$ $(\Lambda^{\mathsf{I}}_j.\overline{\mathsf{PK}}, \Lambda^{\mathsf{I}}_j.\overline{\mathsf{SK}})$*,* $(\Lambda^{\mathsf{R}}_k.\overline{\mathsf{PK}}, \Lambda^{\mathsf{R}}_k.\overline{\mathsf{SK}}) \leftarrow \texttt{AKeyGen}(\mathsf{pp})$*;*
- $(\mathsf{PRK_{out}}, \mathsf{sst_I}, \mathsf{sst_R}, \mathsf{PRK_{out}})$ *resulting on a run of the LI-EDHOC protocol on input* pp *and the previous public key sets* $\overline{\mathsf{PK}}$*;*
- $\mathsf{tr_I}$ *is the transcript of the protocol session execution producing* $\mathsf{sst_I}$ *from the point of view of* $\mathsf{Prox_I}$*;*
- $\mathsf{tr_R}$ *is the transcript of the protocol session execution producing* $\mathsf{sst_R}$ *from the point of view of* $\mathsf{Prox_R}$*;*
- $\mathsf{LIC}.\overline{\mathsf{PK}} \leftarrow \{\mathsf{I}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{Prox_I}.\overline{\mathsf{PK}}, \mathsf{Prox_R}.\overline{\mathsf{PK}}\} \cup \{\Lambda^{\mathsf{I}}_j.\overline{\mathsf{PK}}\}^m_{j=1} \cup \{\Lambda^{\mathsf{R}}_j.\overline{\mathsf{PK}}\}^n_{j=1}$*;*

we have that for any $\mathsf{U} \in \{\mathsf{I}, \mathsf{R}\}$ *and for any proxy instance* $\pi_{\mathsf{Prox_U}}$ *such that* $\pi_{\mathsf{Prox_U}}.\mathsf{tr} = \mathsf{tr_U}$ *and* $\pi_{\mathsf{Prox_U}}.\mathsf{sst} = \mathsf{sst_U}$*:*

$$\mathbb{P}\Big(\texttt{KeyExtract}(\pi_{\mathsf{Prox_U}}, \mathsf{LIC}.\overline{\mathsf{PK}}) = \mathsf{PRK_{out}}\Big) = 1.$$

The set of keys $\mathsf{LIC}.\overline{\mathsf{PK}}$ is there to ensure that the $\texttt{KeyExtract}$ algorithm extracts the correct key, *i.e.,* it may need certain keys to compute $\mathsf{PRK_{out}}$ (and possibly also use some of these keys in the proof to do some verifications).

The adversary has access to all but the $\texttt{oTest}$ oracles and he wants to outputs a tuple $\big(j, \mathsf{sst_d}, \mathsf{I}, \mathsf{R}, \mathsf{Prox_d}, (\Lambda^{\mathsf{d}}_q, \Lambda^{\mathsf{d}}_q.\mathsf{td})^{n_{\mathsf{d}}}_{q=1}\big)$ in such a way that the session keys retrieving by the $\texttt{Open}$ algorithm and the $\texttt{KeyExtract}$ algorithm based on this tuple are different.

Definition 21 (LI-compliance) *We define the* LI-compliance *advantage of a p.p.t. adversary* $\mathcal{A}$ *as:*

$$\mathsf{Adv}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) = \mathbb{P}\left[\begin{array}{l} \big(\mathsf{PRK_{out*}}, \pi^j_{\mathsf{Prox_d}}, \mathsf{LIC}.\overline{\mathsf{PK}}\big) \leftarrow \mathbb{E}^{\mathsf{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \\ \wedge\ \mathsf{PRK_{out}} \leftarrow \texttt{KeyExtract}(\pi^j_{\mathsf{Prox_d}}, \mathsf{LIC}.\overline{\mathsf{PK}}) \\ s.t.\ \mathsf{PRK_{out}} \neq \perp \wedge \mathsf{PRK_{out*}} \neq \perp \wedge \mathsf{PRK_{out}} \neq \mathsf{PRK_{out*}} \end{array}\right].$$

The experiment is depicted in Figure 20*. We then said that the* LI-EDHOC *protocol is* LI-compliant *secure if for any p.p.t. adversary* $\mathcal{A}$*, the advantage* $\mathsf{Adv}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ *is negligible compared to* λ*.*

6.3.3 Non-frameability

In the non-frameability security game, $\mathbb{E}^{\mathsf{NF}}_{\mathsf{LI\text{-}EDHOC},\mathcal{A}}(\lambda)$, the adversary is given access to all oracles except `oTest`. The objective is to associate an honest party P—either an $\mathsf{Initiator}$ or a $\mathsf{Responder}$—with a session state sst resulting from a communication in which P did not reach an accepting state.

More precisely, the adversary wins if they can construct such a party P and a session sst involving valid roles (an $\mathsf{Initiator}$ or a $\mathsf{Responder}$, a proxy, and a group of authorities), such that sst appears to result from a legitimate communication involving P, despite P not having accepted the session. Otherwise, the adversary loses the non-frameability game.

Definition 22 (Non-frameability) *We define the* non-frameability *avantage of a p.p.t. adversary $\mathcal{A}$ as:*

$$\mathsf{Adv}^{\mathrm{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) = \mathbb{P}\Big[\mathbb{E}^{\mathsf{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big].$$

The experiment is depicted in Figure 20. *We then said that the* LI-EDHOC *protocol is* non-frameable *if for any p.p.t. adversary $\mathcal{A}$, the advantage $\mathsf{Adv}^{\mathrm{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ is negligible compared to λ.*

$\mathbb{E}^{\mathsf{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
$\mathsf{ESet} \leftarrow \emptyset, \mathsf{PROXSet} \leftarrow \emptyset, \mathsf{AUTHSet} \leftarrow \emptyset$
$\mathcal{O}_{\mathsf{LIC}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot,\cdot,\cdot), \mathtt{oSend}(\cdot,\cdot), \\ \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot,\cdot), \\ \mathtt{SessionKeyReveal}(\cdot,\cdot), \\ \mathtt{oRevealTD}(\cdot,\cdot,\cdot,\cdot,\cdot) \end{array} \right\}$
$\big(j, \mathsf{sst_d}, \mathsf{I}, \mathsf{R}, \mathsf{Prox_d}, (\Lambda^{\mathsf{d}}_q, \Lambda^{\mathsf{d}}_q.\mathsf{td})^{n_{\mathsf{d}}}_{q=1}\big) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{LIC}}}(\lambda, \mathsf{pp})$
if $\mathsf{Prox_d}.\overline{\mathsf{SK}}[\mathsf{sst_d}].\gamma = 1$ **then**
  return $\perp$
if $\mathtt{Verify}(\mathsf{pp}, \mathsf{sst_d}, \mathsf{I}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{Prox_d}.\overline{\mathsf{PK}}, (\Lambda^{\mathsf{d}}_q.\overline{\mathsf{PK}})^{n_{\mathsf{d}}}_{q=1}) = 0$
  then return $\perp$
if $\mathtt{IdentifySession}(\mathsf{sst_d}, \pi^j_{\mathsf{Prox_d}}) = 0$ **then**
  return $\perp$
$\mathsf{PRK_{out*}} \leftarrow \mathtt{Open}(\mathsf{pp}, \mathsf{sst_d}, (\Lambda^{\mathsf{d}}_q.\overline{\mathsf{PK}})^{n_{\mathsf{d}}}_{q=1}, (\Lambda^{\mathsf{d}}_q.\mathsf{td})^{n_{\mathsf{d}}}_{q=1})$
if $\mathsf{PRK_{out*}} = \perp$ **then**
  return $\perp$
$\mathsf{LIC}.\overline{\mathsf{PK}} \leftarrow \mathsf{I}.\overline{\mathsf{PK}} \cup \mathsf{R}.\overline{\mathsf{PK}} \cup \mathsf{Prox_d}.\overline{\mathsf{PK}} \cup \{\cup(\Lambda^{\mathsf{d}}_q.\overline{\mathsf{PK}})^{n_{\mathsf{d}}}_{q=1}\}$
return $(\mathsf{PRK_{out*}}, \pi^j_{\mathsf{Prox_d}}, \mathsf{LIC}.\overline{\mathsf{PK}})$

$\mathbb{E}^{\mathsf{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
$\mathsf{ESet} \leftarrow \emptyset, \mathsf{PROXSet} \leftarrow \emptyset, \mathsf{AUTHSet} \leftarrow \emptyset$
$b \leftarrow\$ \{0,1\}$
$\mathcal{O}_{\mathsf{NF}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot,\cdot,\cdot), \mathtt{oSend}(\cdot,\cdot), \\ \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot,\cdot), \\ \mathtt{SessionKeyReveal}(\cdot,\cdot), \\ \mathtt{oRevealTD}(\cdot,\cdot,\cdot,\cdot,\cdot) \end{array} \right\}$
$(\mathsf{sst}, \mathsf{P}) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{NF}}}(\lambda, \mathsf{pp})$
if $\exists(\mathsf{I}, \mathsf{R}) \in \mathsf{ESet}^2, \exists n \in \mathbb{N}^*, \exists \mathsf{Prox} \in \mathsf{PROXSet},$
  $\exists(\Lambda_j)^n_{j=1} \in \mathsf{AUTHSet}^n$ **s.t.** :
  $-\ \mathtt{Verify}(\mathsf{pp}, \mathsf{sst}, \mathsf{I}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{Prox}.\overline{\mathsf{PK}}, (\Lambda_j.\overline{\mathsf{PK}})^n_{j=1}) = 1;$
  $-\ \mathsf{P} \in \{\mathsf{I}, \mathsf{R}\}$ **and** $\mathsf{P}.\overline{\mathsf{SK}}[\mathsf{sst}].\gamma = 0;$
  $-\ \forall q \in \mathbb{N}$, **if** $\pi^q_{\mathsf{P}} \neq \perp$: $\mathtt{IdentifySession}(\mathsf{sst}, \pi^q_{\mathsf{P}}) = 0$
    or $\pi^q_{\mathsf{P}}.\alpha = 0;$
  then return 1
else return 0.

Figure 20 – The LI-EDHOC non-frameability and LI-compliance security games (by $\mathsf{P}.\overline{\mathsf{SK}}[\mathsf{sst}]$ we mean the long-term secret key of P associated to the method stored in sst, and by $\mathsf{Prox_d}.\overline{\mathsf{SK}}[\mathsf{sst_d}]$ we mean the secret (long-term) signing key of $\mathsf{Prox_d}$ used to sign $\mathsf{sst_d}$).

6.3.4 Identity protection

To enable lawful interception, each proxy must be aware of the identities of both participants. By assumption each proxy is aware of the identity of its associated endpoint. During the protocol run, thanks to the

additional encryption between the endpoint and its proxy, the later learns the identity of the endpoint's partner. The identity protection property has been defined in [14] in the special case of EDHOC but can be easily generalizable to other protocols. We slightly adapt this definition to take into account the modification we made in LI-EDHOC.

On the Responder **side**. As is the case in [14], we will show that our protocol, LI-EDHOC, is identity protective against passive adversaries for the Responder, *i.e.,* an adversary that only intercepts forwarded messages cannot learn the identity of the Responder, at least, given two Responder identities, distinguish one of the two. Why this restriction to passive adversaries? It's actually quite simple. If $\mathcal{A}$ is active and impersonates the Initiator, then he immediately learns the identity of the Responder to the second message. If $\mathcal{A}$ is active but can simply control the Initiator (without corrupting its long-term keys), then he also learns the identity of the Responder to the second message since sk_2 only depends on g^{xy}. This is why we limit ourselves to a passive adversary.

It goes without saying that if the adversary has access to sst, he learns both identities. This is an assumption we make here (and also for the identity protection of the Initiator): $\mathcal{A}$ never accesses sst (which seems reasonable, given that this information is kept secret by the proxy until an authority requests interception). It also goes without saying that the corruption of an authority leads nowhere. Indeed, the only for $\mathcal{A}$ to obtain information by corrupting an authority is to have access to sst (so far supposed impossible). Finally a trivial attack consists of corruption a proxy, which we will assume is impossible in our security game (and which seems reasonable since in our application case, it is then an operator).

We know define this identity protection notion with more rigor for the Responder.

Definition 23 (Responder identity protection) *In a similar way, we define the* Responder identity protection *advantage of a p.p.t. adversary $\mathcal{A}$ as:*

$$\mathsf{Adv}^{\text{Id-Pro-Resp}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) = \left|\mathbb{P}\left[\mathbb{E}^{\text{Id-Responder-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\right] - \mathbb{P}\left[\mathbb{E}^{\text{Id-Responder-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\right]\right|$$

with the experiences depicted in FIGURE 21. *We then said that the* LI-EDHOC *protocol is* Responder identity protecting *if for any p.p.t. adversary $\mathcal{A}$, the advantage $\mathsf{Adv}^{\text{Id-Pro-Resp}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ is negligible compared to λ.*

On the Initiator **side**. The LI-EDHOC protocol is also identity protective for the Initiator against passive adversaries but also against active adversaries who can control the Responder without corruption its long-term keys. On the other hand, against a fully active adversary $\mathcal{A}$ who impersonates a Responder (with corruption of the long-term keys) the property is not preserved.

We here define this identity protection notion for the Initiator, as we did for the Responder.

Definition 24 (Initiator identity protection) *We define the* Initiator identity protection *advantage of a p.p.t. adversary $\mathcal{A}$ as:*

$$\mathsf{Adv}^{\text{Id-Pro-Init}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) = \left|\mathbb{P}\left[\mathbb{E}^{\text{Id-Initiator-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\right] - \mathbb{P}\left[\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\right]\right|$$

with the experiences depicted in FIGURE 21. *We then said that the* LI-EDHOC *protocol is* Initiator identity protecting *if for any p.p.t. adversary $\mathcal{A}$, the advantage $\mathsf{Adv}^{\text{Id-Pro-Init}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ is negligible compared to λ.*

$\mathbb{E}^{\text{IdPro-Responder-}b}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
$\mathsf{ID_CRED_R}_0, \mathsf{ID_CRED_R}_1 \leftarrow \mathcal{A}(\mathsf{ESet}_{\mathsf{meth}})$
$\mathsf{ID_CRED_R} \leftarrow \mathsf{ID_CRED_R}_b$
$\mathsf{ID_CRED_I} \leftarrow\$ \mathsf{ESet}_{\mathsf{meth}}$
$M_1 \leftarrow \mathtt{InitRun1}(\mathsf{ID_CRED_I})$
$M_2 \leftarrow \mathtt{RespRun1}(\mathsf{ID_CRED_R}, M_1)$
$M_3 \leftarrow \mathtt{InitRun2}(\mathsf{ID_CRED_I}, M_2)$
$M_3' \leftarrow \mathtt{Prox_I Run3}(\mathsf{ID_CRED_I}, M_3)$
$M_4 \leftarrow \mathtt{RespRun2}(\mathsf{ID_CRED_R}, M_3')$
$M \leftarrow (\mathsf{ID_CRED_I}, M_1, M_2, M_3, M_3', M_4)$
$b' \leftarrow \mathcal{A}(M)$
if $b = b'$ **then**
  return 1
else
  return 0

$\mathbb{E}^{\text{IdPro-Initiator-}b}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
$\mathsf{ID_CRED_I}_0, \mathsf{ID_CRED_I}_1 \leftarrow \mathcal{A}(\mathsf{ESet}_{\mathsf{meth}})$
$\mathsf{ID_CRED_I} \leftarrow \mathsf{ID_CRED_I}_b$
$\mathsf{ID_CRED_R} \leftarrow \mathcal{A}(\mathsf{ESet}_{\mathsf{meth}})$
$M_1 \leftarrow \mathtt{InitRun1}(\mathsf{ID_CRED_I})$
$M_2 \leftarrow \mathcal{A}(\mathtt{RespRun1}(\mathsf{ID_CRED_R}))$
$M_3 \leftarrow \mathtt{InitRun2}(\mathsf{ID_CRED_I}, M_2)$
$M_3' \leftarrow \mathtt{Prox_I Run3}(\mathsf{ID_CRED_I}, M_3)$
$M \leftarrow (M_1, M_3, M_3')$
$b' \leftarrow \mathcal{A}(M)$
if $b = b'$ **then**
  return 1
else
  return 0

FIGURE 21 – The LI-EDHOC identity protection security games. By $\mathsf{ESet}_{\mathsf{meth}}$ we mean the set of all endpoints running the protocol with the same method (0,1,2, or 3).

7 Security results

Following the approach of [14], we conduct this security analysis in the random oracle model, where the hash function and the key-derivation functions used across the four protocols are represented as random oracles. Specifically, we denote by $\mathsf{RO}_{\mathcal{H}}$ the oracle corresponding to the hash function, and by RO_{T} and RO_{P} the oracles modeling `HKDF_Extract` and `HKDF_Expand`[12], respectively, as ideal random functions.

We begin this section by establishing the key-security property of our protocol on page 52. Then, on page 81, we demonstrate that the LI-EDHOC protocol satisfies the LI-compliance property. We continue on page 96, with the proof of the non-frameability property. Finally, on page 101, we prove the identity protection properties (for the Responder and the Initiator) of the protocol.

7.1 Key security

The first security result we obtained is now presented in the following statement. It states that our protocols satisfy the key-security property.

Theorem 2 (Key-Secure LI-EDHOC protocol) *Under the assumption that the signature scheme* DS *is* EUF-CMA*-secure, that the proofs and signatures of knowledge are zero-knowledge and extractable, that the* DDH *and the* GDH *problems are hard to solve in* $\mathbb{G}$*, the encryption schemes are secure and injective, then the* LI-EDHOC *protocol, in its four versions, is key-secure. More precisely, for all p.p.t. adversary* $\mathcal{A}$ *making at most* q_{R} *(resp.* $q_{\mathsf{RO}_{\mathsf{T}}}$*,* $q_{\mathsf{RO}_{\mathsf{P}}}$*,* $q_{\mathsf{RO}_{\mathsf{H}}}$*,* q_{RTD}*,* q_{NS} *and* q_{S}*) queries to the* `oRegister` *(resp.* RO_{T}*,* RO_{P}*,*RO_{H}*,* `oRevealTD`*,* `oNewSession` *and* `oSend`*) oracle, we have:*

$$\begin{aligned}\mathsf{Adv}^{\mathrm{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq \frac{3q_{\mathsf{NS}}^2}{2^{l_{\mathsf{C}}+1}p} + \frac{\left(q_{\mathsf{RO}_{\mathsf{T}}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}}\right)^2 + \left(q_{\mathsf{RO}_{\mathsf{T}}} + q_{\mathsf{RO}_{\mathsf{H}}}\right)^2 + \left(q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO}_{\mathsf{H}}}\right)^2}{2^{l_{\mathsf{hash}}+1}} \\ + q_{\mathsf{NS}} \cdot q_{\mathsf{R}}^4 \cdot \Big[\left(2q_{\mathsf{RTD}} + q_{\mathsf{S}}\right) \cdot \epsilon_{\mathsf{SoK}}(\lambda) + q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + 24\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}}(\lambda) + 4\mathsf{Adv}^{\mathrm{EUF\text{-}CMA}}_{\mathsf{DS}}(\lambda) \\ + \left(4q_{\mathsf{RO}_{\mathsf{T}}} + 4q_{\mathsf{NS}}\right) \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda) + 4 \cdot \frac{q_{\mathsf{RO}_{\mathsf{P}}{}^{l_2}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{key}}}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{mac}}}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}} + 6}{2^{l_{\mathsf{hash}}}}\Big]\end{aligned}$$

Proof 2 (of theorem 2) To prove this theorem, we proceed by games hop. As a reminder, here is the key-security security game.

$\mathbb{E}^{\mathsf{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

1 : $\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda), T \leftarrow \emptyset, b \leftarrow\$ \{0,1\}$
2 : $\mathsf{ESet} \leftarrow \emptyset, \mathsf{PROXSet} \leftarrow \emptyset, \mathsf{AUTHSet} \leftarrow \emptyset$
3 : $\mathcal{O}_{\mathsf{KS}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot,\cdot,\cdot), \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot,\cdot), \mathtt{SessionKeyReveal}(\cdot,\cdot), \\ \mathtt{oRevealTD}(\cdot,\cdot,\cdot,\cdot,\cdot), \mathtt{oTest}_b(\cdot,\cdot), \mathtt{oSend}(\cdot,\cdot) \end{array} \right\}$
4 : $(e, \Xi, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{KS}}}(\lambda, \mathsf{pp})$
5 : **if** $\neg\mathsf{Sound}$ **then return** 1
6 : **if** $\neg\mathsf{Fresh}(\pi^e_\Xi)$ **then** $b' \leftarrow\$ \{0,1\}$
7 : **if** $b = b'$ **return** 1, **else return** 0

FIGURE 22 – LI-EDHOC key-security security game.

[12]More precisely, we have several random oracles RO_{P} : $\mathsf{RO}_{\mathsf{P}}{}^{l_2}$, $\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{mac}}}$,$\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}$, $\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{key}}}$, $\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{iv}}}$, depending on the digest size, but for clarity here we simply denote them as RO_{P} (latter we will make the distinction).

Our security model capture the four versions of the protocol into one security experiment. Nevertheless some difference appear at certain stages between the four versions, which leads us to split our analysis into four branches, depending on which authentication method is used for a session. For the first thirteen games, the method used does not intervene in the analysis (except for the twelfth, for notations). Then we will separate our analysis into four branches. The FIGURE 23 depicts the thirteen first steps with some explanation. Then in FIGURE 24 we present the beginning of the four branches and with FIGURE 25 we represent the end of the four branches along with the conclusion.

To begin with, the game $\mathbb{G}_0$ is the reformulation of the original experience. Then with games $\mathbb{G}_1$ and $\mathbb{G}_2$ we ensure uniqueness of the partnering instances (one Initiator instance for one Responder instance). This is useful for the Sound property. Then in games $\mathbb{G}_3$, $\mathbb{G}_4$ and $\mathbb{G}_5$, we simulate the random oracles RO as lists SO (in order to manipulate them later). With games $\mathbb{G}_6$ and $\mathbb{G}_7$ we avoid collisions between three simulated oracles (necessary later to ensure the key-schedule security). Then with game $\mathbb{G}_8$ we try to guess the instance tested by the adversary, as well as its potential partner, and in game $\mathbb{G}_9$ we ensure that this partner exists, and guessed it in game $\mathbb{G}_{10}$. Then with $\mathbb{G}_{11}$ we chose particular authorities in order to ensure that some sessions are partly fresh. Next with games $\mathbb{G}_{12}$ and $\mathbb{G}_{13}$ we use extractors of knowledge to prevent the adversary from providing false signatures or proofs of knowledge.

† **Game $\mathbb{G}_0$:** This game is the original experience $\mathbb{E}^{\mathsf{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{E}^{\mathsf{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_1$:** This game is the same game as the previous one, except that this time, we set bad flags $\mathtt{bad}_1$ and $\mathtt{bad}_2$ to `true` when:

- two honest (either Initiator or Responder) instances pick the same nonce ($\mathsf{C_I}$ or $\mathsf{C_R}$) or the same group element [$\mathtt{bad}_1$];
- an honest Responder instance picks a nonce or a group element already received from an Initiator instance [$\mathtt{bad}_2$].

By "honest" we mean not adversary-controlled. This changes are internal to the game, thus are indistinguishable by the adversary $\mathcal{A}$. We hence have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_2$:** This game is the same as $\mathbb{G}_1$, except that we abort when flags $\mathtt{bad}_1$ or $\mathtt{bad}_2$ are set to `true`. By the identical-until-bad lemma, we have:

$$\Big|\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_2 \Longrightarrow 1\big)\Big| \leq \mathbb{P}(F_1)$$

where F_1 is the event: $\mathtt{bad}_1$ or $\mathtt{bad}_2$ is set to `true` in $\mathbb{G}_1$.
We provide an upper bound for $\mathbb{P}(F_1)$ using the birthday paradox. First, observe that we pick at random q_{NS} pairs of group element and nonce over a space of size $p \times 2^{l_c}$. We may reasonably assume that $q_{\mathsf{NS}} \leq p \times 2^{l_c}$. Thus, the birthday paradox theorem gives us that the probability a collision occurs between two such pairs is:

$$\frac{q_{\mathsf{NS}}(q_{\mathsf{NS}}-1)}{2(2^{l_c}p)} \leq \frac{q_{\mathsf{NS}}^2}{2^{l_c+1}p}.$$

$\mathbb{G}_0$	Reformulation of the original game
$\mathbb{G}_1$	Flagging on invalid sid
$\mathbb{G}_2$	Restricting the possible sid to ensure a part of soundness
$\mathbb{G}_3$	Rewriting of $\mathsf{RO}_{\mathcal{H}}$ as a random list
$\mathbb{G}_4$	Rewriting of RO_{P} as a random list
$\mathbb{G}_5$	Rewriting of RO_{T} as a random list
$\mathbb{G}_6$	Flagging on collisions between random oracles
$\mathbb{G}_7$	Restriction on potential collisions between some RO
$\mathbb{G}_8$	Guessing the tested instance and its potential matching instance
$\mathbb{G}_9$	Ensuring the existence of a matching instance for the tested instance
$\mathbb{G}_{10}$	Guessing the matching instance for the tested instance
$\mathbb{G}_{11}$	Choosing at random two special authorities to ensure a part of freshness
$\mathbb{G}_{12}$	Using extractor on SoK to prevent $\mathcal{A}$ from forging signature of knowledge
$\mathbb{G}_{13}$	Using extractor on NIPoK to prevent $\mathcal{A}$ from forging proof of knowledge on authorities' long-term public keys

FIGURE 23 – Thirteen first games of the game-schedule.

This was for $\mathtt{bad}_1$. Now for $\mathtt{bad}_2$ observe that a $\mathsf{Responder}$ instance may receive at most $q_{\mathsf{NS}} - 1$ pairs of group element and nonce. Since this is true for possibly $q_{\mathsf{NS}} - 1$ $\mathsf{Responder}$ instance, we thus deduce that the probability it picks at random one of these pairs is:

$$\frac{(q_{\mathsf{NS}} - 1)(q_{\mathsf{NS}} - 1)}{2^{l_{\mathsf{C}}} p} \leq \frac{q_{\mathsf{NS}}^2}{2^{l_{\mathsf{C}}} p} = \frac{2q_{\mathsf{NS}}^2}{2^{l_{\mathsf{C}}+1} p}.$$

So, combining these two results, we thus obtain:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_2 \Longrightarrow 1\big)\Big| \leq \frac{3q_{\mathsf{NS}}^2}{2^{l_{\mathsf{C}}+1} p}}$$

where $p = |\mathbb{G}|$ and l_{C} is the length of the nonces.

At the end of this game, we have proved that no more than two endpoint instances can share the same $\mathsf{sid} = (\mathsf{C_I}, \mathsf{C_R}, g^x, g^y)$. Indeed, by absurd, let's assume that there exists three endpoint instances π_u^q, π_v^j, and π_w^k that have matching conversation by pairs. In this situation, we have then three pairs of partnered instances, but they only commonly share (at most) two Diffie-Hellman pairs of exchanged elements. But thanks to bad flags $\mathtt{bad}_1$ or $\mathtt{bad}_2$, after, $\mathbb{G}_2$ we abort if this situation occurs. Contradiction $\lightning$.

Moreover, this game also implies that endpoint instances having matching conversation have opposite roles. Indeed, by absurd, since at most two instances can share the same sid, we have the following implication: if the two endpoint instances having matching conversation, have the same role, they thus have the same Diffie-Hellman element (since its position is fixed in the sid). But thanks to the bad flag $\mathtt{bad}_1$, this is a collision and after game $\mathbb{G}_2$, we abort in this situation. We thus have a contradiction $\lightning$. This analysis ensure a part of the soundness property.

† **Game $\mathbb{G}_3$:** This game is the same as $\mathbb{G}_2$, except that we simulate $\mathsf{RO}_{\mathcal{H}}$ with a list that is empty when the game begins. Then for any new query it picks, stores and returns a random value from $\{0,1\}^{l_{\mathsf{hash}}}$, and returns the same answer if a query is repeated. We then denote $\mathsf{SO}_{\mathcal{H}}$ instead of $\mathsf{RO}_{\mathcal{H}}$. This modification is internal to the game and indistinguishable by the adversary. Thus we have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_3 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_2 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_4$:** This game is the same as $\mathbb{G}_3$ except that here we simulate RO_{P} (which is used to simulate the different expansions in the key-derivation schedule instead of `HKDF_Expand`) with a list that is empty when the game begins. Then for any new query it picks, stores and returns a random value, and returns the same answer if a query is repeated. We then denote SO_{P} instead of RO_{P}. However, remark that SO_{P} picks at random values of different lengths depending on which query is made. In fact we define simulated random oracles by size: $\mathsf{SO}_{\mathsf{P}}{}^{\mathsf{size}}$ for $\mathsf{size} \in \{l_2, l_{\mathsf{id}}, l_{\mathsf{hash}}, l_{\mathsf{key}}, l_{\mathsf{iv}}, l_{\mathsf{mac}}\}$. But this distinction does not impact this or future analysis, we will however specify which one we use if necessary. This modification is internal to the game and indistinguishable by the adversary. Thus we have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_4 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_3 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_5$:** This game is the same as game $\mathbb{G}_4$ except that here we simulate RO_{T} (which is used to simulate the different extractions in the key-derivation schedule instead of `HKDF_Extract`) with a list that is empty when the game begins and which we denote SO_{T} instead of RO_{T}. But this one is slightly different from the two previous ones. Indeed we need, for future reductions against the GDH

problem, that it accepts two types of queries. To do so, the list elements differ from the previous two simulated oracles adn are of the form $(\mathsf{string}, Z, (X, Y); \lambda)$, with at least Z or (X, Y) different from $\perp$. We distinguish two situations. Firstly, when making a normal call to this oracle, $\mathcal{A}$ make a query of the form (string, Z), where string is any bit string, which may represent one of the pseudo-random key or simply be empty, and Z is a group element, possibly a Diffie-Hellman value, according to the protocol run. In that case the challenger checks in the list he kept in memory to find an entry matching with $(\mathsf{string}, Z, *; \nu)$ (here $*$ is either $\perp$ or (X, Y) (if already defined with a Diffie-Hellman couple (X, Y) matching Z)). If the search is successful, the challenger then outputs ν. Otherwise he randomly sets $\nu \leftarrow\$ \{0,1\}^{l_{\mathsf{hash}}}$ and appends the new element $(\mathsf{string}, Z, \perp; \nu)$ to the list.
Secondly, *e.g.,* during reductions, the adversary will also ask queries of the form $(\mathsf{string}, (X, Y))$, where (X, Y) is a pair of group elements. Once again then the challenger checks the list he has in memory for an entry matching with either $(\mathsf{string}, *, (X, Y); \nu)$ or $(\mathsf{str}, Z, *; \nu)$ such that (X, Y, Z) is a Diffie-Hellman tuple. Again, if its search is successful, he outputs ν. Otherwise he randomly sets $\nu \leftarrow\$ \{0,1\}^{l_{\mathsf{hash}}}$ and appends $(\mathsf{string}, \perp, (X, Y); \nu)$ to the list. We must precise that the simulation is perfect and indistinguishable by the adversary, since the challenger can use the DDH oracle provided with the GDH problem to check if a tuple (X, Y, Z) is a Diffie-Hellman tuple (in the second case). We hence have:

$$\boxed{\mathbb{P}(\mathbb{G}_5 \Longrightarrow 1) = \mathbb{P}(\mathbb{G}_4 \Longrightarrow 1).}$$

† **Game $\mathbb{G}_6$:** Since $\mathsf{SO}_{\mathcal{H}}$, ${\mathsf{SO}_\mathsf{P}}^{l_{\mathsf{hash}}}$ and SO_T return a digest of size l_{hash}, we want to avoid collision between this three "simulated" oracles. For this the game $\mathbb{G}_6$ is the same as $\mathbb{G}_5$ except that we set a bad flag $\mathtt{bad_3}$ to $\mathtt{true}$ when a collision on two of this three simulated oracles occurs. This modification is indistinguishable by the adversary $\mathcal{A}$ and so we have:

$$\boxed{\mathbb{P}(\mathbb{G}_6 \Longrightarrow 1) = \mathbb{P}(\mathbb{G}_5 \Longrightarrow 1).}$$

† **Game $\mathbb{G}_7$:** This game is the same as $\mathbb{G}_6$, except that this time we abort when flag $\mathtt{bad_3}$ is set to $\mathtt{true}$. By the identical-until-bad lemma we have:

$$\Big|\mathbb{P}(\mathbb{G}_6 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_7 \Longrightarrow 1)\Big| \leq \mathbb{P}(F_2)$$

where F_2 is the event: a collision occurs between $\mathsf{SO}_{\mathcal{H}}$ and ${\mathsf{SO}_\mathsf{P}}^{l_{\mathsf{hash}}}$, or $\mathsf{SO}_{\mathcal{H}}$ and SO_T, or SO_T and ${\mathsf{SO}_\mathsf{P}}^{l_{\mathsf{hash}}}$.
We provide an upper bound for $\mathbb{P}(F_2)$ using the birthday paradox and the same analysis we made in game $\mathbb{G}_2$ to obtain that:

$$\boxed{\Big|\mathbb{P}(\mathbb{G}_6 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_7 \Longrightarrow 1)\Big| \leq \frac{\left(q_{\mathsf{SO}_\mathsf{T}} + q_{{\mathsf{SO}_\mathsf{P}}^{l_{\mathsf{hash}}}}\right)^2 + \left(q_{\mathsf{SO}_\mathsf{T}} + q_{\mathsf{SO}_{\mathcal{H}}}\right)^2 + \left(q_{{\mathsf{SO}_\mathsf{P}}^{l_{\mathsf{hash}}}} + q_{\mathsf{SO}_{\mathcal{H}}}\right)^2}{2^{l_{\mathsf{hash}}+1}}.}$$

† **Game $\mathbb{G}_8$:** We denote by (e, Ξ, b') the guess of the adversary. With this game, we try to guess in advance the tested instance choosing by $\mathcal{A}$, as well as its potential partner for having matching conversation. This game is the same as the previous one except that we pick at random $e^* \leftarrow\$ [\![1, q_{\mathsf{NS}}]\!]$ and $(p_1, p_2) \leftarrow\$ [\![1, q_{\mathsf{R}}]\!]^2$. Denoting P_j the jth party registered by $\mathcal{A}$, if $e \neq e^*$, $\Xi \neq \mathsf{P}_{p_1}$ and $\Xi' \neq \mathsf{P}_{p_2}$, where $\Xi' = \pi_\Xi^e.\mathsf{PNID}$, then we abort the simulation and return a random bit.
Thus with this game hop, we increase the winning advantage of $\mathcal{A}$ by a factor equaling the correct guessing (e^*, p_1, p_2) of the challenger. This leads us to:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_7 \Longrightarrow 1) - \tfrac{1}{2}\right| \leq q_{\mathsf{NS}} \cdot q_{\mathsf{R}}^2 \cdot \left|\mathbb{P}(\mathbb{G}_8 \Longrightarrow 1) - \tfrac{1}{2}\right|.}$$

† **Game $\mathbb{G}_9$:** This game is the same as the previous one, except that we add an additional failure condition. In this game, we abort if, for the guessed instance π^e_Ξ there exists no integer $e' \in \mathbb{N}$ such that π^e_Ξ and $\pi^{e'}_{\Xi'}$ have matching conversation. Then observe that $\mathcal{A}$ has the same chances of winning $\mathbb{G}_8$ and $\mathbb{G}_9$. Indeed if there exists no matching conversation instance for π^e_Ξ, then $\mathcal{A}$ has a zero winning advantage in game $\mathbb{G}_8$ since in this situation he did not respect the Fresh property and the $\mathtt{oTest}$ condition. So it is clear that we have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_9 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_8 \Longrightarrow 1\big).}$$

As a result of this game, thanks to game $\mathbb{G}_2$, we now have selected an endpoint instance π^e_Ξ for which there exist a unique instance $\pi^{e'}_{\Xi'}$ having matching conversation with.

† **Game $\mathbb{G}_{10}$:** With this game we try to guess in advance the matching instance of π^e_Ξ. For that, this game is the same as game $\mathbb{G}_9$, except that we begin by picking at random $e' \leftarrow\$ [\![1, q_{\mathsf{NS}}]\!]$ and if π^e_Ξ and $\pi^{e'}_{\Xi'}$ do not have matching conversation, then we abort and return a random bit (with $\Xi' = \pi^e_\Xi.\mathsf{PNID}$). As above, we increase the winning probability of $\mathcal{A}$ with this game hop, by a factor equaling the correct guessing probability of the matching instance, which leads us to:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_9 \Longrightarrow 1\big) - \tfrac{1}{2}\right| \leq q_{\mathsf{NS}} \cdot \left|\mathbb{P}\big(\mathbb{G}_{10} \Longrightarrow 1\big) - \tfrac{1}{2}\right|.}$$

† **Game $\mathbb{G}_{11}$:** This game is the same as the previous one, except that we begin by picking two indexes $(\ell_1, \ell_2) \leftarrow\$ [\![1, q_{\mathsf{R}}]\!]^2$. Next:

- ↬ if the ℓ_1^{th} or the ℓ_2^{th} party registered by the adversary thanks to the $\mathtt{oRegister}$ oracle:
  - ⇝ is not an authority or;
  - ⇝ is an authority (then denoted by $\Lambda^{\mathsf{d}*}_{\ell_1}$ or $\Lambda^{\mathsf{d}'*}_{\ell_2}$) but which is corrupted (the adversary called the $\mathtt{oCorrupt}$ oracle on them or maliciously registered them);
- ↬ or if $\Lambda^{\mathsf{d}*}_{\ell_1}$ and $\Lambda^{\mathsf{d}'*}_{\ell_2}$ are in the same set of authorities (*i.e.,* both either $\mathsf{Initiator}$'s authorities, or $\mathsf{Responder}$'s authorities);
- ↬ or if the adversary queries the $\mathtt{oRevealTD}$ oracle with request $(\mathsf{sst}, \pi^e_\Xi, \pi^{e'}_{\Xi'}, \pi^k_w, (\Lambda_o)^n_{o=1}, \ell)$ such that:
  - ⇝ $\Lambda_\ell = \Lambda^{\mathsf{d}*}_{\ell_1}$ and $\mathtt{IdentifySession}(\mathsf{sst_d}, \pi^e_\Xi.\mathsf{sid}) = 1$ or;
  - ⇝ $\Lambda_\ell = \Lambda^{\mathsf{d}'*}_{\ell_2}$ and $\mathtt{IdentifySession}(\mathsf{sst_{d'}}, \pi^{e'}_{\Xi'}.\mathsf{sid}) = 1$;

then we abort and return a random bit.
The natural question now is whether such indexes exist. In fact this is not necessarily the case. But in the end, if $\mathcal{A}$ wants to win the game with a non-zero probability, he must respect the Fresh property, which requires the existence of two such indexes. So if such a situation occurs, we stop the game and return a random bit. With this game hop, the winning advantage of the adversary is increased by a factor equaling the probability of selecting correctly these two authorities. We thus obtain:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{10} \Longrightarrow 1\big) - \tfrac{1}{2}\right| \leq q_{\mathsf{R}}^2 \cdot \left|\mathbb{P}\big(\mathbb{G}_{11} \Longrightarrow 1\big) - \tfrac{1}{2}\right|.}$$

† **Game $\mathbb{G}_{12}$:** We denote by $\mathtt{Ext}_{\mathsf{SoK}}$ the knowledge extractor of the signature of knowledge. In order to prevent the adversary from providing false signatures of knowledge, this game is the same as the previous one, except that we begin by initializing four empty lists $\mathsf{L}_0[\,], \mathsf{L}_1[\,], \mathsf{L}_2[\,], \mathsf{L}_3[\,]$. We describe what happens with L_0, which is easily adaptable for the three others with the appropriate notations.

So every time the oSend oracle needs to generate a signature of knowledge ni of an element $d \leftarrow\$ \mathbb{Z}_p^*$ for group elements g_1, g_2, D_1, and D_2, such that $D_1 \leftarrow g_1^d$, and $D_2 \leftarrow g_2^d$, for the message m, it sets $\mathsf{L}_0[(g_1, D_1, g_2, D_2, m, \mathsf{ni})] \leftarrow d$.
Following this, whenever they need to perform a verification on a valid signature of knowledge: $\mathsf{SoKver}(m, (g_1, D_1, g_2, D_2), \mathsf{ni}) = 1$ in a query made by $\mathcal{A}$ with $\mathsf{L}_0[(g_1, D_1, g_2, D_2, m, \mathsf{ni})] = \perp$, the oSend and the oRevealTD oracles run the key extractor $\mathtt{Ext}_{\mathsf{SoK}}$ on $\mathcal{A}$ in order to extract the witness d that was used to build the signature ni. Now we check if $D_1 \neq g_1^d$ or $D_2 \neq g_2^d$, in other words we check the validity of the witness. If one of these two inequality occurs, then the experiment aborts and returns a random bit, else it sets $\mathsf{L}_0[(g_1, D_1, g_2, D_2, m, \mathsf{ni})] \leftarrow d$.
With this game hop, we point out the possibility of the extractor $\mathtt{Ext}_{\mathsf{SoK}}$ failing when it is called (this happens with probability $\epsilon_{\mathsf{SoK}}(\lambda)$). Because oRevealTD requires (possibly) two calls and oSend one call to the extractor each time $\mathcal{A}$ queries them, we thus have the bound:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{11} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{12} \Longrightarrow 1\big)\Big| \leq (2q_{\mathsf{RTD}} + q_{\mathsf{S}}) \cdot \epsilon_{\mathsf{SoK}}(\lambda).}$$

† **Game $\mathbb{G}_{13}$:** Here we denote by $\mathtt{Ext}_{\mathsf{NIPoK}}$ the knowledge extractor of the zero-knowledge proof of knowledge. This game is the same as the previous one, except that we begin by initializing an empty list $\mathsf{L}'[\,] \leftarrow \emptyset$, and we now describe the modification in the oRegister oracle. When oRegister received a query $(\Lambda, \mathsf{authority}, \{\perp\})$, it generates the associated keys $(\Lambda.\overline{\mathsf{SK}}, \Lambda.\overline{\mathsf{PK}})$ (corresponding to the $\mathsf{type} = \mathsf{authority}$, and remind that for an authority Λ, sets $\Lambda.\overline{\mathsf{SK}}$ and $\Lambda.\overline{\mathsf{PK}}$ contain only one element, respectively $\Lambda.\mathsf{SK}$ and $\Lambda.\mathsf{PK} = (\Lambda.\mathsf{pk}, \Lambda.\mathsf{ni})$, so to simplify notations, in what follows we write $\Lambda.\mathsf{SK}$ and $\Lambda.\mathsf{PK}$ instead of $\Lambda.\overline{\mathsf{SK}}, \Lambda.\overline{\mathsf{PK}}$), and then sets $\mathsf{L}'[\Lambda.\mathsf{PK}] \leftarrow \Lambda.\mathsf{SK}$. In the case the adversary queries oRegister on $(\Lambda, \mathsf{authority}, \mathsf{PK} \neq \perp)$, the oracle sets $\Lambda.\mathsf{PK} \leftarrow \mathsf{PK}$ and parses $\Lambda.\mathsf{PK}$ as $(\Lambda.\mathsf{pk}, \Lambda.\mathsf{ni})$.
In order to complete the process, it checks if $\mathsf{NIPoKver}((g, \Lambda.\mathsf{pk}), \Lambda.\mathsf{ni}) = 1$, and then it runs the extractor $\mathtt{Ext}_{\mathsf{NIPoK}}$ on $\mathcal{A}$ to get the witness $\Lambda.\mathsf{SK}$ for $\Lambda.\mathsf{ni}$. (If not, the game aborts, but we may reasonably assume that the adversary well constructed the public key). If $g^{\Lambda.\mathsf{SK}} \neq \Lambda.\mathsf{pk}$, then the experiment aborts and returns a random bit, else it sets $\mathsf{L}'[\Lambda.\mathsf{PK}] \leftarrow \Lambda.\mathsf{SK}$.

With this modification, we highlight the probability of the extractor $\mathtt{Ext}_{\mathsf{NIPoK}}$ failing when it is called. Since oRegister requires possibly one call to $\mathtt{Ext}_{\mathsf{NIPoK}}$ at each query (this happens with probability $\epsilon_{\mathsf{NIPoK}}(\lambda)$), this implies that we have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{12} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big)\Big| \leq q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda).}$$

From the next games we need to split the analysis into four branches as depicted in FIGURES 24 and 25. In what follows, 0 (resp. 1, 2 and 3) is for two instances running the **Sign-sign** (resp. **Sign-static**, **Static-sign** and **Static-static**) version of the protocol in a session. We will now work with our guessed and tested instance, and its matching peer, and in general, with games $\mathbb{G}_{14}$ and $\mathbb{G}_{15}$ we randomize the lawful interception elements for the tested instance, in order to test their security and prevent the adversary from forging them, and we do the same in games $\mathbb{G}_{16}$ and $\mathbb{G}_{17}$ for the partner instance of the tested instance.

† **Game $\mathbb{G}_{14}^0$:** In this game, we want to randomize the element H for one of the two instances (running the **Sign-sign** version of the protocol) for the lawful interception process. To do so, this game is the same as $\mathbb{G}_{13}$ except that during the session between π_{Ξ}^{e} and $\pi_{\Xi'}^{e'}$, assuming π_{Ξ}^{e} is an Initiator instance, we pick at random, according to the uniform distribution on our group $\mathbb{G}$, the element H_{Ξ} (we will then do the same with $\mathsf{H}_{\Xi'}$). Since it is random, we need to simulate the proof $\mathsf{ni}_{\pi_{\Xi}^{e}}$ using the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the signature of knowledge. We now claim that, by reduction, we can build a distinguisher $\mathcal{B}_1^0$ against the DDH problem for a DDH instance $(\Gamma_*, \Delta_*, \Sigma_*)$.
We thus have:

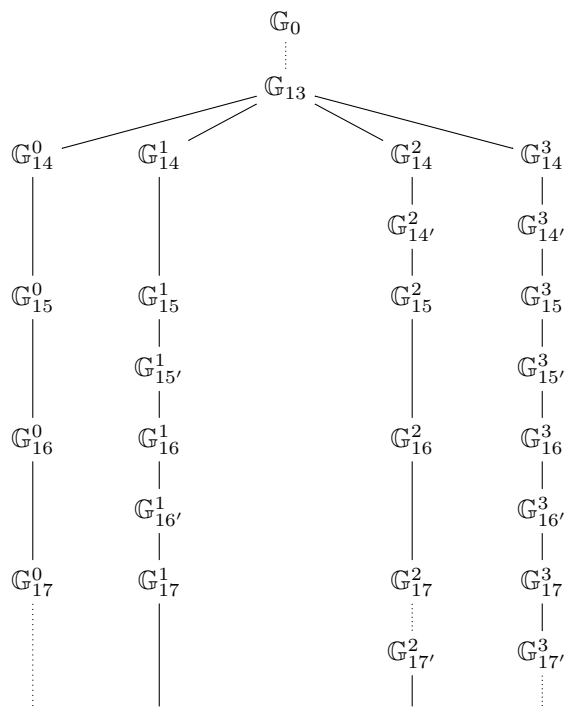

FIGURE 24 – Continuation of the game schedule with separation into four branches. All these games are used to randomize the lawful interception elements.

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{14} \Longrightarrow 1\big)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

‡ **<u>Proof of the claim:</u>** We proceed as in [9], easily adapting the notation and the context. Firstly we denote by $\mathtt{Sim}$. the simulators for the proofs of knowledge, and we now describe the construction of our distinguisher $\mathcal{B}^0_1$.
He begins by setting $\mathsf{pp} \leftarrow (\lambda, \mathbb{G}, p, g)$ and picking at random $b \leftarrow\$ \{0,1\}$. That's for the initialization. Now we describe how $\mathcal{B}^0_1$ runs $\mathcal{A}(\mathsf{pp})$.
It starts by simulating $\mathbb{G}_{13}$ to $\mathcal{A}$ as in the real game with certain exceptions regarding some oracles. We start with the $\mathtt{oRegister}$ oracle.

↬ $\underline{\mathtt{oRegister}(\mathsf{E}, \mathsf{type}, \overline{\mathsf{PK}})}$: if the ℓ_1^{th} (here to make it clear we state that $\Lambda^*_{\ell_1}$ is an $\mathsf{Initiator}$'s authority and $\Lambda^*_{\ell_2}$ is an $\mathsf{Responder}$'s authority) endpoint E registered has $\mathsf{type} \neq \mathsf{authority}$ or if the query is made with $\overline{\mathsf{PK}} \neq \{\perp\}$, then $\mathcal{B}^0_1$ aborts and returns a random bit. Otherwise, it sets $\Lambda^*_{\ell_1} \leftarrow u$, $\Lambda^*_{\ell_1}.\mathsf{pk} \leftarrow \Gamma_*$, $\Lambda^*_{\ell_1}.\mathsf{ni} \leftarrow \mathtt{Sim}_{\mathsf{NIPoK}}(g, \Gamma_*)$; $\Lambda^*_{\ell_1}.\overline{\mathsf{PK}} \leftarrow \{(\Lambda^*_{\ell_1}.\mathsf{pk}, \Lambda^*_{\ell_1}.\mathsf{ni})\}$ and returns $\Lambda^*_{\ell_1}.\overline{\mathsf{PK}}$. Since this set, composing the public keys of an authority, contains only one element, in what follows, we simply denote by $\Lambda.\mathsf{PK}$ this public key, which is of the form $(\Lambda.\mathsf{pk}, \Lambda.\mathsf{ni})$.

We now take care of the $\mathtt{oSend}$ oracle.

↬ $\underline{\mathtt{oSend}(\pi^k_w, m)}$: We have to details different cases.

⇉ Firstly, we consider the case $w = \Xi'$ and $k = e'$. Algorithm $\mathcal{B}^0_1$ sets $y_* \leftarrow y$, and $Y_* \leftarrow g^y$ (in this case (y, g^y) is generated faithfully as in the real protocol).

⇉ Secondly, we take a look at the case $w = \Xi$ and $k = e$ and we detail the modifications. Since w is an $\mathsf{Initiator}$, then it that case, $\mathcal{B}_1^0$ does not generate the exponent x and sets $X \leftarrow \Delta_*$ and $X_* \leftarrow X$. Next it parses $\pi_\Xi^e.\mathsf{AID}$ as $\left(\Lambda_o^{\mathsf{I}}\right)_{o=1}^{n_{\mathsf{I}}}$, sets $\mathsf{SetAu} \leftarrow \left\{\Lambda_o^{\mathsf{I}}\right\}_{o=1}^{n_{\mathsf{I}}} \setminus \left\{\Lambda_{\ell_1}^*\right\}$ and sets:

$$\mathsf{H}_{\mathsf{I}} \leftarrow \prod_{\Lambda \in \mathsf{SetAu}} \left(X_*^{\mathsf{L}'[\Lambda.\mathsf{PK}]}\right)\Sigma_* X_*^{y_*}.$$

He now needs to run $\mathsf{ni}_{\mathsf{I}} \leftarrow \mathtt{Sim}_{\mathsf{SoK}}(m, (g, X_*, \mathsf{h}_{\mathsf{I}}, \mathsf{H}_{\mathsf{I}},))$, where $m = \pi_\Xi^e.\mathsf{AID}\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}\|\mathsf{TH}_2\|\mathsf{TH}_3 \|\mathsf{TH}_4$ in order to simulate the signature of knowledge. Finally it sets $\mathsf{h}_* \leftarrow \mathsf{h}_{\mathsf{I}}$ and $\mathsf{H}_* \leftarrow \mathsf{H}_{\mathsf{I}}$ (with h_{I} faithfully generated).

We now work with the `oRevealTD` oracle. We modify it only if the method contained in sst (more precisely in MCS) is $\mathsf{ID} = 0$.

↬ $\underline{\mathtt{oRevealTD}(\mathsf{sst}_{\mathsf{I}}, \mathsf{ID}_{\mathsf{I}}, \mathsf{ID}_{\mathsf{R}}, \mathsf{Prox}_{\mathsf{I}}, (\Lambda_o^{\mathsf{I}})_{o=1}^{n_{\mathsf{I}}}, \ell)}$: when receiving such a query, $\mathcal{B}_1^0$ parses $\mathsf{sst}_{\mathsf{I}}$ as $\omega'\|X\|Y \|\mathsf{TH}_2\|\mathsf{TH}_3\|\mathsf{TH}_4\|M_1\|m_2\|m_3\|\mathsf{H}_{\mathsf{I}}\|\mathsf{ni}_{\mathsf{I}}\|\mathsf{ID}_{\mathsf{R}}\|n_r\|\mathsf{val}\|\sigma_I$ (with $X = g^x$, $Y = g^y$), and sets $m \leftarrow (\Lambda_o^{\mathsf{I}})_{o=1}^{n_{\mathsf{I}}} \|\mathsf{ID}_{\mathsf{I}}\|\mathsf{ID}_{\mathsf{R}}$, and sets $\mathsf{h}_{\mathsf{I}} \leftarrow \prod_{o=1}^{n_{\mathsf{I}}} \Lambda_o^{\mathsf{I}}.\mathsf{pk}$.
We know have to detail many cases.

⇉ If $\mathtt{IdentifySession}(\mathsf{sst}_{\mathsf{I}}, \pi_\Xi^e.\mathsf{sid}) = 1$ and $\Lambda_\ell^{\mathsf{I}} = \Lambda_{\ell_1}^*$, then $\mathcal{B}_1^0$ aborts and returns a random bit. This is done to respect the key-freshness requirement.

⇉ If $\mathtt{IdentifySession}(\mathsf{sst}_{\mathsf{I}}, \pi_\Xi^e.\mathsf{sid}) = 1$ and $\Lambda_\ell^{\mathsf{I}} \neq \Lambda_{\ell_1}^*$, then in this situation $\mathcal{B}_1^0$ knows the secret key of $\Lambda_\ell^{\mathsf{I}}$. So in this case, it forges a trapdoor by computing $\Lambda_\ell^{\mathsf{I}}.\mathsf{td}_1 \leftarrow X^{\mathsf{L}'[\Lambda_\ell^{\mathsf{I}}.\mathsf{PK}]}$, $\Lambda_\ell^{\mathsf{I}}.\mathsf{td}_0 \leftarrow \mathtt{Sim}_{\mathsf{td}}(g, \Lambda_\ell^{\mathsf{I}}.\mathsf{pk}, X, \Lambda_\ell^{\mathsf{I}}.\mathsf{td}_1)$ and $\Lambda_\ell^{\mathsf{I}}.\mathsf{td} \leftarrow (\Lambda_\ell^{\mathsf{I}}.\mathsf{td}_0, \Lambda_\ell^{\mathsf{I}}.\mathsf{td}_1)$.

⇉ If $\mathtt{IdentifySession}(\mathsf{sst}_{\mathsf{I}}, \pi_\Xi^e.\mathsf{sid}) \neq 1$ and $\Lambda_\ell^{\mathsf{I}} = \Lambda_{\ell_1}^*$, we automatically get $\mathtt{IdentifySession}(\mathsf{sst}_{\mathsf{I}}, \pi_{\Xi'}^{e'}.\mathsf{sid}) \neq 1$, which implies that $X\|Y\|\mathsf{ID}_{\mathsf{I}}\|\mathsf{ID}_{\mathsf{R}} \neq X_*\|Y_*\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}$. In this particular situation, $\mathcal{B}_1^0$ acts as in $\mathbb{G}_{13}$ except that:

⇝ If $(\Lambda_o^{\mathsf{I}})_{o=1}^{n_{\mathsf{I}}}\|\mathsf{ID}_{\mathsf{I}}\|\mathsf{ID}_{\mathsf{R}} \neq \pi_\Xi^e.\mathsf{AID}\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}$, then $\mathcal{B}_1^0$ computes $\Lambda_{\ell_1}^*.\mathsf{td}_1 \leftarrow (\Lambda_{\ell_1}^*.\mathsf{pk})^{\mathsf{L}[(g,X,\mathsf{h}_{\mathsf{I}}Y,\mathsf{H}_{\mathsf{I}},m,\mathsf{ni}_{\mathsf{I}})]}$, runs the simulator $\Lambda_{\ell_1}^*.\mathsf{td}_0 \leftarrow \mathtt{Sim}_{\mathsf{td}}(g, \Lambda_{\ell_1}^*.\mathsf{pk}, X, \Lambda_{\ell_1}^*.\mathsf{td}_1)$ and finally sets $\Lambda_{\ell_1}^*.\mathsf{td} \leftarrow (\Lambda_{\ell_1}^*.\mathsf{td}_0, \Lambda_{\ell_1}^*.\mathsf{td}_1)$.
In this case $\mathsf{L}[(g, X, \mathsf{h}_{\mathsf{I}}Y, \mathsf{H}_{\mathsf{I}}, m, \mathsf{ni}_{\mathsf{I}})]$ is always defined because $m \neq \pi_\Xi^e.\mathsf{AID}\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}$.

⇝ If $(\Lambda_o^{\mathsf{I}})_{o=1}^{n_{\mathsf{I}}}\|\mathsf{ID}_{\mathsf{I}}\|\mathsf{ID}_{\mathsf{R}} = \pi_\Xi^e.\mathsf{AID}\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}$, but $X \neq X^*$, then we must have $Y = Y^*$ and $\mathcal{B}_1^0$ computes $\Lambda_{\ell_1}^*.\mathsf{td}_1 \leftarrow (\Lambda_{\ell_1}^*.\mathsf{pk})^{\mathsf{L}[(g,X,\mathsf{h}_{\mathsf{I}}Y,\mathsf{H}_{\mathsf{I}},m,\mathsf{ni}_{\mathsf{I}})]}$, runs the simulator $\Lambda_{\ell_1}^*.\mathsf{td}_0 \leftarrow \mathtt{Sim}_{\mathsf{td}}(g, \Lambda_{\ell_1}^*.\mathsf{pk}, X, \Lambda_{\ell_1}^*.\mathsf{td}_1)$ and finally sets $\Lambda_{\ell_1}^*.\mathsf{td} \leftarrow (\Lambda_{\ell_1}^*.\mathsf{td}_0, \Lambda_{\ell_1}^*.\mathsf{td}_1)$.
This is possible since $\mathsf{L}[(g, X, \mathsf{h}_{\mathsf{I}}Y, \mathsf{H}_{\mathsf{I}}, m, \mathsf{ni}_{\mathsf{I}})]$ is always defined because $X \neq X^*$.

⇝ If $(\Lambda_o^{\mathsf{I}})_{o=1}^{n_{\mathsf{I}}}\|\mathsf{ID}_{\mathsf{I}}\|\mathsf{ID}_{\mathsf{R}} = \pi_\Xi^e.\mathsf{AID}\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}$, but $X = X^*$, then automatically $Y \neq Y^*$ which implies that:

$$\mathsf{h}_{\mathsf{I}}Y = \Big(\prod_{o=1}^{n_{\mathsf{I}}} \Lambda_o^{\mathsf{I}}.\mathsf{pk}\Big)Y \neq \mathsf{h}_{\mathsf{I}}^*Y^* = \Big(\prod_{o=1}^{n_{\mathsf{I}}} \Lambda_o^{\mathsf{I}}.\mathsf{pk}\Big)Y^*$$

and thus $\mathsf{h}_{\mathsf{I}}Y \neq \mathsf{h}_{\mathsf{I}}^*Y^*$. Then $\mathcal{B}_1^0$ computes $\Lambda_{\ell_1}^*.\mathsf{td}_1 \leftarrow (\Lambda_{\ell_1}^*.\mathsf{pk})^{\mathsf{L}[(g,X,\mathsf{h}_{\mathsf{I}}Y,\mathsf{H}_{\mathsf{I}},m,\mathsf{ni}_{\mathsf{I}})]}$ and runs the simulator $\Lambda_{\ell_1}^*.\mathsf{td}_0 \leftarrow \mathtt{Sim}_{\mathsf{td}}(g, \Lambda_{\ell_1}^*.\mathsf{pk}, X, \Lambda_{\ell_1}^*.\mathsf{td}_1)$ and finally sets $\Lambda_{\ell_1}^*.\mathsf{td} \leftarrow (\Lambda_{\ell_1}^*.\mathsf{td}_0, \Lambda_{\ell_1}^*.\mathsf{td}_1)$.
Here $\mathsf{L}[(g, X, \mathsf{h}_{\mathsf{I}}Y, \mathsf{H}_{\mathsf{I}}, m, \mathsf{ni}_{\mathsf{d}})]$ is always defined because $\mathsf{h}_{\mathsf{I}}Y \neq \mathsf{h}_{\mathsf{I}}^*Y^*$.

After having made all his queries to $\mathcal{B}_1^0$, the adversary $\mathcal{A}$ will return a bit b_* as in the original experience. The distinguisher $\mathcal{B}_1^0$ now compares this bit to the one he samples earlier, and if $b_* = b$, he returns 1,

else he returns 0.
We now explain how the reduction works and how $\mathcal{B}_1^0$ distinguishes. Let $\gamma_*, \delta_* \in \mathbb{Z}_p^*$ such that $\Gamma_* = g^{\gamma_*}$ and $\Delta_* = g^{\delta_*}$. We set $\Lambda_{\ell_1}^*.\mathsf{SK} \leftarrow \gamma_*$.
In our situation Ξ plays the role of the $\mathsf{Initiator}$ in π_Ξ^e, so $\mathcal{B}_1^0$ sets $x_* \leftarrow \delta_*$. With these notations, we have $X_* = g^{x_*}$ and $\Lambda_{\ell_1}^*.\mathsf{pk} \leftarrow g^{\Lambda_{\ell_1}^*.\mathsf{SK}}$ and e.g. if $\Sigma_* = g^{\gamma_*\delta_*}$, then $\Sigma_* = (\Lambda_{\ell_1}^*.\mathsf{pk})^{x_*}$ and:

$$\mathsf{H}_{\mathsf{I}*} = \prod_{\Lambda \in \mathsf{SetAu}} \left(X_*^{\mathsf{L}'[\Lambda.\mathsf{PK}]}\right)\Sigma_* X_*^{y_*} = \prod_{\Lambda \in \mathsf{SetAu}} \left(g^{\mathsf{L}'[\Lambda.\mathsf{PK}]x_*}\right)(\Lambda_{\ell_1}^*.\mathsf{pk})^{x_*} g^{x_* y_*}$$
$$= \left(\prod_{o=1}^{n_\mathsf{I}} (\Lambda_o^\mathsf{I}.\mathsf{pk}) Y_*\right)^{x_*} = (\mathsf{h}_* Y_*)^{x_*}.$$

By construction, and with this analysis, it is clear that $\mathcal{B}_1^0$ perfectly simulates $\mathbb{G}_{13}$ for $\mathcal{A}$. On the other hand, if Σ_* is a random value, then $\mathcal{B}_1^0$ perfectly simulates $\mathbb{G}_{14}^0$ for $\mathcal{A}$. So we deduce that:

$$\looparrowright \ \mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big) = \mathbb{P}\big((\gamma_*, \delta_*) \leftarrow\$ (\mathbb{Z}_p^*)^2, b \leftarrow \mathcal{B}_1^0(g^{\gamma_*}, g^{\delta_*}, g^{\gamma_*\delta_*}) : b = 1\big);$$

$$\looparrowright \ \mathbb{P}\big(\mathbb{G}_{14}^0 \Longrightarrow 1\big) = \mathbb{P}\big((\gamma_*, \delta_*, \sigma_*) \leftarrow\$ (\mathbb{Z}_p^*)^3, b \leftarrow \mathcal{B}_1^0(g^{\gamma_*}, g^{\delta_*}, g^{\sigma_*}) : b = 1\big).$$

So $\mathcal{B}_1^0$ is a well constructed distinguisher and we have:

$$\Big|\mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{14}^0 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda)$$

which concludes the proof of the claim.

† **Game $\mathbb{G}_{15}^0$:** This game is the same as the previous one except that here we assume that π_Ξ^e is a $\mathsf{Responder}$ instance. We easily adapt the previous reduction against the DDH problem with a p.p.t. distinguisher $\mathcal{B}_2^0$ and thus obtain the same bound:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{14}^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{15}^0 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_{16}^0$:** This game works in the same vein as $\mathbb{G}_{14}^0$, but this time we consider $\pi_{\Xi'}^{e'}$ as a $\mathsf{Responder}$ instance and we pick at random the element $\mathsf{H}_{\Xi'}$ and simulate the associated proof. This leads us to the same result:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{15}^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{16}^0 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_{17}^0$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}_{15}^0$, *i.e.,* this time we consider $\pi_{\Xi'}^{e'}$ as an $\mathsf{Initiator}$ instance and we reduce against the DDH problem. This leads us to:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{16}^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{17}^0 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_{14}^1$:** In this game, we want to randomize elements H and H^1 for one of the two instances (running the **Sign-static** version of the protocol) for the lawful interception process. To do so, this game is the same as $\mathbb{G}_{13}$ except that during the session between π_Ξ^e and $\pi_{\Xi'}^{e'}$, assuming π_Ξ^e is an $\mathsf{Initiator}$ instance, we pick at random, according to the uniform distribution on our group $\mathbb{G}$, the elements H_Ξ and H_Ξ^1 (we will then do the same with $\mathsf{H}_{\Xi'}$ and $\mathsf{H}_{\Xi'}^1$). Since it is random, we need to simulate the proof $\mathsf{ni}_{\pi_\Xi^e}$ using the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the signature of knowledge. We now claim that, by reduction, we can build a distinguisher $\mathcal{B}_1^1$ against the DDH problem (this is similar to the proof made in game $\mathbb{G}_{14}^0$).
We thus have:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{14}^1 \Longrightarrow 1\big)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

‡ **Proof of the claim:** We built a distinguisher $\mathcal{B}_1^1$ in the same way as the previous reduction. But here we reduce by modifying two elements: H_Ξ and H_Ξ^1, that why we detail here the modifications. The initialization works exactly in the same way as for $\mathcal{B}_1^0$. We also make the same modification for the `oRegister` and `oRevealTD` oracles, except for some notations. So we now explain the modification related to the `oSend` oracle.

↬ $\mathtt{oSend}(\pi_w^k, m)$: We distinguish two cases.

⇉ We begin by considering the case $w = \Xi'$ and $k = e'$. Algorithm $\mathcal{B}_1^1$ sets $y_* \leftarrow y$, $Y_* \leftarrow g^y$, $r_* \leftarrow r$ and $Y_{1*} \leftarrow g^r$ (where (y, g^y) and (r, g^r) are generated faithfully according to the real protocol).

⇉ Secondly, if $w = \Xi$ and $k = e$, we detail a little bit more the modifications. Since in this case w is an $\mathsf{Initiator}$, $\mathcal{B}_1^1$ does not generate x and sets $X = g^x \leftarrow \Delta_*$ and $X_* \leftarrow X$. Then it parses $\pi_\Xi^e.\mathsf{AID}$ as $\big(\Lambda_o^{\mathsf{I}}\big)_{o=1}^{n_{\mathsf{I}}}$, it sets $\mathsf{SetAu} \leftarrow \big\{\Lambda_o^{\mathsf{I}}\big\}_{o=1}^{n_{\mathsf{I}}} \setminus \big\{\Lambda_{\ell_1}^*\big\}$ and sets:

$$\mathsf{H}_{\mathsf{I}} \leftarrow \prod_{\Lambda \in \mathsf{SetAu}} \Big(X_*^{\mathsf{L}'[\Lambda.\mathsf{PK}]}\Big)\Sigma_* X_*^{y_*}, \mathsf{H}_{\mathsf{I}}^1 \leftarrow \prod_{\Lambda \in \mathsf{SetAu}} \Big(X_*^{\mathsf{L}'[\Lambda.\mathsf{PK}]}\Big)\Sigma_* X_*^{r_*},$$

and (y, g^y), (r, g^r) are normally generated.
It now runs $\mathsf{ni}_{\mathsf{I}} \leftarrow \mathtt{Sim}_{\mathsf{Sok}}(m, (g, X_*, \mathsf{h}_{\mathsf{I}}, \mathsf{H}_{\mathsf{I}}, \mathsf{H}_{\mathsf{I}}^1))$, where $m = \pi_\Xi^e.\mathsf{AID}\|\mathsf{ID}_\Xi\|\mathsf{ID}_{\Xi'}$. Finally it sets $\mathsf{h}_* \leftarrow \mathsf{h}_{\mathsf{I}}$, $\mathsf{H}_* \leftarrow \mathsf{H}_{\mathsf{I}}$ and $\mathsf{H}_*^1 \leftarrow \mathsf{H}_{\mathsf{I}}^1$ (with h_{I} faithfully generated).

At the end of the simulation, $\mathcal{A}$ will return a bit b_* as in the original experience. Algorithm $\mathcal{B}_1^1$ compares this bit to the one he samples at the beginning, and then if $b_* = b$, then he returns 1, else it returns 0. Let's now explain how the reduction works and how $\mathcal{B}_1^1$ distinguishes. Let $(\gamma_*, \delta_*) \in (\mathbb{Z}_p^*)^2$ such that $\Gamma_* = g^{\gamma_*}$ and $\Delta_* = g^{\delta_*}$. We set $\Lambda_{\ell_1}^*.\mathsf{SK} \leftarrow \gamma_*$.
In our situation Ξ plays the role of the $\mathsf{Initiator}$ in π_Ξ^e. In this case $\mathcal{B}_1^1$ sets $x_* \leftarrow \delta_*$. With these notations, we have $X_* = g^{x_*}$ and $\Lambda_{\ell_1}^*.\mathsf{pk} \leftarrow g^{\Lambda_{l_1}^*.\mathsf{SK}}$. If $\Sigma_* = g^{\gamma_*\delta_*}$, then $\Sigma_* = (\Lambda_{\ell_1}^*.\mathsf{pk})^{x_*}$ and:

$$\begin{aligned}\mathsf{H}_{\mathsf{I}*} = \prod_{\Lambda \in \mathsf{SetAu}} \Big(X_*^{\mathsf{L}'[\Lambda.\mathsf{PK}]}\Big)\Sigma_* X_*^{y_*} &= \prod_{\Lambda \in \mathsf{SetAu}} \Big(g^{\mathsf{L}'[\Lambda.\mathsf{PK}]x_*}\Big)(\Lambda_{\ell_1}^*.\mathsf{pk})^{x_*} g^{x_* y_*} \\ &= \Big(\prod_{o=1}^{n_{\mathsf{I}}} (\Lambda_o^{\mathsf{I}}.\mathsf{pk}) Y\Big)^{x_*} = (\mathsf{h}_* Y)^{x_*}\end{aligned}$$

and

$$\mathsf{H}^1_{\mathsf{I}_*} = \prod_{\Lambda \in \mathsf{SetAu}} \left(X_*^{\mathsf{L}'[\Lambda.\mathsf{PK}]}\right)\Sigma_* X_*^{r_*} = \prod_{\Lambda \in \mathsf{SetAu}} \left(g^{\mathsf{L}'[\Lambda.\mathsf{PK}]x_*}\right)(\Lambda^*_{\ell_1}.\mathsf{pk})^{x_*} g^{x_* r_*}$$
$$= \left(\prod_{o=1}^{n_{\mathsf{I}}} (\Lambda^{\mathsf{I}}_o.\mathsf{pk}) Y_1\right)^{x_*} = (\mathsf{h}_* Y_1)^{x_*}$$

By construction, it is clear that in this case, $\mathbb{G}_{13}$ is perfectly simulated for $\mathcal{A}$ by $\mathcal{B}^1_1$. On the other hand, if Σ_* is a random value, then $\mathbb{G}^1_{14}$ is perfectly simulated for $\mathcal{A}$ by $\mathcal{B}^1_1$. We then deduce that:

$\looparrowright$ $\mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big) = \mathbb{P}\big((\gamma_*, \delta_*) \leftarrow\$ (\mathbb{Z}_p^*)^2, b \leftarrow \mathcal{B}^1_1(g^{\gamma_*}, g^{\delta_*}, g^{\gamma_* \delta_*}) : b = 1\big)$;

$\looparrowright$ $\mathbb{P}\big(\mathbb{G}^1_{14} \Longrightarrow 1\big) = \mathbb{P}\big((\gamma_*, \delta_*, \sigma_*) \leftarrow\$ (\mathbb{Z}_p^*)^3, b \leftarrow \mathcal{B}^1_1(g^{\gamma_*}, g^{\delta_*}, g^{\sigma_*}) : b = 1\big)$.

So here again, $\mathcal{B}^1_1$ is a well constructed distinguisher and we have:

$$\Big|\mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{14} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda)$$

which concludes the proof of the claim.

† **Game $\mathbb{G}^1_{15}$:** This game is the same as the previous one except that here we assume that π^e_Ξ is a Responder instance. We here first pick at random the element H_Ξ and reduce against the DDH problem as in game $\mathbb{G}^0_{14}$. We thus have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^1_{14} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{15} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^1_{15'}$:** This game is the same as the previous one, still assuming that π^e_Ξ is a Responder instance. We here pick at random the second element H^1_Ξ. As in the previous game, we have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^1_{15} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{15'} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^1_{16}$:** This game works in the same vein as $\mathbb{G}^1_{15}$, but this time we consider $\pi^{e'}_{\Xi'}$ as a Responder instance and we pick at random the element $\mathsf{H}_{\Xi'}$ and simulate this associated proof. This leads us to the same result as in game $\mathbb{G}^1_{15}$:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^1_{15'} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{16} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^1_{16'}$:** This game works in the same vein as $\mathbb{G}^1_{15'}$, still considering $\pi^{e'}_{\Xi'}$ as a Responder instance. We pick at random the element $\mathsf{H}^1_{\Xi'}$ and simulate the associated proof. This leads us to the same result:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^1_{16} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{16'} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^1_{17}$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}^0_{14}$, *i.e.,* this time we consider $\pi^{e'}_{\Xi'}$ as an Initiator instance and we pick at random elements $\mathsf{H}_{\Xi'}$ and $\mathsf{H}^1_{\Xi'}$. Following what we did in game $\mathbb{G}^1_{14}$, with a p.p.t. distinguisher $\mathcal{B}^1_4$ we obtain:

$$\left|\mathbb{P}(\mathbb{G}^1_{16'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^1_{17} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^2_{14}$:** In this game, we want to randomize the element H for one of the two instances (running the **Static-sign** version of the protocol) for the lawful interception process. Here again, this game is the same as $\mathbb{G}_{13}$, and we assume that π^e_Ξ is an $\mathsf{Initiator}$ instance. We pick at random, according to the uniform distribution on our group $\mathbb{G}$, the element H_Ξ. Since it is random, we need to simulate the proof $\mathsf{ni}_{\pi^e_\Xi}$ using the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the signature of knowledge. By reduction, we can build a distinguisher $\mathcal{B}^2_1$ against the DDH problem (this is similar to the proof made in game $\mathbb{G}^0_{14}$). We thus have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_{13} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{14} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^2_{14'}$:** This game is the same as the previous one except that here we pick at random the element H^1_Ξ (still assuming that π^e_Ξ is an $\mathsf{Initiator}$ instance). We obtain the same bound as in game $\mathbb{G}^1_{15'}$:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^1_{14} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^1_{14'} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^2_{15}$:** This game is the same as the previous one and work in the same vein as game $\mathbb{G}^0_{15}$. Here we assume that π^e_Ξ is a $\mathsf{Responder}$ instance. We pick at random the two elements H_Ξ and H^1_Ξ. Using arguments of game $\mathbb{G}^1_{14}$, we obtain the following bound:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{14'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{15} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^2_{16}$:** This game works in the same vein as $\mathbb{G}^2_{15}$, but this time we consider $\pi^{e'}_{\Xi'}$ as a $\mathsf{Responder}$ instance and we pick at random elements $\mathsf{H}_{\Xi'}$ and H^1_Ξ, and we simulate the associated proof. This leads us to the same result:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{15} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{16} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^2_{17}$:** This game works in the same vein as $\mathbb{G}^2_{14}$. Here we consider $\pi^{e'}_{\Xi'}$ as an $\mathsf{Initiator}$ instance, we pick at random the element $\mathsf{H}_{\Xi'}$ and simulate the associated proof. Again we have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{16} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{17} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^2_{17'}$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}^2_{14'}$, *i.e.,* this time we consider $\pi^{e'}_{\Xi'}$ as an $\mathsf{Initiator}$ instance and we pick at random the element $\mathsf{H}^1_{\Xi'}$. This leads us to:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{17} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{17'} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^3_{14}$:** In this game, we want to randomize element H H^1 for one of the two instances (running the **Static-static** version of the protocol) for the lawful interception process. Here again, this game is the same as $\mathbb{G}_{13}$, and we assume that π^e_Ξ is an $\mathsf{Initiator}$ instance. We pick at random, according to the uniform distribution on our group $\mathbb{G}$, elements H_Ξ and H^1_Ξ. Since it is random, we need to simulate the proof $\mathsf{ni}_{\pi^e_\Xi}$ using the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the signature of knowledge. By reduction, we can build a distinguisher $\mathcal{B}^3_1$ against the DDH problem (this is similar to the proof made in game $\mathbb{G}^0_{14}$). We thus have:

$$\left|\mathbb{P}(\mathbb{G}_{13} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{14} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{14'}$:** This game is the same as the previous one except that here we pick at random the element H^2_{Ξ} (still assuming that π^e_{Ξ} is an Initiator instance). We obtain the same bound as in game $\mathbb{G}^2_{14}$ thanks to a reduction against the DDH problem. However, there are two slight modifications to do in the reduction. The first is in the `oRegister` oracle. When the endpoint Ξ is registered, $\mathcal{B}^3_{1'}$ embeds a part of its DDH challenge into static Diffie-Hellman key "g^i" of Ξ. The second one is due to the fact that Ξ cannot be corrupt on its long-term key (otherwise $\mathcal{A}$ cannot win the game). So if $\mathcal{A}$ queries the `oCorrupt` oracle on Ξ for this static Diffie-Hellman key, $\mathcal{B}^3_{1'}$ aborts and returns a random bit. The rest of the reduction is identical, except for the notations. We thus have:

$$\left|\mathbb{P}(\mathbb{G}^3_{14} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{14'} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{15}$:** This game is the same as the previous one and work in the same vein as game $\mathbb{G}^0_{15}$. Here we assume that π^e_{Ξ} is a Responder instance. We pick at random the two elements H_{Ξ} and H^1_{Ξ}. Using arguments of game $\mathbb{G}^0_{15}$, we obtain the following bound:

$$\left|\mathbb{P}(\mathbb{G}^3_{14'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{15} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{15'}$:** This game is the same as the previous one (we assume that π^e_{Ξ} is a Responder instance) but this time we pick at random the element H^2_{Ξ}. Again (with the explanations of the game $\mathbb{G}^3_{14'}$) we obtain the following bound:

$$\left|\mathbb{P}(\mathbb{G}^3_{15} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{15'} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{16}$:** This game works in the same vein as $\mathbb{G}^3_{15}$, but this time we consider $\pi^{e'}_{\Xi'}$ as a Responder instance and we pick at random elements $\mathsf{H}_{\Xi'}$ and H^1_{Ξ}, and we simulate the associated proof. This leads us to:

$$\left|\mathbb{P}(\mathbb{G}^3_{15'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{16} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{16'}$:** This game works in the same vein as $\mathbb{G}^3_{15'}$ (we continue considering $\pi^{e'}_{\Xi'}$ as a Responder instance) but here we pick at random the element $\mathsf{H}^2_{\Xi'}$ (and we simulate the associated proof). This leads us to:

$$\left|\mathbb{P}(\mathbb{G}^3_{16} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{16'} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{17}$:** This game works in the same vein as $\mathbb{G}^3_{16}$. Here we consider $\pi^{e'}_{\Xi'}$ as an Initiator instance, we pick at random elements $\mathsf{H}_{\Xi'}$ and $\mathsf{H}^1_{\Xi'}$, and simulate the associated proof. Again we have:

$$\left|\mathbb{P}(\mathbb{G}^3_{16'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{17} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

† **Game $\mathbb{G}^3_{17'}$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}^3_{14'}$, *i.e.,* this time we consider $\pi^{e'}_{\Xi'}$ as an Initiator instance and we pick at random the element $\mathsf{H}^2_{\Xi'}$. This leads us to:

$$\left|\mathbb{P}\big(\mathbb{G}^3_{17} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^3_{17'} \Longrightarrow 1\big)\right| \leq \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{A}}(\lambda).$$

We are now close to the conclusion and we continue on our four branches. In games $\mathbb{G}_{18}$ and $\mathbb{G}_{19}$, and in games $\mathbb{G}_{20}$ and $\mathbb{G}_{21}$, we prevent the adversary from forging signatures, either on Initiator or Responder side. Next, in games $\mathbb{G}_{22}$, $\mathbb{G}_{23}$, $\mathbb{G}_{24}$ and $\mathbb{G}_{24}$ we test the key-derivation schedule security with element g^{xy} (or g^{iy} or g^{rx}) and ensure that the adversary is not able to predict the key(s), taking into account the fact that he obtains information from proxies if the latter are malicious. Finally with game $\mathbb{G}_{26}$ we finish testing the security of the key-derivation schedule and we conclude the proof with the fresh and sound sessions, reducing the winning advantage of $\mathcal{A}$ to $\frac{1}{2}$.

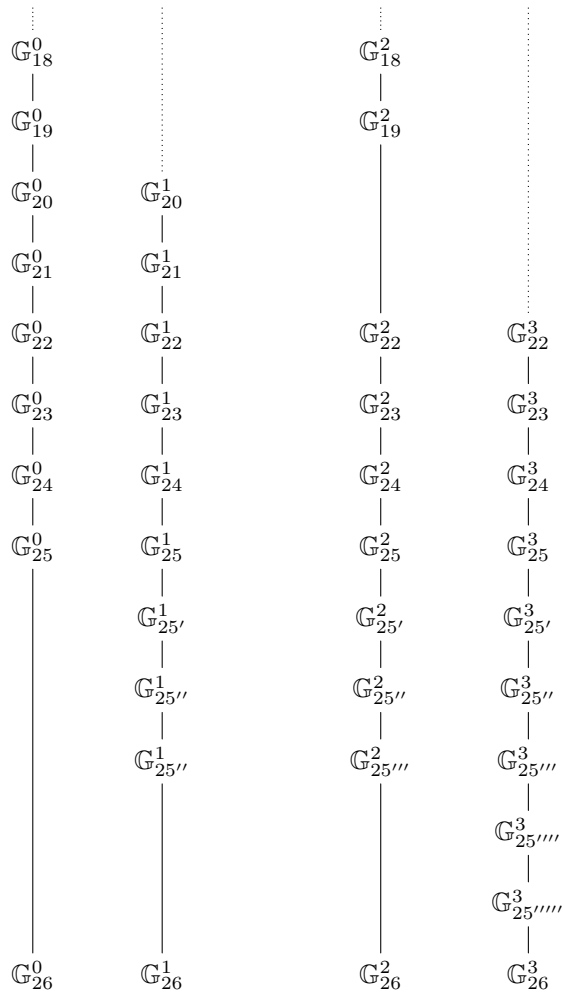

FIGURE 25 – End of the game-schedule with the four branches and the conclusion. With games $\mathbb{G}_{18}$, $\mathbb{G}_{19}$,$\mathbb{G}_{20}$ and $\mathbb{G}_{21}$ we are flagging the false signatures σ_2 or σ_3 and aborting if one is received. Then with the following games, depending on the version, we prevent the adversary from attacking the key-derivation schedule. Finally we conclude with game $\mathbb{G}_{26}$ by ensuring soundness and freshness, and reducing the adversary's advantage to $\frac{1}{2}$.

† **Game $\mathbb{G}^0_{18}$:** With next games, we want to prevent the adversary from forging signatures. This game is the same as the previous one, except that for each Responder E registered, we create an empty list and we store every message for which a signature σ_2 is generated by an honest instance π^k_{E}, $k \in \mathbb{N}$ (*i.e.,* not corrupted and not adversary-controlled). We then set a bad flag $\mathtt{bad_{sign}}$ to $\mathtt{true}$ when an (non-malicious) Initiator instance received a valid signature σ_2 from an uncorrupted (adversary-controlled) Responder's instance for a message which was not already generated by an honest Responder's instance. This change is internal to the game, and hence remains indistinguishable by $\mathcal{A}$. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}^0_{17} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}^0_{18} \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}^0_{19}$:** This game is the same as game $\mathbb{G}^0_{18}$, but we abort when the bad flag $\mathtt{bad_{sign}}$ is set to $\mathtt{true}$. By the identical-until-bad lemma we hence have:

$$\Big|\mathbb{P}\big(\mathbb{G}^0_{18} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{19} \Longrightarrow 1\big)\Big| \leq \mathbb{P}(F^0_3),$$

where F^0_3 is the event: the flag $\mathtt{bad_{sign}}$ is set to $\mathtt{true}$ in game $\mathbb{G}^0_{18}$.
To provide an upper bound for $\mathbb{P}(F^0_3)$, we use an hybrid argument, and claim that, by reduction, we can build p.p.t. adversaries against the EUF-CMA property of our digital signature scheme DS. We thus have:

$$\mathbb{P}(F^0_3) \leq q_{\mathsf{R_R}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda),$$

(where $q_{\mathsf{R_R}}$ denotes the number of Responder registered (and similarly $q_{\mathsf{R_I}}$ denotes the number of Initiator registered, and we have $q_{\mathsf{R_R}} + q_{\mathsf{R_I}} = q_{\mathsf{R}}$)) and hence:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^0_{18} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{19} \Longrightarrow 1\big)\Big| \leq q_{\mathsf{R_R}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda).}$$

‡ **Proof of the claim:** As announced, we prove this claim with an hybrid argument. For that we consider the sequence of hybrids games $\mathbb{H}_0, \cdots, \mathbb{H}_{q_{\mathsf{R_R}}}$, where $q_{\mathsf{R_R}}$ denotes the number of Responder registered (and similarly $q_{\mathsf{R_I}}$ denotes the number of Initiator registered, and we have $q_{\mathsf{R_R}} + q_{\mathsf{R_I}} = q_{\mathsf{R}}$). The game $\mathbb{H}_0$ is the game $\mathbb{G}^0_{18}$. We assume, without loss of generality, that we number each Responder in order of registration (*e.g.,* R_1 is the first Responder registered, etc. Then for all $\xi \in [\![1, q_{\mathsf{R_R}}]\!]$, game $\mathbb{H}_\xi$ is the same as game $\mathbb{H}_{\xi-1}$, except that we abort when the bad flag $\mathtt{bad_{sign}}$ is raised for an R_ξ's instance. By reduction, we build a p.p.t. algorithm $\mathcal{B}^0_{5,\xi}$ against the EUF-CMA property of DS, such that:

$$\Big|\mathbb{P}\big(\mathbb{H}_{\xi-1} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{H}_\xi \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda).$$

So to build $\mathcal{B}^0_{5,\xi}$, we first make the following assumption: there exists an adversary $\mathcal{A}$ that wins the game $\mathbb{H}_{\xi-1}$ with a non negligible probability, then denoted $\epsilon_{\mathcal{A}}(\lambda)$, by forging a signature σ_2 on one Responder R_ξ's instance.
We now present the construction of $\mathcal{B}^0_{5,\xi}$. To begin with, the challenger generates a pair of public/private signature keys $(\widetilde{\mathsf{ssk}}, \widetilde{\mathsf{spk}})$ with the $\mathsf{DS}.\mathtt{Gen}$ algorithm, and gives $\widetilde{\mathsf{spk}}$ to $\mathcal{B}^0_{5,\xi}$. Algorithm $\mathcal{B}^0_{5,\xi}$ wins if it sends to its challenger a message/signature (m, σ) he did not generate, and such that $\mathsf{DS}.\mathtt{Verif}(\widetilde{\mathsf{spk}}, m, \sigma) = 1$. So $\mathcal{B}^0_{5,\xi}$ will start by initializing an empty list $\mathsf{L_S} \leftarrow \emptyset$, and then simulate $\mathbb{H}_{\xi-1}$ to $\mathcal{A}(\mathsf{pp})$ faithfully, except for certain points. Mainly we make two following modifications:

- Firstly, when $\mathcal{A}$ registered Responder R_ξ, $\mathcal{B}^0_{5,\xi}$ sets $\mathsf{R}_\xi.\overline{\mathsf{PK}} \ni \mathsf{R}_\xi.\mathsf{spk} \leftarrow \widetilde{\mathsf{spk}}$.

- In a second time, when calling the $\mathtt{oSend}$ oracle on input (π^q_u, m), if $u = \mathsf{R}_\xi$, $\mathcal{B}^0_{5,\xi}$ simulates $\mathtt{oSend}$ faithfully, but queries its $\mathtt{Sign}$ oracle (instead of the $\mathsf{DS}.\mathtt{Sign}$ algorithm): $\sigma \leftarrow \mathtt{Sign}(m)$ and adds (m, σ) to $\mathsf{L_S}$.

Remark that $\mathcal{B}^0_{5,\xi}$ can still answer all the corruption queries, excepted for the Responder R_ξ. Thus if $\mathcal{A}$ asks $\mathcal{B}^0_{5,\xi}$ for a corruption query on the particular Responder, $\mathcal{B}^0_{5,\xi}$ stops the simulation.
Thus if $\mathcal{A}$ wins the game simulated by $\mathcal{B}^0_{5,\xi}$, having forged a signature under the conditions of $\mathbb{H}^0_{\xi-1}$ (i.e forging the signature σ_2 on one R_ξ's instance), then $\mathcal{B}^0_{5,\xi}$ wins its game by construction (because $\mathcal{A}$ will have generated a pair $(m,\sigma) \notin \mathsf{L_S}$) and breaks the EUF-CMA security of DS. So in this case we have $\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{B}^0_{5,\xi}}(\lambda) \geq \epsilon_{\mathcal{A}}(\lambda)$.
We thus built well our p.p.t. adversary $\mathcal{B}^0_{5,\xi}$. Repeating this on each hybrid game hop, we have :

$$\begin{aligned}\Big|\mathbb{P}\big(\mathbb{H}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{H}_{q_{\mathsf{R_R}}} \Longrightarrow 1\big)\Big| &\leq \Big|\mathbb{P}\big(\mathbb{H}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{H}_1 \Longrightarrow 1\big)\Big| + \cdots \\ &\quad + \Big|\mathbb{P}\big(\mathbb{H}_{q_{\mathsf{R_R}}-1} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{H}_{q_{\mathsf{R_R}}} \Longrightarrow 1\big)\Big| \\ &\leq q_{\mathsf{R_R}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda)\end{aligned}$$

Observe that after game $\mathbb{H}_{q_{\mathsf{R_R}}}$, $\mathcal{A}$ is no more able to forge a signature σ_2, as well as in game $\mathbb{G}^0_{19}$. We deduce that $\mathbb{P}\big(\mathbb{H}_{q_{\mathsf{R_R}}} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}^0_{19} \Longrightarrow 1\big)$ and since $\mathbb{H}_0 = \mathbb{G}^0_{18}$, we thus obtain that :

$$\mathbb{P}(F^0_3) \leq q_{\mathsf{R_R}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda)$$

as announced, and this concludes the proof of the claim.

With this game hop, we prevent the adversary from forging signatures σ_2. With the following two, we prevent the latter from forging signatures σ_3.

† **Game $\mathbb{G}^0_{20}$:** This game is the same as the previous one, except that for each Initiator E' registered, we create an empty list and we store every message for which a signature σ_3 is generated by an honest instance $\pi^{k'}_{\mathsf{E}'}$, $k' \in \mathbb{N}$ (*i.e.,* not corrupted and not adversary-controlled). We then set a bad flag $\mathtt{bad}_{\mathtt{sign}'}$ to `true` when an honest Responder instance received a valid signature σ_3 from an uncorrupted (adversary-controlled) Initiator's instance for a message which was not already generated by an honest Initiator's instance.
This change is internal to the game, and hence remains indistinguishable by $\mathcal{A}$. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}^0_{20} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}^0_{19} \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}^0_{21}$:** In the same vein as for game $\mathbb{G}^0_{19}$, this game is the same as game $\mathbb{G}^0_{20}$, but we abort when the bad flag $\mathtt{bad}_{\mathtt{sign}'}$ is set to `true`. By the identical-until-bad lemma we hence have:

$$\Big|\mathbb{P}\big(\mathbb{G}^0_{20} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{21} \Longrightarrow 1\big)\Big| \leq \mathbb{P}(F^0_{3'}),$$

where $F^0_{3'}$ is the event: the flag $\mathtt{bad}_{\mathtt{sign}'}$ is set to `true` in game $\mathbb{G}^0_{20}$.
Here again, to provide an upper bound for $\mathbb{P}(F^0_{3'})$, we use an hybrid argument, and we claim that, by reduction, we can build p.p.t. adversaries against the EUF-CMA property of our digital signature scheme DS. This is done in a similar way as for $\mathbb{G}^0_{19}$ and we obtain:

$$\mathbb{P}(F^0_{3'}) \leq q_{\mathsf{R_I}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda),$$

and hence:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^0_{20} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{21} \Longrightarrow 1\big)\Big| \leq q_{\mathsf{R_I}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^1_{20}$:** This game is the same as game $\mathbb{G}^1_{17}$ and works in the same vein as game $\mathbb{G}^0_{20}$. For each Initiator E' registered, we create an empty list and we store every message for which a signature σ_3 is generated by an honest instance $\pi^{k'}_{\mathsf{E}'}$, $k' \in \mathbb{N}$ (*i.e.,* not corrupted and not adversary-controlled). We then set a bad flag $\mathtt{bad}_{\mathtt{sign}'}$ to `true` when an honest Responder instance received a valid signature σ_3 from an uncorrupted (adversary-controlled) Initiator's instance for a message which was not already generated by an honest Initiator's instance.
This change is internal to the game, and hence remains indistinguishable by $\mathcal{A}$. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}^1_{20} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}^1_{17} \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}^1_{21}$:** This game is the same as the previous one and works in the same vein as game $\mathbb{G}^1_{20}$, but we abort when the bad flag $\mathtt{bad}_{\mathtt{sign}'}$ is set to `true`. By the identical-until-bad lemma we hence have:

$$\Big|\mathbb{P}\big(\mathbb{G}^0_{20} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{21} \Longrightarrow 1\big)\Big| \leq \mathbb{P}(F^1_{3'}),$$

where $F^1_{3'}$ is the event: the flag $\mathtt{bad}_{\mathtt{sign}'}$ is set to `true` in game $\mathbb{G}^1_{20}$. With an hybrid argument, we obtain:

$$\mathbb{P}(F^1_{3'}) \leq q_{\mathsf{R_I}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda),$$

and hence:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^1_{20} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{21} \Longrightarrow 1\big)\Big| \leq q_{\mathsf{R_I}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^2_{18}$:** This game is the same as game $\mathbb{G}^2_{17'}$, except that for each Responder E registered, we create an empty list and we store every message for which a signature σ_2 is generated by an honest instance π^k_{E}, $k \in \mathbb{N}$ (*i.e.,* not corrupted and not adversary-controlled). We then set a bad flag $\mathtt{bad}_{\mathtt{sign}}$ to `true` when an (non-malicious) Initiator instance received a valid signature σ_2 from an uncorrupted (adversary-controlled) Responder's instance for a message which was not already generated by an honest Responder's instance. This change is internal to the game, and hence remains indistinguishable by $\mathcal{A}$. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}^2_{17'} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}^2_{18} \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}^2_{19}$:** This game is the same as game $\mathbb{G}^2_{18}$ and works in the same vein as game $\mathbb{G}^0_{19}$: but we abort when the bad flag $\mathtt{bad}_{\mathtt{sign}}$ is set to `true`. We hence obtain:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}^2_{18} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^2_{19} \Longrightarrow 1\big)\Big| \leq q_{\mathsf{R_R}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}^0_{22}$:** With the following games, we want the prevent the adversary from obtaining the session key through the key derivation schedule. So we begin with this game, which is the same as the previous one, but we add a failure condition: if the adversary, during the tested session, queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}, then we abort and return a random bit. We then denote F^0_4 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}. If the event F^0_4 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\Big|\mathbb{P}\big(\mathbb{G}^0_{21} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^0_{22} \Longrightarrow 1\big)\Big| \leq \mathbb{P}(F^0_4).$$

Let's now provide a bound for $\mathbb{P}(F^0_4)$. For that we construct, by reduction, an adversary $\mathcal{B}^0_6$ against the GDH problem. By the identical-until-bad lemma we hence claim that:

$$\mathbb{P}(F^0_4) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\text{GDH}}_{\mathbb{G}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^0_{21} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^0_{22} \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

‡ **Proof of the claim:** This reduction is very similar to the reduction against the GDH problem proposed in [14]. We construct a p.p.t adversary $\mathcal{B}^0_6$ against the GDH problem.
On input a GDH challenge $(g, A = g^a, B = g^b)$ and having access to a DDH oracle, returning 1 if a tuple (g, g^a, g^b, g^c) is a Diffie-Hellman tuple, and 0 otherwise, our algorithm $\mathcal{B}^0_6$ must return g^{ab} in a polynomial time. To achieve this, $\mathcal{B}^0_6$ is going to simulate $\mathbb{G}^0_{22}$ for $\mathcal{A}(\mathsf{pp})$ faithfully, except for certain points.
Firstly, in all Initiator and Responder instances, $\mathcal{B}^0_6$ embeds its challenge into the ephemeral sent Diffie-Hellman value as follows: it samples uniformly at random (for each instance) $k, k' \in \mathbb{Z}_p$ and sets $X \leftarrow Ag^k$ and $Y \leftarrow Bg^{k'}$ (with $A = g^a$ and $B = g^b$).
If the event F^0_4 occurs, then the value Z queried to the $\mathsf{SO_T}$ oracle will equal $g^{(a+k)(b+k')}$. To detect that, $\mathcal{B}^0_6$ can compute $ZX^{-k'}B^{-k}$ which, if F^0_4 occurs, will equal g^{ab} and test this equality by calling its DDH oracle on $(X, Y, ZX^{-k'}B^{-k})$, which will return 1 if F^0_4 occurs. So when the event happens $\mathcal{B}^0_6$ wins.
To finally bound $\mathbb{P}(F^0_4)$, we remark that each time the $\mathsf{SO_T}$ oracle is queried on input Z, the event F^0_4 can happen (on a single pair (X, Y)). This implies that possibly $q_{\mathsf{RO_T}}$ DDH-oracle queries might be useful to detect the event. Hence the result:

$$\mathbb{P}(F^0_4) \leq q_{\mathsf{RO_T}} \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda)$$

which concludes the proof of the claim.

† **Game $\mathbb{G}^0_{23}$:** This game is the same as the previous one, except that in this game we simulate the key generation of $\mathsf{PRK_{2e}}$ for some chosen instances. Namely if our tested instance π^e_Ξ is an honest Initiator's instance receiving (g^y, c_2) from an honest Responder's instance (its partner $\pi^{e'}_{\Xi'}$), and conversely if our tested instance is an honest Responder's instance receiving m_1 from an honest Initiator's instance (its partner), we simulate the generation of the key $\mathsf{PRK_{2e}}$ thanks to a private oracle, denoted $\mathsf{SO_{PRK_{2e}}}$, and working the same way as $\mathsf{SO_T}$, but inaccessible to the adversary. Here we prove the security of x and y in the key-derivation schedule; in normal times, $\mathsf{PRK_{2e}}$ is generated thanks to $\mathsf{SO_T}$ and $\mathsf{TH_2}$ (which is always authentic since the two instances get the same m_1) and g^{xy}.
One way to detect the modification is for the adversary to ask $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}. Thanks to the event F^0_4 the simulation is already stopped in this situation. Another way to detect it is to query ${\mathsf{SO_P}}^{l_2}$, or ${\mathsf{SO_P}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO_P}}^{l_{\mathsf{iv}}}$, or ${\mathsf{SO_P}}^{l_{\mathsf{mac}}}$, or ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$ on the correct value $\mathsf{PRK_{2e}}$ which unpredictable at this stage. We denote F^0_5 the event: $\mathcal{A}$ queries ${\mathsf{SO_P}}^{l_2}$, or ${\mathsf{SO_P}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO_P}}^{l_{\mathsf{iv}}}$, or ${\mathsf{SO_P}}^{l_{\mathsf{mac}}}$, or ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$ on the correct value $\mathsf{PRK_{2e}}$ for a session in which the latter is unpredictable. If F^0_5 happens, we stop the simulation and return a random bit. We can provide now an upper bound for $\mathbb{P}(F^0_5)$. For that we look at the probability of $\mathcal{A}$ querying one of these four oracles on the correct $\mathsf{PRK_{2e}}$ in our tested session. Since for the adversary, $\mathsf{PRK_{2e}}$ is here unpredictable, we obtain the following probability: $\frac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK_{2e}}$ has size l_{hash}) for the guess of $\mathsf{PRK_{2e}}$. Now $\mathcal{A}$ can make at most $q_{{\mathsf{RO_P}}^{l_2}}$ (resp. $q_{{\mathsf{RO_P}}^{l_{\mathsf{key}}}}$, $q_{{\mathsf{RO_P}}^{l_{\mathsf{iv}}}}$, and $q_{{\mathsf{RO_P}}^{l_{\mathsf{mac}}}}$) calls to ${\mathsf{SO_P}}^{l_2}$ (resp. ${\mathsf{SO_P}}^{l_{\mathsf{key}}}$, ${\mathsf{SO_P}}^{l_{\mathsf{iv}}}$ and ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$), we thus have:

$$\mathbb{P}(F^0_5) \leq \frac{q_{{\mathsf{RO_P}}^{l_2}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{mac}}}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}}.$$

By the identical-until-bad lemma, we obtain:

$$\left|\mathbb{P}(\mathbb{G}^0_{22} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^0_{23} \Longrightarrow 1)\right| \leq \mathbb{P}(F^0_5)$$

and thus:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^0_{22} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^0_{23} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^0_{24}$:** This game is the same as $\mathbb{G}^0_{23}$, but once again we simulates the generation of $\mathsf{PRK_{2e}}$ for some other instances. Namely, if our tested instance is an honest Initiator's instance receiving a forged second message (g^y, c_2) from an adversary controlled Responder's instance, but not corrupted on its long term signing key with the `oCorrupt` oracle, we simulate the generation of $\mathsf{PRK_{2e}}$ thanks to the private oracle $\mathsf{SO_{PRK_{2e}}}$, still inaccessible to the adversary.
Now let's see what happens in this situation. Firstly, we assume that g^y is forged. In this case $\mathsf{TH_2}$ is different from the value obtained by an honest Responder instance (*i.e.,* an Responder instance simulated by the challenger) thanks to the absence of collisions because we computed it using $\mathsf{SO_H}$ (according to game $\mathbb{G}_7$). Secondly, if g^y is not forged, then the symmetric key $\mathsf{sk_2}$ is authentic. We deduce that the ciphertext c_2 is forged, since the XOR-ing encryption scheme used here verifies the injective property when the key is fixed. In both cases, m_2 and then $\mathsf{TH_3}$ are different from the expected values obtained with an honest Responder's instance (as before, but this time in the second run of the Responder) (note $\mathsf{TH_2}$ and $\mathsf{TH_3}$ are common values for Initiator and Responder instances, even if the tuple is forged; and we use these values to compute $\mathsf{PRK_{2e}}$ or in a calculation involving $\mathsf{PRK_{2e}}$).
As for the previous game, the use of the private oracle $\mathsf{SO_{PRK_{2e}}}$ can possibly be detected by the adversary in two ways: he queried $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}. But in this situation, thanks to the event F^0_4, the simulation is already stopped. Or as is the previous game by picking at random a value $\mathsf{PRK_{2e*}}$ from $\{0,1\}^{l_{\mathsf{hash}}}$ and querying $\mathsf{SO_P}$ on this value. In this situation, thanks to the event F^0_5, we already stopped the simulation. We hence have:

$$\boxed{\mathbb{P}(\mathbb{G}^0_{24} \Longrightarrow 1) = \mathbb{P}(\mathbb{G}^0_{23} \Longrightarrow 1).}$$

† **Game $\mathbb{G}^0_{25}$:** This game is the same as the previous one, and here again we simulate the generation of the key $\mathsf{PRK_{2e}}$ thanks to the private oracle $\mathsf{SO_{PRK_{2e}}}$ for some instances. Namely, if our tested instance in an honest Responder's instance receiving a forged message m_1 from an adversary controlled Initiator's instance, but not corrupted with `oCorrupt` on the long term signing key.
Since m_1 is forged, thanks to the absence of collisions in $\mathsf{SO}_{\mathcal{H}}$, elements $\mathsf{TH_2}$ and $\mathsf{TH_3}$ are different from the values computed by in an honest Initiator's instance (*i.e.,* simulated by the challenger).
Again, $\mathcal{A}$ detects the use of $\mathsf{SO_{PRK_{2e}}}$: if he queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}, and here again, thanks to the event F^0_4, the simulation is already stopped. Or if he queried $\mathsf{SO_P}$ on a random value from $\{0,1\}^{l_{\mathsf{hash}}}$, in which case the simulation is also already stopped. We deduce that:

$$\boxed{\mathbb{P}(\mathbb{G}^0_{25} \Longrightarrow 1) = \mathbb{P}(\mathbb{G}^0_{24} \Longrightarrow 1).}$$

† **Game $\mathbb{G}^0_{26}$:** This game is the same as $\mathbb{G}^0_{25}$, but here we finally focus on the generation of $\mathsf{PRK_{out}}$ if the tested instance is fresh. Indeed if the tested instance does not respect the Fresh property, then the adversary cannot win the game with a non-zero advantage. So in this situation we generate $\mathsf{PRK_{out}}$ thanks to a private oracle $\mathsf{SO_{PRK_{out}}}$ for these sessions. Note that the game $\mathbb{G}_8$ ensures the points related to the authorities for all the sessions in the fresh definition. The rest of the definition means that no call to the `oCorrupt` oracle (on the long term signing key) occurred before each party `accepted`. We can retranslate this by saying that the Initiator's instance is not corrupted before receiving the second message and the Responder's instance is not corrupted before receiving the third message. By chance, the three previous games have been constructed to ensure that in this case, the key $\mathsf{PRK_{2e}}$ is generated using the private oracle $\mathsf{SO_{PRK_{2e}}}$, and so unpredictable by the adversary.
Since $\mathsf{PRK_{out}}$ is computed using $\mathsf{PRK_{2e}}$, the adversary may detected the use of the private oracle $\mathsf{SO_{PRK_{out}}}$ (in one of the two instances involved in the session) if he asked the $\mathsf{SO_P}^{l_{\mathsf{hash}}}$ oracle on the (public but

unpredictable) value $\mathsf{PRK_{2e}}$. Let's denote F_6^0 this event. If F_6^0 occurs we stop the simulation by returning a random bit. By the identical-until-bad lemma we obtain:

$$\left|\mathbb{P}\big(\mathbb{G}_{25}^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{26}^0 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_6^0).$$

We now need to provide a bound for $\mathbb{P}(F_6^0)$. Formally we look at the probability of the adversary querying ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$ on the correct value $\mathsf{PRK_{2e}}$ in one session. Since $\mathsf{PRK_{2e}}$ is unpredictable, the probability we are looking for is hence $\frac{1}{2^{l_{\mathsf{hash}}}}$ (because the digests output by the oracle $\mathsf{SO_{PRK_{2e}}}$ are of size l_{hash}). Because $\mathcal{A}$ makes at most $q_{{\mathsf{RO_P}}^{l_{\mathsf{hash}}}}$ queries to ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$, we thus have:

$$\mathbb{P}(F_6^0) \leq \frac{q_{{\mathsf{RO_P}}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}},$$

and hence:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{25}^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{26}^0 \Longrightarrow 1\big)\right| \leq \frac{q_{{\mathsf{RO_P}}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}_{22}^1$:** This game is the same as the previous one (game $\mathbb{G}_{21}^1$) and works in the same vein as game $\mathbb{G}_{22}^0$ by adding a failure condition: if the adversary, during the tested session, queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xr}, then we abort and return a random bit. We then denote F_4^1 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xr} (with g^x the public Diffie-Hellman element of a challenger-controlled Initiator instance and r a (adversary-controlled, not corrupted) Responder instance long-term secret key). If the event F_4^1 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}\big(\mathbb{G}_{21}^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{22}^1 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_4^1).$$

Let's now provide a bound for $\mathbb{P}(F_4^1)$. For that we construct, by reduction, an adversary $\mathcal{B}_6^1$ against the GDH problem. By the identical-until-bad lemma we hence claim that:

$$\mathbb{P}(F_4^1) \leq q_{\mathsf{NS}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{21}^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{22}^1 \Longrightarrow 1\big)\right| \leq q_{\mathsf{NS}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda).}$$

‡ **Proof of the claim:** We want to provide an upper bound for $\mathbb{P}(F_4^1)$. Nothing more simple, by reduction we built an adversary $\mathcal{B}_6^1$ against the GDH problem.
On input a GDH challenge $(g, A = g^a, B = g^b)$ and having access to a DDH oracle, returning 1 if a tuple (g, g^a, g^b, g^c) is a Diffie-Hellman tuple, and 0 otherwise, our algorithm mus returns g^{ab} in a polynomial time. For that, $\mathcal{B}_6^1$ simulates $\mathbb{G}_{22}^1$ for $\mathcal{A}(\mathsf{pp})$ as follows, mainly faithfully, but with some modifications. Firstly, in every honest Initiator instances, $\mathcal{B}_6^1$ embeds its challenge into the shared Diffie-Hellman value: it picks at random $k \in \mathbb{Z}_p$ (for each instance) and sets $X \leftarrow Ag^k$. Then he samples $k' \leftarrow\$ \mathbb{Z}_p$ for each Responder and sets $R \leftarrow Bg^{k'}$. Even if b (from g^b) remains unknown by $\mathcal{B}_6^1$, the latter can continue the simulation by querying the $\mathsf{SO_T}$ oracle on input (X, R).
If F_4^1 occurs on that particular Responder instance, then the value Z queried to the $\mathsf{SO_T}$ oracle will equal $g^{(a+k)b}$. To detect that, $\mathcal{B}_6^1$ can compute $ZR^{-k}X^{-k'}$, which if F_4^1 occurs will equal $g^{ab}g^{kb}g^{-kb}g^{ak'}g^{-ak'} = g^{ab}$, and test this equality with its DDH oracle on input $(X, R, ZR^{-k}X^{-k'})$, which will return 1 since F_4^1 occurs. So when the event happens, $\mathcal{B}_6^1$ wins.

If F_4^1 occurs on another Responder instance, then $\mathcal{B}_6^1$ stops the simulation.
For each Responder instance involved in a session, the event F_4^1 can be detected. Thus we have:

$$\mathbb{P}(F_4^1) \leq q_{\mathsf{NS}} \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda)$$

and this concludes the proof of the claim.

† **Game $\mathbb{G}_{23}^1$:** This game is the same as the previous one ($\mathbb{G}_{22}^1$) and we work in the same vein as $\mathbb{G}_{23}^0$. Once again, if our tested instance π_{Ξ}^{e} is an honest Initiator's instance receiving (g^y, c_2) from an adversary controlled Responder's instance (its partner $\pi_{\Xi'}^{e'}$), but which is not corrupt (note that at this point the LI elements are chosen at random, thus they cannot be forged), the challenger simulates $\mathsf{PRK}_{\mathsf{3e2m}}$ for the Initiator instance with a private oracle $\mathsf{SO}_{\mathsf{PRK}_{\mathsf{3e2m}}}$. As before, this key is unpredictable by the adversary. Note that here we prove the security of r and g^{rx} in the key-derivation schedule; in normal times, $\mathsf{PRK}_{\mathsf{3e2m}}$ is generated with SO_{T}, $\mathsf{salt}_{\mathsf{3e2m}}$ and g^{rx}. One way to detect the modification is for the adversary to ask SO_{T} on the correct Diffie-Hellman value g^{xr}. Thanks to the event F_4^1 the simulation is already stopped in this situation. Another way to detect it is to query ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK}_{\mathsf{3e2m}}$ which unpredictable at this stage. We denote F_5^1 the event: $\mathcal{A}$ queries ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$,or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK}_{\mathsf{3e2m}}$ for a session in which the latter is unpredictable. If F_5^1 happens, we stop the simulation and return a random bit. We can provide now an upper bound for $\mathbb{P}(F_5^1)$. For that we look at the probability of $\mathcal{A}$ querying one of these four oracles on the correct $\mathsf{PRK}_{\mathsf{3e2m}}$ in our tested session. Since for the adversary, $\mathsf{PRK}_{\mathsf{3e2m}}$ is here unpredictable, we obtain the following probability: $\frac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK}_{\mathsf{3e2m}}$ has size l_{hash}) for the guess of $\mathsf{PRK}_{\mathsf{3e2m}}$. Now $\mathcal{A}$ can make at most $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}}$ (resp. $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}}$, $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}}$, and $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}$) calls to ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$ (resp. ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$ and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$), we thus have:

$$\mathbb{P}(F_5^1) \leq \frac{q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

By the identical-until-bad lemma, we obtain:

$$\left|\mathbb{P}\big(\mathbb{G}_{22}^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{23}^1 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_5^1)$$

and thus:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{22}^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{23}^1 \Longrightarrow 1\big)\right| \leq \frac{q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}_{24}^1$:** This game is the same as $\mathbb{G}_{23}^1$, except that we keep modifying the key generation for some instances. This time, if our tested instance is an honest Responder instance receiving a forged message m_1 from an adversary-controlled Initiator's instance, we simulate $\mathsf{PRK}_{\mathsf{3e2m}}$ (for the Responder's instance) thanks to a private oracle $\mathsf{SO}_{\mathsf{PRK}_{\mathsf{3e2m}}}$. In normal situations, $\mathsf{PRK}_{\mathsf{3e2m}}$ is generated with SO_{T}, $\mathsf{salt}_{\mathsf{3e2m}}$ and g^{rx}).
We now make the following observation: because m_1 is forged, elements $\mathsf{TH}_2, \mathsf{TH}_3$ and $\mathsf{salt}_{\mathsf{3e2m}}$ are different from those expected by an honest Initiator instance, thanks to the absence of collisions (since game $\mathbb{G}_7$). In this situation, the adversary detect the use of the private oracle, if he queries SO_{T} on the correct Diffie-Hellman value g^{rx} - and thanks to the event F_4^1, the simulation is already stopped - or if $\mathcal{A}$ queries SO_{P}s on the correct value $\mathsf{PRK}_{\mathsf{3e2m}}$ - and thanks to the event F_5^1, the simulation is already stopped. So this change becomes indistinguishable by the adversary. So we have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_{23}^1 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_{24}^1 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}^1_{25}$:** This game is the same as game $\mathbb{G}^1_{24}$, except that we apply the modification of the game $\mathbb{G}^0_{22}$ to attack the security of x and y in the key-derivation schedule. We perform the same analysis and reduction, and thus obtain, thanks to a p.p.t. algorithm $\mathcal{B}^1_7$:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_{24} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{25} \Longrightarrow 1\big)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

† **Game $\mathbb{G}^1_{25'}$:** This game is the same as game $\mathbb{G}^1_{25}$, except that we apply the modification of the game $\mathbb{G}^0_{23}$ to again attack the security of x and y in the key-derivation schedule. To detect that change, $\mathcal{A}$ has two options: queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}, which is as usual excluded at this stage or pick a random $\mathsf{PRK_{2e*}}$ from $\{0,1\}^{l_{\mathsf{hash}}}$, hopping for $\mathsf{PRK_{2e*}} = \mathsf{PRK_{2e}}$. Similarly we denote $F^1_{5''}$ the event: $\mathcal{A}$ queries ${\mathsf{SO_P}}^{l_2}$ on the correct value $\mathsf{PRK_{2e}}$ for a session in which the latter is unpredictable (here only ${\mathsf{SO_P}}^{l_2}$ uses $\mathsf{PRK_{2e}}$). If $F^1_{5''}$ happens, we stop the simulation and return a random bit. In a similar way as in game $\mathbb{G}^0_{23}$, we provide an upper bound for $\mathbb{P}(F^1_{5''})$ with:

$$\mathbb{P}(F^1_{5''}) \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}}.$$

By the identical-until-bad lemma, we obtain:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_{25} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{25'} \Longrightarrow 1\big)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^1_{25''}$:** This game is the same as the previous one, except that if π^e_Ξ is an honest Initiator's instance receiving the second message (g^y, c_2) (not necessarily forged) from an adversary-controlled Responder's instance which is not corrupted, we simulate the generation of the key $\mathsf{PRK_{3e2m}}$ for the Initiator's instance thanks to a private oracle $\mathsf{SO_{PRK_{3e2m}}}$ (instead of using $\mathsf{SO_T}$).
This is a minor modification compare to the previous games: a difference appears only if the second message is authentic, which can be compare to saying that we are in an honest session, in which case the key $\mathsf{PRK_{2e}}$, computed by the Initiator instance, is unpredictable since generated by a private oracle, and thus different from the public one, expected by the Responder instance, with a probability of $\frac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK_{2e}}$ is of size l_{hash}).
Once again a previous analysis applies: there is no collision in ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$, thus $\mathsf{salt_{3e2m}}$ obtained by the Initiator instance is different from the value obtained by an honest Responder instance.
Adversary $\mathcal{A}$ may be able to detect the use of $\mathsf{SO_{PRK_{3e2m}}}$ if he queries $\mathsf{SO_T}$ on the correct Diffie-Hellman element g^{rx} or $\mathsf{SO_P}$s on the correct value $\mathsf{PRK_{3e2m}}$. Because of events F^1_4 and F^1_5, these situations cannot happen since we already stop the simulation in these cases. We thus have:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_{25''} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{25'} \Longrightarrow 1\big)\right| \leq \frac{1}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^1_{25'''}$:** This game is the same as the previous one, and once is not customary, if π^e_Ξ is an honest Responder's instance receiving m_1 from an adversary-controlled Initiator's instance which is not corrupted, the challenger simulates $\mathsf{PRK_{3e2m}}$ thanks to a private oracle $\mathsf{SO_{PRK_{3e2m}}}$.
We can perform a similar analysis as in the previous game, just reverting the role: a difference appears only if the first message is not forged, which can be compare to saying that we are in an honest session, in which case $\mathsf{PRK_{2e}}$ is generated thanks to a private oracle, and thus different from the public one expected by the Initiator instance with probability $\frac{1}{2^{l_{\mathsf{hash}}}}$.
And again, there are no collisions in ${\mathsf{SO_P}}^{l_{\mathsf{hash}}}$, so $\mathsf{salt_{3e2m}}$ is different from the value obtained by an honest Initiator's instance. So $\mathcal{A}$ detect the use of this private oracle if he asked the $\mathsf{SO_T}$ oracle on the correct Diffie-Hellman value g^{rx} or queried $\mathsf{SO_P}$s on the correct value $\mathsf{PRK_{3e2m}}$ which is impossible thanks to the events F^1_4 and F^1_5 (which already stopped the simulation in this case). We thus have:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_{25''} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{25'''} \Longrightarrow 1\big)\right| \leq \frac{1}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^1_{26}$:** This game is the same as $\mathbb{G}^1_{25'''}$, but here we finally focus on the generation of $\mathsf{PRK_{out}}$ if the tested instance is fresh. Indeed if the tested instance does not respect the Fresh property, then the adversary cannot win the game with a non-zero advantage. So in this situation we generate $\mathsf{PRK_{out}}$ thanks to a private oracle $\mathsf{SO_{PRK_{out}}}$ for these sessions. Note that the game $\mathbb{G}_8$ ensures the points related to the authorities for all the sessions in the fresh definition. The rest of the definition means that no call to the `oCorrupt` oracle (on the long term signing key) occurred before each party `accepted`. We can retranslate this by saying that the Initiator's instance is not corrupted before receiving the second message and the Responder's instance is not corrupted before receiving the third message. By chance, the three previous games have been constructed to ensure that in this case, the key $\mathsf{PRK_{3e2m}}$ is generated using the private oracle $\mathsf{SO_{PRK_{3e2m}}}$, and so unpredictable by the adversary.
Since $\mathsf{PRK_{out}}$ is computed using $\mathsf{PRK_{3e2m}}$, the adversary may detected the use of the private oracle $\mathsf{SO_{PRK_{out}}}$ (in one of the two instances involved in the session) if he asked the $\mathsf{SO_P}^{l_{\mathsf{hash}}}$ oracle on the (public but unpredictable) value $\mathsf{PRK_{2e}}$. Let's denote F^1_6 this event. If F^1_6 occurs we stop the simulation by returning a random bit. By the identical-until-bad lemma we obtain:

$$\left|\mathbb{P}\big(\mathbb{G}^1_{25'''} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{26} \Longrightarrow 1\big)\right| \leq \mathbb{P}(F^1_6).$$

As we done it for the game $\mathbb{G}^0_{26}$, we bound $\mathbb{P}(F^1_6)$ by:

$$\mathbb{P}(F^1_6) \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}},$$

and hence:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_{25'''} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{26} \Longrightarrow 1\big)\right| \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^2_{22}$:** This game is the same as $\mathbb{G}^2_{21}$ and we keep going on the same vein as $\mathbb{G}^1_{22}$. If the adversary, during the tested session (the session between π^e_Ξ and $\pi^{e'}_{\Xi'}$), queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{iy}, then we abort and return a random bit. We then denote F^2_4 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{iy} (with g^y the public Diffie-Hellman element of a challenger-controlled Responder instance and i a (adversary-controlled, not corrupted) Initiator instance long-term secret key). If the event F^2_4 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}\big(\mathbb{G}^2_{21} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^2_{22} \Longrightarrow 1\big)\right| \leq \mathbb{P}(F^2_4).$$

As in game $\mathbb{G}^1_{22}$ we construct, by reduction, an adversary $\mathcal{B}^2_6$ against the GDH problem and we bound $\mathbb{P}(F^2_4)$ by:

$$\mathbb{P}(F^2_4) \leq q_{\mathsf{NS}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^2_{21} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^2_{22} \Longrightarrow 1\big)\right| \leq q_{\mathsf{NS}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

† **Game $\mathbb{G}^2_{23}$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}^1_{23}$, but compare to the later, here we reversed the role of the Initiator and the Responder. This time, if our tested instance is an honest Responder instance receiving a forged message m_1 from and adversary-controlled Initiator's

instance which is non-corrupted, and which is still non-corrupted when sending c_3, the challenger generates the key $\mathsf{PRK}_{\mathsf{4e3m}}$ (for the Responder instance) thanks to a private oracle $\mathsf{SO}_{\mathsf{PRK}_{\mathsf{4e3m}}}$. Here we prove the security of i and g^{iy} in the key-derivation schedule; in normal times, $\mathsf{PRK}_{\mathsf{4e3m}}$ is generated with SO_{T}, $\mathsf{salt}_{\mathsf{4e3m}}$ and g^{iy}) (note that here we don't need to restrict the use of the `oCorrupt` oracle, after $\mathcal{A}$ returns its guess).
We perform now an analysis of what is happening in this case. Because m_1 is forged, thanks to the absence of collisions, elements $\mathsf{TH}_2, \mathsf{TH}_3$ and $\mathsf{salt}_{\mathsf{4e3m}}$ do not equal values expected from an honest Initiator instance.
The adversary $\mathcal{A}$ may be able to detect this modification if he asks SO_{T} on the correct Diffie-Hellman value g^{iy}. Thanks to the event F_4^2 the simulation is already stopped in this situation. Another way to detect it is to query ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK}_{\mathsf{4e3m}}$ which unpredictable at this stage. We denote F_5^2 the event: $\mathcal{A}$ queries ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$,or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK}_{\mathsf{3e2m}}$ for a session in which the latter is unpredictable. If F_5^1 happens, we stop the simulation and return a random bit. We can provide now an upper bound for $\mathbb{P}(F_5^2)$. For that we look at the probability of $\mathcal{A}$ querying one of these four oracles on the correct $\mathsf{PRK}_{\mathsf{4e3m}}$ in our tested session. Since for the adversary, $\mathsf{PRK}_{\mathsf{4e3m}}$ is here unpredictable, we obtain the following probability: $\frac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK}_{\mathsf{4e3m}}$ has size l_{hash}) for the guess of $\mathsf{PRK}_{\mathsf{4e3m}}$. Now $\mathcal{A}$ can make at most $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}}$ (resp. $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}}$, $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}}$, and $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}$) calls to ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$ (resp. ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$ and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$), we thus have:

$$\mathbb{P}(F_5^1) \leq \frac{q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

By the identical-until-bad lemma, we obtain:

$$\left|\mathbb{P}\big(\mathbb{G}_{22}^2 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{23}^2 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_5^2)$$

and thus:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{22}^2 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{23}^2 \Longrightarrow 1\big)\right| \leq \frac{q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}_{24}^2$:** This game is the same as game $\mathbb{G}_{23}^2$ and works in the same vein as $\mathbb{G}_{24}^0$. Namely if π_{Ξ}^{e} is an honest Initiator's instance receiving a forged tuple (g^y, c_2) from an adversary-controlled Responder's instance which is non-corrupted, the challenger simulates $\mathsf{PRK}_{\mathsf{4e3m}}$ for the Initiator's instance with a private oracle $\mathsf{SO}_{\mathsf{PRK}_{\mathsf{4e3m}}}$.
The same analysis as in game $\mathbb{G}_{24}^0$ (with the appropriate notations) can be performed. Again, in this situation, the adversary detect the use of the private oracle, if he queries SO_{T} on the correct Diffie-Hellman value g^{iy} - and thanks to the event F_4^2, the simulation is already stopped - or if $\mathcal{A}$ queries SO_{P}s on the correct value $\mathsf{PRK}_{\mathsf{4e3m}}$ - and thanks to the event F_5^2, the simulation is already stopped. So this change becomes indistinguishable by the adversary. So we have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_{24}^2 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_{23}^2 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_{25}^2$:** This game is the same as game $\mathbb{G}_{24}^2$, except that we apply the modification of the game $\mathbb{G}_{22}^0$ to attack the security of x and y in the key-derivation schedule. We perform the same analysis and reduction, and thus obtain, thanks to a p.p.t. algorithm $\mathcal{B}_7^2$:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{24}^2 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{25}^2 \Longrightarrow 1\big)\right| \leq q_{\mathsf{RO}_{\mathsf{T}}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda).}$$

† **Game $\mathbb{G}^2_{25'}$:** This game is the same as the previous one, except that we apply the modification of the game $\mathbb{G}^0_{23}$ to again attack the security of x and y in the key-derivation schedule. To detect that change, $\mathcal{A}$ has two options: queries $\mathsf{SO_T}$ on the correct DIFFIE-HELLMAN value g^{xy}, which is as usual excluded at this stage or pick a random $\mathsf{PRK_{2e*}}$ from $\{0,1\}^{l_{\mathsf{hash}}}$, hopping for $\mathsf{PRK_{2e*}} = \mathsf{PRK_{2e}}$. Similarly we denote $F^2_{5''}$ the event: $\mathcal{A}$ queries ${\mathsf{SO_P}}^{l_2}$ on the correct value $\mathsf{PRK_{2e}}$ for a session in which the latter is unpredictable (here only ${\mathsf{SO_P}}^{l_2}$ uses $\mathsf{PRK_{2e}}$). If $F^2_{5''}$ happens, we stop the simulation and return a random bit. In a similar way as in game $\mathbb{G}^0_{23}$, we provide an upper bound for $\mathbb{P}(F^2_{5''})$ with:

$$\mathbb{P}(F^2_{5''}) \leq \frac{q_{{\mathsf{RO_P}}^{l_2}}}{2^{l_{\mathsf{hash}}}}.$$

By the identical-until-bad lemma, we obtain:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{25} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{25'} \Longrightarrow 1)\right| \leq \frac{q_{{\mathsf{RO_P}}^{l_2}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^2_{25''}$:** This game is the same as $\mathbb{G}^2_{25'}$, except that if π^e_Ξ is an honest $\mathsf{Responder}$ instance receiving c_3 (not necessarily forged) from an adversary-controlled $\mathsf{Initiator}$'s instance which is not corrupted, the challenger simulates $\mathsf{PRK_{4e3m}}$ thanks to a private oracle $\mathsf{SO_{PRK_{4e3m}}}$.
We can now perform the same deduction as in the game $\mathbb{G}^1_{25''}$ (with the appropriate notations) which leads to:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{25'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{25''} \Longrightarrow 1)\right| \leq \frac{1}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^2_{25'''}$:** This game is the same as $\mathbb{G}^2_{25''}$, and works in the same way as $\mathbb{G}^1_{25'''}$, with the appropriate notations. We thus obtain the same results:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{25''} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{25'''} \Longrightarrow 1)\right| \leq \frac{1}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^2_{26}$:** This game is the same as $\mathbb{G}^2_{25'''}$, but here we finally focus on the generation of $\mathsf{PRK_{out}}$ if the tested instance is fresh. As in game $\mathbb{G}^1_{26}$ we define the event F^2_6 and bound $\mathbb{P}(F^2_6)$ as in game $\mathbb{G}^0_{26}$. Here again by the identical-until-bad lemma we obtain:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{25'''} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{26} \Longrightarrow 1)\right| \leq \frac{q_{{\mathsf{RO_P}}^{l_{\mathsf{hash}}}}}{2^{l_{\mathsf{hash}}}}.}$$

† **Game $\mathbb{G}^3_{22}$:** This game follows $\mathbb{G}^3_{17'}$ and works in the same way as $\mathbb{G}^1_{22}$, with the appropriate notations. After defining the event F^3_4, we obtain the same results and thus have, thanks to a p.p.t. algorithm $\mathcal{B}^3_6$:

$$\left|\mathbb{P}(\mathbb{G}^3_{17'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{22} \Longrightarrow 1)\right| \leq q_{\mathsf{NS}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}^3_{23}$:** This game is the same as the previous one, and we apply the same modification as in $\mathbb{G}^1_{23}$ (with the appropriate notations), defining the event F^3_5. Again we bound $\mathbb{P}(F^3_5)$ and thanks to the identical-until-bad lemma we obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}^3_{22} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{23} \Longrightarrow 1)\right| \leq \frac{q_{{\mathsf{RO_P}}^{l_{\mathsf{hash}}}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO_P}}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

† **Game $\mathbb{G}^3_{24}$:** This game is the same as the previous one and works in the same way as $\mathbb{G}^2_{22}$, with the appropriate notations. We here define the event $F^3_{5'}$ and we bound $\mathbb{P}(F^3_{5'})$ in the same way. We thus obtain:

$$\left|\mathbb{P}(\mathbb{G}^3_{23} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{24} \Longrightarrow 1)\right| \leq q_{\mathsf{NS}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}^3_{25}$:** This game is the same as the previous one, and we apply the same modification as in $\mathbb{G}^2_{23}$ (with the appropriate notations), defining the event $F^3_{5''}$. Again we obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}^3_{24} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{25} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

† **Game $\mathbb{G}^3_{25'}$:** Here again, this game is the same as the previous one, and works in the same vein as $\mathbb{G}^0_{22}$. This leads again, thanks to a p.p.t. algorithm $\mathcal{B}^3_8$, to:

$$\left|\mathbb{P}(\mathbb{G}^3_{25} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{25'} \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}^3_{25''}$:** This game is the same as the previous one, and we apply the same modification as in game $\mathbb{G}^2_{25'}$ (with the appropriate notations). We thus obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}^3_{25'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{25''} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}}.$$

† **Game $\mathbb{G}^3_{25'''}$:** This game is the same as the previous one and works in the same way as $\mathbb{G}^1_{25''}$ (by using the private oracle $\mathsf{SO}_{\mathsf{PRK}_{3e2m}}$). This leads us to bound the advantage of the adversary for this game hop in the same way:

$$\left|\mathbb{P}(\mathbb{G}^3_{25''} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{25'''} \Longrightarrow 1)\right| \leq \frac{1}{2^{l_{\mathsf{hash}}}}.$$

† **Game $\mathbb{G}^3_{25''''}$:** This game is the same as $\mathbb{G}^3_{25'''}$, except that we simulate the generation of PRK_{4e3m} with the private oracle $\mathsf{SO}_{\mathsf{PRK}_{4e3m}}$ for π^e_Ξ if it is an honest $\mathsf{Initiator}$'s instance receiving a second message (not necessarily forged) from an adversary-controlled $\mathsf{Responder}$'s instance (which is not corrupted on its long term static DH key). But thanks to the previous game, we know that PRK_{3e2m} is random from the point of view of $\mathcal{A}$, and as it is involved in the computation of salt_{4e3m} and PRK_{4e3m}, the latter are already unpredictable (for the $\mathsf{Initiator}$ and $\mathsf{Responder}$ instances thanks to the events introduced in the previous games, that's why the adversary cannot detect it). So this modification is indistinguishable by $\mathcal{A}$ and we thus have:

$$\mathbb{P}(\mathbb{G}_{25'''} \Longrightarrow 1) = \mathbb{P}(\mathbb{G}_{25''''} \Longrightarrow 1).$$

† **Game $\mathbb{G}^3_{25'''''}$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}^2_{25''}$. We deduce the same result and so we have:

$$\left|\mathbb{P}(\mathbb{G}^3_{25''''} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{25'''''} \Longrightarrow 1)\right| \leq \frac{1}{2^{l_{\mathsf{hash}}}}.$$

† **Game $\mathbb{G}^3_{26}$:** This game is the same as $\mathbb{G}^3_{25''''}$, but here we finally focus on the generation of $\mathsf{PRK}_{\mathsf{out}}$ if the tested instance is fresh. As in game $\mathbb{G}^2_{26}$ we define the event F^3_6 and bound $\mathbb{P}(F^3_6)$ as in game $\mathbb{G}^0_{26}$. Here again by the identical-until-bad lemma we obtain:

$$\left|\mathbb{P}\big(\mathbb{G}^3_{25''''} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^3_{26} \Longrightarrow 1\big)\right| \leq \frac{q_{\mathsf{RO_P}}{}^{l_{\mathsf{hash}}}}{2^{l_{\mathsf{hash}}}}.$$

We can now conclude the proof and for that we analyze the final advantage of $\mathcal{A}$ when playing game $\mathbb{G}_{26}$ (this explanation, expect for the notations, is valid for the four games $\mathbb{G}^0_{26}$, $\mathbb{G}^1_{26}$, $\mathbb{G}^2_{26}$ and $\mathbb{G}^3_{26}$). For that we take a look at the situations in which $\mathcal{A}$ has a non-zero advantage after having return his guessing bit. It is the case when:

- $\rightsquigarrow$ Sound is `false` or;
- $\rightsquigarrow$ $\mathsf{Fresh}(\pi^e_\Xi)$ is `true` and $b = b'$.

Indeed:

- $\looparrowright$ If Sound is `false`, then the experience returns 1 and thus the advantage of $\mathcal{A}$ is greater than 0.
- $\looparrowright$ Let's observe that if $\mathsf{Fresh}(\pi^e_\Xi)$ is `false`, then the experience sets $b' \leftarrow\$ \{0,1\}$. So the event $b = b'$ happens, in this case, with probability $\frac{1}{2}$ and so the advantage of $\mathcal{A}$ is 0.
  So if $\mathsf{Fresh}(\pi^e_\Xi)$ is `true` and if $b = b'$, then $\mathcal{A}$ wins the game $\mathbb{G}_{26}$ (if $b \neq b'$, then $\mathcal{A}$ loses the game and it means that its advantage is 0) and thus its advantage is greater than 0.

So we need to check if here $\mathcal{A}$ respects soundness. Informally, we raised a flag `Sound` to `false` if:

(•) Three honest instances hold the same sid, or;

(••) Two partnered instances involved in the same session hold different session keys.

Now observe that thanks to the game $\mathbb{G}_2$, (•) is impossible. So we take a closer look to (••). With honest instances, or simply controlled by the adversary (but not corrupted), with the previous construction, we do not infringe on the soundness property (thanks to the private oracles). Now if we consider two partnered instances, with at least on corrupted, since they share the session identifier with the beginning of the exchange process, they derive the same session key $\mathsf{PRK}_{\mathsf{out}}$ thanks to the multiples oracles SO. In other words, agreeing on the sid implies deriving the same $\mathsf{PRK}_{\mathsf{out}}$. Subsequently in game $\mathbb{G}_{26}$, the flag `Sound` is always set to `true`.
We can finally bound the adversary's advantage in game $\mathbb{G}_{26}$. For that observe that for any fresh sessions, if the adversary queries the $\mathtt{oTest}_b$ oracle, in the case $b = 0$, since we used private oracles, it finally outputs a random key. Moreover, in the case $b = 1$, the outputed session key is random too. So, ultimately, the probability that $\mathcal{A}$ correctly guesses and distinguishes between the two keys in game $\mathbb{G}_{26}$ is $\dfrac{1}{2}$ (we here look at the probability that $b = b'$), *i.e.*,:

$$\boxed{\mathbb{P}\big(\mathbb{G}_{26} \Longrightarrow 1\big) = \frac{1}{2}.}$$

So finally, combining the four branches and the main branch if the beginning, we have:

$$
\begin{aligned}
\mathsf{Adv}^{\mathrm{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) &= \left|\mathbb{P}\big(\mathbb{E}^{\mathsf{KS}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big) - \frac{1}{2}\right| = \left|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{26} \Longrightarrow 1\big)\right| \\
&\leq \left|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big)\right| + \cdots + \left|\mathbb{P}\big(\mathbb{G}_{25} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{26} \Longrightarrow 1\big)\right| \\
&\leq \frac{3q_{\mathsf{NS}}^2}{2^{l_{\mathsf{C}}+1}p} + \frac{\big(q_{\mathsf{RO_T}} + q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}}\big)^2 + \big(q_{\mathsf{RO_T}} + q_{\mathsf{RO_H}}\big)^2 + \big(q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_H}}\big)^2}{2^{l_{\mathsf{hash}}+1}} \\
&\quad + q_{\mathsf{NS}} \cdot q_{\mathsf{R}}^4 \cdot \Big[\big(2q_{\mathsf{RTD}} + q_{\mathsf{S}}\big) \cdot \epsilon_{\mathsf{SoK}}(\lambda) + q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + 24\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}}(\lambda) + 4\mathsf{Adv}^{\mathrm{EUF\text{-}CMA}}_{\mathsf{DS}}(\lambda) \\
&\quad + \big(4q_{\mathsf{RO_T}} + 4q_{\mathsf{NS}}\big) \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda) + 4 \cdot \frac{q_{\mathsf{RO_P}^{l_2}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + 6}{2^{l_{\mathsf{hash}}}}\Big]
\end{aligned}
$$

and this concludes the proof. ■

7.2 LI-compliance

The second security result, whose we have ensured the validity, is now stated with the following theorem. With it, we guarantee that the four versions of our protocol respect the LI-compliance security property.

Theorem 3 (LI-compliant LI-EDHOC protocol) *Under the assumption that our signature scheme* DS *is* EUF-CMA*-secure, that proofs and signatures of knowledge are zero-knowledge and extractable, the* LI-EDHOC *protocol, in its four versions, is LI-compliant. More precisely, for all p.p.t. adversary* $\mathcal{A}$ *making at most* q_{R} *queries to the* `oRegister` *oracle and* q_{S} *queries to the* `oSend` *oracle, we have:*

$$\mathsf{Adv}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq q_{\mathsf{R}}\Big(\mathsf{Adv}^{\mathrm{EUF\text{-}CMA}}_{\mathsf{DS}}(\lambda) + q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + q_{\mathsf{S}} \cdot \epsilon_{\mathsf{SoK}}(\lambda)\Big).$$

Before starting the proof, we just remind the LI-compliance security game.

$\mathbb{E}^{\mathsf{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$1: \mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda), \mathsf{ESet} \leftarrow \emptyset, \mathsf{PROXSet} \leftarrow \emptyset, \mathsf{AUTHSet} \leftarrow \emptyset$

$2: \mathcal{O}_{\mathsf{LIC}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot,\cdot,\cdot), \mathtt{oSend}(\cdot,\cdot), \\ \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot,\cdot), \\ \mathtt{SessionKeyReveal}(\cdot,\cdot), \\ \mathtt{oRevealTD}(\cdot,\cdot,\cdot,\cdot,\cdot) \end{array} \right\}$

$3: \big(j, \mathsf{sst_d}, \mathsf{I}, \mathsf{R}, \mathsf{Prox_d}, (\Lambda^{\mathsf{d}}_q, \Lambda^{\mathsf{d}}_q.\mathsf{td})^{n_{\mathsf{d}}}_{q=1}\big) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{LIC}}}(\lambda, \mathsf{pp})$

$4:$ **if** $\mathsf{Prox_d}.\overline{\mathsf{SK}}[\mathsf{sst_d}].\gamma = 1$ **then return** $\perp$

$5:$ **if** $\mathtt{Verify}(\mathsf{pp}, \mathsf{sst_d}, \mathsf{I}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{Prox_d}.\overline{\mathsf{PK}}, (\Lambda^{\mathsf{d}}_q.\overline{\mathsf{PK}})^{n_{\mathsf{d}}}_{q=1}) = 0$ **then return** $\perp$

$6:$ **if** $\mathtt{IdentifySession}(\mathsf{sst_d}, \pi^j_{\mathsf{Prox_d}}) = 0$ **then return** $\perp$

$7: \mathsf{PRK_{out*}} \leftarrow \mathtt{Open}(\mathsf{pp}, \mathsf{sst_d}, (\Lambda^{\mathsf{d}}_q.\overline{\mathsf{PK}})^{n_{\mathsf{d}}}_{q=1}, (\Lambda^{\mathsf{d}}_q.\mathsf{td})^{n_{\mathsf{d}}}_{q=1})$

$8:$ **if** $\mathsf{PRK_{out*}} = \perp$ **then return** $\perp$

$9: \mathsf{LIC}.\overline{\mathsf{PK}} \leftarrow \mathsf{I}.\overline{\mathsf{PK}} \cup \mathsf{R}.\overline{\mathsf{PK}} \cup \mathsf{Prox_d}.\overline{\mathsf{PK}} \cup \{\cup(\Lambda^{\mathsf{d}}_q.\overline{\mathsf{PK}})^{n_{\mathsf{d}}}_{q=1}\}$

$10:$ **return** $(\mathsf{PRK_{out*}}, \pi^j_{\mathsf{Prox_d}}, \mathsf{LIC}.\overline{\mathsf{PK}})$

FIGURE 26 – LI-compliance security game for the LI-EDHOC protocol (by $\mathsf{Prox_d}.\overline{\mathsf{SK}}[\mathsf{sst_d}]$ we mean the secret (long-term) signing key of $\mathsf{Prox_d}$ used to sign $\mathsf{sst_d}$).

Proof 3 (of theorem 3) Since the four versions of the protocol have some differences, we need, at some point, to split the proof of the theorem into four branches. The scheme is as follows: we first describe the four key-extractor algorithms $\mathtt{KeyExtract}(\pi_{\mathsf{Prox}}, \mathsf{LIC}.\overline{\mathsf{PK}})$, and then we proceed by games hop, with four (main) games. First, the game $\mathbb{G}_0$ is the reformulation of the original game. Then with game $\mathbb{G}_1$ and $\mathbb{G}_2$ we select at random a particular honest proxy against which $\mathcal{A}$ will fight, and we restrict the winning condition of the adversary according to this particular proxy. Then in game $\mathbb{G}_3$ we use the NIPoK extractor in order to prevent the adversary from forging a proof of knowledge. Finally in game $\mathbb{G}_4$ we use the SoK extractor in order to prevent the adversary from forging a signature of knowledge, and we conclude by reducing the winning probability of the adversary to 0. This last game will also be split into four similar games to cover all four versions. The game schedule is depicted in FIGURE 27 below.

We start by describing the four (deterministic, unbounded (otherwise it would be possible to determine the key in polynomial time, which would be problematic)) key-extractor algorithms $\mathtt{KeyExtract}(\pi_{\mathsf{Prox}}, \mathsf{LIC}.\overline{\mathsf{PK}})$, which given as input a proxy instance π_{Prox} and a set of public keys (generated as is the definition), outputs the same session key $\mathsf{PRK_{out}}$ as the $\mathsf{Initiator}$ and the $\mathsf{Responder}$ instances having matching conversation in

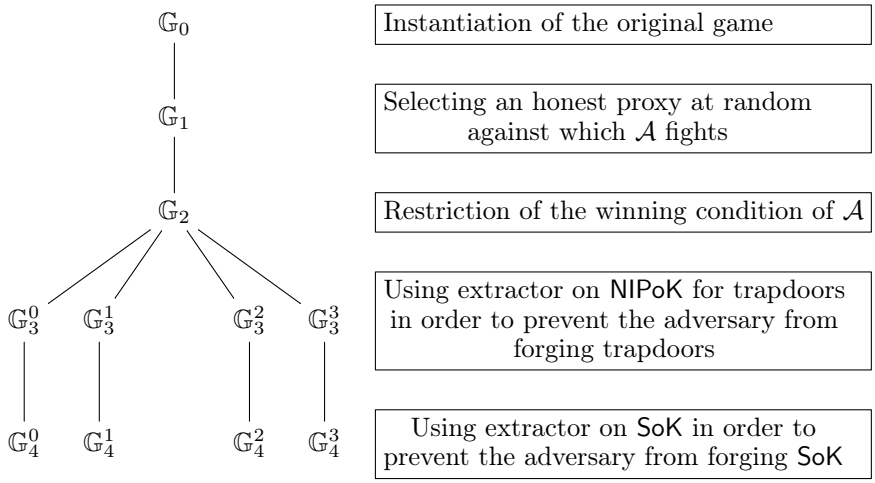

FIGURE 27 – Game-schedule for the LI-compliance property

the same session as the proxy. We depicted the four algorithms in one picture (FIGURE 28), framing or highlighting the differences that appear between the different versions:

$\texttt{KeyExtract}(\pi_{\mathsf{Prox}}, \mathsf{LIC}.\overline{\mathsf{PK}})$

1 : Parse the instance transcript $\pi_{\mathsf{Prox}}.\mathsf{tr}$,
parse $\pi_{\mathsf{Prox}}.\mathsf{PNID}$ as (I, R) and $\pi_{\mathsf{Prox}}.\mathsf{AID}$ as $(\Lambda_q)_{q=1}^n$.

2 : Set $y_1 = 0$. While $g^{y_1} \neq g^y$, do $y_1 = y_1 + 1$.

3 : Set $r_1 = 0$. While $g^{r_1} \neq g^r$, do $r_1 = r_1 + 1$.

4 : Set $i_1 = 0$. While $g^{i_1} \neq g^i$, do $i_1 = i_1 + 1$.

5 : Compute $g^{xy} \leftarrow (g^x)^{y_1} = (g^x)^y$, $g^{xr} \leftarrow (g^x)^{r_1} = (g^x)^r$,
$g^{yi} \leftarrow (g^y)^{i_1} = (g^y)^i$ and $\mathsf{PRK}_{\mathsf{out}}$ with the additional data contained in the transcript.

6 : Return $\mathsf{PRK}_{\mathsf{out}}$.

FIGURE 28 – The key-extractor algorithm. The hatched boxes $\cdots$ contain the additional elements for the ID = 1 version, the boxes $\cdots$ contain the additional elements for the ID = 2 version and the highlighted texts $\cdots$ indicate the additional elements for the ID = 3 version.

Depending on the version and on whose side we stand, the transcript of the instance $\pi_{\mathsf{Prox}}.\mathsf{tr}$ is parsed differently in the previous algorithm. In order, on the side of the Initiator we have:

- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|\mathsf{H_I}\|\mathsf{ni_I}\|S_\mathsf{I}\|c_4\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_I}$;

- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|\mathsf{H_I}\|\mathsf{H_I^1}\|\mathsf{ni_I}\|S_\mathsf{I}\|c_4\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_I}$;

- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|\mathsf{H_I}\|\mathsf{H_I^1}\|\mathsf{ni_I}\|S_\mathsf{I}\|c_4\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_I}$;

- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|\mathsf{H_I}\|\mathsf{H_I^1}\|\mathsf{H_I^2}\|\mathsf{ni_I}\|S_\mathsf{I}\|c_4\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_I}$.

And on the side of the Responder, always in order, we have:

- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|c_4\|\mathsf{H_R}\|\mathsf{ni_R}\|S_\mathsf{R}\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_R}$;
- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|c_4\|\mathsf{H_R}\|\mathsf{H_R^1}\|\mathsf{ni_R}\|S_\mathsf{R}\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_R}$;
- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|c_4\|\mathsf{H_R}\|\mathsf{H_R^1}\|\mathsf{ni_R}\|S_\mathsf{R}\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_R}$;
- $\pi_{\mathsf{Prox}}.\mathsf{tr} = \mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|c_4\|\mathsf{H_R}\|\mathsf{H_R^1}\|\mathsf{H_R^2}\|\mathsf{ni_R}\|S_\mathsf{R}\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_R}$.

We can now focus on the games hop.

† **Game $\mathbb{G}_0$:** This game is the original experience $\mathbb{E}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. We thus have:

$$\boxed{\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_0) = \mathbb{P}\Big(\mathcal{A} \text{ wins } \mathbb{E}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)\Big).}$$

† **Game $\mathbb{G}_1$:** This game is the same game as $\mathbb{G}_0$ except that we begin by picking at random $s \leftarrow\$ [\![1, q_\mathsf{R}]\!]$. Let P_j be the $j-$th oracle party output by `oRegister`. If $\mathcal{A}$ returns a tuple $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ such that $\mathsf{P}_s \neq \mathsf{Prox}_*$, then we abort and return 0. With this game hop, the winning advantage of the adversary is increased by a factor equaling the probability of guessing correctly the proxy P_s. We hence have:

$$\boxed{\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_0) \leq q_\mathsf{R}\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_1).}$$

† **Game $\mathbb{G}_2$:** Let $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ be the guess of $\mathcal{A}$. This game is the same as the previous one except that, using $\mathsf{sid}_* = (\mathsf{C}_{\mathsf{I},*}, \mathsf{C}_{\mathsf{R},*}, g_*^x, g_*^y)$ constructed from sst_* ($\mathsf{C}_{\mathsf{I},*}$ is drawn from m_{1*} and $\mathsf{C}_{\mathsf{R},*}$ from m_{2*}), if for all $k \in \mathbb{N}$, $\pi^k_{\mathsf{Prox}_*}.\mathsf{sid} \neq \mathsf{sid}_*$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{PNID} \neq (\mathsf{I}_*, \mathsf{R}_*)$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{AID} \neq (\Lambda_{*,q})_{q=1}^n$, then $\mathbb{G}_2$ aborts and returns 0. In this way, games $\mathbb{G}_1$ and $\mathbb{G}_2$ work the same way, unless the event F occurs, where F is the event: $\mathcal{A}$ returns a guess $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ such that for all $k \in \mathbb{N}$, $\pi^k_{\mathsf{Prox}_*}.\mathsf{sid} \neq \mathsf{sid}_*$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{PNID} \neq (\mathsf{I}_*, \mathsf{R}_*)$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{AID} \neq (\Lambda_{*,q})_{q=1}^n$.
By the identical-until-bad lemma we have:

$$\Big|\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_1) - \mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_2)\Big| \leq \mathbb{P}(F).$$

We now provide an upper bound for $\mathbb{P}(F)$. We claim that:

$$\mathbb{P}(F) \leq \mathsf{Adv}^{\mathrm{EUF\text{-}CMA}}_{\mathsf{DS}}(\lambda)$$

and hence we have:

$$\boxed{\Big|\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_1) - \mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_2)\Big| \leq \mathsf{Adv}^{\mathrm{EUF\text{-}CMA}}_{\mathsf{DS}}(\lambda).}$$

‡ **Proof of the claim:** We prove the claim by reduction, constructing a p.p.t. adversary $\mathcal{B}_1$ that breaks the EUF-CMA security property of DS.
Before constructing $\mathcal{B}_1$, assume that an adversary $\mathcal{A}$ wins game $\mathbb{G}_1$ with a non negligible probability $\epsilon_{\mathcal{A}}(\lambda)$ by returning a guess $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ such that for all $k \in \mathbb{N}$, $\pi^k_{\mathsf{Prox}_*}.\mathsf{sid} \neq \mathsf{sid}_*$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{PNID} \neq (\mathsf{I}_*, \mathsf{R}_*)$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{AID} \neq (\Lambda_{*,q})_{q=1}^n$. Note that if $\mathcal{A}$ wins $\mathbb{G}_1$, then $\mathsf{Prox}_*.\gamma = 0$ and $\mathtt{Verify}(\mathsf{pp}, \mathsf{sst}_*, \mathsf{I}_*.\mathsf{PK}, \mathsf{R}_*.\mathsf{PK}, \mathsf{Prox}_*.\mathsf{PK}, (\Lambda_{*,q})_{q=1}^n) = 1$, and according to $\mathbb{G}_1$, if $\mathsf{Prox}_* \neq \mathsf{P}_s$, the game returns 0.
We now present the construction of our adversary $\mathcal{B}_1$.
At the beginning, the challenger generates a pair of public/private signature keys $(\widehat{\mathsf{ssk}}, \widehat{\mathsf{spk}})$ with the $\mathsf{DS}.\mathtt{Gen}$ algorithm, and gives to $\mathcal{B}_1$ the public signing key $\widehat{\mathsf{spk}}$. Algorithm $\mathcal{B}_1$ wins the game if it sends back to its challenger a message/signature pair (m, σ) he did not generate and such that $\mathsf{DS}.\mathtt{Verif}(\widehat{\mathsf{spk}}, m, \sigma) = 1$.
In order to achieve this, $\mathcal{B}_1$ initializes an empty list $\mathsf{L_S} \leftarrow \emptyset$, and then simulates $\mathbb{G}_1$ to $\mathcal{A}(\mathsf{pp})$ faithfully, but with three exceptions:

- $\mathtt{oRegister}(\mathsf{P}, \mathsf{type}, \overline{\mathsf{PK}})$: if the s-th register query has $\mathsf{type} \neq \mathsf{proxy}$ ($\mathbb{G}_1$ requirement) or $\overline{\mathsf{PK}} \neq \{\perp\}$ (experiment requirement : we want this proxy to be honest, *i.e.,* not malicious , then $\mathcal{B}_1$ aborts. Else $\mathcal{B}_1$ sets $\mathsf{P}_s \leftarrow \mathsf{P}$ and $\mathsf{P}_s.\overline{\mathsf{PK}} \ni \mathsf{P}_s.\mathsf{spk} \leftarrow \widehat{\mathsf{spk}}$.

- $\mathtt{oSend}(\pi^q_{\mathsf{P}}, m)$: if $\mathsf{P} = \mathsf{Prox}$, $\mathcal{B}_1$ simulates $\mathtt{oSend}$ faithfully, but queries its $\mathtt{Sign}$ oracle (instead of the $\mathsf{DS}.\mathtt{Sign}$ algorithm) if needed: $\sigma \leftarrow \mathtt{Sign}(\tau)$, then adds (τ, σ) to $\mathsf{L_S}$ and sets $\mathsf{sst} \leftarrow \tau \| \sigma$ (as in the original protocol).

- $\mathsf{Corrupt}(\mathsf{P}, \ell)$: if $\mathsf{P} = \mathsf{P}_s$ and $\mathsf{P}.\overline{\mathsf{PK}}[l] = \widehat{\mathsf{spk}}$, then $\mathcal{B}_1$ aborts .

Finally $\mathcal{A}$ returns $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$, and $\mathcal{B}_1$ parses sst_* as $\tau_* \| \sigma_*$, sets $\mathsf{sid}_* = (\mathsf{C}_{\mathsf{I},*}, \mathsf{C}_{\mathsf{R},*}, g_*^x, g_*^y)$, and returns (τ_*, σ_*) to its challenger.
Note that the simulation of $\mathbb{G}_1$ by $\mathcal{B}_1$ for $\mathcal{A}$ is perfect.
We now consider the situation in which $\mathcal{A}$ wins the game $\mathbb{G}_1$ with respect to the conditions imposed for this reduction. In this case we have $\forall k \in \mathbb{N}$, $\pi^k_{\mathsf{Prox}_*}.\mathsf{sid} \neq \mathsf{sid}_*$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{PNID} \neq (\mathsf{I}_*, \mathsf{R}_*)$, or $\pi^k_{\mathsf{Prox}_*}.\mathsf{AID} \neq (\Lambda_{*,q})_{q=1}^n$. By interpreting this, we see that sst_* was returned by $\mathtt{oSend}$ on a run of the protocol in which no instances of Prox_* was involved. This thus means that $(\tau_*, \sigma_*) \notin \mathsf{L}_S$.
Moreover, at the beginning of the proof of this claim, we remind that in this situation (*i.e.,* $\mathcal{A}$ wins), we must have $\mathsf{Prox}_* = \mathsf{P}_s$ but also $\mathtt{Verify}(\mathsf{pp}, \mathsf{sst}_*, \mathsf{I}_*.\mathsf{PK}, \mathsf{R}_*.\mathsf{PK}, \mathsf{Prox}_*.\mathsf{PK}, (\Lambda_{*,q})_{q=1}^n) = 1$. Thanks to the description of algorithm $\mathtt{Verify}$, this previous equality means that we have in particular $\mathsf{DS}.\mathtt{Verif}(\mathsf{Prox}_*.\mathsf{spk}, \tau_*, \sigma_*) = 1$. In other words, σ_* is a valid signature for the message τ_* with the challenged key $\widehat{\mathsf{spk}}$. Bringing together these analysis, we obtain that if $\mathcal{A}$ wins (with the imposed conditions), *i.e.,* if F happens, then $\mathcal{B}_1$ wins its game, and breaks the EUF-CMA security of DS. This leads to $\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{B}_1}(\lambda) \geq \epsilon_{\mathcal{A}}(\lambda)$, and so:

$$\mathbb{P}(F) \leq \epsilon_{\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{B}_1}(\lambda)$$

which concludes the proof of the claim.

† **Game $\mathbb{G}_3$:** Let $\mathtt{Ext}_{\mathsf{td}}$ be the extractor of $\mathsf{NIPoK}\{d \ : \ D_1 = g_1^d \wedge D_2 = g_2^d\}$ in the $\mathtt{TDGen}$ algorithm, and $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ be the guess of $\mathcal{A}$. In order to prevent the adversary from forging proofs of knowledge (to ensure the security of the trapdoors) we insert in this game the extractor. So this game is the same game as $\mathbb{G}_2$, except that $\mathbb{G}_3$ starts by initializing an empty list $\mathsf{L}[\,] \leftarrow \emptyset$ and:

- $\rightsquigarrow$ For each query to `oRevealTD` on input $(\mathsf{sst}, \mathsf{I}, \mathsf{R}, \mathsf{Prox}, (\Lambda_q)_{q=1}^n, \ell)$ (to get $\Lambda_\ell.\mathsf{td}$), game $\mathbb{G}_3$ sets $\mathsf{L}[\Lambda_\ell.\mathsf{td}] \leftarrow \Lambda_\ell.\mathsf{SK}$;

- $\rightsquigarrow$ After that $\mathcal{A}$ returns its guess, $\mathbb{G}_3$ sets $\omega_* \leftarrow (\Lambda_{*,q})_{q=1}^n \| \mathsf{ID}_*$ (either $\mathsf{ID}_{\mathsf{I}*}$ or $\mathsf{ID}_{\mathsf{R}*}$) and parses sst_* as $\omega_* \| \mathsf{MCS}_* \| X_* \| Y_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{ni}_* \| \mathsf{pid} \| n_{r*} \| \mathsf{val}_* \| \sigma_*$ (with pid either $\mathsf{ID}_{\mathsf{R}*}$ or $\mathsf{ID}_{\mathsf{I}*}$), and sets $\mathsf{A}_0 \leftarrow X_*$ and $\mathsf{A}_1 \leftarrow Y_*$.

- $\rightsquigarrow$ For each $q \in [\![1, n]\!]$ such that $\mathsf{L}[\Lambda_{*,q}.\mathsf{td}] = \perp$, it parses $\Lambda_{*,q}.\mathsf{PK} \in \Lambda_{*,q}.\overline{\mathsf{PK}}$ as $(\Lambda_{*,q}.\mathsf{pk}, \Lambda_{*,q}.\mathsf{ni})$ and $\Lambda_{*,q}.\mathsf{td}$ as $(\Lambda_{*,q}.\mathsf{td}_0, \Lambda_{*,q}.\mathsf{td}_1)$.
  If $\mathsf{NIPoKver}((g, \Lambda_{*,q}.\mathsf{pk}, \mathsf{A}_\mathsf{b}, \Lambda_{*,q}.\mathsf{td}_1), \Lambda_{*,q}.\mathsf{td}_0) = 1$ (with $b = 0$ or 1 depending on the point of view), then $\mathbb{G}_3$ runs $\mathtt{Ext}_{\mathsf{td}}(\lambda)$ on $\mathcal{A}$ in order to extract a witness δ for the proof $\Lambda_{*,q}.\mathsf{td}_0$ and sets $\mathsf{L}[\Lambda_{*,q}.\mathsf{td}] \leftarrow \delta$. Else $\mathbb{G}_3$ aborts and returns 0.
  Following this, if $g^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]} \neq \Lambda_{*,q}.\mathsf{pk}$ or $\mathsf{A}_\mathsf{b}{}^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]} \neq \Lambda_{*,q}.\mathsf{td}_1$, then $\mathbb{G}_3$ aborts and returns 0.

So with this game hop we capture the possibility of $\mathtt{Ext}_{\mathsf{td}}$ failing when extracting a valid witness, in any of the (at most) q_R calls. We deduce that:

$$\boxed{\Big|\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_2\big) - \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_3\big)\Big| \leq q_\mathsf{R} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda).}$$

Note that this game uses the notations of the $\mathsf{ID} = 0$ version of the protocol (some we may have write $\mathbb{G}_3^0$ instead of $\mathbb{G}_3$). But for the three other versions, the same analysis can be performed, leading to the same result, because only the notations for NIPoK, $\mathsf{NIPoKver}$, sst and $\Lambda_{*,q}.\mathsf{td}$ change (but it does not impact the above analysis). Thus, in all cases, we obtain this conclusion.

† **Game $\mathbb{G}_4^0$:** Let $\mathtt{Ext}_{\mathsf{SoK}}$ be the extractor of the signature of knowledge. In order to prevent the adversary from providing a forged signature of knowledge, this game is the same as the previous one, except that it starts by initializing an empty list $\mathsf{L}'[\,] \leftarrow \emptyset$ and:

- $\rightsquigarrow$ Each time the `oSend` oracle generates a SoK: $\mathsf{ni} \leftarrow \mathsf{SoK}_m\{z : (Z = g^z \wedge \mathsf{H} = (\mathsf{hA})^z)\}$, it sets $\mathsf{L}'[(g, Z, \mathsf{hA}, \mathsf{H}, m, \mathsf{ni})] \leftarrow z$ (where A is an ephemeral DH secret of the point of view considered);

- $\rightsquigarrow$ After $\mathcal{A}$ returns its guess, $\mathbb{G}_4^0$ sets $\omega_* \leftarrow (\Lambda_{*,q})_{q=1}^n \| \mathsf{ID}_*$ (either $\mathsf{ID}_{\mathsf{I}*}$ or $\mathsf{ID}_{\mathsf{R}*}$) and parses sst_* as $\omega_* \| \mathsf{MCS}_* \| X_* \| Y_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{ni}_* \| \mathsf{pid}_* \| n_{r*} \| \mathsf{val}_* \| \sigma_*$ (with pid_* either $\mathsf{ID}_{\mathsf{R}*}$ or $\mathsf{ID}_{\mathsf{I}*}$), sets $\mathsf{A}_0 \leftarrow X_*$ and $\mathsf{A}_1 \leftarrow Y_*$, parses $\Lambda_{*,q}.\mathsf{PK} \in \Lambda_{*,q}.\overline{\mathsf{PK}}$ as $(\Lambda_{*,q}.\mathsf{pk}, \Lambda_{*,q}.\mathsf{ni})$ and $\Lambda_{*,q}.\mathsf{td}$ as $(\Lambda_{*,q}.\mathsf{td}_0, \Lambda_{*,q}.\mathsf{td}_1)$, and sets $\mathsf{h}_* \leftarrow \prod_{q=1}^n \Lambda_{*,q}.\mathsf{pk}$.
  If $\mathsf{L}'[(g, \mathsf{A}_\mathsf{b}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}}, \mathsf{H}_*, m_*, \mathsf{ni}_*)] = \perp$ (here $m_* = \omega_* \| \mathsf{pid}_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*}$), then if $\mathsf{SoKver}(m_*, (g, \mathsf{A}_\mathsf{b}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}}, \mathsf{H}_*), \mathsf{ni}_*) = 1$, $\mathbb{G}_4^0$ runs $\mathtt{Ext}_{\mathsf{SoK}}(\lambda)$ on $\mathcal{A}$, in order to extract a witness η for the proof ni_* and sets $\mathsf{L}'[(g, \mathsf{A}_\mathsf{b}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}}, \mathsf{H}_*, m_*, \mathsf{ni}_*)] \leftarrow \eta$. Else it aborts, returning 0 (which leads `Verify` to output 0 for sst_* in this case, since the SoKver algorithm will have returns 0). Following this, if $\mathsf{H}_* \neq (\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}})^{\mathsf{L}'[(g, \mathsf{A}_\mathsf{b}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}}, \mathsf{H}_*, m_*, \mathsf{ni}_*)]}$ or if $\mathsf{A}_\mathsf{b} \neq g^{\mathsf{L}'[(g, \mathsf{A}_\mathsf{b}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}}, \mathsf{H}_*, m_*, \mathsf{ni}_*)]}$, then $\mathbb{G}_4^0$ aborts and returns 0.

In this game we pointed out the probability of $\mathtt{Ext}_{\mathsf{SoK}}$ failing to extract a valid witness when it is called in any of the (at most) q_S calls. We thus obtain:

$$\boxed{\Big|\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_3^0\big) - \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_4^0\big)\Big| \leq q_\mathsf{S} \cdot \epsilon_{\mathsf{SoK}}(\lambda).}$$

To conclude this branch of the game schedule, we now show that $\mathcal{A}$ always loses the game $\mathbb{G}_4^0$ against the `KeyExtract` algorithm we build. For that we reason through the absurd and we assume that the adversary $\mathcal{A}$ wins the game $\mathbb{G}_4^0$ with a non-zero probability. Denoting $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ the guess of $\mathcal{A}$, we parse $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{tr}$ as $\mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|\mathsf{H_I}\|\mathsf{ni_I}\|S_\mathsf{I}\|c_4\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_I}$ (or as $\mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|c_4\|\mathsf{H_R}\|\mathsf{ni_R}\|S_\mathsf{R}\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_R}$ depending on the point of view), $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{PNID}$ as (I, R) and $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{AID}$ as $(\Lambda_q)_{q=1}^n$. On the other hand, we parse sst_* as $(\Lambda_{*,q})_{q=1}^{n_*}\|\mathsf{ID}_*\|\mathsf{MCS}_*\|X_*\|Y_*\|\mathsf{TH}_{2*}\|\mathsf{TH}_{3*}\|\mathsf{TH}_{4*}\|M_{1*}\|m_{2*}\|m_{3*}\|\mathsf{H}_*\|\mathsf{ni}_*\|\mathsf{pid}_*\|n_{r*}\|\mathsf{val}_*\|\sigma_*$. Thanks to the game $\mathbb{G}_2$, we know that $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{sid} = (\mathsf{C}_{\mathsf{I},*}, \mathsf{C}_{\mathsf{R},*}, X_*, Y_*)$, $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{PNID} = (\mathsf{I}_*, \mathsf{R}_*)$ and $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{AID} = (\Lambda_{*,q})_{q=1}^{n_*}$, which implies that:

$$X_* = X \text{ and } Y_* = Y,\ \mathsf{I}_* = \mathsf{I}, \mathsf{R}_* = \mathsf{R},\ \text{and } (\Lambda_q)_{q=1}^n = (\Lambda_{*,q})_{q=1}^{n_*}.$$

By construction, the `KeyExtract` algorithm returns ($\mathsf{PRK_{out}}$ but temporarily) $\mathsf{k} = (g^x)^y = X^y$ where $Y = g^y$. At this point, note that an ambiguous element is n_{r*}. Indeed, $\mathcal{A}$ may have compute a key with some random n_r and return a false n_r' to the proxy which stores this value in sst_*. This is always possible, but useless for the adversary. Indeed, since our proxy is supposed to be honest, he will perform the test $\mathsf{TH_5} = \mathcal{H}(\mathsf{TH}_{4*}, \mathsf{val}_*, n_{r*})$ to check that he received the correct n_{r*} (the proxy has compute himself $\mathsf{TH_4}$, and if he couldn't the session would not have run until the end, so we would have `IdentifySession` returning 0). Despite this test, $\mathcal{A}$ may still compute $\mathsf{PRK_{out}}$ with another random. But in this situation, this computed key $\mathsf{PRK_{out}}$ is not valid or useless since only computable by one endpoint. And, in anyway, the `Open` algorithm checks this problem with val_*, n_{r*} and k, returning $\perp$ if $\mathcal{A}$ cheats on this value and making $\mathcal{A}$ loosing the game in this situation. So we now assume that n_{r*} is not a fake value. We set $\mathsf{h} \leftarrow \prod_{q=1}^n \Lambda_q.\mathsf{pk}$ and $m_* = (\Lambda_q)_{q=1}^n\|\mathsf{ID}_*\|\mathsf{pid}_*\|\mathsf{TH}_{2*}\|\mathsf{TH}_{3*}\|\mathsf{TH}_{4*}$. We also define a_0 and a_1 so that $\mathsf{A_b} = g^{a_\mathsf{b}}$.
Running the `Open` algorithm on input $(\mathsf{pp}, \mathsf{sst}_*, (\Lambda_{*,q}.\mathsf{td})_{q=1}^{n_*}, (\Lambda_{*,q}.\mathsf{PK})_{q=1}^{n_*})$ gives us:

$$\begin{aligned}
\mathsf{k}_* &= \frac{\mathsf{H}_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td_1}} = \frac{(\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}})^{\mathsf{L}'[(g,\mathsf{A_b},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}},\mathsf{H}_*,m_*,\mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} \mathsf{A_b}^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g,\mathsf{A_b},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}},\mathsf{H}_*,m_*,\mathsf{ni}_*)]}(g^{a_{1-\mathsf{b}}})^{\mathsf{L}'[(g,\mathsf{A_b},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}},\mathsf{H}_*,m_*,\mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} g^{a_\mathsf{b}\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*}(\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g,\mathsf{A_b},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}},\mathsf{H}_*,m_*,\mathsf{ni}_*)]}\right)\mathsf{A_b}^{a_{1-\mathsf{b}}}}{\prod_{q=1}^{n_*}(\Lambda_{*,q}.\mathsf{pk})^{a_\mathsf{b}}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g,\mathsf{A_b},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}},\mathsf{H}_*,m_*,\mathsf{ni}_*)]}\right)\mathsf{k}}{\prod_{q=1}^{n_*}(\Lambda_{*,q}.\mathsf{pk})^{a_\mathsf{b}}} \\
&= \frac{\left(\prod_{q=1}^{n_*} \mathsf{A_b}^{(\Lambda_{*,q}.\mathsf{SK})}\right)\mathsf{k}}{\prod_{q=1}^{n_*}(\Lambda_{*,q}.\mathsf{pk})^{a_\mathsf{b}}} = \mathsf{k}
\end{aligned}$$

This implies that $\mathsf{k}_* = \mathsf{k}$ (and thus $\mathsf{PRK_{out*}} = \mathsf{PRK_{out}}$ (note that $\mathcal{A}$ cannot cheat on transcript hashes TH_* by returning different ones in sst_* from those used in the protocol (and stored in tr), otherwise `Verify` detects it and $\mathcal{A}$ looses) with probability 1, and so $\mathcal{A}$ fails the game $\mathbb{G}_4^0$. Absurd!
So $\mathcal{A}$ always fails the game $\mathbb{G}_4^0$, and thus we have:

$$\boxed{\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_4^0) = 0.}$$

† **Game $\mathbb{G}_4^1$:** Again we denote by $\mathtt{Ext}_{\mathsf{SoK}}$ be the extractor of the signature of knowledge. This game has the same purpose as $\mathbb{G}_4^0$, and works as the previous one, except that it starts by initializing an empty list $\mathsf{L}'[\,] \leftarrow \emptyset$ and:

$\rightsquigarrow$ Each time the oSend oracle generates an SoK: $\mathsf{ni} \leftarrow \mathsf{SoK}_m\{z, (Z = g^z \wedge \mathsf{H} = (\mathsf{h}Y)^z) \wedge \mathsf{H}^1 = (\mathsf{h}R)^z\}$, it sets $\mathsf{L}'[(g, Z, \mathsf{h}Y, \mathsf{H}, \mathsf{h}R, \mathsf{H}^1, m, \mathsf{ni})] \leftarrow z$ (or $\mathsf{ni} \leftarrow \mathsf{SoK}_m\{(z, z'), (Z = g^z \wedge \mathsf{H} = (\mathsf{h}X)^z) \wedge Z' = g^{z'} \wedge \mathsf{H}^1 = (\mathsf{h}X)^{z'}\}$, it sets $\mathsf{L}'[(g, Z, \mathsf{h}X, \mathsf{H}, Z', \mathsf{h}X, \mathsf{H}^1, m, \mathsf{ni})] \leftarrow (z, z')$, depending on the side we consider (but this does not impact the analysis));

$\rightsquigarrow$ After $\mathcal{A}$ returns its guess, $\mathbb{G}_4^1$ sets $\omega_* \leftarrow (\Lambda_{*,q})_{q=1}^n \| \mathsf{ID}_*$ (either $\mathsf{ID}_{\mathsf{I}*}$ or $\mathsf{ID}_{\mathsf{R}*}$) and parses sst_* as $\omega_* \| \mathsf{MCS}_* \| X_* \| Y_* \| R_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{H}_*^1 \| \mathsf{ni}_* \| \mathsf{pid}_* \| n_{r*} \| \mathsf{val}_* \| \sigma_*$ (with pid_* either $\mathsf{ID}_{\mathsf{R}*}$ or $\mathsf{ID}_{\mathsf{I}*}$), sets $\mathsf{A}_0 \leftarrow X_*$ and $\mathsf{A}_1 \leftarrow Y_*$, parses $\Lambda_{*,q}.\mathsf{PK} \in \Lambda_{*,q}.\overline{\mathsf{PK}}$ as $(\Lambda_{*,q}.\mathsf{pk}, \Lambda_{*,q}.\mathsf{ni})$ and $\Lambda_{*,q}.\mathsf{td}$ as $(\Lambda_{*,q}.\mathsf{td}_0, \Lambda_{*,q}.\mathsf{td}_1, \Lambda_{*,q}.\mathsf{td}_2)$, and sets $\mathsf{h}_* \leftarrow \prod_{q=1}^n \Lambda_{*,q}.\mathsf{pk}$.
If $\mathsf{L}'[(g, X_*, \mathsf{h}_*Y_*, \mathsf{H}_*, \mathsf{h}_*R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)] = \perp$ (or $\mathsf{L}'[(g, Y_*, \mathsf{h}_*X_*, \mathsf{H}_*, R_*, \mathsf{h}_*X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)] = \perp$), then:

- If $\mathsf{SoKver}(m_*, (g, X_*, \mathsf{h}_*Y_*, \mathsf{H}_*, R_*, \mathsf{h}_*R_*, \mathsf{H}_*^1), \mathsf{ni}_*) = 1$ (or if $\mathsf{SoKver}(m_*, (g, Y_*, \mathsf{h}_*X_*, \mathsf{H}_*, R_*, \mathsf{h}_*X_*, \mathsf{H}_*^1), \mathsf{ni}_*) = 1$), $\mathbb{G}_4^1$ runs $\mathtt{Ext}_{\mathsf{SoK}}(\lambda)$ on $\mathcal{A}$, in order to extract a witness η (or witnesses (η, η')) for the proof ni_* and sets $\mathsf{L}'[(g, X_*, \mathsf{h}_*Y_*, \mathsf{H}_*, \mathsf{h}_*R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)] \leftarrow \eta$ (or $\mathsf{L}'[(g, Y_*, \mathsf{h}_*X_*, \mathsf{H}_*, R_*, \mathsf{h}_*X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)] \leftarrow (\eta, \eta')$). Else it aborts, returning 0 (for the same reason, in this situation Verify fails on sst_*);

- Following this, if $\mathsf{H}_* \neq (\mathsf{h}_*Y_*)^{\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)]}$ or if $\mathsf{H}_*^1 \neq (\mathsf{h}_*R_*)^{\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)]}$ or if $X_* \neq g^{\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)]}$ (*i.e.,* extraction fails) (or if $\mathsf{H}_* \neq (\mathsf{h}_*X_*)^{\mathsf{L}'[(g,Y_*,\mathsf{h}_*X_*,\mathsf{H}_*,R_*,\mathsf{h}_*X_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)][0]}$ or if $\mathsf{H}_*^1 \neq (\mathsf{h}_*X_*)^{\mathsf{L}'[(g,Y_*,\mathsf{h}_*X_*,\mathsf{H}_*,R_*,\mathsf{h}_*X_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)][1]}$ or if $Y_* \neq g^{\mathsf{L}'[(g,Y_*,\mathsf{h}_*X_*,\mathsf{H}_*,R_*,\mathsf{h}_*X_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)][0]}$ or if $R_* \neq g^{\mathsf{L}'[(g,Y_*,\mathsf{h}_*X_*,\mathsf{H}_*,R_*,\mathsf{h}_*X_*,\mathsf{H}_*^1,m_*,\mathsf{ni}_*)][1]}$), then $\mathbb{G}_4^1$ aborts and returns 0.

Again, with this game we pointed out the probability of $\mathtt{Ext}_{\mathsf{SoK}}$ failing to extract valid witness(es) in any of (at most) q_{S} calls. We thus deduce that:

$$\boxed{\left|\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_3^1) - \mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_4^1)\right| \leq q_{\mathsf{S}} \cdot \epsilon_{\mathsf{SoK}}(\lambda).}$$

As previously, to conclude this branch of the game schedule, we now show that $\mathcal{A}$ always loses the game $\mathbb{G}_4^1$ against the KeyExtract algorithm we build. For that we reason through the absurd and we assume that the adversary $\mathcal{A}$ wins the game $\mathbb{G}_4^1$ with a non-zero probability. Denoting $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ the guess of $\mathcal{A}$, we parse $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{tr}$ as $\mathsf{MCS} \| g^x \| \mathsf{C}_\mathsf{I} \| \mathsf{EAD}_1 \| g^y \| c_2 \| c_3 \| \mathsf{H}_\mathsf{I} \| \mathsf{H}_\mathsf{I}^1 \| \mathsf{ni}_\mathsf{I} \| S_\mathsf{I} \| c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{sst}_\mathsf{I}$ (or as $\mathsf{MCS} \| g^x \| \mathsf{C}_\mathsf{I} \| \mathsf{EAD}_1 \| g^y \| c_2 \| c_3 \| c_4 \| \mathsf{H}_\mathsf{R} \| \mathsf{H}_\mathsf{R}^1 \| \mathsf{ni}_\mathsf{R} \| S_\mathsf{R} \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{sst}_\mathsf{R}$ depending on the point of view), $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{PNID}$ as (I, R) and $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{AID}$ as $(\Lambda_q)_{q=1}^n$. On the other hand, we parse sst_* as $(\Lambda_{*,q})_{q=1}^{n_*} \| \mathsf{ID}_* \| \mathsf{MCS}_* \| X_* \| Y_* \| R_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{H}_*^1 \| \mathsf{ni}_* \| \mathsf{pid}_* \| n_{r*} \| \mathsf{val}_* \| \sigma_*$.
Thanks to the game $\mathbb{G}_2$, we know that $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{sid} = (\mathsf{C}_{\mathsf{I}*}, \mathsf{C}_{\mathsf{R}*}, X_*, Y_*)$, $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{PNID} = (\mathsf{I}_*, \mathsf{R}_*)$ and $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{AID} = (\Lambda_{*,q})_{q=1}^{n_*}$, which implies that:

$$X_* = X \text{ and } Y_* = Y,\ \mathsf{I}_* = \mathsf{I}, \mathsf{R}_* = \mathsf{R}, \text{ and } (\Lambda_q)_{q=1}^n = (\Lambda_{*,q})_{q=1}^{n_*}$$

and note that since $\mathsf{R}_* = \mathsf{R}$, we also have $R_* = R$.
By construction, the KeyExtract algorithm returns ($\mathsf{PRK}_{\mathsf{out}}$ but temporarily) $\mathsf{k} = (g^x)^y = X^y$ where $Y = g^y$ and $\mathsf{k}' = (g^x)^r = X^r$ where $R = g^r$. At this point, as in $\mathbb{G}_4^0$, we have the same ambiguity on n_r, and we solved it in the same way. We set $\mathsf{h} \leftarrow \prod_{q=1}^n \Lambda_q.\mathsf{pk}$ and $m_* = (\Lambda_q)_{q=1}^n \| \mathsf{ID}_* \| \mathsf{pid}_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*}$. We also define a_0 and a_1 so that $\mathsf{A}_\mathsf{b} = g^{a_\mathsf{b}}$ and r_1 so that $R_* = g^{r_1}$.

Running the $\mathtt{Open}$ algorithm on input $(\mathsf{pp}, \mathsf{sst}_*, (\Lambda_{*,q}.\mathsf{td})_{q=1}^{n_*}, (\Lambda_{*,q}.\mathsf{PK})_{q=1}^{n_*})$ gives us:

$$\begin{aligned}
\mathsf{k}_* &= \frac{\mathsf{H}_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* Y_*)^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} X_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]} (g^{a_1})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} g^{a_0 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}\right) X_*^{a_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK}) \mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} X_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} = \mathsf{k}
\end{aligned}$$

and

$$\begin{aligned}
\mathsf{k}'_* &= \frac{\mathsf{H}_*^1}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* R_*)^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} X_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]} (g^{r_1})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} g^{a_0 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}\right) X_*^{r_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK}) \mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, \mathsf{h}_* R_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)]}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} X_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} = \mathsf{k}'
\end{aligned}$$

if we are on the $\mathsf{Initiator}$ point of view, or, if we are the $\mathsf{Responder}$ point of view:

$$\begin{aligned}
\mathsf{k}_* &= \frac{\mathsf{H}_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* X_*)^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} \mathsf{Y}_*{}^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]} (g^{a_0})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} g^{a_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}\right) \mathsf{Y}_*{}^{a_0}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK}) \mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} Y_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} = \mathsf{k}
\end{aligned}$$

and

$$
\begin{aligned}
\mathsf{k}'_* &= \frac{\mathsf{H}^1_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_2} = \frac{(\mathsf{h}_* X_*)^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} R_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]} (g^{a_0})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} g^{r_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}\right) R_*^{a_0}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{r_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, R_*, \mathsf{h}_* X_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{r_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} R_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{r_1}} = \mathsf{k}'
\end{aligned}
$$

This implies that $\mathsf{k}_* = \mathsf{k}$ and $\mathsf{k}'_* = \mathsf{k}'$ (and thus $\mathsf{PRK}_{\mathsf{out}*} = \mathsf{PRK}_{\mathsf{out}}$) with probability 1, and so $\mathcal{A}$ fails the game $\mathbb{G}_4^1$. Absurd!
So $\mathcal{A}$ always fails the game $\mathbb{G}_4^1$, and thus we have:

$$\boxed{\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_4^1) = 0.}$$

† **Game $\mathbb{G}_4^2$:** In a similar way as for previous games, we denote by $\mathtt{Ext}_{\mathsf{SoK}}$ the extractor of the signature of knowledge. Again this game is the same as the previous one, except that it starts by initializing an empty list $\mathsf{L}'[\,] \leftarrow \emptyset$ and:

- $\rightsquigarrow$ Each time the `oSend` oracle generates an SoK: $\mathsf{ni} \leftarrow \mathsf{SoK}_m\{(z, z'), (Z = g^z \wedge \mathsf{H} = (\mathsf{h}Y)^z) \wedge Z' = g^{z'} \wedge \mathsf{H}^1 = (\mathsf{h}Y)^{z'}\}$, it sets $\mathsf{L}'[(g, Z, \mathsf{h}Y, \mathsf{H}, Z', \mathsf{h}Y, \mathsf{H}^1, m, \mathsf{ni})] \leftarrow (z, z')$ (or $\mathsf{ni} \leftarrow \mathsf{SoK}_m\{z, (Z = g^z \wedge \mathsf{H} = (\mathsf{h}X)^z) \wedge \mathsf{H}^1 = (\mathsf{h}I)^z\}$, it sets $\mathsf{L}'[(g, Z, \mathsf{h}X, \mathsf{H}, \mathsf{h}I, \mathsf{H}^1, m, \mathsf{ni})] \leftarrow z$ depending on the side we consider (but this does not impact the analysis));

- $\rightsquigarrow$ After $\mathcal{A}$ returns its guess, $\mathbb{G}_4^2$ sets $\omega_* \leftarrow (\Lambda_{*,q})_{q=1}^n \| \mathsf{ID}_*$ (either $\mathsf{ID}_{\mathsf{I}*}$ or $\mathsf{ID}_{\mathsf{R}*}$) and parses sst_* as $\omega_* \| \mathsf{MCS}_* \| X_* \| Y_* \| I_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{H}^1_* \| \mathsf{ni}_* \| \mathsf{pid}_* \| n_{r*} \| \mathsf{val}_* \| \sigma_*$ (with pid_* either $\mathsf{ID}_{\mathsf{R}*}$ or $\mathsf{ID}_{\mathsf{I}*}$), sets $\mathsf{A}_0 \leftarrow X_*$ and $\mathsf{A}_1 \leftarrow Y_*$, parses $\Lambda_{*,q}.\mathsf{PK}$ as $(\Lambda_{*,q}.\mathsf{pk}, \Lambda_{*,q}.\mathsf{ni})$ and $\Lambda_{*,q}.\mathsf{td}$ as $(\Lambda_{*,q}.\mathsf{td}_0, \Lambda_{*,q}.\mathsf{td}_1, \Lambda_{*,q}.\mathsf{td}_2)$, and sets $\mathsf{h}_* \leftarrow \prod_{q=1}^n \Lambda_{*,q}.\mathsf{pk}$.
  If $\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)] = \perp$ (or $\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)] = \perp$), then:
  - If $\mathsf{SoKver}(m_*, (g, X*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*), \mathsf{ni}_*) = 1$ (or if $\mathsf{SoKver}(m_*, (g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, I_*, \mathsf{h}_* I_*, \mathsf{H}^1_*), \mathsf{ni}_*) = 1$), $\mathbb{G}_4^2$ runs $\mathtt{Ext}_{\mathsf{SoK}}(\lambda)$ on $\mathcal{A}$ in order to extract witnesses (η, η') (or a witness η) for the proof ni_* and sets $\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)] \leftarrow (\eta, \eta')$ (or $\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)] \leftarrow \eta$). Else it aborts, returning 0 (in this situation again `Verify` fails on sst_*);
  - Following this, if $\mathsf{H}_* \neq (\mathsf{h}_* Y_*)^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][0]}$ or if $\mathsf{H}^1_* \neq (\mathsf{h}_* Y_*)^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}$ or if $X_* \neq g^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][0]}$ or if $I_* \neq g^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}$ (*i.e.,* extraction fails) (or if $\mathsf{H}_* \neq (\mathsf{h}_* X_*)^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}$ or if $\mathsf{H}^1_* \neq (\mathsf{h}_* I_*)^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}$ or if $Y_* \neq g^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}$), then $\mathbb{G}_4^2$ aborts and returns 0.

With this game hop, we again pointed out the probability of $\mathtt{Ext}_{\mathsf{SoK}}$ failing to extract valid witness(es) in any of (at most) q_{S} calls. We thus have:

$$\boxed{\left|\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_3^2\big) - \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_4^2\big)\right| \leq q_{\mathsf{S}} \cdot \epsilon_{\mathsf{SoK}}(\lambda).}$$

Here again to conclude this branch of the game schedule, we show that $\mathcal{A}$ always loses the game $\mathbb{G}_4^2$ against the `KeyExtract` algorithm we build. For that we reason through the absurd and we assume that the adversary $\mathcal{A}$ wins the game $\mathbb{G}_4^2$ with a non-zero probability. Denoting $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,i}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ the guess of $\mathcal{A}$, we parse $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{tr}$ as $\mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|\mathsf{H_I}\|\mathsf{H_I^1}\|\mathsf{ni_I}\|S_\mathsf{I}\|c_4\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_I}$ (or as $\mathsf{MCS}\|g^x\|\mathsf{C_I}\|\mathsf{EAD_1}\|g^y\|c_2\|c_3\|c_4\|\mathsf{H_R}\|\mathsf{H_R^1}\|\mathsf{ni_R}\|S_\mathsf{R}\|n_r\|\mathsf{val}\|\mathsf{TH_5}\|\mathsf{sst_R}$ depending on the point of view), $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{PNID}$ as (I, R) and $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{AID}$ as $(\Lambda_q)_{q=1}^n$. On the other hand, we parse sst_* as $(\Lambda_{*,q})_{q=1}^{n_*}\|\mathsf{ID}_*\|\mathsf{MCS}_*\|X_*\|Y_*\|I_*\|\mathsf{TH_{2*}}\|\mathsf{TH_{3*}}\|\mathsf{TH_{4*}}\|M_{1*}\|m_{2*}\|m_{3*}\|\mathsf{H}_*\|\mathsf{H}_*^1\|\mathsf{ni}_*\|\mathsf{pid}_*\|n_{r*}\|\mathsf{val}_*\|\sigma_*$.
Thanks to the game $\mathbb{G}_2$, we know that $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{sid} = (\mathsf{C_{I*}}, \mathsf{C_{R*}}, X_*, Y_*)$, $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{PNID} = (\mathsf{I}_*, \mathsf{R}_*)$ and $\pi^{j_*}_{\mathsf{Prox}_*}.\mathsf{AID} = (\Lambda_{*,q})_{q=1}^{n_*}$, which implies that:

$$X_* = X \text{ and } Y_* = Y,\ \mathsf{I}_* = \mathsf{I}, \mathsf{R}_* = \mathsf{R},\ \text{and } (\Lambda_q)_{m=1}^n = (\Lambda_{*,q})_{q=1}^{n_*}$$

and note that since $\mathsf{I}_* = \mathsf{I}$, we also have $I_* = I$. By construction, the `KeyExtract` algorithm returns ($\mathsf{PRK_{out}}$ but temporarily) $\mathsf{k} = (g^x)^y = X^y$ where $Y = g^y$ and $\mathsf{k}' = (g^y)^i = Y^i$ where $I = g^i$. At this point, as in $\mathbb{G}_4^0$, we have the same ambiguity on n_r, and we solve it in the same way. We set $\mathsf{h} \leftarrow \prod_{q=1}^n \Lambda_q.\mathsf{pk}$ and $m_* = (\Lambda_q)_{q=1}^n\|\mathsf{ID}_*\|\mathsf{pid}_*\|\mathsf{TH_{2*}}\|\mathsf{TH_{3*}}\|\mathsf{TH_{4*}}$. We also define a_0 and a_1 so that $\mathsf{A_b} = g^{a_\mathsf{b}}$ and i_1 so that $I_* = g^{i_1}$.
Running the `Open` algorithm on input $(\mathsf{pp}, \mathsf{sst}_*, (\Lambda_{*,q}.\mathsf{td})_{q=1}^{n_*}, (\Lambda_{*,q}.\mathsf{PK})_{q=1}^{n_*})$ gives us:

$$\begin{aligned}
\mathsf{k}_* &= \frac{\mathsf{H}_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td_1}} = \frac{(\mathsf{h}_* Y_*)^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} \mathsf{X}_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]} (g^{a_1})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} g^{a_0 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}\right) \mathsf{X}_*^{a_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}_*^1, m_*, \mathsf{ni}_*)][0]}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} X_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} = \mathsf{k}
\end{aligned}$$

and

$$
\begin{aligned}
\mathsf{k}'_* &= \frac{\mathsf{H}^1_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_2} = \frac{(\mathsf{h}_* Y_*)^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} I_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, X_*, \mathsf{h}_* y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]} (g^{a_1})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} g^{i_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}\right) I_*^{a_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{i_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g, X_*, \mathsf{h}_* Y_*, \mathsf{H}_*, I_*, \mathsf{h}_* Y_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)][1]}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{i_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} I_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{i_1}} = \mathsf{k}'
\end{aligned}
$$

if we are on the Initiator point of view, or, if we are the Responder point of view:

$$
\begin{aligned}
\mathsf{k}_* &= \frac{\mathsf{H}_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* X_*)^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} Y_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]} (g^{a_0})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} g^{a_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}\right) Y_*^{a_0}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} Y_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} = \mathsf{k}
\end{aligned}
$$

and

$$
\begin{aligned}
\mathsf{k}'_* &= \frac{\mathsf{H}^1_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* I_*)^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} Y_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\mathsf{h}_*^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]} (g^{i_1})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}}{\prod_{q=1}^{n_*} g^{a_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}\right) Y_*^{i_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g, Y_*, \mathsf{h}_* X_*, \mathsf{H}_*, \mathsf{h}_* I_*, \mathsf{H}^1_*, m_*, \mathsf{ni}_*)]}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} Y_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} = \mathsf{k}'
\end{aligned}
$$

This implies that $\mathsf{k}_* = \mathsf{k}$ and $\mathsf{k}'_* = \mathsf{k}'$ (and thus $\mathsf{PRK}_{\mathsf{out}*} = \mathsf{PRK}_{\mathsf{out}}$) with probability 1, and so $\mathcal{A}$ fails the game $\mathbb{G}_4^2$. Absurd!
So $\mathcal{A}$ always fails the game $\mathbb{G}_4^2$, and thus we obtain that:

$$\boxed{\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_4^2) = 0.}$$

† **Game $\mathbb{G}_4^3$:** For the last time, we consider $\mathtt{Ext}_{\mathsf{SoK}}$ the extractor of the signature of knowledge. Always to prevent the adversary from providing a false signature of knowledge, this game is the same as the previous one, except that it starts by initializing an empty list $\mathsf{L}'[\,] \leftarrow \emptyset$ and:

- $\rightsquigarrow$ Each time the $\mathtt{oSend}$ oracle generates an SoK: $\mathsf{ni} \leftarrow \mathsf{SoK}_m\{(z, z'), (Z = g^z \wedge \mathsf{H} = (\mathsf{hA})^z) \wedge \mathsf{H}^1 = (\mathsf{hB})^z \wedge Z' = g^{z'} \wedge \mathsf{H}^2 = (\mathsf{hA})^{z'}\}$ (with A the ephemeral DH shared element, and B the long term static DH element of the endpoint considered), it sets $\mathsf{L}'[(g, Z, \mathsf{hA}, \mathsf{H}, \mathsf{hB}, \mathsf{H}^1, Z', \mathsf{hA}, \mathsf{H}^2, m, \mathsf{ni})] \leftarrow (z, z')$;

- $\rightsquigarrow$ After $\mathcal{A}$ returns its guess, $\mathbb{G}_4^3$ sets $\omega_* \leftarrow (\Lambda_{*,q})_{q=1}^n \| \mathsf{ID}_*$ (either $\mathsf{ID}_{\mathsf{I}*}$ or $\mathsf{ID}_{\mathsf{R}*}$) and parses sst_* as $\omega_* \| \mathsf{MCS}_* \| X_* \| Y_* \| R_* \| I_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{H}_*^1 \| \mathsf{H}_*^2 \| \mathsf{ni}_* \| \mathsf{pid}_* \| n_{r*} \| \mathsf{val}_* \| \sigma_*$ (with pid_* either $\mathsf{ID}_{\mathsf{R}*}$ or $\mathsf{ID}_{\mathsf{I}*}$), sets $\mathsf{A}_0 \leftarrow X_*$ and $\mathsf{A}_1 \leftarrow Y_*$,, and sets $\mathsf{B}_0 \leftarrow I_*$ and $\mathsf{B}_1 \leftarrow R_*$, parses $\Lambda_{*,q}.\mathsf{PK}$ as $(\Lambda_{*,q}.\mathsf{pk}, \Lambda_{*,q}.\mathsf{ni})$ and $\Lambda_{*,q}.\mathsf{td}$ as $(\Lambda_{*,q}.\mathsf{td}_0, \Lambda_{*,q}.\mathsf{td}_1, \Lambda_{*,q}.\mathsf{td}_2)$, and sets $\mathsf{h}_* \leftarrow \prod_{q=1}^n \Lambda_{*,q}.\mathsf{pk}$.
  If $\mathsf{L}'[(g, \mathsf{A}_{\mathsf{b}*}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*}, \mathsf{H}_*, \mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*}, \mathsf{H}_*^1, \mathsf{B}_{\mathsf{b}*}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*}, \mathsf{H}_*^2, m_*, \mathsf{ni}_*)] = \perp$, then:
  - If $\mathsf{SoKver}(m_*, (g, \mathsf{A}_{\mathsf{b}*}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*}, \mathsf{H}_*, \mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*}, \mathsf{H}_*^1, \mathsf{B}_{\mathsf{b}*}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*}, \mathsf{H}_*^2), \mathsf{ni}_*) = 1$, $\mathbb{G}_4^3$ runs $\mathtt{Ext}_{\mathsf{SoK}}(\lambda)$ on $\mathcal{A}$, extracting witnesses (η, η') for the proof ni_* and sets $\mathsf{L}'[(g, \mathsf{A}_{\mathsf{b}*}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*}, \mathsf{H}_*, \mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*}, \mathsf{H}_*^1, \mathsf{B}_{\mathsf{b}*}, \mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*}, \mathsf{H}_*^2, m_*, \mathsf{ni}_*)] \leftarrow (\eta, \eta')$. Else it aborts, returning 0 (here again we have that $\mathtt{Verify}$ fails on sst_* for the same reason);
  - If $\mathsf{H}_* \neq (\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*})^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{hA}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}_*^1,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*^2,m_*,\mathsf{ni}_*)][0]}$ or if $\mathsf{H}_*^1 \neq (\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*})^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{hB}_{1-\mathsf{b}*},\mathsf{H}_*^1,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*^2,m_*,\mathsf{ni}_*)][0]}$ or if $\mathsf{H}_*^2 \neq (\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*})^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}_*^1,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*^2,m_*,\mathsf{ni}_*)][1]}$ or if $\mathsf{A}_{\mathsf{b}*} \neq g^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}_*^1,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*^2,m_*,\mathsf{ni}_*)][0]}$ or if $\mathsf{B}_{\mathsf{b}*} \neq g^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}_*^1,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*^2,m_*,\mathsf{ni}_*)][1]}$ (*i.e.,* extraction fails), then $\mathbb{G}_4^3$ aborts and returns 0.

For the last time, with this game we pointed out the probability of $\mathtt{Ext}_{\mathsf{SoK}}$ failing to extract valid witnesses in any of (at most) q_{S} calls. We thus obtain:

$$\boxed{\left|\mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_3^3) - \mathbb{P}(\mathcal{A} \text{ wins } \mathbb{G}_4^3)\right| \leq q_{\mathsf{S}} \cdot \epsilon_{\mathsf{SoK}}(\lambda).}$$

To conclude this branch of the game schedule (and the proof), we again show that $\mathcal{A}$ always loses the game $\mathbb{G}_4^3$ against the $\mathtt{KeyExtract}$ algorithm we build. For that we reason through the absurd and we assume that the adversary $\mathcal{A}$ wins the game $\mathbb{G}_4^3$ with a non-zero probability. Denoting $(j_*, \mathsf{sst}_*, \mathsf{I}_*, \mathsf{R}_*, \mathsf{Prox}_*, (\Lambda_{*,q}, \Lambda_{*,q}.\mathsf{td})_{q=1}^n)$ the guess of $\mathcal{A}$, we parse $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{tr}$ as $\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1 \| g^y \| c_2 \| c_3 \| \mathsf{H}_{\mathsf{I}} \| \mathsf{H}_{\mathsf{I}}^1 \| \mathsf{H}_{\mathsf{I}}^2 \| \mathsf{ni}_{\mathsf{I}} \| S_{\mathsf{I}} \| c_4 \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{sst}_{\mathsf{I}}$ (or as $\mathsf{MCS} \| g^x \| \mathsf{C}_{\mathsf{I}} \| \mathsf{EAD}_1 \| g^y \| c_2 \| c_3 \| c_4 \| \mathsf{H}_{\mathsf{R}} \| \mathsf{H}_{\mathsf{R}}^1 \| \mathsf{H}_{\mathsf{R}}^2 \| \mathsf{ni}_{\mathsf{R}} \| S_{\mathsf{R}} \| n_r \| \mathsf{val} \| \mathsf{TH}_5 \| \mathsf{sst}_{\mathsf{R}}$ depending on the point of view), $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{PNID}$ as (I, R) and $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{AID}$ as $(\Lambda_q)_{q=1}^n$. On the other hand, we parse sst_* as $\omega_* \| \mathsf{MCS}_* \| X_* \| Y_* \| R_* \| I_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*} \| M_{1*} \| m_{2*} \| m_{3*} \| \mathsf{H}_* \| \mathsf{H}_*^1 \| \mathsf{H}_*^2 \| \mathsf{ni}_* \| \mathsf{pid}_* \| n_{r*} \| \mathsf{val}_* \| \sigma_*$. Thanks to the game $\mathbb{G}_2$, we know that $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{sid} = (\mathsf{C}_{\mathsf{I}*}, \mathsf{C}_{\mathsf{R}*}, X_*, Y_*)$, $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{PNID} = (\mathsf{I}_*, \mathsf{R}_*)$ and $\pi_{\mathsf{Prox}_*}^{j_*}.\mathsf{AID} = (\Lambda_{*,q})_{q=1}^{n_*}$, which implies that:

$$X_* = X \ , \ Y_* = Y \ , \ \mathsf{I}_* = \mathsf{I}, \mathsf{R}_* = \mathsf{R}, \text{ and } (\Lambda_q)_{q=1}^n = (\Lambda_{*,q})_{q=1}^{n_*}$$

and note that since $\mathsf{I}_* = \mathsf{I}$ and $\mathsf{R}_* = \mathsf{R}$, we have $I_* = I$ and $R_* = R$. By construction, the `KeyExtract` algorithm returns ($\mathsf{PRK_{out}}$ but temporarily) $\mathsf{k} = (g^x)^y = X^y$ where $Y = g^y$ and $\mathsf{k}' = (g^x)^r = X^r$ where $R = g^r$ and $\mathsf{k}'' = (g^y)^i = Y^i$ where $I = g^i$. As in the three previous games, we have the same ambiguity with element n_r, and we solve it in the same way. We set $\mathsf{h} \leftarrow \prod_{q=1}^{n} \Lambda_q.\mathsf{pk}$ and $m_* = (\Lambda_q)_{q=1}^{n} \| \mathsf{ID}_* \| \mathsf{pid}_* \| \mathsf{TH}_{2*} \| \mathsf{TH}_{3*} \| \mathsf{TH}_{4*}$. We also define a_0 and a_1 so that $\mathsf{A_b} = g^{a_\mathsf{b}}$, r_1 so that $R_* = g^{r_1}$ and i_1 so that $I_* = g^{i_1}$.
Running the `Open` algorithm on input $(\mathsf{pp}, \mathsf{sst}_*, (\Lambda_{*,q}.\mathsf{td})_{q=1}^{n_*}, (\Lambda_{*,q}.\mathsf{PK})_{q=1}^{n_*})$ gives us:

$$
\begin{aligned}
\mathsf{k}_* &= \frac{\mathsf{H}_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* \mathsf{A}_{1-\mathsf{b}*})^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}^1_*,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} \mathsf{A_b}^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\mathsf{h}_* g^{a_{1-\mathsf{b}}}\right)^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}^1_*,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} g^{a_\mathsf{b}\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}^1_*,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}\right) \mathsf{A_b}^{a_{1-\mathsf{b}}}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_\mathsf{b}}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g,\mathsf{A}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}_*,\mathsf{h}_*\mathsf{B}_{1-\mathsf{b}*},\mathsf{H}^1_*,\mathsf{B}_{\mathsf{b}*},\mathsf{h}_*\mathsf{A}_{1-\mathsf{b}*},\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_\mathsf{b}}} \\
&= \frac{\left(\prod_{q=1}^{n_*} \mathsf{A_b}^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_\mathsf{b}}} = \mathsf{k}
\end{aligned}
$$

and if we are on the $\mathsf{Initiator}$ point of view we will have:

$$
\begin{aligned}
\mathsf{k}'_* &= \frac{\mathsf{H}^1_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* R_*)^{\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_*Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} X_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\mathsf{h}_* g^{r_1}\right)^{\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_*Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} g^{a_0\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_*Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}\right) X_*^{r_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g,X_*,\mathsf{h}_*Y_*,\mathsf{H}_*,\mathsf{h}_*R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_*Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} X_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_0}} = \mathsf{k}'
\end{aligned}
$$

and

$$
\begin{aligned}
\mathsf{k}''_* &= \frac{\mathsf{H}^2_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_2} = \frac{(\mathsf{h}_* Y_*)^{\mathsf{L}'[(g,X_*,\mathsf{h}_* Y_*,\mathsf{H}_*,\mathsf{h}_* R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_* Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} I_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\mathsf{h}_* g^{a_1}\right)^{\mathsf{L}'[(g,X_*,\mathsf{h}_* Y_*,\mathsf{H}_*,\mathsf{h}_* R_*,\mathsf{H}^1,I_*,\mathsf{h}_* Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} g^{i_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.LIpk)^{\mathsf{L}'[(g,X_*,\mathsf{h}_* Y_*,\mathsf{H}_*,\mathsf{h}_* R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_* Y_*,\mathsf{H}^2,m_*,\mathsf{ni}_*)][1]}\right) I_*^{a_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{i_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g,X_*,\mathsf{h}_* Y_*,\mathsf{H}_*,\mathsf{h}_* R_*,\mathsf{H}^1_*,I_*,\mathsf{h}_* Y_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}\right) \mathsf{k}''}{\prod_{q=1}^{n_*} (\Lambda_{*,}.\mathsf{pk})^{i_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} I_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}''}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{i_1}} = \mathsf{k}''
\end{aligned}
$$

or if we are the Responder point of view we will have:

$$
\begin{aligned}
\mathsf{k}'_* &= \frac{\mathsf{H}^2_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_2} = \frac{(\mathsf{h}_* X_*)^{\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} R_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(g^{a_0}\right)^{\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}}{\prod_{q=1}^{n_*} g^{r_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}\right) R_*^{a_0}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{r_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][1]}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{r_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} R_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}'}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{r_1}} = \mathsf{k}'
\end{aligned}
$$

and

$$
\begin{aligned}
\mathsf{k}''_* &= \frac{\mathsf{H}^1_*}{\prod_{q=1}^{n_*} \Lambda_{*,q}.\mathsf{td}_1} = \frac{(\mathsf{h}_* I_*)^{\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} Y_*^{\mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\mathsf{h}_* g^{\mathsf{i}_1}\right)^{\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}}{\prod_{q=1}^{n_*} g^{a_1 \mathsf{L}[\Lambda_{*,q}.\mathsf{td}]}} \\
&= \frac{\left(\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}\right) Y_*^{i_1}}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} g^{(\Lambda_{*,q}.\mathsf{SK})\mathsf{L}'[(g,Y_*,\mathsf{h}_* X_*,\mathsf{H}_*,\mathsf{h}_* I_*,\mathsf{H}^1_*,R_*,\mathsf{h}_* X_*,\mathsf{H}^2_*,m_*,\mathsf{ni}_*)][0]}\right) \mathsf{k}''}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} \\
&= \frac{\left(\prod_{q=1}^{n_*} Y_*^{(\Lambda_{*,q}.\mathsf{SK})}\right) \mathsf{k}''}{\prod_{q=1}^{n_*} (\Lambda_{*,q}.\mathsf{pk})^{a_1}} = \mathsf{k}''
\end{aligned}
$$

This implies that $\mathsf{k}_* = \mathsf{k}$, $\mathsf{k}'_* = \mathsf{k}'$ and $\mathsf{k}''_* = \mathsf{k}''$ (and thus $\mathsf{PRK}_{\mathsf{out}*} = \mathsf{PRK}_{\mathsf{out}}$) with probability 1, and so $\mathcal{A}$ fails the game $\mathbb{G}_4^3$. Absurd!
So $\mathcal{A}$ always fails the game $\mathbb{G}_4^3$, and thus we have:

$$\boxed{\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_4^3\big) = 0.}$$

So finally, for each of the four versions of the protocol, we have:

$$\begin{aligned}
\mathsf{Adv}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) &= \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{E}^{\mathrm{LIC}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)\big) = \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_0\big) \\
&\leq q_{\mathsf{R}}\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_1\big) \\
&\leq q_{\mathsf{R}}\Big(\Big|\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_1\big) - \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_2\big)\Big| \\
&\quad + \Big|\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_2\big) - \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_3\big)\Big| \\
&\quad + \Big|\mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_3\big) - \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_4\big)\Big| + \mathbb{P}\big(\mathcal{A} \text{ wins } \mathbb{G}_4\big)\Big) \\
&\leq q_{\mathsf{R}}\Big(\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda) + q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + q_{\mathsf{S}} \cdot \epsilon_{\mathsf{SoK}}(\lambda)\Big)
\end{aligned}$$

and this concludes the proof. ■

7.3 Non-frameability.

With this third theorem, we also verify that our four protocols respect the non-frameability security property.

Theorem 4 (Non-Frameable LI-EDHOC protocol) *Under the assumption that our signature scheme* DS *is* EUF-CMA*-secure, the* **Sign-sign** *version of the* LI-EDHOC *protocol is non-frameable. Moreover, assuming that our proofs and signatures of knowledge are zero-knowledge and extractable, and that the* Oracle-DLog *problem is hard in* $\mathbb{G}$*, then the three other versions of the* LI-EDHOC *protocol are non-frameable. More precisely, for all p.p.t. adversary* $\mathcal{A}$ *making at most* q_{R} *queries to the* `oRegister` *oracle, we have (for the* **Sign-sign** *method):*

$$\mathsf{Adv}^{\text{NF-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq q_{\mathsf{R}} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda)$$

and (for the three others methods):

$$\mathsf{Adv}^{\text{NF-1,2,3}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq q_{\mathsf{R}} \cdot \Big(\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda) + q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + \epsilon_{\mathsf{SoK}}(\lambda) + \mathsf{Adv}^{\text{Oracle-DLog}}_{\mathbb{G}}(\lambda)\Big).$$

Before starting the proof, we just remind the non-frameability security game.

$\mathbb{E}^{\mathsf{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

1 : $\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda), b \leftarrow\$ \{0,1\}, \mathsf{ESet} \leftarrow \emptyset, \mathsf{PROXSet} \leftarrow \emptyset, \mathsf{AUTHSet} \leftarrow \emptyset$

2 : $\mathcal{O}_{\mathsf{NF}} \leftarrow \left\{ \begin{array}{l} \mathtt{oRegister}(\cdot,\cdot,\cdot), \mathtt{oSend}(\cdot,\cdot), \\ \mathtt{oNewSession}(\cdot,\cdot,\cdot,\cdot), \\ \mathtt{oCorrupt}(\cdot,\cdot), \\ \mathtt{SessionKeyReveal}(\cdot,\cdot), \\ \mathtt{oRevealTD}(\cdot,\cdot,\cdot,\cdot,\cdot) \end{array} \right\}$

3 : $(\mathsf{sst}, \mathsf{P}) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{NF}}}(\lambda, \mathsf{pp})$

4 : **if** $\exists(\mathsf{I},\mathsf{R}) \in \mathsf{ESet}^2, \exists n \in \mathbb{N}^*, \exists \mathsf{Prox} \in \mathsf{PROXSet}, \exists(\Lambda_j)_{j=1}^n \in \mathsf{AUTHSet}^n$ **s.t.** :

5 : $\quad -\ \mathtt{Verify}(\mathsf{pp}, \mathsf{sst}, \mathsf{I}.\overline{\mathsf{PK}}, \mathsf{R}.\overline{\mathsf{PK}}, \mathsf{Prox}.\overline{\mathsf{PK}}, (\Lambda_j.\overline{\mathsf{PK}})_{j=1}^n) = 1;$

6 : $\quad -\ \mathsf{P} \in \{\mathsf{I},\mathsf{R}\}$ **and** $\mathsf{P}.\overline{\mathsf{SK}}[\mathsf{sst}].\gamma = 0;$

7 : $\quad -\ \forall q \in \mathbb{N}$, **if** $\pi^q_{\mathsf{P}} \neq \perp$: $\mathtt{IdentifySession}(\mathsf{sst}, \pi^q_{\mathsf{P}}) = 0$ **or** $\pi^q_{\mathsf{P}}.\alpha = 0;$

8 : $\quad$ **then return** 1

9 : **else return** 0.

FIGURE 29 – Non-frameability security game for the LI-EDHOC protocol (by $\mathsf{P}.\overline{\mathsf{SK}}[\mathsf{sst}]$ we mean the long-term key of P associated to the method stored in sst).

Proof 4 (of theorem 4) In the non-frameability game, the adversary aims to convince the verifier that a given endpoint participated in a session, despite this being false. We proceed by games hop and we separate this proof in two parts. First we treat the case for a *signing party*, and we subsequently treat the case for a *static party*. However the first two games are common to both. The game $\mathbb{G}_0$ is simply the original experiment, and then in game $\mathbb{G}_1$ we chose at random an honest endpoint, against which $\mathcal{A}$ will try to forge a session state. After choosing this particular endpoint, we divide the proof into two branches, depending on the authentication method used by the endpoint. If it is a signature authentication, with game $\mathbb{G}_2^{\mathbf{Sign}}$, we raised a bad flag whenever $\mathcal{A}$ forges a signature, and finally with game $\mathbb{G}_3^{\mathbf{Sign}}$, we abort if the bad flag is raised and we reduce against the EUF-CMA security property of DS. Otherwise, we selected an endpoint using static DH authentication, and with game $\mathbb{G}_2^{\mathbf{Static}}$, we prevent the adversary from forging proof of knowledge on the authorities' long-term public keys with the extractor. In game $\mathbb{G}_3^{\mathbf{Static}}$ we simulate the signature of knowledge for the chosen endpoint (with the simulator) and extracting them (with the extractor) in game $\mathbb{G}_4^{\mathbf{Static}}$, to finally reduce against the Oracle-DLog problem. The game schedule is depicted in FIGURE 30 below.

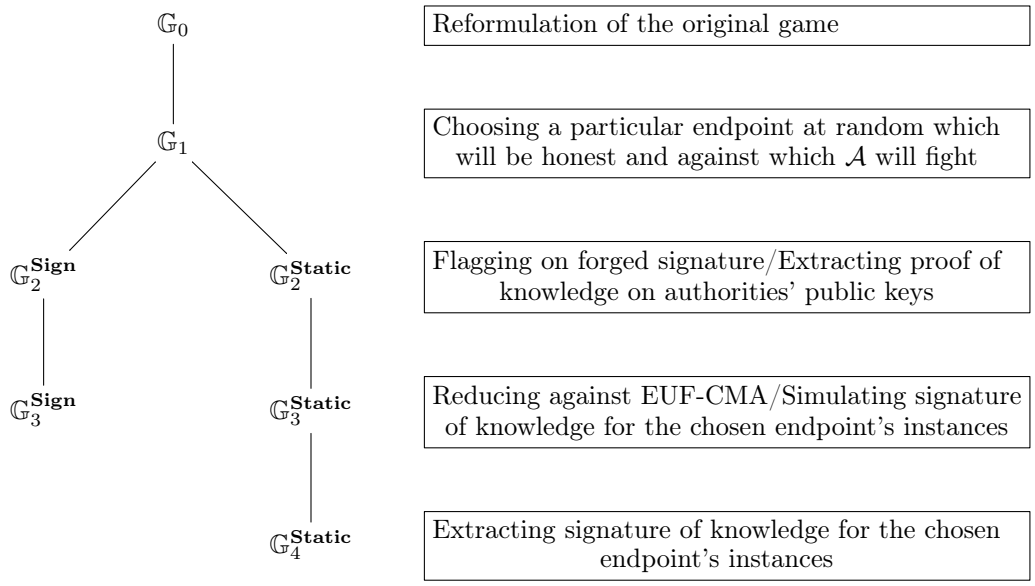

FIGURE 30 – Game-schedule for the non-frameability property.

† **Game $\mathbb{G}_0$:** This game is the original experience $\mathbb{E}^{\mathrm{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$.We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{E}^{\mathsf{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_1$:** This game is the same as the previous one, except that it starts by picking $\ell \leftarrow\$ [\![1, q_{\mathsf{R}}]\!]$. We denote by $\mathsf{P_j}$ the $j-$th party registered.
If $\mathcal{A}$ returns $(\mathsf{sst}, \mathsf{P})$ with $\mathsf{P} \neq \mathsf{P}_\ell$, then $\mathbb{G}_1$ aborts and returns 0. With this game hop, the winning advantage of the adversary is increased by a factor equaling the probability of guessing correctly the party P_ℓ. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) \leq q_{\mathsf{R}} \cdot \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big).}$$

We now split the proof into two branches.

Proof for a signing party. From now on, for this first branch we assume that party P_ℓ uses the signing method. Thus it provides a signature which is recorded in the session state and subsequently verified. The associated signature must be valid for the adversary to win the experiment in such a case. We show below that the adversary can only frame party P_ℓ by producing a forgery for P_j's signing key. This is prevented based on the EUF-CMA security property of the signature scheme as we elaborate below.

† **Game $\mathbb{G}_2^{\mathbf{Sign}}$:** This game is the same as the previous one, except that the challenger records all the messages signed by party P_ℓ which has been picked in game $\mathbb{G}_1$. If $\mathcal{A}$ returns $(\mathsf{sst}, \mathsf{P})$ with $\mathsf{P} = \mathsf{P}_\ell)$, we parsed sst as $\mathsf{sst} = \omega \| \mathsf{MCS} \| g^x \| g^y \| \mathsf{TH}_2 \| \mathsf{TH}_3 \| \mathsf{TH}_4 \| M_1 \| m_2 \| m_3 \| \mathsf{H_d} \| \mathsf{ni} \| \mathsf{pid_d} \| n_r \| \mathsf{val} \| \sigma$. In the one hand, assuming that party P_ℓ is an $\mathsf{Initiator}$ in this session, we parse m_3 as $(\mathsf{ID_I} \| \sigma_3 \| \mathsf{EAD}_3)$ we check that σ_3 is a valid signature on a signed message of the recorded list. Otherwise the experiment raised a bad flag $\mathtt{bad_{Init}}$ to $\mathtt{true}$. On the other hand, assuming that party P_ℓ is a $\mathsf{Responder}$, we parse m_2 as $(\mathsf{C_R} \| \mathsf{ID_R} \| \sigma_2 \| \mathsf{EAD}_2)$ and do the same checks on σ_2. The experiment raised a bad flag $\mathtt{bad_{Resp}}$ to $\mathtt{true}$

if the message was not honestly signed by the honest party. This modification is internal to the game, and we obtain:

$$\boxed{\mathbb{P}\big(\mathbb{G}_2^{\mathbf{Sign}} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_3^{\mathbf{Sign}}$:** This game is the same as the previous one, except that we abort when bad flags $\mathtt{bad}_{\mathtt{Init}}$ or $\mathtt{bad}_{\mathtt{Init}}$ is set to $\mathtt{true}$. Let's denote F this event. We claim that, by reduction, we can build a p.p.t. adversary $\mathcal{B}_{\mathbf{Sign}}$ against the EUF-CMA property of DS in order to bound $\mathbb{P}(F)$, and by the identical-until-bad lemme we thus have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_2^{\mathbf{Sign}} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_3^{\mathbf{Sign}} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathsf{DS},\mathcal{A}}^{\text{EUF-CMA}}(\lambda).}$$

‡ **Proof of the claim:** The reduction is direct and similar to the previous ones on the EUF-CMA assumption. We first assume for contradiction that we have a p.p.t. adversary $\mathcal{A}$ that wins against game $\mathbb{G}_2^{\mathbf{Sign}}$ but not game $\mathbb{G}_3^{\mathbf{Sign}}$ with a non-zero probability $\epsilon_{\mathcal{A}}(\lambda)$. Hence, $\mathcal{A}$ triggered the failure condition added in game $\mathbb{G}_2^{\mathbf{Sign}}$ by returning a valid signature that has not been produced by party P_ℓ. Based on the challenger of the $\mathbb{E}_{\mathsf{DS},\mathcal{A}}^{\text{EUF-CMA}}(\lambda)$ experiment, we build a p.p.t. algorithm $\mathcal{B}_{\mathbf{Sign}}$ returning a forgery as follows.
The challenger generates a pair of private/public signature keys $(\widetilde{\mathsf{ssk}}, \widetilde{\mathsf{spk}})$ thanks to $\mathsf{DS}.\mathtt{Gen}$, and sends $\widetilde{\mathsf{spk}}$ to $\mathcal{B}_{\mathbf{Sign}}$. Then algorithm $\mathcal{B}_{\mathbf{Sign}}$ initializes an empty list $\mathsf{L} \leftarrow \emptyset$, and then simulates $\mathbb{G}_2^{\mathbf{Sign}}$ to $\mathcal{A}$ faithfully, with two exceptions. Firstly, when $\mathcal{A}$ registered (honestly) party P_ℓ, $\mathcal{B}_{\mathbf{Sign}}$ sets $\mathsf{P}_\ell.\overline{\mathsf{PK}} \ni \mathsf{P}_\ell.\mathsf{spk} \leftarrow \widetilde{\mathsf{spk}}$. Secondly, whenever $\mathcal{A}$ queries $\mathtt{oSend}$, all required signatures are generated through the signature oracle $\mathtt{Sign}$ provided by the EUF-CMA experiment.
Finally, $\mathcal{A}$ returns a sst from which $\mathcal{B}_{\mathbf{Sign}}$ extract a signature σ_2 or σ_3 depending on the role of the party P_ℓ, and the associated MAC t used as a message to sign. If $\mathcal{A}$ wins the game by returning a valid signature that has not been produced by party P_ℓ, *i.e.,* if F occurs, we have by construction of the game and the algorithm $\mathtt{Verify}$, that $\mathsf{DS}.\mathtt{Verif}(\mathsf{P}_\ell.\mathsf{spk}, t, \sigma) = 1$ and $(t, \sigma) \notin \mathsf{L}$. Thus $\mathcal{B}_{\mathbf{Sign}}$ breaks the EUF-CMA security of DS and we have:

$$\mathsf{Adv}_{\mathsf{DS},\mathcal{B}_{\mathbf{Sign}}}^{\text{EUF-CMA}}(\lambda) \geq \epsilon_{\mathcal{A}}(\lambda) \geq \mathbb{P}(F)$$

which concludes the proof of the claim.

To conclude this branch of the proof, we now show that has a zero probability of winning game $\mathbb{G}_3^{\mathbf{Sign}}$. For that observe that in game $\mathbb{G}_3^{\mathbf{Sign}}$ the adversary is left with the objective to return $(\mathsf{sst}, \mathsf{P})$, where $\mathsf{P} = \mathsf{P}_\ell$ and sst must contains a signature σ_2 or σ_3 (depending on the role of P_ℓ in the associated statement) which is on a message that has been produced by party P_ℓ. By construction, this signature is verified by the $\mathtt{Verify}$ algorithm. Since $\mathcal{A}$ is not able anymore to forged this signature, in order to respect the winning condition of $\mathtt{Verify}$ returning 1, this signature must have been generated by P_ℓ. But this implies that there exists an integer $q \in \mathbb{N}$ such that $\pi_{\mathsf{P}_\ell}^q \neq \perp$ and $\mathtt{IdentifySession}(\mathsf{sst}, \pi_{\mathsf{P}_\ell}^q) = 1$, which contradict the last winning condition of the game. Thus $\mathcal{A}$ is unable to win game $\mathbb{G}_3^{\mathbf{Sign}}$. We thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_3^{\mathbf{Sign}} \Longrightarrow 1\big) = 0}$$

Proof for a static party. From now on, for this second branch, we assume that party P_ℓ uses the static authentication method. Thus, no signature is registered in the session state by the endpoint P_ℓ, only a SoK produced by the latter, involving its long term static DH secret. This signature of knowledge is verified in the final verification step of the non-frameability experiment. We now elaborate why this SoK cannot have been generated by the adversary for party P_ℓ.

† **Game $\mathbb{G}_2^{\mathbf{Static}}$:** Let's denote $\mathtt{Ext}_{\mathsf{NIPoK}}$ the knowledge extractor of the zero-knowledge proof of knowledge. In order to prevent the adversary from forging proofs of knowledge on authorities' long-term public keys, this game operate similarly to game $\mathbb{G}_1$, except that we begin by initializing an empty list $\mathsf{L}'[\,] \leftarrow \emptyset$, and we slightly modify the `oRegister` oracle. When $\mathcal{A}$ queries `oRegister` on input $(\Lambda, \mathsf{authority}, \{\perp\})$, it generates the associated keys $(\Lambda.\overline{\mathsf{SK}}, \Lambda.\overline{\mathsf{PK}})$ (to simplify notations, in what follows we write $\Lambda.\mathsf{SK}$ and $\Lambda.\mathsf{PK}$ instead of $\Lambda.\overline{\mathsf{SK}}, \Lambda.\overline{\mathsf{PK}}$, since these sets contain only one element each) and then sets $\mathsf{L}'[\Lambda.\mathsf{PK}] \leftarrow \Lambda.\mathsf{SK}$. Now in case the adversary queries `oRegister` on $(\Lambda, \mathsf{authority}, \mathsf{PK} \neq \perp)$, which in this experiment is the most likely, the oracle sets $\Lambda.\mathsf{PK} \leftarrow \mathsf{PK}$ and parses $\Lambda.\mathsf{PK}$ as $(\Lambda.\mathsf{pk}, \Lambda.\mathsf{ni})$.
To complete the process correctly, it checks if $\mathsf{NIPoKver}((g, \Lambda.\mathsf{pk}), \Lambda.\mathsf{ni}) = 1$ (if not, the game aborts), and then it runs the extractor $\mathtt{Ext}_{\mathsf{NIPoK}}$ on $\mathcal{A}$ to get the witness $\Lambda.\mathsf{SK}$ for $\Lambda.\mathsf{ni}$. Following this, if $g^{\Lambda.\mathsf{SK}} \neq \Lambda.\mathsf{pk}$, then the experiment aborts and returns a random bit, else it sets $\mathsf{L}'[\Lambda.\mathsf{PK}] \leftarrow \Lambda.\mathsf{SK}$.
With this modification, we point out the probability of the extractor $\mathtt{Ext}_{\mathsf{NIPoK}}$ failing when called to extract a valid witness. Since `oRegister` requires possibly one call to $\mathtt{Ext}_{\mathsf{NIPoK}}$ at each query, this implies that:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_2^{\mathbf{Static}} \Longrightarrow 1\big)\right| \leq q_{\mathsf{R}} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda).}$$

† **Game $\mathbb{G}_3^{\mathbf{Static}}$:** Let's denote by $\mathtt{Sim}_{\mathsf{SoK}}$ the simulator of the signature of knowledge. This game is the same as $\mathbb{G}_8$, except that each time the `oSend` oracle is queried to produce signature of knowledge for party P_ℓ, the signature of knowledge ni is simulate using $\mathtt{Sim}_{\mathsf{SoK}}$. Since by assumption the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ and the SoK algorithm follow the same probability distribution, this modification is indistinguishable by $\mathcal{A}$, and thus we have :

$$\boxed{\mathbb{P}\big(\mathbb{G}_3^{\mathbf{Static}} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_2^{\mathbf{Static}} \Longrightarrow 1\big).}$$

Game $\mathbb{G}_4^{\mathbf{Static}}$: This game is the same as the previous one, except that we added an additional failure condition: during the verification of the values $(\mathsf{sst}, \mathsf{P})$ returned by the adversary, the state sst contains either a SoK ni_{I} or ni_{R} depending wether sst corresponds to a $\mathsf{Initiator}$'s or $\mathsf{Responder}$'s proxy state. After a successful verification of the SoK, the challenger uses the extractor algorithm of the SoK, denoted $\mathtt{Ext}_{\mathsf{SoK}}$, to recover the secret (x^*, i^*) (for a SoK ni_{I}) or (y^*, r^*) (for a SoK ni_{R}). He then checks the validity of this secret (as in game $\mathbb{G}_9$ of the proof of theorem 2), if it fails, stops the simulation.
With this game hop, we highlight the possibility of the extractor $\mathtt{Ext}_{\mathsf{SoK}}$ failing when extracting a valid witness. Since the extractor is called once in the game, we thus have the bound:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_3^{\mathbf{Static}} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_4^{\mathbf{Static}} \Longrightarrow 1\big)\right| \leq \epsilon_{\mathsf{SoK}}(\lambda).}$$

To conclude this branch of the proof, we now need to bound $\mathbb{P}\big(\mathbb{G}_4^{\mathbf{Static}} \Longrightarrow 1\big)$. To do so, we build a reduction on the Oracle-DLog problem.
So we consider game $\mathbb{G}_4^{\mathbf{Static}}$ and will construct an intermediary p.p.t. algorithm $\mathcal{B}_{\mathbf{Static}}$ which simulates it based on the challenger of the Oracle-DLog experiment. We also consider an adversary $\mathcal{A}$ and demonstrate that if $\mathcal{A}$ wins with non negligible probability $\epsilon_{\mathcal{A}}(\lambda)$ against game $\mathbb{G}_4^{\mathbf{Static}}$, then we can build an adversary against the Oracle-DLog problem that we assumed hard.

The challenger of the Oracle-DLog experiment starts by sending a value U to $\mathcal{B}_{\mathbf{Static}}$. With only one exception, algorithm $\mathcal{B}_{\mathbf{Static}}$ generate the parameter and keys as prescribed in experiment $\mathbb{G}_4^{\mathbf{Static}}$ and as it was done by the challenger of $\mathbb{G}_4^{\mathbf{Static}}$. The exception is when $\mathcal{B}_{\mathbf{Static}}$ sets the keys of the static endpoint P_ℓ. There $\mathcal{B}_{\mathbf{Static}}$ only sets the long-term public key as U (usually denoted by g^i or g^r), and the long-term secret key is set to $\perp$. Algorithm $\mathcal{B}_{\mathbf{Static}}$ subsequently reproduce the action of the challenger of game $\mathbb{G}_4^{\mathbf{Static}}$ for any query which does not involve endpoint P_ℓ. For any query involving

P_ℓ following differences are observed: on receiving g^x or g^y (depending on the Initiator or Responder role of P_ℓ) from another endpoint starting an handshake process, $\mathcal{B}_{\textbf{Static}}$ send V (either g^x or g^y) as a query to the oracle ${\mathcal{O}_{\mathsf{CDH}}}^U$ of the Oracle-DLog challenger. This call returns an element W that can be used as g^{iy} or g^{rx}, once again with a notation depending the role of endpoint P_ℓ in this session. Subsequently, based on this element, and also based on the extracted authorities' secret keys, $\mathcal{B}_{\textbf{Static}}$ computes either:

- $\mathsf{H}^1_\mathsf{R} = (\mathsf{h}_\mathsf{R} g^x)^r = U^{\sum_{j=1}^{n_\mathsf{R}} \Lambda^\mathsf{R}_j.\mathsf{SK}} g^{rx}$, where $U = g^r$, for P_ℓ as a Responder in a **Sign-static** method;
- $\mathsf{H}^1_\mathsf{I} = (\mathsf{h}_\mathsf{I} g^y)^i = U^{\sum_{j=1}^{n_\mathsf{I}}} g^{iy}$, where $U = g^i$, for P_ℓ as an Initiator in a **Static-sign** method;
- $\mathsf{H}^2_\mathsf{I} = (\mathsf{h}_\mathsf{I} g^y)^i = U^{\sum_{j=1}^{n_\mathsf{I}}} g^{iy}$, where $U = g^i$, for P_ℓ as an Initiator in a **Static-static** method;
- $\mathsf{H}^2_\mathsf{R} = (\mathsf{h}_\mathsf{R} g^x)^r = U^{\sum_{j=1}^{n_\mathsf{R}} \Lambda^\mathsf{R}_j.\mathsf{SK}} g^{rx}$, where $U = g^r$, for P_ℓ as a Responder in a **Static-static** method.

As the SoK is simulated, $\mathcal{B}_{\textbf{Static}}$ can still generate it. This allows to fully simulate P_ℓ. Based on the final witness extraction a value i^* or r^* extracted from a winning adversary $\mathcal{A}$ against the simulation of $\mathcal{B}_{\textbf{Static}}$ of game $\mathbb{G}_4^{\textbf{Static}}$, the extracted witness can be sent to the challenger of the Oracle-DLog as an answer to the Oracle-DLog problem. Thus the chances of $\mathcal{B}_{\textbf{Static}}$ winning against the Oracle-DLog problem are the same as the chances of $\mathcal{A}$ winning against $\mathbb{G}_4^{\textbf{Static}}$ as it was perfectly simulated, *i.e.*:

$$\mathsf{Adv}^{\text{Oracle-DLog}}_{\mathbb{G},\mathcal{B}_{\textbf{Static}}}(\lambda) = \epsilon_{\mathcal{A}}(\lambda).$$

So we finally obtain that:

$$\boxed{\mathbb{P}\big(\mathbb{G}_4^{\textbf{Static}} \Longrightarrow 1\big) = \mathsf{Adv}^{\text{Oracle-DLog}}_{\mathbb{G},\mathcal{A}}(\lambda).}$$

So to conclude the proof in its entirety, we have for the **Sign-sign** version of the LI-EDHOC protocol:

$$\begin{aligned}
\mathsf{Adv}^{\text{NF-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) &= \mathbb{P}\big(\mathbb{E}^{\text{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) \\
&\leq q_\mathsf{R} \cdot \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) \\
&= q_\mathsf{R} \cdot \mathbb{P}\big(\mathbb{G}_2^{\textbf{Sign}} \Longrightarrow 1\big) \\
&\leq q_\mathsf{R} \cdot \Big(\Big|\mathbb{P}\big(\mathbb{G}_2^{\textbf{Sign}} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_3^{\textbf{Sign}} \Longrightarrow 1\big)\Big| + \mathbb{P}\big(\mathbb{G}_3^{\textbf{Sign}} \Longrightarrow 1\big)\Big) \\
&\leq q_\mathsf{R} \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda).
\end{aligned}$$

And for the **Static-static** version:

$$\begin{aligned}
\mathsf{Adv}^{\text{NF-3}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) &= \mathbb{P}\big(\mathbb{E}^{\text{NF}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) \\
&\leq q_\mathsf{R} \cdot \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) \\
&\leq q_\mathsf{R} \cdot \Big(\Big|\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_2^{\textbf{Static}} \Longrightarrow 1\big)\Big| + \cdots + \mathbb{P}\big(\mathbb{G}_4^{\textbf{Static}} \Longrightarrow 1\big)\Big) \\
&\leq q_\mathsf{R} \cdot \Big(q_\mathsf{R} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + \epsilon_{\mathsf{SoK}}(\lambda) + \mathsf{Adv}^{\text{Oracle-DLog}}_{\mathbb{G},\mathcal{A}}(\lambda)\Big).
\end{aligned}$$

Finally combining both for the **Sign-static** and **Static-sign** versions:

$$\mathsf{Adv}^{\text{NF-1,2}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq q_\mathsf{R} \cdot \Big(\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda) + q_\mathsf{R} \cdot \epsilon_{\mathsf{NIPoK}}(\lambda) + \epsilon_{\mathsf{SoK}}(\lambda) + \mathsf{Adv}^{\text{Oracle-DLog}}_{\mathbb{G},\mathcal{A}}(\lambda)\Big).$$

And this concludes the proof. ■

7.4 Identity protection.

We now present the two following statements, which state that our protocols satisfy the Initiator and Responder identities protection between proxies, respectively against active and passive adversaries. In this situation the considered adversary only has access to the $\mathsf{RO}_{\mathcal{H}}$, RO_{T} and $\mathsf{RO}_{\mathsf{P}}s$ oracles (indispensable to simulate the protocol run), but not to those described in FIGURE 18.

7.4.1 Responder identity protection

We begin with the Responder side.

Theorem 5 (Responder **identity protecting LI-EDHOC protocol)** *Under the assumption that the* DDH *and the* GDH *problems are hard to solve in* $\mathbb{G}$*, that our two encryption schemes* $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ *and* $(\mathsf{KGen}'', \mathsf{Enc}'', \mathsf{Dec}'')$ *are* IND*-secure and that our encryption scheme* $(\mathsf{KGen}', \mathsf{Enc}', \mathsf{Dec}')$ *is an* AEAD*-encryption scheme* IND-CPA*-secure, then the* LI-EDHOC *protocol, in its four versions, protect the* Responder*'s identity against passive adversaries. More precisely, for all p.p.t. adversary* $\mathcal{A}$ *making at most* $q_{\mathsf{RO}_{\mathsf{T}}}$ *(resp.* $q_{\mathsf{RO}_{\mathsf{P}}}$ *and* $q_{\mathsf{RO}_{\mathcal{H}}}$*) queries to the* RO_{T} *(resp.* RO_{P} *and* $\mathsf{RO}_{\mathcal{H}}$*) oracle, we have:*

$$\begin{aligned}\mathsf{Adv}^{\text{Id-Pro-Resp}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq\ & \frac{(q_{\mathsf{SO}_{\mathsf{T}}} + q_{\mathsf{SO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}})^2 + (q_{\mathsf{SO}_{\mathsf{T}}} + q_{\mathsf{SO}_{\mathcal{H}}})^2 + (q_{\mathsf{SO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}} + q_{\mathsf{SO}_{\mathcal{H}}})^2}{2^{l_{\mathsf{hash}}+1}} + 12\mathsf{Adv}^{\text{DDH}}_{\mathbb{G}}(\lambda) \\ & + 8q_{\mathsf{RO}_{\mathsf{T}}} \cdot \mathsf{Adv}^{\text{GDH}}_{\mathbb{G}}(\lambda) + 5 \cdot \frac{q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{key}}}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 4 \cdot \frac{q_{\mathsf{RO}_{\mathsf{P}}{}^{l_2}} + q_{\mathsf{RO}_{\mathsf{P}}{}^{l_{\mathsf{hash}}}} + 2}{2^{l_{\mathsf{hash}}}} \\ & + 4 \cdot \Big(8\mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}) + \mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec})}(l_2) + \mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD}}(l_{\mathsf{key}})\Big)\end{aligned}$$

Proof 5 (of theorem 5) To begin with we remind here, in FIGURE 31 the Responder identity protection security games.

$\mathbb{E}^{\text{IdPro-Responder-}b}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

1 : $\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
2 : $\mathsf{ID_CRED_R}_0, \mathsf{ID_CRED_R}_1 \leftarrow \mathcal{A}(\mathsf{ESet}_{\mathsf{meth}})$
3 : $\mathsf{ID_CRED_R} \leftarrow \mathsf{ID_CRED_R}_b$
4 : $\mathsf{ID_CRED_I} \leftarrow\$ \mathsf{ESet}_{\mathsf{meth}}$
5 : $M_1 \leftarrow \mathtt{InitRun1}(\mathsf{ID_CRED_I})$
6 : $M_2 \leftarrow \mathtt{RespRun1}(\mathsf{ID_CRED_R}, M_1)$
7 : $M_3 \leftarrow \mathtt{InitRun2}(\mathsf{ID_CRED_I}, M_2)$
8 : $M_3' \leftarrow \mathtt{Prox_I Run3}(\mathsf{ID_CRED_I}, M_3)$
9 : $M_4 \leftarrow \mathtt{RespRun2}(\mathsf{ID_CRED_R}, M_3')$
10 : $M \leftarrow (\mathsf{ID_CRED_I}, M_1, M_2, M_3, M_3', M_4)$
11 : $b' \leftarrow \mathcal{A}(M)$
12 : **if** $b = b'$ **then return** 1
13 : **else return** 0

FIGURE 31 – The LI-EDHOC Responder identity protection security games. By $\mathsf{ESet}_{\mathsf{meth}}$ we mean the set of all endpoints running the protocol with the same method (0,1,2, or 3).

We proceed by game hops which are depicted in FIGURES 32 to 34 below. Firstly, game $\mathbb{G}_0$ is a reformulation of the original experiment $\mathbb{E}^{\text{IdPro-Responder-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. Then with games $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_3$ we simulates random oracles RO as list, in order to manipulate them later. We next prevent collisions on some oracles with games $\mathbb{G}_4$ and $\mathbb{G}_5$. We now need to divide our analysis into four branches, in order to distinguish between the four

versions of the protocol (denoting games with exponent $0, 1, 2$ or 3 to indicate the method used in the current game). With games $\mathbb{G}_6$, $\mathbb{G}_{6'}$, $\mathbb{G}_7$ and $\mathbb{G}'_7$, we randomize the H elements in order to prevent the adversary from obtaining informations about g^{xy} (or g^{iy} or g^{xr}), so he cannot compute on his side some keys as sk_2 or sk_3. With the pack of games $\mathbb{G}_8$ and $\mathbb{G}_9$ we randomize the key-schedule elements to limit the capacity of $\mathcal{A}$ in predicting the encryption keys (and thus potentially learning the identity of the $\mathsf{Responder}$). With game $\mathbb{G}_{10}$ we randomize the encryption keys sk_2 and sk_3 to make them unpredictable to $\mathcal{A}$. Finally with game $\mathbb{G}_{11}$ we turn the game into the original experiment $\mathbb{E}^{\text{IdPro-Responder-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$, so we can conclude on the advantage of $\mathcal{A}$.

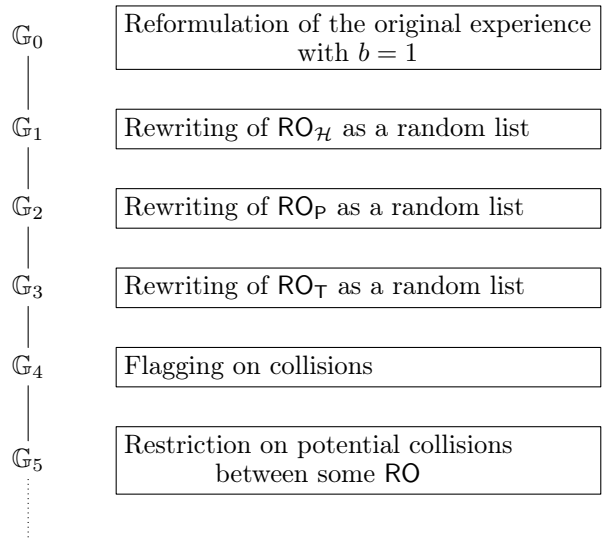

FIGURE 32 – First steps of the game-schedule for the $\mathsf{Responder}$ identity protection security property.

† **Game $\mathbb{G}_0$:** This game is the original experience $\mathbb{E}^{\text{IdPro-Responder-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. As usually we thus have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) = \mathbb{P}\Big(\mathbb{E}^{\text{IdPro-Responder-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big).}$$

† **Game $\mathbb{G}_1$:** This game is the same as the previous one except that we make the same modification as in the game $\mathbb{G}_3$ of the key-secure property demonstration (on page 55); namely we simulate $\mathsf{RO}_{\mathcal{H}}$ thanks to a list (with memory) denoted $\mathsf{SO}_{\mathcal{H}}$. We thus obtain the same conclusion:

$$\boxed{\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_2$:** Similarly, this game works as the previous one, except that we make the same modification as in the game $\mathbb{G}_4$ of the key-secure property demonstration (on page 55); namely we simulate RO_{P} thanks to a list (again with memory) denoted SO_{P} (still depending on the digest size, but again it does not impact the analysis). We thus obtain the same result:

$$\boxed{\mathbb{P}\big(\mathbb{G}_2 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_3$:** Similarly, this game is the same as the previous one, except that this time we make the same modification as in the game $\mathbb{G}_5$ of the key-secure property demonstration (on page 55); namely we simulate RO_{T} thanks to a special list (with memory) denoted SO_{T} (following the same construction as in $\mathbb{G}_5$). We thus obtain:

$$\boxed{\mathbb{P}(\mathbb{G}_3 \Longrightarrow 1) = \mathbb{P}(\mathbb{G}_2 \Longrightarrow 1).}$$

† **Game $\mathbb{G}_4$:** In a future game, we will work on the key-derivation schedule. With this game, and the next one, we want to avoid collision between some simulated random oracles. So this game is the same as the previous one, except that this time we make the same modification as in the game $\mathbb{G}_6$ of the key-secure property demonstration (on page 56), raising a bad flag $\mathtt{bad_3}$ to $\mathtt{true}$ when a collision between $\mathsf{SO}_{\mathcal{H}}$ and SO_{T}, or SO_{T} and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or $\mathsf{SO}_{\mathcal{H}}$ and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$ occurs. This change is indistinguishable by the adversary and we obtain:

$$\boxed{\mathbb{P}(\mathbb{G}_4 \Longrightarrow 1) = \mathbb{P}(\mathbb{G}_3 \Longrightarrow 1).}$$

† **Game $\mathbb{G}_5$:** This game is the same as the previous one, except that this time we make the same modification as in the game $\mathbb{G}_7$ of the key-secure property demonstration (on page 56), aborting when the bad flag $\mathtt{bad_3}$ is set to $\mathtt{true}$. Using the same analysis as is game $\mathbb{G}_7$, we bound the winning advantage of the adversary with this game hop as follows:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_4 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_5 \Longrightarrow 1)\right| \leq \frac{(q_{\mathsf{SO}_{\mathsf{T}}}+q_{{\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}})^2+(q_{\mathsf{SO}_{\mathsf{T}}}+q_{\mathsf{SO}_{\mathcal{H}}})^2+(q_{{\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}}+q_{\mathsf{SO}_{\mathcal{H}}})^2}{2^{l_{\mathsf{hash}}+1}}.}$$

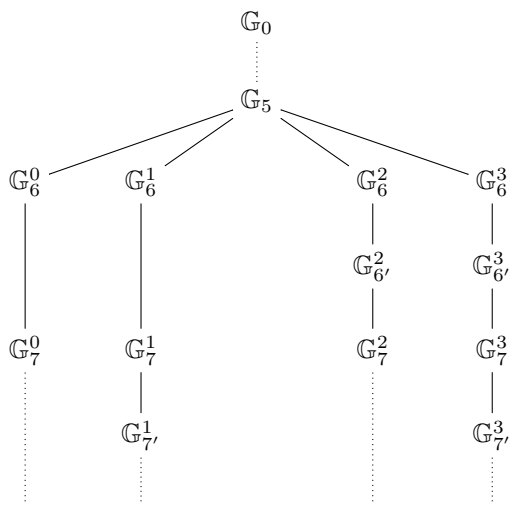

FIGURE 33 – Continuation of the game-schedule for the Responder identity protection security property. With these games we randomize the H elements.

† **Game $\mathbb{G}_6^0$:** With the next game pack, we prevent the adversary from obtaining information from the H elements. This game is the same as the previous one except that we pick at random, according to the uniform distribution on the group $\mathbb{G}$, the element $\mathsf{H_I}$ of the Initiator (and of course simulate the proof $\mathsf{ni_I}$ with the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the SoK). We claim that, by reduction, we can build a p.p.t. algorithm $\mathcal{B}_1^0$ against the DDH problem, and thus we have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_5 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_6^0 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

‡ **Proof of the claim:** The challenger gives a DDH challenge $(\Gamma = g^{\gamma}, \Delta = g^{\delta}, \Sigma)$ to $\mathcal{B}_1^0$, with either $\Sigma = g^{\gamma\delta}$ or $\Sigma = g^{\sigma}$ with $\sigma \neq \gamma\delta$, and $\mathcal{B}_1^0$ tries to distinguish. To do so $\mathcal{B}_1^0$ simulates game $\mathbb{G}_5$ to $\mathcal{A}$ faithfully, except on certain points:

$\rightsquigarrow$ When registering all authorities (to generate the element $\mathsf{h_I}$), $\mathcal{B}_1^0$ chooses $\Lambda_* \in (\Lambda_j^{\mathsf{I}})_{j=1}^{n_{\mathsf{I}}}$ and sets $\Lambda_*.\mathsf{pk} \leftarrow \Delta = g^\delta$ (and he simulates the associated proof thanks to the simulator $\mathtt{Sim}_{\mathsf{NIPoK}}$);

$\rightsquigarrow$ When simulating the protocol run, $\mathcal{B}_1^0$ sets $(g^x =:)X \leftarrow \Gamma = g^\gamma$, and:

$$\mathsf{H_I} \leftarrow \prod_{\Lambda \in (\Lambda_j^{\mathsf{I}})_{j=1}^{n_{\mathsf{I}}} \setminus \Lambda_*} (X^{\Lambda.\mathsf{SK}}) \Sigma X^y.$$

Now observe that if $\Sigma = g^{\gamma\delta}$, $\mathsf{H_I}$ is well formed as describe by the protocol, and in this case (since $\mathcal{B}_1^0$ did not have to answer $\mathcal{A}$'s queries) $\mathcal{B}_1^0$ perfectly simulates game $\mathbb{G}_5$ to $\mathcal{A}$. On the other hand, if $\Sigma = g^\sigma$, with $\sigma \leftarrow\$ \mathbb{Z}_p^*$, $\mathsf{H_I}$ is random, and in that case $\mathcal{B}_1^0$ perfectly simulates game $\mathbb{G}_6^0$ to $\mathcal{A}$. So we deduce that:

$\looparrowright$ $\mathbb{P}\big(\mathbb{G}_5 \Longrightarrow 1\big) = \mathbb{P}\big((\gamma, \delta) \leftarrow\$ (\mathbb{Z}_p^*)^2, b \leftarrow \mathcal{B}_1^0(g^\gamma, g^\delta, g^{\gamma\delta}) : b = 1\big)$;

$\looparrowright$ $\mathbb{P}\big(\mathbb{G}_6^0 \Longrightarrow 1\big) = \mathbb{P}\big((\gamma, \delta, \sigma) \leftarrow\$ (\mathbb{Z}_p^*)^3, b \leftarrow \mathcal{B}_1^0(g^\gamma, g^\delta, g^\sigma) : b = 1\big)$.

So $\mathcal{B}_1^0$ is a well constructed distinguisher and we have:

$$\Big|\mathbb{P}\big(\mathbb{G}_5 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_6^0 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda)$$

which concludes the proof of the claim.

† **Game $\mathbb{G}_7^0$:** This game is the same as the previous one except that this time we pick at random, according to the uniform distribution on the group $\mathbb{G}$, the element $\mathsf{H_R}$ of the $\mathsf{Responder}$ (and of course simulate the proof $\mathsf{ni_R}$ with the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the SoK). As in the previous game, by reduction, we can build a p.p.t. algorithm $\mathcal{B}_2^0$ against the DDH problem, and thus we have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_6^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_7^0 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_6^1$:** This game is the same as the previous one (gamme $\mathbb{G}_5$) except that this time we pick at random, according to the uniform distribution on the group $\mathbb{G}$, elements $\mathsf{H_I}$ and $\mathsf{H_I^1}$ of the $\mathsf{Initiator}$ (and of course simulate the proof $\mathsf{ni_I}$ with the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the SoK). As in game $\mathbb{G}_6^0$, by reduction (slightly adapted), we can build a p.p.t. algorithm $\mathcal{B}_1^1$ against the DDH problem, and thus we have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_5 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_6^1 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_7^1$:** This game is the same as the previous one, and works in the same vein as game $\mathbb{G}_7^0$. Thanks to a p.p.t. algorithm $\mathcal{B}_2^1$, we have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_6^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_7^1 \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_{7'}^1$:** This game is the same as the previous one, and works in the same vein as game $\mathbb{G}_7^0$, but here we pick at random the element $\mathsf{H_R^1}$ (and simulate the associated proof). Thanks to a p.p.t. algorithm $\mathcal{B}_{2'}^1$ (instead of imbedding the challenge $\Gamma = g^\gamma$ in $X = g^x$, we set $R := g^r \leftarrow g^\gamma$ (since $\mathcal{A}$ cannot corrupt the $\mathsf{Responder}$ this does not lead to any problem)), we have:

$$\left|\mathbb{P}(\mathbb{G}_7^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{7'}^1 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

† **Game $\mathbb{G}_6^2$:** This game is the same as the previous one (game $\mathbb{G}_5$), and works in the same vein as game $\mathbb{G}_6^0$. Thanks to a p.p.t. algorithm $\mathcal{B}_1^1$, we have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_5 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_6^2 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_{6'}^2$:** This game is the same as the previous one, and works in the same vein as games $\mathbb{G}_6^0$ and $\mathbb{G}_{7'}^1$. Thanks to a p.p.t. algorithm $\mathcal{B}_{1'}^1$, we have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_6^2 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{6'}^2 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_7^2$:** This game is the same as the previous one, except that this time we pick at random, according to the uniform distribution on the group $\mathbb{G}$, elements H_R and H_R^1 of the $\mathsf{Responder}$ (and of course simulate the proof ni_R with the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the SoK). As in game $\mathbb{G}_6^1$, by reduction (slightly adapted), we can build a p.p.t. algorithm $\mathcal{B}_2^2$ against the DDH problem, and thus we have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_{6'}^2 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_7^2 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).}$$

† **Game $\mathbb{G}_6^3$:** This game is the same as the previous one (game $\mathbb{G}_5$), and works in the same vein as game $\mathbb{G}_6^1$. Thanks to a p.p.t. algorithm $\mathcal{B}_1^3$, we have:

$$\left|\mathbb{P}(\mathbb{G}_5 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_6^3 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

† **Game $\mathbb{G}_{6'}^3$:** This game is the same as the previous one, and works in the same vein as game $\mathbb{G}_{6'}^2$. Thanks to a p.p.t. algorithm $\mathcal{B}_{1'}^3$, we have:

$$\left|\mathbb{P}(\mathbb{G}_6^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{6'}^3 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

† **Game $\mathbb{G}_7^3$:** This game is the same as the previous one, and works in the same vein as game $\mathbb{G}_7^2$. Thanks to a p.p.t. algorithm $\mathcal{B}_2^3$, we have:

$$\left|\mathbb{P}(\mathbb{G}_{6'}^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_7^3 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

† **Game $\mathbb{G}_{7'}^3$:** This game is the same as the previous one, and works in the same vein as game $\mathbb{G}_{7'}^1$. Thanks to a p.p.t. algorithm $\mathcal{B}_{2'}^3$, we have:

$$\left|\mathbb{P}(\mathbb{G}_7^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{7'}^3 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

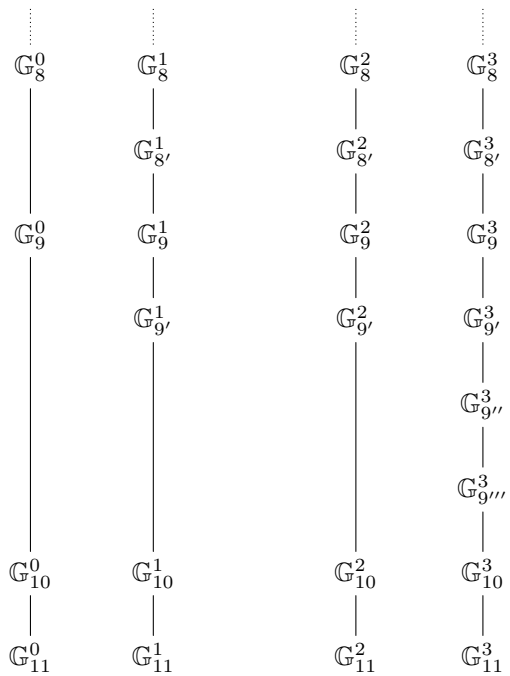

FIGURE 34 – Continuation of the game-schedule for the $\mathsf{Responder}$ identity protection security property. With these games we randomize the key-derivation schedule elements in order to prevent the adversary to obtain the encryptions keys sk_2 and sk_3, and we conclude the security game by modifying the game $\mathbb{G}_{10}$ in the original experiment with bit $b = 0$.

† **Game $\mathbb{G}_8^0$:** This game is the same as game $\mathbb{G}_7^0$, except that we added a failure condition: if the adversary, during the protocol session, queries $\mathsf{SO_T}$ on the correct DIFFIE-HELLMAN value g^{xy}, we then abort and return a random bit. We then denote F_1^0 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct DIFFIE-HELLMAN value g^{xy}. If the event F_1^0 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}\big(\mathbb{G}_7^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_8^0 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_1^0).$$

Let's now provide a bound for $\mathbb{P}(F_1^0)$. For that we construct, by reduction, an adversary $\mathcal{B}_3^0$ against the GDH problem. We hence claim that:

$$\mathbb{P}(F_1^0) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_7^0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_8^0 \Longrightarrow 1\big)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda).}$$

‡ **Proof of the claim:** This reduction is very similar to the reduction against the GDH problem proposed in [14] and also the one we made in the key-security property proof. So we briefly explain the construction of a p.p.t adversary $\mathcal{B}_3^0$ against the GDH problem.
On input a GDH challenge $(g, A = g^a, B = g^b)$ and having access to a DDH oracle, returning 1 if a tuple (g, g^a, g^b, g^c) is a DIFFIE-HELLMAN tuple, and 0 otherwise, our algorithm $\mathcal{B}_3^0$ must return g^{ab} in

a polynomial time. To achieve this, $\mathcal{B}_3^0$ is going to simulate $\mathbb{G}_8^0$ for $\mathcal{A}(\mathsf{pp})$ faithfully, except for certain points.
Firstly, $\mathcal{B}_3^0$ embeds its challenge into the ephemeral sent DIFFIE-HELLMAN value of each endpoint as follows: it samples uniformly at random $k, k' \in \mathbb{Z}_p$ and sets $X \leftarrow Ag^k$ and $Y \leftarrow Bg^{k'}$ (with $A = g^a$ and $B = g^b$).
If the event F_1^0 occurs, then the value Z queried to the $\mathsf{SO_T}$ oracle will equal $g^{(a+k)(b+k')}$. To detect that, $\mathcal{B}_3^0$ can compute $ZX^{-k'}B^{-k}$ which, if F_1^0 occurs, will equal g^{ab} and test this equality by calling its DDH oracle on $(X, Y, ZX^{-k'}B^{-k})$, which will return 1 if F_1^0 occurs. So when the event happens $\mathcal{B}_3^0$ wins.
To finally bound $\mathbb{P}(F_1^0)$, we remark that each time the $\mathsf{SO_T}$ oracle is queried on input Z, the event F_1^0 can happen (on a single pair (X, Y)). This implies that possibly $q_{\mathsf{RO_T}}$ DDH-oracle queries might be useful to detect the event. Hence the result:

$$\mathbb{P}(F_1^0) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda)$$

which concludes the proof of the claim.

† **Game $\mathbb{G}_9^0$:** This game is the same as the previous one, except that in this game we simulate the key generation of $\mathsf{PRK_{2e}}$ for our running session. Namely we simulate the generation of the key $\mathsf{PRK_{2e}}$ thanks to a private oracle, denoted $\mathsf{SO_{PRK_{2e}}}$, and working the same way as $\mathsf{SO_T}$, but inaccessible to $\mathcal{A}$ (in normal times, $\mathsf{PRK_{2e}}$ is generated thanks to $\mathsf{SO_T}$, $\mathsf{TH_2}$ and g^{xy}).
One way to detect the modification is for the adversary to ask $\mathsf{SO_T}$ on the correct DIFFIE-HELLMAN value g^{xy}. Thanks to the event F_1^0 the simulation is already stopped in this situation. Another way to detect it is to query $\mathsf{SO_P}^{l_2}$, or $\mathsf{SO_P}^{l_{\mathsf{key}}}$, or $\mathsf{SO_P}^{l_{\mathsf{iv}}}$, or $\mathsf{SO_P}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK_{2e}}$ which unpredictable at this stage. We denote F_2^0 the event: $\mathcal{A}$ queries $\mathsf{SO_P}^{l_2}$, or $\mathsf{SO_P}^{l_{\mathsf{key}}}$, or $\mathsf{SO_P}^{l_{\mathsf{iv}}}$,or $\mathsf{SO_P}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK_{2e}}$ for a session in which the latter is unpredictable. If F_2^0 happens, we stop the simulation and return a random bit. We can provide now an upper bound for $\mathbb{P}(F_2^0)$. For that we look at the probability of $\mathcal{A}$ querying one of these four oracles on the correct $\mathsf{PRK_{2e}}$ in our protocol session. Since for the adversary, $\mathsf{PRK_{2e}}$ is here unpredictable, we obtain the following probability: $\frac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK_{2e}}$ has size l_{hash}) for the guess of $\mathsf{PRK_{2e}}$. Now $\mathcal{A}$ can make at most $q_{\mathsf{RO_P}^{l_2}}$ (resp. $q_{\mathsf{RO_P}^{l_{\mathsf{key}}}}$, $q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}}$, and $q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}$) calls to $\mathsf{SO_P}^{l_2}$ (resp. $\mathsf{SO_P}^{l_{\mathsf{key}}}$, $\mathsf{SO_P}^{l_{\mathsf{iv}}}$ and $\mathsf{SO_P}^{l_{\mathsf{hash}}}$), we thus have:

$$\mathbb{P}(F_2^0) \leq \frac{q_{\mathsf{RO_P}^{l_2}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

Another way to detect the modification is on elements S_{I} and S_{R}. But since he cannot corrupt proxies, this will not help him in its distinguishing task, except if he wins the experiment $\mathbb{E}^{\mathrm{IND}}_{(\mathsf{KGen''},\mathsf{Enc''},\mathsf{Dec''}),\mathcal{A}}(l_{\mathsf{key}})$ (since elements contained in these ciphertexts are unpredictable), which is supposed difficult.
Note that this game is very similar to the game $\mathbb{G}_{23}^0$ of the key-security property proof, and so by the identical-until-bad lemma, we obtain:

$$\left|\mathbb{P}(\mathbb{G}_8^0 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_9^0 \Longrightarrow 1)\right| \leq \mathbb{P}(F_2^0) + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen''},\mathsf{Enc''},\mathsf{Dec''})}(l_{\mathsf{key}})$$

and thus:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_8^0 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_9^0 \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen''},\mathsf{Enc''},\mathsf{Dec''})}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}_8^1$:** This game is the same as the previous one (game $\mathbb{G}_{7'}^1$) and works in the same vein as game $\mathbb{G}_8^0$ by adding a failure condition: if the adversary, during the protocol session, queries $\mathsf{SO_T}$ on the

correct DIFFIE-HELLMAN value g^{xr}, then we abort and return a random bit. We then denote F_1^1 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct DIFFIE-HELLMAN value g^{xr} (with g^x the public DIFFIE-HELLMAN element of a challenger-controlled $\mathsf{Initiator}$ instance and r a challenger-controlled $\mathsf{Responder}$ instance long-term secret key). If the event F_1^1 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}\big(\mathbb{G}_{7'}^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_8^1 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_1^1).$$

Let's now provide a bound for $\mathbb{P}(F_1^1)$. For that we construct, by reduction, an adversary $\mathcal{B}_3^1$ against the GDH problem and we claim that:

$$\mathbb{P}(F_1^1) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_{7'}^1 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_8^1 \Longrightarrow 1\big)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda).}$$

‡ **<u>Proof of the claim:</u>** We want to provide an upper bound for $\mathbb{P}(F_1^1)$. As we did in the previous branch, by reduction we built an adversary $\mathcal{B}_3^1$ against the GDH problem.
On input a GDH challenge $(g, A = g^a, B = g^b)$ and having access to a DDH oracle, returning 1 if a tuple (g, g^a, g^b, g^c) is a DIFFIE-HELLMAN tuple, and 0 otherwise, our algorithm must returns g^{ab} in a polynomial time. For that, $\mathcal{B}_3^1$ simulates $\mathbb{G}_8^1$ for $\mathcal{A}(\mathsf{pp})$ as follows, mainly faithfully, but with some modifications. Firstly, $\mathcal{B}_3^1$ embeds its challenge into the shared DIFFIE-HELLMAN value: it picks at random $k \in \mathbb{Z}_p$ and sets $X \leftarrow Ag^k$. Then for the chosen $\mathsf{Responder}$, he sets $R \leftarrow B$.
Even if b (from g^b) remains unknown by $\mathcal{B}_3^1$, the latter can continue the simulation by querying the $\mathsf{SO_T}$ oracle on input (X, R).
If F_1^1 occurs on that particular $\mathsf{Responder}$ instance, then the value Z queried to the $\mathsf{SO_T}$ oracle will equal $g^{(a+k)b}$. To detect that, $\mathcal{B}_3^1$ can compute ZR^{-k}, which if F_1^1 occurs will equal $g^{ab}g^{kb}g^{-kb} = g^{ab}$, and test this equality with its DDH oracle on input (X, R, ZR^{-k}), which will return 1 since F_1^1 occurs. So when the event happens, $\mathcal{B}_3^1$ wins.
To finally bound $\mathbb{P}(F_1^1)$, we remark that each time the $\mathsf{SO_T}$ oracle is queried on input Z, the event F_1^1 can happen. Again, this implies that possibly $q_{\mathsf{RO_T}}$ DDH-oracle queries might be useful to detect the event.Thus we have:

$$\mathbb{P}(F_4^1) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda)$$

and this concludes the proof of the claim.

† **<u>Game $\mathbb{G}_{8'}^1$</u>:** This game is the same as the previous one and we work in the same vein as $\mathbb{G}_9^0$. The challenger simulates $\mathsf{PRK_{3e2m}}$ for both endpoints with a private oracle $\mathsf{SO_{PRK_{3e2m}}}$. As before, this key is unpredictable by the adversary (in normal times, $\mathsf{PRK_{3e2m}}$ is generated with $\mathsf{SO_T}$, $\mathsf{salt_{3e2m}}$ and g^{rx}).
One way to detect the modification is for the adversary to ask $\mathsf{SO_T}$ on the correct DIFFIE-HELLMAN value g^{xr}. Thanks to the event F_1^1 the simulation is already stopped in this situation. Another way to detect it is to query $\mathsf{SO_P}^{l_{\mathsf{hash}}}$, or $\mathsf{SO_P}^{l_{\mathsf{key}}}$, or $\mathsf{SO_P}^{l_{\mathsf{iv}}}$, or $\mathsf{SO_P}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK_{3e2m}}$ which unpredictable at this stage. We denote $F_{1'}^1$ the event: $\mathcal{A}$ queries $\mathsf{SO_P}^{l_{\mathsf{hash}}}$, or $\mathsf{SO_P}^{l_{\mathsf{key}}}$, or $\mathsf{SO_P}^{l_{\mathsf{iv}}}$,or $\mathsf{SO_P}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK_{3e2m}}$ for a session in which the latter is unpredictable. If $F_{1'}^1$ happens, we stop the simulation and return a random bit. We can provide now an upper bound for $\mathbb{P}(F_{1'}^1)$. For that we look at the probability of $\mathcal{A}$ querying one of these four oracles on the correct $\mathsf{PRK_{3e2m}}$ in our protocol session. Since for the adversary, $\mathsf{PRK_{3e2m}}$ is here unpredictable, we obtain the following probability: $\dfrac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK_{3e2m}}$ has size l_{hash}) for the guess of $\mathsf{PRK_{3e2m}}$. Now $\mathcal{A}$ can make at most $q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}}$ (resp.

$q_{\mathsf{RO_P}^{l_{\mathsf{key}}}}$, $q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}}$, and $q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}$) calls to $\mathsf{SO_P}^{l_{\mathsf{hash}}}$ (resp. $\mathsf{SO_P}^{l_{\mathsf{key}}}$, $\mathsf{SO_P}^{l_{\mathsf{iv}}}$ and $\mathsf{SO_P}^{l_{\mathsf{hash}}}$), we thus have:

$$\mathbb{P}(F^1_{1'}) \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

Again, another way to detect the modification is on elements S_{I} and S_{R}, thus by the identical-until-bad lemma, we obtain:

$$\left|\mathbb{P}\big(\mathbb{G}^1_8 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{8'} \Longrightarrow 1\big)\right| \leq \mathbb{P}(F^1_{1'}) + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}})$$

and thus:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_8 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{8'} \Longrightarrow 1\big)\right| \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}^1_9$:** This game is the same as game $\mathbb{G}^1_{8'}$, except that we apply the modification of the game $\mathbb{G}^0_8$. We perform the same analysis and reduction, and thus obtain, thanks to a p.p.t. algorithm $\mathcal{B}^1_4$:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_{8'} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_9 \Longrightarrow 1\big)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

† **Game $\mathbb{G}^1_{9'}$:** This game is the same as the previous one, except that we apply the modification of the game $\mathbb{G}^0_9$. To detect that change, $\mathcal{A}$ has two options: queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}, which is as usual excluded at this stage or pick a random PRK_{2e*} from $\{0,1\}^{l_{\mathsf{hash}}}$, hopping for $\mathsf{PRK}_{2e*} = \mathsf{PRK}_{2e}$. Similarly we denote $F^1_{2'}$ the event: $\mathcal{A}$ queries $\mathsf{SO_P}^{l_2}$ on the correct value PRK_{2e} for a session in which the latter is unpredictable (here only $\mathsf{SO_P}^{l_2}$ uses PRK_{2e}). If $F^1_{2'}$ happens, we stop the simulation and return a random bit. In a similar way as in game $\mathbb{G}^0_9$, we provide an upper bound for $\mathbb{P}(F^1_{2'})$ with:

$$\mathbb{P}(F^1_{2'}) \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}}.$$

Here again $\mathcal{A}$ can detect the modification on S_{I} and S_{R}. By the identical-until-bad lemma, we obtain:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^1_9 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^1_{9'} \Longrightarrow 1\big)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}^2_8$:** This game is the same as $\mathbb{G}^2_7$ and we keep going on the same vein as $\mathbb{G}^1_8$. If the adversary, during the session queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{iy}, then we abort and return a random bit. We then denote F^2_1 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{iy}. If the event F^2_1 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}\big(\mathbb{G}^2_7 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^2_8 \Longrightarrow 1\big)\right| \leq \mathbb{P}(F^2_1).$$

As in game $\mathbb{G}^1_8$ we construct, by reduction, an adversary $\mathcal{B}^2_3$ against the GDH problem and we bound $\mathbb{P}(F^2_1)$ by:

$$\mathbb{P}(F^2_1) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}^2_7 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}^2_8 \Longrightarrow 1\big)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

† **Game $\mathbb{G}^2_{8'}$:** This game is the same as the previous one and works in the same vein as $\mathbb{G}^1_{8'}$. Here the challenger generates the key $\mathsf{PRK}_{\mathsf{4e3m}}$ for both endpoints thanks to a private oracle $\mathsf{SO}_{\mathsf{PRK}_{\mathsf{4e3m}}}$ (in normal times, $\mathsf{PRK}_{\mathsf{4e3m}}$ is generated with SO_{T}, $\mathsf{salt}_{\mathsf{4e3m}}$ and g^{iy}). We perform now an analysis of what is happening in this case, and we obtain, similarly to game $\mathbb{G}^1_{8'}$:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_8 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{8'} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{key}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}^2_9$:** This game is the same as the previous one, except that we apply the modification of the game $\mathbb{G}^1_9$. We perform the same analysis and reduction, and thus obtain, thanks to a p.p.t. algorithm $\mathcal{B}^2_4$:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_{8'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_9 \Longrightarrow 1)\right| \leq q_{\mathsf{RO}_\mathsf{T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

† **Game $\mathbb{G}^2_{9'}$:** This game is the same as the previous one, except that we apply the modification of the game $\mathbb{G}^1_{9'}$. We can perform the same analysis and hence obtain thanks to the identical-until-bad lemma:

$$\boxed{\left|\mathbb{P}(\mathbb{G}^2_9 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^2_{9'} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO}_\mathsf{P}{}^{l_2}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}^3_8$:** This game follows $\mathbb{G}^3_{7'}$ and works in the same way as $\mathbb{G}^1_8$, with the appropriate notations. After defining the event F^3_1, we obtain the same results and thus have, thanks to a p.p.t. algorithm $\mathcal{B}^3_3$:

$$\left|\mathbb{P}(\mathbb{G}^3_{7'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_8 \Longrightarrow 1)\right| \leq q_{\mathsf{RO}_\mathsf{T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}^3_{8'}$:** This game is the same as the previous one, and we apply the same modification as in $\mathbb{G}^1_{8'}$ (with the appropriate notations), defining the event $F^3_{1'}$. Again we bound $\mathbb{P}(F^3_{1'})$ and thanks to the identical-until-bad lemma we obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}^3_8 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{8'} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{key}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).$$

† **Game $\mathbb{G}^3_9$:** This game is the same as the previous one and works in the same way as $\mathbb{G}^2_8$, with the appropriate notations. We thus obtain:

$$\left|\mathbb{P}(\mathbb{G}^3_{8'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_9 \Longrightarrow 1)\right| \leq q_{\mathsf{RO}_\mathsf{T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}^3_{9'}$:** This game is the same as the previous one, and we apply the same modification as in $\mathbb{G}^2_{8'}$ (with the appropriate notations). Again we obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}^3_9 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{9'} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{key}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO}_\mathsf{P}{}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).$$

† **Game $\mathbb{G}^3_{9''}$:** Here again, this game is the same as the previous one, and works in the same vein as $\mathbb{G}^1_9$. This leads again, thanks to a p.p.t. algorithm $\mathcal{B}^3_5$, to:

$$\left|\mathbb{P}(\mathbb{G}^3_{9'} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{9''} \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}^3_{9'''}$:** This game is the same as the previous one, and we apply the same modification as in game $\mathbb{G}^1_{9'}$ (with the appropriate notations). We thus obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}^3_{9''} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}^3_{9'''} \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}} l_2}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).$$

We can now conclude the four branches. We explicit games $\mathbb{G}_{10}$ and $\mathbb{G}_{11}$ instead of games $\mathbb{G}^{0,1,2,3}_{10}$ and $\mathbb{G}^{0,1,2,3}_{11}$ because, in the two cases, the four games are very similar and to make it lighten, we just explain, in parentheses, what can change.

† **Game $\mathbb{G}_{10}$:** This game is the same as the previous one, except that, we generate the keys sk_2 and sk_3 thanks to private oracles $\mathsf{SO}_{\mathsf{sk}_2}$ and $\mathsf{SO}_{\mathsf{sk}_3}$, which work in the same vein as $\mathsf{SO_P}^{l_2}$ and $\mathsf{SO_P}^{l_{\mathsf{key}}}$, but which are not accessible to $\mathcal{A}$ (usually sk_2 and sk_3 are obtain using $\mathsf{PRK}_{\mathsf{2e}}$ (or $\mathsf{PRK}_{\mathsf{3e2m}}$ or $\mathsf{PRK}_{\mathsf{4e3m}}$)). Once again the adversary may detect this modification only if he queries $\mathsf{SO_P}^{l_2}$ or $\mathsf{SO_P}^{l_{\mathsf{key}}}$ on the public (randomly generated) key $\mathsf{PRK}_{\mathsf{2e}}$ (or $\mathsf{PRK}_{\mathsf{3e2m}}$ or $\mathsf{PRK}_{\mathsf{4e3m}}$). Since there is $2^{l_{\mathsf{hash}}}$ possibility for $\mathsf{PRK}_{\mathsf{2e}}$ (or $\mathsf{PRK}_{\mathsf{3e2m}}$ or $\mathsf{PRK}_{\mathsf{4e3m}}$), the probability the adversary made these particular queries is then $\frac{2}{2^{l_{\mathsf{hash}}}}$. Note that here again $\mathcal{A}$ may detect a change in the ciphertexts S_{I} and S_{R}, but we can make the same analysis we did before. So finally we have:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_9 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{10} \Longrightarrow 1)\right| \leq \frac{2}{2^{l_{\mathsf{hash}}}} + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}''),\mathcal{A}}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}_{11}$:** This game works in the same vein as the previous one, except that we replace the choice of $\mathsf{ID_CRED_R} \leftarrow \mathsf{ID_CRED_R}_b$ by $\mathsf{ID_CRED_R} \leftarrow \mathsf{ID_CRED_R}_{1-b}$. So here we simulate the experiment $\mathbb{E}^{\text{Id-Responder-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ for $\mathcal{A}$ (despite everything, we remain in the context where $\mathsf{PRK}_{\mathsf{2e}}$ (or $\mathsf{PRK}_{\mathsf{3e2m}}$ or $\mathsf{PRK}_{\mathsf{4e3m}}$), sk_2 and sk_3 are simulated thanks to private oracles).
In this situation, the adversary will receive different ciphertexts c_2 and c_3, and also S_{I} and S_{R} (and also c_4, but there is nothing to learn from c_4 (or its decryption)). So he detects the changes we made, but this will not help him in its distinguishing task. Indeed, here he knows that the four messages did not come from the same $\mathsf{Responder}$ and he wants to succeed in distinguishing them. But since the keys sk_2 and sk_3 are random at this point, the adversary is forced to win the experiments $\mathbb{E}^{\mathrm{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec}),\mathcal{A}}(l_2)$, $\mathbb{E}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}''),\mathcal{A}}(l_{\mathsf{key}})$ and $\mathbb{E}^{\text{IND-CPA}}_{\mathrm{AEAD},\mathcal{A}}(l_{\mathsf{key}})$ (that is why we did not randomize the initialization vector IV_3), which were previously assumed to be very difficult to win. So finally we bound the winning advantage of the adversary with this game hop as follows:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_{10} \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{11} \Longrightarrow 1)\right| \leq \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec}),\mathcal{A}}(l_2) + \mathsf{Adv}^{\text{IND-CPA}}_{\mathrm{AEAD},\mathcal{A}}(l_{\mathsf{key}}) + 2\mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

So finally, to summarize, we have:

$$
\begin{aligned}
\mathsf{Adv}^{\text{Id-Pro-Resp}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) &= \Big|\mathbb{P}\Big(\mathbb{E}^{\text{Id-Responder-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big) - \mathbb{P}\Big(\mathbb{E}^{\text{Id-Responder-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big)\Big| \\
&= \Big|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{11} \Longrightarrow 1\big)\Big| \\
&\leq \Big|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big)\Big| + \cdots + \Big|\mathbb{P}\big(\mathbb{G}_{10} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{11} \Longrightarrow 1\big)\Big| \\
&\leq \frac{(q_{\mathsf{SO_T}} + q_{\mathsf{SO_P}^{l_{\mathsf{hash}}}})^2 + (q_{\mathsf{SO_T}} + q_{\mathsf{SO}_{\mathcal{H}}})^2 + (q_{\mathsf{SO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{SO}_{\mathcal{H}}})^2}{2^{l_{\mathsf{hash}}+1}} + 12\mathsf{Adv}^{\text{DDH}}_{\mathbb{G}}(\lambda) \\
&\quad + 8q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\text{GDH}}_{\mathbb{G}}(\lambda) + 5 \cdot \frac{q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + 4 \cdot \frac{q_{\mathsf{RO_P}^{l_2}} + q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + 2}{2^{l_{\mathsf{hash}}}} \\
&\quad + 4 \cdot \Big(8\mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}) + \mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen},\mathsf{Enc},\mathsf{Dec})}(l_2) + \mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD}}(l_{\mathsf{key}})\Big)
\end{aligned}
$$

and this concludes the proof. ■

7.4.2 Initiator identity protection

We continue with that property on the Initiator side.

Theorem 6 (Inititator **identity protecting LI-EDHOC protocol)** *Under the assumption that the* DDH *and the* GDH *problems are hard to solve in* $\mathbb{G}$*, that our encryption scheme* $(\mathsf{KGen}'', \mathsf{Enc}'', \mathsf{Dec}'')$ *is* IND-*secure, that our encryption scheme* $(\mathsf{KGen}', \mathsf{Enc}', \mathsf{Dec}')$ *in an* AEAD-*encryption scheme* IND-CPA-*secure, and that our digital signature scheme* DS *is* EUF-CMA$-$*secure, then the* $\mathsf{LI-EDHOC}$ *protocol, in its four versions, protect the* Initiator*'s identity against (partial) active adversaries. More precisely, for all p.p.t. adversary* $\mathcal{A}$ *making at most* $q_{\mathsf{RO_T}}$ *(resp.* $q_{\mathsf{RO_P}}$ *and* $q_{\mathsf{RO}_{\mathcal{H}}}$*) queries to the* $\mathsf{RO_T}$ *(resp.* $\mathsf{RO_P}$ *and* $\mathsf{RO}_{\mathcal{H}}$*) oracle, denoting* N *the number of endpoints, we have:*

$$
\begin{aligned}
\mathsf{Adv}^{\text{Id-Pro-Init}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \leq{}& \frac{(q_{\mathsf{RO_T}} + q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}})^2 + (q_{\mathsf{RO_T}} + q_{\mathsf{RO}_{\mathcal{H}}})^2 + (q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO}_{\mathcal{H}}})^2}{2^{l_{\mathsf{hash}}+1}} + N \cdot \Big[3\mathsf{Adv}^{\text{DDH}}_{\mathbb{G}}(\lambda) \\
&+ 5q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\text{GDH}}_{\mathbb{G}}(\lambda) + 3 \cdot \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + \frac{q_{\mathsf{RO_P}^{l_2}} + 1}{2^{l_{\mathsf{hash}}-1}} \\
&+ 2\mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD}}(l_{\mathsf{key}}) + 9\mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}) + 2\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda)\Big]
\end{aligned}
$$

Proof 6 (of theorem 6) To begin with, here again, in FIGURE 35, we remind the Initiator identity protection security games.

We proceed by game hops which are depicted in FIGURES 36 and 37 below. Firstly, game $\mathbb{G}_0$ is a reformulation of the original experiment $\mathbb{E}^{\text{IdPro-Initiator-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. Then with games $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_3$ we simulates random oracles RO as list, in order to manipulate them later, as in the previous proof. We next prevent collisions on some oracles with games $\mathbb{G}_4$ and $\mathbb{G}_5$, and with game $\mathbb{G}_6$ we try to guess in advance the adversary chosen Responder. We now need to divide our analysis into three branches, in order to distinguish between methods 0 and 2, 1, and 3 of the protocol (denoting games with exponent 0/2, 1, and 3 to indicate the method used in the current game). On the first branch corresponding to methods 0 and 2, with games $\mathbb{G}_7^{0/2}$ and $\mathbb{G}_8^{0/2}$ we prevent $\mathcal{A}$ from forging the Responder signature σ_2, and we conclude this branch with the game $\mathbb{G}_9^{0/2}$ corresponding the original experiment $\mathbb{E}^{\text{IdPro-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. The two other branches correspond to methods 1 and 3, but are very similar. With games $\mathbb{G}_7^{1/3}$ we randomize the H elements in order to prevent the adversary from obtaining informations about g^{xy} (or g^{iy} or g^{xr}). With the pack of games $\mathbb{G}_8^{1/3}$, $\mathbb{G}_9^{1/3}$, $\mathbb{G}_{10}^{1/3}$ and $\mathbb{G}_{11}^{1/3}$ we randomize the key-schedule elements to limit the capacity of $\mathcal{A}$ in predicting the encryption keys (and thus potentially learning the identity of the Initiator). Next with game $\mathbb{G}_{12}^{1/3}$ we randomize the encryption key $\mathsf{sk_3}$ to make it unpredictable to $\mathcal{A}$. Finally with game $\mathbb{G}_{13}^{1/3}$ we turn the game into the original experiment $\mathbb{E}^{\text{IdPro-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$, so we can conclude on the advantage of $\mathcal{A}$.

$\mathbb{E}^{\text{IdPro-Initiator-}b}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$

$\mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda)$
$\mathsf{ID_CRED_I}_0, \mathsf{ID_CRED_I}_1 \leftarrow \mathcal{A}(\mathsf{ESet}_{\mathsf{meth}})$
$\mathsf{ID_CRED_I} \leftarrow \mathsf{ID_CRED_I}_b$
$\mathsf{ID_CRED_R} \leftarrow \mathcal{A}(\mathsf{ESet}_{\mathsf{meth}})$
$M_1 \leftarrow \mathtt{InitRun1}(\mathsf{ID_CRED_I})$
$M_2 \leftarrow \mathcal{A}(\mathtt{RespRun1}(\mathsf{ID_CRED_R}))$
$M_3 \leftarrow \mathtt{InitRun2}(\mathsf{ID_CRED_I}, M_2)$
$M_3' \leftarrow \mathtt{Prox_IRun3}(\mathsf{ID_CRED_I}, M_3)$
$M \leftarrow (M_1, M_3, M_3')$
$b' \leftarrow \mathcal{A}(M)$
if $b = b'$ **then return** 1
else return 0

FIGURE 35 – The LI-EDHOC Initiator identity protection security games. By $\mathsf{ESet}_{\mathsf{meth}}$ we mean the set of all endpoints running the protocol with the same method (0,1,2, or 3).

† **Game $\mathbb{G}_0$:** This first game is the original experiment $\mathbb{E}^{\text{Id-Initiator-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$. As usually we have:

$$\boxed{\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{E}^{\text{Id-Initiator-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_1$:** As in the Responder identity protection proof, this game is the same as the previous one except that we simulate $\mathsf{RO}_{\mathcal{H}}$ thanks to a list denoted $\mathsf{SO}_{\mathcal{H}}$. As usually we obtain:

$$\boxed{\mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_2$:** Again, this game works as the previous one, except that we simulate RO_{P} thanks to a list denoted SO_{P} (still depending on the digest size, but again it does not impact the analysis). We thus obtain:

$$\boxed{\mathbb{P}\big(\mathbb{G}_2 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_3$:** To finish with the simulation of random oracles, this game is the same as the previous one, except that this time we simulate RO_{T} thanks to a special list denoted SO_{T} (following the same construction as in $\mathbb{G}_5$ in the key-security property proof (on page 55)). We obtain as before:

$$\boxed{\mathbb{P}\big(\mathbb{G}_3 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_2 \Longrightarrow 1\big).}$$

† **Game $\mathbb{G}_4$:** With this game and the next one, we want to avoid collision between some simulated random oracles. So this game is the same as the previous one, except that this time we raise a bad flag $\mathtt{bad_3}$ to $\mathtt{true}$ when a collision between $\mathsf{SO}_{\mathcal{H}}$ and SO_{T}, or SO_{T} and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or $\mathsf{SO}_{\mathcal{H}}$ and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$ occurs. This change is indistinguishable by the adversary and we obtain, as before:

$$\boxed{\mathbb{P}\big(\mathbb{G}_4 \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_3 \Longrightarrow 1\big).}$$

$\mathbb{G}_0$	Reformulation of the original experience
$\mathbb{G}_1$	Rewriting of $\mathsf{RO}_{\mathcal{H}}$ as a random list
$\mathbb{G}_2$	Rewriting of RO_{P} as a random list
$\mathbb{G}_3$	Rewriting of RO_{T} as a random list
$\mathbb{G}_4$	Flagging on collisions
$\mathbb{G}_5$	Restriction on potential collisions between some RO
$\mathbb{G}_6$	Guessing the adversary chosen Responder

FIGURE 36 – First steps of the game-schedule for the Initiator identity protection property.

† **Game $\mathbb{G}_5$:** This game is the same as the previous one, except that this time we abort when the bad flag $\mathtt{bad}_3$ is set to `true`. Using the same analysis as is game $\mathbb{G}_7$ of the key-secure property proof (on page 56), we bound the winning advantage of the adversary with this game hop as follows:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_4 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_5 \Longrightarrow 1\big)\Big| \leq \frac{(q_{\mathsf{SO}_\mathsf{T}}+q_{\mathsf{SO}_\mathsf{P}}\ell_{\mathsf{hash}})^2+(q_{\mathsf{SO}_\mathsf{T}}+q_{\mathsf{SO}_{\mathcal{H}}})^2+(q_{\mathsf{SO}_\mathsf{P}}\ell_{\mathsf{hash}}+q_{\mathsf{SO}_{\mathcal{H}}})^2}{2^{\ell_{\mathsf{hash}}+1}}.}$$

† **Game $\mathbb{G}_6$:** This game is the same as the previous one ($\mathbb{G}_5$), but here we try to guess in advance the Responder identity ID_CRED_R chosen by $\mathcal{A}$ at the beginning of the game. With this game hop, the winning advantage of $\mathcal{A}$ is increased by a factor equaling the probability of guessing correctly the Responder. Denoting $N = |\mathsf{ESet}_{\mathsf{meth}}|$, we thus have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_5 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big)\Big| \leq N \cdot \Big|\mathbb{P}\big(\mathbb{G}_6 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big)\Big|.}$$

From now on, we separate our analysis into three branches. The first one concerns the versions of the protocol in which the Responder uses a signature to authenticate (*i.e.,* methods 0 and 2), and the second and the third ones concern the two other versions (respectively methods 1 and 3). The game hops steps are depicted in FIGURE 37 below.

So we start with the first branch, *i.e.,* methods 0 and 2.

† **Game $\mathbb{G}_7^{0/2}$:** This game is the same as the previous one, except that we raised a bad flag $\mathtt{bad}_{\mathtt{sign}}$ to `true` when $\mathcal{A}$ forges the signature σ_2 in `RespRun2`. This change is internal to the game, and thus indistinguishable by $\mathcal{A}$. This leads to:

$$\boxed{\mathbb{P}\big(\mathbb{G}_7^{0/2} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_6 \Longrightarrow 1\big).}$$

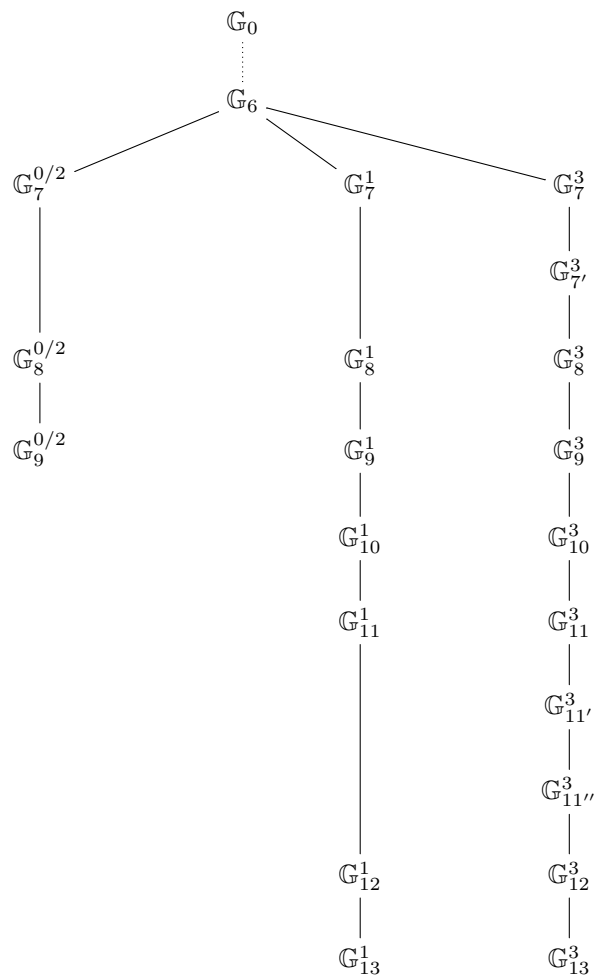

FIGURE 37 – Continuation of the game-schedule for the Initiator identity protection property.

† **Game $\mathbb{G}_8^{0/2}$:** This game is the same as game $\mathbb{G}_7^{0/2}$, except that we abort and return a random bit when the bad flag $\texttt{bad}_{\texttt{sign}}$ is raised to `true`. By the identical-until-bad lemma we have:

$$\left|\mathbb{P}\big(\mathbb{G}_7^{0/2} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_8^{0/2} \Longrightarrow 1\big)\right| \leq \mathbb{P}(F_1^{0/2})$$

where $F_1^{0/2}$ is the event: $\texttt{bad}_{\texttt{sign}}$ is raised to `true` in game $\mathbb{G}_7^{0/2}$.
We claim that by reduction we can build a p.p.t. adversary $\mathcal{B}_1^{0/2}$ against the EUF-CMA-security property of our digital signature scheme DS, which leads to:

$$\mathbb{P}(F_1^{0/2}) \leq \mathsf{Adv}_{\mathsf{DS},\mathcal{A}}^{\text{EUF-CMA}}(\lambda)$$

and thus to:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_7^{0/2} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_8^{0/2} \Longrightarrow 1\big)\right| \leq \mathsf{Adv}_{\mathsf{DS},\mathcal{A}}^{\text{EUF-CMA}}(\lambda).}$$

‡ **Proof of the claim:** We begin by assuming that there exists an adversary $\mathcal{A}$ that wins the game $\mathbb{G}_7^{0/2}$ with a non negligible advantage $\epsilon_{\mathcal{A}}(\lambda)$, by forging the signature σ_2. We can now present the

construction of $\mathcal{B}_1^{0/2}$. To begin with, the challenger generates a pair of public/private signature keys $(\widetilde{\mathsf{ssk}}, \widetilde{\mathsf{spk}})$ with the $\mathsf{DS.Gen}$ algorithm, and gives the public signing key $\widetilde{\mathsf{spk}}$ to $\mathcal{B}_1^{0/2}$. Algorithm $\mathcal{B}_1^{0/2}$ wins if it sends to its challenger a message/signature pair (m, σ) he did not generate, and such that $\mathsf{DS.Verif}(\widetilde{\mathsf{spk}}, m, \sigma) = 1$, in a polynomial time.
So $\mathcal{B}_1^{0/2}$ will start by simulating $\mathbb{G}_7^{0/2}$ to $\mathcal{A}$ faithfully, except for one point: if we denote by R the guessed $\mathsf{Responder}$, then $\mathcal{B}_1^{0/2}$ sets $\mathsf{R.spk} \leftarrow \widetilde{\mathsf{spk}}$.
Now when $\mathcal{B}_1^{0/2}$ simulates the continuation of the protocol run to $\mathcal{A}$, if the signature verification is successful, since R is not corrupted on its long-term signing keys, $\mathcal{B}_1^{0/2}$ knows that $\mathcal{A}$ forges the signature σ_2 on message $m = (l_{\mathsf{sig}} \| \mathsf{CTX}_2 \| t_2)$, and so $\mathcal{B}_1^{0/2}$ wins and breaks the EUF-CMA security of DS. So in this case we have $\mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{B}_1^{0/2}}(\lambda) \geq \epsilon_{\mathcal{A}}(\lambda)$.
We thus built well our p.p.t. adversary $\mathcal{B}_1^{0/2}$ which leads us to:

$$\mathbb{P}(F_1^{0/2}) \leq \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS},\mathcal{A}}(\lambda)$$

and this concludes the proof of the claim.

† **Game $\mathbb{G}_9^{0/2}$:** This game is the same as the previous one, except that we replace the choice of $\mathsf{ID_CRED_I} \leftarrow \overline{\mathsf{ID_CRED_I}}_b$ by $\mathsf{ID_CRED_I} \leftarrow \mathsf{ID_CRED_I}_{1-b}$. As in the previous proof, we here simulate the experiment $\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ for $\mathcal{A}$. Since in this situation $\mathcal{A}$ can anymore forge the signature σ_2, then he cannot obtain the continuation of the protocol run, *i.e.,* messages M_3 and M_3', and so he cannot detect this change. Moreover, M_1 is not link to $\mathsf{ID_CRED_I}$, and so $\mathcal{A}$ cannot detect this change here again. So finally we have:

$$\boxed{\mathbb{P}\big(\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_9^{0/2} \Longrightarrow 1\big) = \mathbb{P}\big(\mathbb{G}_8^{0/2} \Longrightarrow 1\big).}$$

So finally, to summarize and finalize this branch, we have:

$$\begin{aligned}
\mathsf{Adv}^{\text{Id-Pro-Init}}_{\mathsf{LI-EDHOC}\ 0/2,\mathcal{A}}(\lambda) &= \left|\mathbb{P}\Big(\mathbb{E}^{\text{Id-Initiator-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big) - \mathbb{P}\Big(\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big)\right| \\
&= \left|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_9^{0/2} \Longrightarrow 1\big)\right| \\
&\leq \left|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big)\right| + \cdots + \left|\mathbb{P}\big(\mathbb{G}_8^{0/2} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_9^{0/2} \Longrightarrow 1\big)\right| \\
&\leq \frac{(q_{\mathsf{SO_T}} + q_{\mathsf{SO_P}^{l_{\mathsf{hash}}}})^2 + (q_{\mathsf{SO_T}} + q_{\mathsf{SO}_{\mathcal{H}}})^2 + (q_{\mathsf{SO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{SO}_{\mathcal{H}}})^2}{2^{l_{\mathsf{hash}}+1}} + 2N \cdot \mathsf{Adv}^{\text{EUF-CMA}}_{\mathsf{DS}}(\lambda)
\end{aligned}$$

We now focus on the two remaining branches, *i.e.,* methods 1 and 3.

† **Game $\mathbb{G}_7^1$:** With the next game pack, we prevent the adversary from obtaining information from the $\overline{\mathsf{H}}$ elements. This game is the same as the previous one (gamme $\mathbb{G}_6$) except that this time we pick at random, according to the uniform distribution on the group $\mathbb{G}$, elements $\mathsf{H_I}$ and $\mathsf{H_I^1}$ of the $\mathsf{Initiator}$ (and of course simulate the proof $\mathsf{ni_I}$ with the simulator $\mathsf{Sim_{SoK}}$ associated to the SoK). As in game $\mathbb{G}_6^0$ of the $\mathsf{Responder}$ identity protection property proof, by reduction (slightly adapted), we can build a p.p.t. algorithm $\mathcal{B}_1^1$ against the DDH problem, and thus we have:

$$\boxed{\left|\mathbb{P}\big(\mathbb{G}_6 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_7^1 \Longrightarrow 1\big)\right| \leq \mathsf{Adv}^{\text{DDH}}_{\mathcal{A}}(\lambda).}$$

† **Game $\mathbb{G}_7^3$:** This game is the same as the previous one (game $\mathbb{G}_6$), and works in the same vein as game $\mathbb{G}_7^1$. Thanks to a p.p.t. algorithm $\mathcal{B}_1^3$, we have:

$$\left|\mathbb{P}(\mathbb{G}_6 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_7^3 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

† **Game $\mathbb{G}_{7'}^3$:** This game is the same as the previous one, except that this time we pick at random, according to the uniform distribution on the group $\mathbb{G}$, the element $\mathsf{H}_{\mathsf{I}}^2$ of the Initiator (and of course simulate the proof ni_{I} with the simulator $\mathtt{Sim}_{\mathsf{SoK}}$ associated to the SoK). As in game $\mathbb{G}_6^0$ of the Responder identity protection property proof, by reduction (slightly adapted), we can build a p.p.t. algorithm $\mathcal{B}_{1'}^3$ against the DDH problem, and thus we have:

$$\left|\mathbb{P}(\mathbb{G}_7^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{7'}^3 \Longrightarrow 1)\right| \leq \mathsf{Adv}_{\mathcal{A}}^{\mathrm{DDH}}(\lambda).$$

† **Game $\mathbb{G}_8^1$:** This game is the same as the previous one (game $\mathbb{G}_7^1$) and works in the same vein as game $\mathbb{G}_8^1$ of the Responder identity protection property proof, *i.e.,* by adding a failure condition: if the adversary, during the protocol session, queries SO_{T} on the correct Diffie-Hellman value g^{xr}, then we abort and return a random bit. We then denote F_1^1 the event: $\mathcal{A}$ queries SO_{T} on the correct Diffie-Hellman value g^{xr} (with g^x the public Diffie-Hellman element of a challenger-controlled Initiator instance and r an adversary-controlled, not corrupted, Responder instance long-term secret key). If the event F_1^1 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}(\mathbb{G}_7^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_8^1 \Longrightarrow 1)\right| \leq \mathbb{P}(F_1^1).$$

As we did in game $\mathbb{G}_8^1$ of the Responder identity protection property proof, we construct, by reduction, an adversary $\mathcal{B}_2^1$ against the GDH problem to bound $\mathbb{P}(F_1^1)$ and we claim that:

$$\mathbb{P}(F_1^1) \leq q_{\mathsf{RO}_{\mathsf{T}}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_7^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_8^1 \Longrightarrow 1)\right| \leq q_{\mathsf{RO}_{\mathsf{T}}} \cdot \mathsf{Adv}_{\mathbb{G}}^{\mathrm{GDH}}(\lambda).}$$

† **Game $\mathbb{G}_9^1$:** This game is the same as the previous one and we work in the same vein as $\mathbb{G}_9^0$ of the Responder identity protection property proof. The challenger simulates $\mathsf{PRK}_{\mathsf{3e2m}}$ for the Initiator with a private oracle $\mathsf{SO}_{\mathsf{PRK}_{\mathsf{3e2m}}}$. As before, this key is unpredictable by the adversary (in normal times, $\mathsf{PRK}_{\mathsf{3e2m}}$ is generated with SO_{T}, $\mathsf{salt}_{\mathsf{3e2m}}$ and g^{rx}).
One way to detect the modification is for the adversary to ask SO_{T} on the correct Diffie-Hellman value g^{xr}. Thanks to the event F_1^1 the simulation is already stopped in this situation. Another way to detect it is to query ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK}_{\mathsf{3e2m}}$ which unpredictable at this stage. We denote F_2^1 the event: $\mathcal{A}$ queries ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$,or ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}$ on the correct value $\mathsf{PRK}_{\mathsf{3e2m}}$ for a session in which the latter is unpredictable. If F_2^1 happens, we stop the simulation and return a random bit. We can provide now an upper bound for $\mathbb{P}(F_2^1)$. For that we look at the probability of $\mathcal{A}$ querying one of these four oracles on the correct $\mathsf{PRK}_{\mathsf{3e2m}}$ in our protocol session. Since for the adversary, $\mathsf{PRK}_{\mathsf{3e2m}}$ is here unpredictable, we obtain the following probability: $\frac{1}{2^{l_{\mathsf{hash}}}}$ (since $\mathsf{PRK}_{\mathsf{3e2m}}$ has size l_{hash}) for the guess of $\mathsf{PRK}_{\mathsf{3e2m}}$. Now $\mathcal{A}$ can make at most $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}}$ (resp. $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}}$, $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}}$, and $q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}$) calls to ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$ (resp. ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{key}}}$, ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}$ and ${\mathsf{SO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}$), we thus have:

$$\mathbb{P}(F_2^1) \leq \frac{q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{hash}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{key}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{iv}}}} + q_{{\mathsf{RO}_{\mathsf{P}}}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}}.$$

Again, another way to detect the modification is on the element S_{I}, thus by the identical-until-bad lemma, we obtain:

$$\left|\mathbb{P}(\mathbb{G}_8^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_9^1 \Longrightarrow 1)\right| \leq \mathbb{P}(F_2^1) + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}})$$

and thus:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_8^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_9^1 \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}_{10}^1$:** This game is the same as the previous one and works in the same vein as game $\mathbb{G}_8^0$ of the Responder identity protection property proof, *i.e.,* we added a failure condition: if the adversary, during the protocol session, queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}, we then abort and return a random bit. We then denote F_3^1 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}. If the event F_3^1 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}(\mathbb{G}_9^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{10}^1 \Longrightarrow 1)\right| \leq \mathbb{P}(F_3^1).$$

To provide a bound for $\mathbb{P}(F_3^1)$ we construct, by reduction, an adversary $\mathcal{B}_3^1$ against the GDH problem, as we did for game $\mathbb{G}_8^0$ in the Responder identity protection property proof. We hence claim that:

$$\mathbb{P}(F_3^1) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda),$$

and finally:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_9^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{10}^1 \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).}$$

† **Game $\mathbb{G}_{11}^1$:** This game is the same as the previous one, except that in this game we simulate the key generation of $\mathsf{PRK_{2e}}$ for our Initiator in the running session. Namely we simulate the generation of the key $\mathsf{PRK_{2e}}$ thanks to a private oracle, denoted $\mathsf{SO_{PRK_{2e}}}$, and working the same way as $\mathsf{SO_T}$, but inaccessible to $\mathcal{A}$ (in normal times, $\mathsf{PRK_{2e}}$ is generated thanks to $\mathsf{SO_T}$, $\mathsf{TH_2}$ and g^{xy}).
To detect that change, $\mathcal{A}$ has two options: queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{xy}, which is as usual excluded at this stage or pick a random $\mathsf{PRK_{2e*}}$ from $\{0,1\}^{l_{\mathsf{hash}}}$, hopping for $\mathsf{PRK_{2e*}} = \mathsf{PRK_{2e}}$. Similarly we denote F_4^1 the event: $\mathcal{A}$ queries $\mathsf{SO_P}^{l_2}$ on the correct value $\mathsf{PRK_{2e}}$ for a session in which the latter is unpredictable (here only $\mathsf{SO_P}^{l_2}$ uses $\mathsf{PRK_{2e}}$). If F_4^1 happens, we stop the simulation and return a random bit. In a similar way as in game $\mathbb{G}_9^0$ of the Responder identity protection property proof, we provide an upper bound for $\mathbb{P}(F_4^1)$ with:

$$\mathbb{P}(F_4^1) \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}}.$$

Here again $\mathcal{A}$ can detect the modification on S_{I}. By the identical-until-bad lemma, we obtain:

$$\boxed{\left|\mathbb{P}(\mathbb{G}_{10}^1 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{11}^1 \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}} + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}_8^3$:** This game follows $\mathbb{G}_{7'}^3$ and works in the same way as $\mathbb{G}_8^1$, with the appropriate notations. After defining the event F_1^3, we obtain the same results and thus have, thanks to a p.p.t. algorithm $\mathcal{B}_2^3$:

$$\left|\mathbb{P}(\mathbb{G}_{7'}^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_8^3 \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}_9^3$:** This game is the same as the previous one, and we apply the same modification as in $\mathbb{G}_9^1$ (with the appropriate notations), defining the event F_2^3. Again we bound $\mathbb{P}(F_2^3)$ and thanks to the identical-until-bad lemma we obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}_8^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_9^3 \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).$$

† **Game $\mathbb{G}_{10}^3$:** This game is the same as the previous one and works in the same way as game $\mathbb{G}_8^2$ of the Responder identity protection property proof: if the adversary, during the session queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{iy}, then we abort and return a random bit. We then denote F_3^3 the event: $\mathcal{A}$ queries $\mathsf{SO_T}$ on the correct Diffie-Hellman value g^{iy}. If the event F_3^3 occurs, then challenger stops the simulation by returning a random bit. By the identical-until-bad lemma, we thus have:

$$\left|\mathbb{P}(\mathbb{G}_9^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{10}^3 \Longrightarrow 1)\right| \leq \mathbb{P}(F_3^3).$$

As in game $\mathbb{G}_8^2$ (of the Responder identity protection property proof) we construct, by reduction, an adversary $\mathcal{B}_3^3$ against the GDH problem and we bound $\mathbb{P}(F_3^3)$ by:

$$\mathbb{P}(F_3^3) \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda),$$

and finally:

$$\left|\mathbb{P}(\mathbb{G}_9^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{10}^3 \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}_{11}^3$:** This game is the same as the previous one, except that here the challenger generates the key $\mathsf{PRK_{4e3m}}$ for the Initiator thanks to a private oracle $\mathsf{SO_{PRK_{4e3m}}}$ (in normal times, $\mathsf{PRK_{4e3m}}$ is generated with $\mathsf{SO_T}$, $\mathsf{salt_{4e3m}}$ and g^{iy}). We perform now an analysis of what is happening in this case, and we obtain, similarly to game $\mathbb{G}_{8'}^2$ of the Responder identity protection property proof:

$$\left|\mathbb{P}(\mathbb{G}_{10}^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{11}^3 \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).$$

† **Game $\mathbb{G}_{11'}^3$:** Here again, this game is the same as the previous one, and works in the same vein as $\mathbb{G}_9^1$. This leads again, thanks to a p.p.t. algorithm $\mathcal{B}_4^3$, to:

$$\left|\mathbb{P}(\mathbb{G}_{11}^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{11'}^3 \Longrightarrow 1)\right| \leq q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda).$$

† **Game $\mathbb{G}_{11''}^3$:** This game is the same as the previous one, and we apply the same modification as in game $\mathbb{G}_{10}^1$ (with the appropriate notations). We thus obtain the same result:

$$\left|\mathbb{P}(\mathbb{G}_{11'}^3 \Longrightarrow 1) - \mathbb{P}(\mathbb{G}_{11''}^3 \Longrightarrow 1)\right| \leq \frac{q_{\mathsf{RO_P}^{l_2}}}{2^{l_{\mathsf{hash}}}} + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).$$

We can now conclude the two remaining branches. We explicit games $\mathbb{G}_{12}$ and $\mathbb{G}_{13}$ instead of games $\mathbb{G}_{12}^{1,3}$ and $\mathbb{G}_{13}^{1,3}$ because, in the two cases, the two games are very similar.

† **Game $\mathbb{G}_{12}$:** This game is the same as the previous one, except that, we generate the key $\mathsf{sk_3}$ thanks to a private oracle $\mathsf{SO_{sk_3}}$ which works in the same vein as $\mathsf{SO_P}^{l_{\mathsf{key}}}$, but which is not accessible to $\mathcal{A}$ (usually $\mathsf{sk_3}$ is obtain using $\mathsf{PRK_{3e2m}}$ or $\mathsf{PRK_{4e3m}}$). Once again the adversary may detect this modification only

if he queries $\mathsf{SO_P}^{l_{\mathsf{key}}}$ on the public (randomly generated) key $\mathsf{PRK_{3e2m}}$ or $\mathsf{PRK_{4e3m}}$. Since there is $2^{l_{\mathsf{hash}}}$ possibility for $\mathsf{PRK_{3e2m}}$ or $\mathsf{PRK_{4e3m}}$, the probability the adversary made these particular queries is then $\frac{1}{2^{l_{\mathsf{hash}}}}$. Note that here again $\mathcal{A}$ may detect a change in the ciphertext S_{I}, but we can make the same analysis we did before. So finally we have:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{11} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{12} \Longrightarrow 1\big)\Big| \leq \frac{1}{2^{l_{\mathsf{hash}}}} + \mathsf{Adv}^{\mathrm{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}''),\mathcal{A}}(l_{\mathsf{key}}).}$$

† **Game $\mathbb{G}_{13}$:** This game is the same as the previous one, except that we replace the choice of $\mathsf{ID_CRED_I} \leftarrow \mathsf{ID_CRED_I}_b$ by $\mathsf{ID_CRED_I} \leftarrow \mathsf{ID_CRED_I}_{1-b}$. As in the previous proof, we here simulate the experiment $\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda)$ for $\mathcal{A}$ (and as before we remain in the context where $\mathsf{PRK_{3e2m}}$, $\mathsf{PRK_{4e3m}}$ and $\mathsf{sk_3}$ are simulated thanks to private oracles).
In this situation, the adversary will receive a different ciphertext c_3 (and different M_1, but as we explain earlier this does not help him to distinguish since M_1 is not linked to the $\mathsf{Initiator}$ identity). Then $\mathcal{A}$ detects this modification, but this will not help him to distinguish. Indeed here he knows that the two messages didn't come from the same $\mathsf{Initiator}$ and he wants to succeed in distinguishing them. But since the key $\mathsf{sk_3}$ is random at this point, $\mathcal{A}$ is forced to win the experiment $\mathbb{E}^{\text{IND-CPA}}_{\text{AEAD},\mathcal{A}}(l_{\mathsf{key}})$, which we previously assumed to be very difficult, if he wants to wins the game (that is why we didn't simulate (randomly) the initiator vector $\mathsf{IV_3}$). He will also receive a different ciphertext S_{I}, but again since the key $\mathsf{sk_3}$ is random he is forced to win the experiment $\mathbb{E}^{\text{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}''),\mathcal{A}}(l_{\mathsf{key}})$ which again was assumed to be very difficult to win. So finally we bound the winning advantage of the adversary with this game hop as follows:

$$\boxed{\Big|\mathbb{P}\big(\mathbb{G}_{12} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big)\Big| \leq \mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD},\mathcal{A}}(l_{\mathsf{key}}) + \mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}'')}(l_{\mathsf{key}}).}$$

So finally, to summarize, we have:

$$\begin{aligned}
\mathsf{Adv}^{\text{Id-Pro-Init}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) &= \Big|\mathbb{P}\Big(\mathbb{E}^{\text{Id-Initiator-1}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big) - \mathbb{P}\Big(\mathbb{E}^{\text{Id-Initiator-0}}_{\mathsf{LI-EDHOC},\mathcal{A}}(\lambda) \Longrightarrow 1\Big)\Big| \\
&= \Big|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big)\Big| \\
&\leq \Big|\mathbb{P}\big(\mathbb{G}_0 \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_1 \Longrightarrow 1\big)\Big| + \cdots + \Big|\mathbb{P}\big(\mathbb{G}_{12} \Longrightarrow 1\big) - \mathbb{P}\big(\mathbb{G}_{13} \Longrightarrow 1\big)\Big| \\
&\leq \frac{(q_{\mathsf{SO_T}} + q_{\mathsf{SO_P}^{l_{\mathsf{hash}}}})^2 + (q_{\mathsf{SO_T}} + q_{\mathsf{SO}_{\mathcal{H}}})^2 + (q_{\mathsf{SO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{SO}_{\mathcal{H}}})^2}{2^{l_{\mathsf{hash}}+1}} + N \cdot \Big[3\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}}(\lambda) \\
&\quad + 5q_{\mathsf{RO_T}} \cdot \mathsf{Adv}^{\mathrm{GDH}}_{\mathbb{G}}(\lambda) + 3 \cdot \frac{q_{\mathsf{RO_P}^{l_{\mathsf{hash}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{key}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{iv}}}} + q_{\mathsf{RO_P}^{l_{\mathsf{mac}}}}}{2^{l_{\mathsf{hash}}}} + \frac{q_{\mathsf{RO_P}^{l_2}} + 1}{2^{l_{\mathsf{hash}}-1}} \\
&\quad + 2\mathsf{Adv}^{\text{IND-CPA}}_{\text{AEAD},\mathcal{A}}(l_{\mathsf{key}}) + 9\mathsf{Adv}^{\text{IND}}_{(\mathsf{KGen}'',\mathsf{Enc}'',\mathsf{Dec}''),\mathcal{A}}(l_{\mathsf{key}})\Big]
\end{aligned}$$

and this concludes the proof. ■

8 Conclusion

Mobile environments come with a compulsory Lawful Interception (LI) clause for mobile network operators (MNOs). Naïve LI, however, is intrusive, essentially forcing the operator to eavesdrop on all communications.

Our work provides *fine-grained, session-specific* LI for the complex EDHOC protocol, which is being standardized for mobile use in the LAKE IETF working group. Our LI-EDHOC scheme provably guarantees AKE security, non-frameability, and a proof of LI key-recovery for EDHOC's 4 authentication methods, while preserving identity-protection and explicit authentication. Evaluations on two different ARM Cortex-M platforms, show that our protocol remains efficient in constrained environments, especially when the signatures of knowledge benefit from hardware acceleration. Even better performances are likely to be obtained for signature-based authentication methods, or if the processor were optimized for ECC computations.

9 Acknowledgments

This work has been partially supported by the French National Research Agency under the France 2030 label (NF-HiSec ANR-22-PEFT-0009) and through the PRIvacy-preserving tools for VAlidation and Security of Queries (PRIVASIQ) project (ANR-23-CE39-0008). The views reflected herein do not necessarily reflect the opinion of the French government.

References

[1] 3GPP. Lawful interception architecture and functions. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3182, 2018.

[2] 3GPP. Lawful interception requirements. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3181, 2018.

[3] 3GPP. Protocols and procedures for lawful interception. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3183, 2019.

[4] Harold Abelson, Ross Anderson, Steven M. Bellovin, Josh Benaloh, Matt Blaze, Whitfield Diffie, John Gilmore, Matthew Green, Susan Landau, Peter G. Neumann, Ronald L. Rivest, Jeffrey I. Schiller, Bruce Schneier, Michael A. Specter, and Daniel J. Weitzner. Keys under doormats. *Comms. of the ACM*, 58(10):24–26, 2015.

[5] Ghada Arfaoui, Olivier Blazy, Xavier Bultel, Pierre-Alain Fouque, Thibaut Jacques, Adina Nedelcu, and Cristina Onete. How to (legally) keep secrets from mobile operators. In *Proceedings of ESORICS*, pages 23–43, 2021.

[6] Mihir Bellare and Ronald L. Rivest. Translucent cryptography—an alternative to key escrow, and its implementation via fractional oblivious transfer. *Journal of Cryptology*, 12(2):117–139, 1999.

[7] Mihir Bellare and Phillip Rogaway. Code-based game-playing proofs and the security of triple encryption. Cryptology ePrint Archive, Paper 2004/331, 2004.

[8] Felipe Boeira, Mikael Asplund, and Marinho Barcellos. Provable non-frameability for 5g lawful interception. In *Proceedings of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '23, page 109–120. Association for Computing Machinery, 2023.

[9] Xavier Bultel and Cristina Onete. Pairing-free secure-channel establishment in mobile networks with fine-grained lawful interception. In *Proceedings of ACM SAC*, page 968–970, 2022.

[10] Jan Camenisch and Markus Stadler. Efficient group signature schemes for large groups. In *Advances in Cryptology — CRYPTO '97*, 1997.

[11] Jan Camenisch and Markus Stadler. Proof systems for general statements about discrete logarithms. *Technical Report/ETH Zurich, Department of Computer Science*, 260, 1997.

[12] Global Encryption Coalition. Joint statement on the dangers of the may 2024 council of the EU comrpomise proposal on EU CSAM. https://www.globalencryption.org/2024/05/joint-statement-on-the-dangers-of-the-may-2024-council-of-the-eu-compromise-proposal-on-eu-csam/, 2024.

[13] Baptiste Cottier and David Pointcheval. Security analysis of the edhoc protocol, 2022.

[14] Baptiste Cottier and David Pointcheval. Security analysis of improved edhoc protocol. In Guy-Vincent Jourdan, Laurent Mounier, Carlisle Adams, Florence Sèdes, and Joaquin Garcia-Alfaro, editors, *Foundations and Practice of Security*, pages 3–18, Cham, 2023. Springer Nature Switzerland.

[15] EU Council. Resolution on security through encryption and security despite encryption. https://data.consilium.europa.eu/doc/document/ST-13084-2020-REV-1/en/pdf, 2020.

[16] FairTrials. Short update: Police in germany defend the use of contact tracing for criminal investigations. https://www.fairtrials.org/news/short-update-police-germany-defend-use-contact-tracing-criminal-investigations, 2020.

[17] Lorenzo Franceschi-Bicchierai. The 10 biggest revelations from edward snowden's leaks. `https://mashable.com/2014/06/05/edward-snowden-revelations/?europe=true`, 2014.

[18] Felix Günther and Marc Ilunga Tshibumbu Mukendi. Careful with mac-then-sign: A computational analysis of the edhoc lightweight authenticated key exchange protocol. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pages 773–796, 2023.

[19] Xu He, Lixiang Li, and Haipeng Peng. A key escrow-free kp-abe scheme and its application in standalone authentication in iot. *IEEE Internet of Things Journal*, 11(7):11381–11394, 2024.

[20] Xianghong Hu, Xueming Li, Xin Zheng, Yuan Liu, and Xiaoming Xiong. A high speed processor for elliptic curve cryptography over nist prime field. *IET Circuits, Devices & Systems*, 16:350–359, 02 2022.

[21] Meijuan Huang, Yutian Liu, Bo Yang, Yanqi Zhao, and Mingrui Zhang. Efficient revocable attribute-based encryption with data integrity and key escrow-free. *Information*, 15:32, 01 2024.

[22] Ingrid Huso, Marco Olivieri, Leonardo Galgano, Adnan Rashid, Giuseppe Piro, and Gennaro Boggia. Design and implementation of a looking-forward lawful interception architecture for future mobile communication systems. *Comput. Networks*, 249:110518, 2024.

[23] Charlie Jacomme, Elise Klein, Steve Kremer, and Maïwenn Racouchot. A comprehensive, formal and automated analysis of the EDHOC protocol. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5881–5898, Anaheim, CA, August 2023. USENIX Association.

[24] Jake Januzelli and Jiayu Xu. A complete characterization of one-more assumptions in the algebraic group model. Cryptology ePrint Archive, Paper 2024/1954, 2024.

[25] Hugo Krawczyk. SIGMA: The 'SIGn-and-MAc' Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols. In *Advances in Cryptology - CRYPTO 2003*, volume 2729 of *LNCS*, pages 400–425, 2003.

[26] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, May 2010.

[27] Moxie Marlinspike and Trevor Perrin. The X3DH key agreement protocol. `https://signal.org/docs/specifications/x3dh/`, 2016.

[28] Elsa Lopez Perez, Thomas Watteyne, Rafael Marin-Lopez, Cristina Onete, and Mališa Vučinić. Pre-shared key authentication with edhoc: The security-performance tradeoff, 2025. awaiting for publication.

[29] Elsa López Pérez, Inria Göran Selander, John Preuß Mattsson, Thomas Watteyne, and Mališa Vučinić. Edhoc is a new security handshake standard: An overview of security analysis, 2024.

[30] 379 scientists and researchers from 36 countries. Open letter on the position of scientists and researchers on the updated version of the EU's proposed Child Sexual Abuse Regulation. `https://homes.esat.kuleuven.be/~preneel/Open_letter_CSAR_aug24_still_unacceptable.pdf`, 2024.

[31] G. Selander, J. Preuss Mattsson, and F. Palombini. Ephemeral Diffie-Hellman over COSE (EDHOC). RFC 9528, 2024.

[32] Göran Selander, John Preuß Mattsson, and Francesca Palombini. Ephemeral Diffie-Hellman Over COSE (EDHOC). RFC 9528, March 2024.

[33] Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Paper 2004/332, 2004.

[34] UN. Universal Declaration of Human Rights. https://www.un.org/en/universal-declaration-human-rights/, 1948.

[35] UN. Special rapporteur on privacy. https://www.ohchr.org/en/special-procedures/, 2021.

[36] Giuseppe Ungaro, Francesco Ricchitelli, Ingrid Huso, Giuseppe Piro, and Gennaro Boggia. Design and implementation of a lawful interception architecture for b5g systems based on key escrow. In *2022 IEEE Conference on Standards for Communications and Networking (CSCN)*, pages 207–207, 2022.

[37] Charles Wright and Mayank Varia. Crypto crumple zones: Enabling limited access without mass surveillance. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 288–306, 2018.